



RTL Design Sherpa

RTL AMBA Shared Infrastructure Module Reference — Rev 0.90

July 2026

Table of Contents

1 GAXI (Generic AXI) Modules.....	48
1.1 Overview.....	49
1.1.1 Key Features.....	49
1.2 Module Categories.....	49
1.2.1 Skid Buffers (Elastic Buffering).....	49
1.2.2 FIFOs (Deep Buffering).....	49
1.2.3 Test-Code Multi-Instance Helpers.....	50
1.3 Protocol Overview.....	50
1.3.1 Interface Signals.....	50
1.3.2 Handshake Rules.....	50
1.4 Quick Start.....	51
1.4.1 Basic Skid Buffer (Zero-Latency).....	51
1.4.2 Synchronous FIFO (Deep Buffering).....	51
1.4.3 Asynchronous FIFO (Clock Domain Crossing).....	52
1.4.4 Type-Safe Struct Buffer.....	52
1.4.5 Drop FIFO (Packet Processing).....	53
1.5 Testing.....	54
1.6 Design Patterns.....	54
1.6.1 Pattern 1: Timing Closure (Pipeline Stage).....	54
1.6.2 Pattern 2: Rate Adaptation.....	54
1.6.3 Pattern 3: Clock Domain Crossing.....	55
1.7 Performance Characteristics.....	55

1.7.1 Latency.....	55
1.7.2 Throughput.....	56
1.7.3 Resource Utilization (Typical).....	56
1.8 Common Use Cases.....	56
1.8.1 Video Processing (Skid Buffer).....	56
1.8.2 Network Packet Processing (Drop FIFO).....	57
1.8.3 Clock Domain Crossing (Async FIFO).....	57
1.9 Parameter Selection Guidelines.....	57
1.9.1 Buffer Depth (DEPTH).....	57
1.9.2 Output Mode (REGISTERED).....	58
1.9.3 CDC Synchronizer Stages (N_FLOP_CROSS).....	58
1.10 Related Documentation.....	58
1.10.1 Comprehensive Guide.....	58
1.10.2 Protocol Specifications.....	58
1.10.3 RTL Design Sherpa Documentation.....	58
1.10.4 Source Code.....	58
1.11 Design Notes.....	59
1.11.1 When to Use GAXI vs AXI4-Stream.....	59
1.11.2 Buffer vs FIFO Selection.....	59
1.12 Common Issues and Debugging.....	59
1.12.1 Issue 1: Data Loss.....	59
1.12.2 Issue 2: Timing Violation.....	59
1.12.3 Issue 3: CDC Metastability.....	59
1.13 Navigation.....	60

2 GAXI Drop FIFO Sync.....	60
2.1 Overview.....	60
2.2 Module Parameters.....	60
2.3 Interface Signals.....	61
2.3.1 Clock and Reset.....	61
2.3.2 Write Interface (GAXI Master → FIFO).....	61
2.3.3 Read Interface (FIFO → GAXI Slave).....	61
2.3.4 Drop Interface.....	62
2.3.5 Status Signals.....	62
2.4 Drop Operation Behavior.....	62
2.4.1 Drop by Count.....	62
2.4.2 Drop All.....	63
2.4.3 Simultaneous Operations.....	63
2.5 Timing Diagrams.....	63
2.5.1 Fill FIFO.....	63
2.5.2 Drop by Count.....	63
2.5.3 Drop All.....	64
2.5.4 Comprehensive Scenario.....	64
2.6 Usage Examples.....	64
2.6.1 Basic Write/Read.....	64
2.6.2 Drop Operation.....	65
2.7 Test Coverage.....	66
2.7.1 Running Tests.....	66
2.8 Design Notes.....	67

2.8.1 FIFO Depth Requirements.....	67
2.8.2 Registered vs. Mux Mode.....	67
2.8.3 Drop Latency.....	67
2.8.4 Almost Full/Empty Margins.....	67
2.9 Related Modules.....	67
2.10 Revision History.....	68
3 GAXI Asynchronous FIFO.....	68
3.1 Overview.....	68
3.1.1 Key Features.....	68
3.2 Module Interface.....	68
3.3 Parameters.....	69
3.4 Functional Description.....	71
3.4.1 Why Gray Code?.....	72
3.4.2 Dependencies.....	72
3.5 Usage Examples.....	72
3.5.1 Example 1: Basic CDC FIFO.....	72
3.5.2 Example 2: High-Speed to Low-Speed CDC.....	73
3.6 Timing Characteristics.....	73
3.7 Design Considerations.....	74
3.7.1 Depth Sizing for Clock Ratio.....	74
3.7.2 Reset Synchronization.....	74
3.7.3 Metastability Protection.....	74
3.8 Error Checking.....	75
3.9 Testing.....	75

3.10 Common Issues.....	76
3.10.1 Issue 1: Metastability.....	76
3.10.2 Issue 2: Pointer Synchronization Failure.....	76
3.10.3 Issue 3: Underflow/Overflow.....	76
3.11 Related Modules.....	76
3.12 References.....	76
4 GAXI Synchronous FIFO.....	76
4.1 Overview.....	77
4.1.1 Key Features.....	77
4.2 Module Interface.....	77
4.3 Parameters.....	77
4.4 Functional Description.....	78
4.4.1 Read Modes.....	78
4.4.2 Dependencies.....	79
4.5 Usage Examples.....	79
4.5.1 Example 1: Mux Mode (Low Latency).....	79
4.5.2 Example 2: Flop Mode (Timing Closure).....	79
4.6 Timing Characteristics.....	80
4.7 Resource Utilization.....	80
4.8 Testing.....	81
4.9 Related Modules.....	81
5 GAXI Register Slice.....	81
5.1 Overview.....	81
5.1.1 Key Features.....	81

5.2 Module Interface.....	82
5.3 Parameters.....	82
5.4 Functional Description.....	82
5.4.1 Architecture.....	82
5.4.2 Ready/Valid Protocol.....	83
5.4.3 Transfer Scenarios.....	83
5.4.4 Storage Update Logic.....	84
5.5 Usage Examples.....	85
5.5.1 Example 1: Pipeline Timing Isolation.....	85
5.5.2 Example 2: AXI Channel Register Slice.....	85
5.5.3 Example 3: Multiple Register Slices for Deep Pipelining.....	85
5.6 Timing Characteristics.....	86
5.6.1 Latency Guarantee.....	86
5.7 Comparison with Related Modules.....	87
5.7.1 gaxi_regslice vs gaxi_skid_buffer.....	87
5.7.2 gaxi_regslice vs gaxi_fifo_sync.....	87
5.8 Resource Utilization.....	88
5.8.1 FPGA Resources (Typical).....	88
5.9 Testing.....	88
5.9.1 Test Coverage.....	88
5.9.2 Run Tests.....	88
5.10 Design Considerations.....	88
5.10.1 Port Compatibility.....	88
5.10.2 Status Signals.....	89

5.10.3 Assertions.....	89
5.11 Related Modules.....	89
5.12 Common Use Cases.....	89
5.12.1 1. Breaking Long Combinatorial Paths.....	89
5.12.2 2. AXI Interface Pipelining.....	90
5.12.3 3. CDC Preparation.....	90
6 GAXI Skid Buffer.....	90
6.1 Overview.....	91
6.1.1 Key Features.....	91
6.2 Module Interface.....	91
6.3 Parameters.....	92
6.4 Ports.....	92
6.4.1 Clock and Reset.....	92
6.4.2 Write Interface.....	92
6.4.3 Read Interface.....	92
6.4.4 Status.....	93
6.5 Functional Description.....	93
6.5.1 Operation Modes.....	93
6.5.2 Internal State Machine.....	96
6.5.3 Ready/Valid Logic.....	97
6.6 Timing Characteristics.....	97
6.6.1 Latency.....	97
6.6.2 Throughput.....	97
6.6.3 Critical Paths.....	98

6.7 Comprehensive Timing Examples.....	98
6.7.1 Scenario 1: Zero-Latency Bypass.....	98
6.7.2 Scenario 2: Burst Write Until Full.....	99
6.7.3 Scenario 3: Simultaneous Read/Write.....	99
6.7.4 Scenario 4: Burst Read Until Empty.....	100
6.7.5 Scenario 5: Fill Then Drain Pattern.....	101
6.7.6 Scenario 6: Alternating Read/Write.....	102
6.8 Usage Examples.....	103
6.8.1 Example 1: Basic Pipeline Stage.....	103
6.8.2 Example 2: Backpressure Absorption.....	104
6.8.3 Example 3: Monitoring Occupancy.....	104
6.9 Design Considerations.....	105
6.9.1 Depth Selection.....	105
6.9.2 Timing Closure.....	105
6.9.3 Area vs. Performance Trade-offs.....	106
6.10 Error Checking.....	106
6.11 Testing.....	107
6.11.1 Test File.....	107
6.11.2 WaveDrom Test.....	107
6.11.3 Running Tests.....	107
6.12 Common Issues.....	107
6.12.1 Issue 1: Timing Violation on Bypass Path.....	107
6.12.2 Issue 2: Buffer Overflow.....	107
6.12.3 Issue 3: Unexpected Latency.....	108

6.13 Related Modules.....	108
6.14 References.....	108
7 GAXI Asynchronous Skid Buffer.....	108
7.1 Overview.....	108
7.1.1 Key Features.....	108
7.2 Module Interface.....	109
7.2.1 Internal Components.....	109
7.3 Parameters.....	110
7.4 Usage Examples.....	110
7.4.1 Example 1: Basic Async Skid Buffer.....	110
7.4.2 Example 2: Pipeline Stage with CDC.....	111
7.5 Timing Characteristics.....	111
7.5.1 Latency Components.....	111
7.5.2 Throughput.....	112
7.6 Design Considerations.....	112
7.6.1 When to Use This Module.....	112
7.6.2 Depth Selection.....	112
7.6.3 Reset Synchronization.....	112
7.7 Resource Utilization.....	112
7.8 Testing.....	113
7.9 Common Issues.....	113
7.9.1 Issue 1: CDC Violations.....	113
7.9.2 Issue 2: Write-Side Stalls.....	113
7.9.3 Issue 3: Unexpected Latency.....	113

7.10 Related Modules.....	113
7.11 References.....	114
8 GAXI Skid Buffer (Double-Drain).....	114
8.1 Overview.....	114
8.1.1 Key Features.....	114
8.2 Module Purpose.....	115
8.3 Parameters.....	115
8.4 Port Groups.....	116
8.4.1 Clock and Reset.....	116
8.4.2 Write Interface (input side).....	116
8.4.3 Read Interface (output side).....	116
8.4.4 Status.....	117
8.5 Functional Description.....	117
8.5.1 Transfer Detection.....	117
8.5.2 Storage Update.....	117
8.5.3 Ready/Valid Logic.....	118
8.5.4 Output Assignments.....	118
8.5.5 Legality Assertion.....	118
8.6 Usage Example.....	118
8.7 Design Notes.....	119
8.8 Related Modules.....	119
8.8.1 Used By.....	119
8.8.2 Uses.....	119
8.8.3 See Also.....	120

8.9 References.....	120
8.9.1 Source Code.....	120
8.9.2 Documentation.....	120
8.10 Navigation.....	120
9 gaxi_skid_buffer_struct.....	120
9.1 Overview.....	120
9.1.1 Key Features.....	121
9.2 Module Declaration.....	121
9.3 Parameters.....	122
9.4 Ports.....	122
9.4.1 Clock and Reset.....	122
9.4.2 Write Interface.....	122
9.4.3 Read Interface.....	122
9.4.4 Status.....	123
9.5 Usage Examples.....	123
9.5.1 Example 1: AXI Read Address Channel.....	123
9.5.2 Example 2: Custom Packet with Mixed Fields.....	124
9.5.3 Example 3: Simple Data Array.....	125
9.6 Behavior.....	125
9.6.1 Identical to gaxi_skid_buffer.....	125
9.6.2 Key Difference: Type Safety.....	125
9.7 Advantages Over Base Skid Buffer.....	126
9.7.1 Advantage 1: Type Safety.....	126
9.7.2 Advantage 2: No Manual Width Calculation.....	126

9.7.3 Advantage 3: Cleaner Code.....	126
9.8 Design Patterns.....	127
9.8.1 Pattern 1: AXI Channel Buffering.....	127
9.8.2 Pattern 2: Pipeline Stages with Complex Data.....	127
9.9 Testing.....	127
9.10 Synthesis Notes.....	128
9.10.1 Resource Usage.....	128
9.10.2 Type Requirements.....	128
9.11 Related Modules.....	128
9.12 When to Use.....	128
9.13 Navigation.....	129
10 Shared Infrastructure Documentation Status.....	129
10.1 Completion Status.....	129
10.1.1 Completed Documentation.....	129
10.1.2 New shared infrastructure (no dedicated page, covered above).....	132
10.1.3 Remaining Documentation (15 modules).....	132
10.2 Documentation Template.....	133
10.3 Design Notes.....	135
10.3.1 [Important design aspect 1].....	135
10.4 Related Modules.....	135
10.4.1 Used By.....	135
10.4.2 Uses.....	135
10.5 References.....	135
10.5.1 Specifications.....	135

10.5.2 Source Code.....	135
10.6 Navigation.....	135
10.6.1 axi_monitor_filtered.....	145
10.6.2 axi_monitor_trans_mgr.....	146
10.6.3 axi_monitor_reporter.....	147
10.6.4 axi_monitor_timeout.....	147
10.6.5 axi_monitor_timer.....	148
10.6.6 amba_clock_gate_ctrl.....	148
10.7 Arbitration.....	149
10.7.1 monbus_arbiter.....	149
10.7.2 arbiter_monbus_common.....	150
10.7.3 arbiter_rr_pwm_monbus.....	150
10.7.4 arbiter_wrr_pwm_monbus.....	151
10.8 AXI Utilities.....	151
10.8.1 axi_gen_addr.....	151
10.8.2 axi_master_rd_splitter.....	152
10.8.3 axi_master_wr_splitter.....	152
10.8.4 axi_split_combi.....	153
10.8.5 cdc_4_phase_handshake.....	153
10.9 Testing.....	154
10.10 Design Notes.....	155
10.10.1 Monitor Infrastructure.....	155
10.10.2 When to Use Each Module.....	155
10.11 Configuration Best Practices.....	155

10.11.1 Monitor Configuration.....	155
10.11.2 Timeout Configuration.....	156
10.11.3 Clock Gating Configuration.....	156
10.12 Performance Characteristics.....	156
10.12.1 Monitor Overhead.....	156
10.12.2 CDC Latency.....	157
10.12.3 Arbitration.....	157
10.13 Synthesis Notes.....	157
10.13.1 Clock Gating.....	157
10.13.2 CDC Constraints.....	157
10.14 Related Documentation.....	157
10.14.1 RTL Design Sherpa Documentation.....	157
10.14.2 Monitor Package.....	158
10.14.3 Source Code.....	158
10.15 Summary.....	158
10.16 Navigation.....	158
11 AMBA Clock Gate Controller.....	158
11.1 Overview.....	159
11.1.1 Key Features.....	159
11.2 Module Purpose.....	159
11.3 Parameters.....	160
11.4 Port Groups.....	160
11.4.1 Clock and Reset.....	160
11.4.2 Configuration Interface.....	160

11.4.3 Activity Monitoring.....	160
11.4.4 Clock Gating Outputs.....	161
11.5 Functional Description.....	161
11.5.1 Activity Detection Logic.....	161
11.5.2 Idle Signal Generation.....	162
11.5.3 Clock Gating Control Integration.....	162
11.5.4 Gating Threshold Behavior.....	162
11.5.5 Gating/Ungating Timing.....	163
11.6 Usage Example.....	163
11.6.1 Basic Integration with AXI Interface.....	163
11.6.2 Multi-Master Arbitration with Clock Gating.....	164
11.6.3 Dynamic Threshold Adjustment.....	165
11.7 Design Notes.....	165
11.7.1 Wakeup Signal Registration.....	165
11.7.2 Activity Signal Selection.....	166
11.7.3 Integration with ICG Cells.....	166
11.7.4 Power Measurement Methodology.....	166
11.7.5 Performance Considerations.....	167
11.8 Related Modules.....	167
11.8.1 Used By.....	167
11.8.2 Uses.....	167
11.8.3 See Also.....	167
11.9 References.....	168
11.9.1 Source Code.....	168

11.9.2 Documentation.....	168
11.9.3 Industry Standards.....	168
11.10 Navigation.....	168
12 AXI4 DMA Observer.....	168
12.1 Overview.....	169
12.2 What's inside.....	170
12.3 Parameters.....	170
12.4 Port surface.....	172
12.5 axi_bus_meter integration.....	172
12.5.1 Read-side channel attribution: runtime rid → channel map.....	173
12.5.2 Write-side channel attribution: optional sideband.....	173
12.5.3 Outputs.....	173
12.6 Pass-through correctness.....	174
12.7 Test.....	174
12.8 Related Modules.....	174
13 AXI4 DMA Slaves.....	175
13.1 Overview.....	175
13.1.1 Key Features.....	175
13.2 Module Purpose.....	176
13.3 Parameters.....	176
13.4 Port Groups.....	177
13.4.1 Clock and Reset.....	177
13.4.2 Read-Side Telemetry.....	178
13.4.3 Write-Side Telemetry.....	178

13.4.4 AXI4 Read Slave (AR + R).....	179
13.4.5 AXI4 Write Slave (AW + W + B).....	179
13.4.6 Status.....	180
13.5 Functional Description.....	180
13.5.1 Structural Composition.....	180
13.5.2 Read Side (LFSR Source).....	180
13.5.3 Write Side (CRC Sink).....	181
13.5.4 Shared CRC vs Independent Resets.....	181
13.6 Usage Example.....	181
13.7 Design Notes.....	182
13.8 Related Modules.....	182
13.8.1 Used By.....	182
13.8.2 Uses.....	183
13.8.3 See Also.....	183
13.9 References.....	183
13.9.1 Source Code.....	183
13.9.2 Documentation.....	183
13.10 Navigation.....	183
14 AXI4 Master Read CRC Checker.....	183
14.1 Overview.....	184
14.1.1 Key Features.....	184
14.2 Module Purpose.....	184
14.3 Parameters.....	185
14.4 Port Groups.....	186

14.4.1 Clock and Reset.....	186
14.4.2 Configuration (CSR) — same shape as the write generator.....	186
14.4.3 Telemetry.....	187
14.4.4 M-Side AXI4 Read (out to fabric/controller).....	188
14.4.5 Debug Observability (active when DBG_FIFO_DEPTH > 0).....	188
14.5 Functional Description.....	189
14.5.1 Workload FSM.....	189
14.5.2 Decoupled AR and R Address Generation.....	189
14.5.3 Expected Data: LFSR vs Address Hash.....	189
14.5.4 Pipelined Compare.....	190
14.5.5 CRC Accumulation.....	190
14.5.6 AR-ID Generation.....	190
14.5.7 Out-of-Order Completion — Known Limitation.....	190
14.5.8 Optional Debug FIFO.....	190
14.5.9 Standard Protocol Handler.....	191
14.6 Usage Example.....	191
14.7 Design Notes.....	192
14.8 Related Modules.....	192
14.8.1 Used By.....	192
14.8.2 Uses.....	193
14.8.3 See Also.....	193
14.9 References.....	193
14.9.1 Source Code.....	193
14.9.2 Documentation.....	193

14.10 Navigation.....	193
15 AXI4 Master Write Pattern Generator.....	194
15.1 Overview.....	194
15.1.1 Key Features.....	194
15.2 Module Purpose.....	194
15.3 Parameters.....	195
15.4 Port Groups.....	196
15.4.1 Clock and Reset.....	196
15.4.2 Configuration (CSR).....	197
15.4.3 Telemetry.....	198
15.4.4 M-Side AXI4 Write (out to fabric/controller).....	198
15.5 Functional Description.....	199
15.5.1 Workload FSM.....	199
15.5.2 Decoupled AW and W Address Generation.....	199
15.5.3 Data Path: LFSR vs Address Hash.....	199
15.5.4 Hash Pipeline and Staging FIFO.....	200
15.5.5 AW-ID Generation.....	200
15.5.6 CRC Accumulation and Completion.....	200
15.5.7 Standard Protocol Handler.....	200
15.6 Usage Example.....	201
15.7 Design Notes.....	202
15.8 Related Modules.....	202
15.8.1 Used By.....	202
15.8.2 Uses.....	202

15.8.3 See Also.....	203
15.9 References.....	203
15.9.1 Source Code.....	203
15.9.2 Documentation.....	203
15.10 Navigation.....	203
16 AXI4 Slave Read Pattern Generator.....	203
16.1 Overview.....	203
16.1.1 Key Features.....	204
16.2 Module Purpose.....	204
16.3 Parameters.....	205
16.4 Port Groups.....	206
16.4.1 Clock and Reset.....	206
16.4.2 Test Control.....	206
16.4.3 Per-Channel Telemetry.....	206
16.4.4 AXI4 Read Address Channel (AR).....	207
16.4.5 AXI4 Read Data Channel (R).....	208
16.4.6 Status.....	208
16.5 Functional Description.....	208
16.5.1 LFSR Pattern Generation.....	208
16.5.2 Data Replication to the Bus Width.....	208
16.5.3 Per-Channel CRC-32.....	209
16.5.4 Burst FSM and Gapless Back-to-Back Bursts.....	209
16.5.5 Standard Protocol Handler.....	209
16.6 Usage Example.....	209

16.7 Design Notes.....	210
16.8 Related Modules.....	211
16.8.1 Used By.....	211
16.8.2 Uses.....	211
16.8.3 See Also.....	211
16.9 References.....	211
16.9.1 Source Code.....	211
16.9.2 Documentation.....	211
16.10 Navigation.....	212
17 AXI4 Slave Write CRC Checker.....	212
17.1 Overview.....	212
17.1.1 Key Features.....	212
17.2 Module Purpose.....	213
17.3 Parameters.....	213
17.4 Port Groups.....	214
17.4.1 Clock and Reset.....	214
17.4.2 Test Control.....	214
17.4.3 Per-Channel Telemetry.....	215
17.4.4 AXI4 Write Address Channel (AW).....	215
17.4.5 AXI4 Write Data Channel (W).....	216
17.4.6 AXI4 Write Response Channel (B).....	216
17.4.7 Status.....	217
17.5 Functional Description.....	217
17.5.1 Data Slice Extraction.....	217

17.5.2 Per-Channel CRC-32.....	217
17.5.3 Write Burst FSM and Gapless Bursts.....	217
17.5.4 Back-to-Back B Response Id Latching.....	217
17.5.5 Standard Protocol Handler.....	218
17.6 Usage Example.....	218
17.7 Design Notes.....	219
17.8 Related Modules.....	219
17.8.1 Used By.....	219
17.8.2 Uses.....	219
17.8.3 See Also.....	220
17.9 References.....	220
17.9.1 Source Code.....	220
17.9.2 Documentation.....	220
17.10 Navigation.....	220
18 AXI Bus Meter.....	220
18.1 Overview.....	220
18.1.1 Key Features.....	221
18.2 Module Purpose.....	221
18.3 Parameters.....	222
18.4 Port Groups.....	222
18.4.1 Clock and Reset.....	222
18.4.2 Window Control.....	222
18.4.3 AXI Bus Snoop.....	222
18.4.4 Aggregate Counters.....	223

18.4.5 Per-Channel Counters.....	223
18.5 Functional Description.....	224
18.5.1 Bucket Classification.....	224
18.5.2 Aggregate Counters.....	224
18.5.3 Per-Channel Counters and Overflow Stickies.....	224
18.5.4 Recommended Wiring.....	225
18.5.5 Clear and Freeze Semantics.....	225
18.6 Usage Example.....	225
18.7 Design Notes.....	226
18.7.1 Passive Observer.....	226
18.7.2 Counter Width Rationale.....	226
18.7.3 Verilator Output Drivers.....	226
18.7.4 Window Alignment.....	227
18.8 Related Modules.....	227
18.8.1 Used By.....	227
18.8.2 Uses.....	227
18.8.3 See Also.....	227
18.9 References.....	227
18.9.1 Source Code.....	227
18.9.2 Documentation.....	227
18.10 Navigation.....	228
19 AXI Burst Address Generator.....	228
19.1 Overview.....	228
19.1.1 Key Features.....	228

19.2 Parameters.....	228
19.3 Ports.....	229
19.4 Functional Description.....	229
19.4.1 Increment Calculation.....	229
19.4.2 Burst Type Handling.....	229
19.4.3 Aligned Address Output.....	230
19.5 Usage Example.....	230
19.6 Design Notes.....	230
19.6.1 Data Width Conversion.....	230
19.6.2 WRAP Burst Alignment.....	231
19.6.3 Pure Combinational.....	231
19.7 Related Modules.....	231
19.7.1 Used By.....	231
19.7.2 Related Infrastructure.....	231
19.8 References.....	231
19.8.1 Specifications.....	231
19.8.2 Source Code.....	231
19.9 Navigation.....	231
20 AXI Master Read Splitter.....	232
20.1 Overview.....	232
20.1.1 Key Features.....	232
20.2 Module Purpose.....	232
20.3 Parameters.....	233
20.4 Port Groups.....	233

20.4.1 Clock and Reset.....	233
20.4.2 Configuration Interface.....	234
20.4.3 Master AXI Read Interface (Downstream).....	234
20.4.4 Slave AXI Read Interface (Upstream / FUB).....	235
20.4.5 Split Information Interface.....	237
20.5 Functional Description.....	237
20.5.1 Transaction Splitting Overview.....	237
20.5.2 State Machine Architecture.....	238
20.5.3 Boundary Crossing Detection.....	238
20.5.4 AR Channel Forwarding Logic.....	238
20.5.5 Ready Signal Management.....	238
20.5.6 R Channel Handling.....	239
20.5.7 Split Information Tracking.....	239
20.6 Usage Example.....	239
20.6.1 Basic Integration (4KB Boundary Alignment).....	239
20.6.2 Transaction Splitting Example Scenario.....	241
20.7 Design Notes.....	241
20.7.1 AXI Protocol Assumptions.....	241
20.7.2 Performance Characteristics.....	242
20.7.3 Error Handling.....	242
20.7.4 Integration Considerations.....	242
20.8 Related Modules.....	243
20.8.1 Used By.....	243
20.8.2 Uses.....	243

20.8.3 See Also.....	243
20.9 References.....	243
20.9.1 Source Code.....	243
20.9.2 Documentation.....	243
20.10 Navigation.....	244
21 AXI Master Write Splitter.....	244
21.1 Overview.....	244
21.1.1 Key Features.....	244
21.2 Module Purpose.....	244
21.3 Parameters.....	245
21.4 Port Groups.....	246
21.4.1 Clock and Reset.....	246
21.4.2 Configuration Interface.....	246
21.4.3 Master AXI Write Interface (Downstream).....	246
21.4.4 Slave AXI Write Interface (Upstream / FUB).....	248
21.4.5 Split Information Interface.....	250
21.5 Functional Description.....	250
21.5.1 Write Transaction Splitting Overview.....	250
21.5.2 State Machine Architecture.....	251
21.5.3 Write Data Channel Management.....	251
21.5.4 Response Consolidation Logic.....	251
21.5.5 Boundary Crossing Detection.....	252
21.5.6 AW Channel Forwarding Logic.....	252
21.5.7 Ready Signal Management.....	253

21.6 Usage Example.....	253
21.6.1 Basic Integration (4KB Boundary, Response Consolidation).....	253
21.6.2 Response Consolidation Example.....	255
21.7 Design Notes.....	255
21.7.1 AXI Protocol Assumptions.....	255
21.7.2 Response Consolidation Details.....	255
21.7.3 Performance Characteristics.....	256
21.7.4 Error Handling.....	256
21.7.5 Integration Considerations.....	256
21.8 Related Modules.....	257
21.8.1 Used By.....	257
21.8.2 Uses.....	257
21.8.3 See Also.....	257
21.9 References.....	257
21.9.1 Source Code.....	257
21.9.2 Documentation.....	257
21.10 Navigation.....	258
22 AXI Performance Latency Histogram.....	258
22.1 Overview.....	258
22.1.1 Key Features.....	258
22.2 Module Purpose.....	258
22.3 Parameters.....	259
22.4 Port Groups.....	260
22.4.1 Clock and Reset.....	260

22.4.2 Window Control.....	260
22.4.3 Command Channel (AR for reads, AW for writes).....	260
22.4.4 Read Data Channel (R) — used when IS_READ=1.....	260
22.4.5 Write Response Channel (B) — used when IS_READ=0.....	261
22.4.6 CSR-Indexed Readout.....	261
22.5 Functional Description.....	261
22.5.1 Metrics.....	261
22.5.2 Transaction Matching.....	261
22.5.3 Log2 Binning.....	262
22.5.4 Free-Running Timestamp.....	262
22.5.5 Four-Stage Update Pipeline.....	262
22.5.6 Window Control.....	262
22.5.7 Readout.....	263
22.6 Usage Example.....	263
22.7 Design Notes.....	264
22.7.1 Snoop-Only, Base Untouched.....	264
22.7.2 Timestamp Not Frozen.....	264
22.7.3 FIFO Depth vs Outstanding.....	264
22.7.4 Pipeline Drain Before Readback.....	264
22.8 Related Modules.....	264
22.8.1 Used By.....	264
22.8.2 Uses.....	264
22.8.3 See Also.....	265
22.9 References.....	265

22.9.1 Source Code.....	265
22.9.2 Documentation.....	265
22.10 Navigation.....	265
23 AXI Split Combinational Logic.....	265
23.1 Overview.....	265
23.1.1 Key Features.....	266
23.2 Module Purpose.....	266
23.3 Parameters.....	266
23.4 Port Groups.....	267
23.4.1 Clock and Reset (For Assertions Only).....	267
23.4.2 Input Signals.....	267
23.4.3 Output Signals.....	268
23.5 Functional Description.....	268
23.5.1 Boundary Crossing Detection Algorithm.....	268
23.5.2 Split Length Calculation.....	269
23.5.3 Remaining Length Calculation.....	269
23.5.4 State Machine Helper.....	270
23.6 Design Assumptions.....	270
23.6.1 Assumption 1: Address Alignment.....	270
23.6.2 Assumption 2: Fixed Transfer Size.....	270
23.6.3 Assumption 3: Incrementing Bursts Only.....	270
23.6.4 Assumption 4: No Address Wraparound.....	271
23.7 Usage Example.....	271
23.7.1 Integration with Read Splitter.....	271

23.7.2 Example Calculation Walkthrough.....	272
23.8 Design Notes.....	273
23.8.1 Synthesis Optimization.....	273
23.8.2 Assertion Coverage.....	274
23.8.3 Timing Considerations.....	274
23.8.4 Debug Support.....	274
23.9 Related Modules.....	275
23.9.1 Used By.....	275
23.9.2 Dependencies.....	275
23.9.3 See Also.....	275
23.10 References.....	275
23.10.1 Source Code.....	275
23.10.2 Documentation.....	275
23.11 Navigation.....	276
24 AXI-Stream Master Pattern Generator.....	276
24.1 Overview.....	276
24.1.1 Key Features.....	276
24.2 Module Purpose.....	276
24.3 Parameters.....	277
24.4 Port Groups.....	278
24.4.1 Clock and Reset.....	278
24.4.2 Configuration / Control.....	278
24.4.3 Per-Channel Telemetry.....	279
24.4.4 AXI-Stream Master (m_axis).....	279

24.5 Functional Description.....	280
24.5.1 Scheduling FSM.....	280
24.5.2 Sequential Per-Channel Scheduling.....	280
24.5.3 Per-Channel LFSR + CRC (Verbatim from the AXI4 blocks).....	280
24.5.4 Stream Outputs and tlast Cadence.....	281
24.5.5 Backpressure Insensitivity.....	281
24.6 Usage Example.....	281
24.7 Design Notes.....	282
24.8 Related Modules.....	282
24.8.1 Used By.....	282
24.8.2 Uses.....	282
24.8.3 See Also.....	282
24.9 References.....	283
24.9.1 Source Code.....	283
24.9.2 Documentation.....	283
24.10 Navigation.....	283
25 AXI-Stream Slave Pattern Checker.....	283
25.1 Overview.....	283
25.1.1 Key Features.....	284
25.2 Module Purpose.....	284
25.3 Parameters.....	285
25.4 Port Groups.....	286
25.4.1 Clock and Reset.....	286
25.4.2 Configuration / Control.....	286

25.4.3 Telemetry.....	286
25.4.4 AXI-Stream Slave (s_axis).....	287
25.5 Functional Description.....	288
25.5.1 Readiness and Beat Handshake.....	288
25.5.2 Per-Channel LFSR + CRC Regeneration (Verbatim from the Generator)	288
25.5.3 Per-Beat Compare and Sticky Error.....	288
25.5.4 Packet Counting.....	288
25.6 Usage Example.....	288
25.7 Design Notes.....	289
25.8 Related Modules.....	290
25.8.1 Used By.....	290
25.8.2 Uses.....	290
25.8.3 See Also.....	290
25.9 References.....	290
25.9.1 Source Code.....	290
25.9.2 Documentation.....	290
25.10 Navigation.....	291
26 AXIS Bus Meter.....	291
26.1 Overview.....	291
26.1.1 Key Features.....	291
26.2 Module Purpose.....	292
26.3 Parameters.....	292
26.4 Port Groups.....	293

26.4.1 Clock and Reset.....	293
26.4.2 Window Control.....	293
26.4.3 AXIS Bus Snoop.....	293
26.4.4 Aggregate Cycle Buckets.....	293
26.4.5 Aggregate AXIS-Native Throughput (productive beats only).....	294
26.4.6 Per-Channel Cycle Buckets (binned by tid).....	294
26.5 Functional Description.....	295
26.5.1 Bucket Classification.....	295
26.5.2 Payload-Byte Popcount.....	295
26.5.3 AXIS-Native Throughput Counters.....	295
26.5.4 Per-Channel Cycle Buckets.....	295
26.5.5 Clear and Freeze Semantics.....	296
26.6 Usage Example.....	296
26.7 Design Notes.....	297
26.7.1 Why Byte/Packet Counters Are Window-Independent.....	297
26.7.2 Per-Channel Idle Is Not Attributed.....	297
26.7.3 tstrb Handling.....	297
26.7.4 Counter Widths.....	297
26.8 Related Modules.....	297
26.8.1 Used By.....	297
26.8.2 Uses.....	298
26.8.3 See Also.....	298
26.9 References.....	298
26.9.1 Source Code.....	298

26.9.2 Documentation.....	298
26.10 Navigation.....	298
27 Clock-Gated Variants Guide.....	298
27.1 Overview.....	299
27.2 Key Principle.....	299
27.3 Gating Infrastructure.....	299
27.3.1 amba_clock_gate_ctrl (rtl/amba/shared/).	299
27.3.2 clock_gate_ctrl (rtl/common/).	300
27.4 Additional Parameter (All _cg Modules).....	300
27.5 Additional Ports (All _cg Modules).....	301
27.5.1 Configuration Inputs.....	301
27.5.2 Status Outputs.....	301
27.5.3 Ports That Do Not Exist.....	302
27.6 Clock Gating Behavior.....	302
27.6.1 Activity Detection.....	302
27.6.2 Gating Conditions (All Must Be True).....	302
27.6.3 Counting Convention.....	302
27.6.4 Register Stages by Family.....	303
27.6.5 Gating Latency.....	304
27.6.6 Wake-Up Behavior.....	305
27.6.7 Ready Forcing While Gated.....	305
27.7 Usage Patterns.....	306
27.7.1 Pattern 1: Aggressive Gating.....	306
27.7.2 Pattern 2: Runtime-Adjustable Threshold.....	306

27.7.3 Pattern 3: Gating Bypassed.....	307
27.8 Measuring Gated Cycles.....	307
27.9 Synthesis and Portability.....	308
27.9.1 The ICG Cell.....	308
27.10 Verification Considerations.....	309
27.10.1 Functional Verification.....	309
27.10.2 Gating Verification.....	309
27.11 Available Clock-Gated Variants.....	309
27.11.1 Transport Modules (22).....	309
27.11.2 Monitor Modules (12).....	310
27.12 Known Gaps.....	311
27.13 Related Documentation.....	311
27.14 Navigation.....	312
28 SDPRAM Core.....	312
28.1 Overview.....	312
28.1.1 Key Features.....	312
28.2 Module Purpose.....	313
28.3 Parameters.....	313
28.4 Port Groups.....	314
28.4.1 Clock and Reset.....	314
28.4.2 FUB Write Side (AW + W + B).....	314
28.4.3 FUB Read Side (AR + R).....	315
28.4.4 Bulk-Clear Control.....	316
28.4.5 Observation.....	316

28.5 Functional Description.....	317
28.5.1 FUB Interface Contract.....	317
28.5.2 Write Path — Burst Tracker.....	317
28.5.3 Read Path — Burst Tracker.....	317
28.5.4 Address Generation.....	317
28.5.5 BRAM Array.....	318
28.5.6 Bulk-Clear FSM.....	318
28.5.7 Observation Outputs.....	318
28.6 Usage Example.....	318
28.7 Design Notes.....	319
28.7.1 No Reset on the RAM Array.....	319
28.7.2 One Wire Format, Four Wrappers.....	320
28.7.3 WRAP Bursts.....	320
28.7.4 Single Outstanding Burst Per Side.....	320
28.8 Related Modules.....	320
28.8.1 Used By.....	320
28.8.2 Uses.....	320
28.8.3 See Also.....	320
28.9 References.....	321
28.9.1 Source Code.....	321
28.9.2 Documentation.....	321
28.10 Navigation.....	321
29 Simple Dual-Port BRAM Slave Family.....	321
29.1 Overview.....	321

29.2 Why four wrappers + a backend?.....	322
29.3 Common Parameters.....	322
29.4 Burst Support.....	323
29.5 Bulk Clear.....	323
29.6 Debug / Observation Outputs.....	324
29.7 Use in the Monitor System.....	324
29.8 Test.....	324
29.9 Migration.....	325
29.10 Related Modules.....	325
30 SDPRAM Slave — AXI4 Write / AXI4 Read.....	326
30.1 Overview.....	326
30.1.1 Key Features.....	326
30.2 Module Purpose.....	326
30.3 Parameters.....	327
30.4 Port Groups.....	328
30.4.1 Clock and Reset.....	328
30.4.2 AXI4 Slave Write (AW + W + B).....	328
30.4.3 AXI4 Slave Read (AR + R).....	329
30.4.4 Bulk-Clear Control and Debug.....	331
30.5 Functional Description.....	331
30.5.1 Structure.....	331
30.5.2 Field Pass-Through and Tie-Offs.....	332
30.5.3 Memory Behavior.....	332
30.5.4 Debug Taps.....	332

30.5.5 WRAP Assertion.....	332
30.6 Usage Example.....	332
30.7 Design Notes.....	333
30.7.1 Native AXI4, No Emulation.....	333
30.7.2 One Backend, Four Wrappers.....	333
30.7.3 WRAP Support Is Sim-Gated.....	333
30.8 Related Modules.....	334
30.8.1 Used By.....	334
30.8.2 Uses.....	334
30.8.3 See Also.....	334
30.9 References.....	334
30.9.1 Source Code.....	334
30.9.2 Documentation.....	334
30.10 Navigation.....	334
31 SDPRAM Slave — AXI4 Write / AXIL Read.....	335
31.1 Overview.....	335
31.1.1 Key Features.....	335
31.2 Module Purpose.....	335
31.3 Parameters.....	336
31.4 Port Groups.....	336
31.4.1 Clock and Reset.....	336
31.4.2 AXI4 Slave Write (AW + W + B).....	337
31.4.3 AXIL Slave Read (AR + R).....	338
31.4.4 Bulk-Clear Control and Debug.....	339

31.5 Functional Description.....	339
31.5.1 Structure.....	339
31.5.2 AXIL Read Bridge.....	340
31.5.3 Write Path and Tie-Offs.....	340
31.5.4 Memory Behavior.....	340
31.5.5 WRAP Assertion.....	340
31.6 Usage Example.....	340
31.7 Design Notes.....	341
31.7.1 Single-Beat Adaptation Lives in One Place.....	341
31.7.2 One Backend, Four Wrappers.....	341
31.7.3 Read Beat Size.....	341
31.8 Related Modules.....	342
31.8.1 Used By.....	342
31.8.2 Uses.....	342
31.8.3 See Also.....	342
31.9 References.....	342
31.9.1 Source Code.....	342
31.9.2 Documentation.....	342
31.10 Navigation.....	342
32 SDPRAM Slave — AXIL Write / AXI4 Read.....	343
32.1 Overview.....	343
32.1.1 Key Features.....	343
32.2 Module Purpose.....	343
32.3 Parameters.....	344

32.4 Port Groups.....	344
32.4.1 Clock and Reset.....	344
32.4.2 AXIL Slave Write (AW + W + B).....	345
32.4.3 AXI4 Slave Read (AR + R).....	345
32.4.4 Bulk-Clear Control and Debug.....	347
32.5 Functional Description.....	348
32.5.1 Structure.....	348
32.5.2 AXIL Write Bridge.....	348
32.5.3 Read Path and Tie-Offs.....	348
32.5.4 Memory Behavior.....	348
32.5.5 WRAP Assertion.....	348
32.6 Usage Example.....	348
32.7 Design Notes.....	349
32.7.1 Descriptor-RAM Fit.....	349
32.7.2 Single-Beat Adaptation Lives in One Place.....	350
32.7.3 One Backend, Four Wrappers.....	350
32.8 Related Modules.....	350
32.8.1 Used By.....	350
32.8.2 Uses.....	350
32.8.3 See Also.....	350
32.9 References.....	350
32.9.1 Source Code.....	350
32.9.2 Documentation.....	350
32.10 Navigation.....	351

33 SDPRAM Slave — AXIL Write / AXIL Read.....	351
33.1 Overview.....	351
33.1.1 Key Features.....	351
33.2 Module Purpose.....	352
33.3 Parameters.....	352
33.4 Port Groups.....	353
33.4.1 Clock and Reset.....	353
33.4.2 AXIL Slave Write (AW + W + B).....	353
33.4.3 AXIL Slave Read (AR + R).....	354
33.4.4 Bulk-Clear Control and Debug.....	354
33.5 Functional Description.....	355
33.5.1 Structure.....	355
33.5.2 Single-Beat Defaults on Both Sides.....	355
33.5.3 Memory Behavior.....	356
33.5.4 No WRAP Assertion.....	356
33.6 Usage Example.....	356
33.7 Design Notes.....	357
33.7.1 The Only “Fake” Fields, In One Place Per Side.....	357
33.7.2 Monitor-Bus Pairing.....	357
33.7.3 One Backend, Four Wrappers.....	357
33.8 Related Modules.....	357
33.8.1 Used By.....	357
33.8.2 Uses.....	357
33.8.3 See Also.....	357

33.9 References.....	358
33.9.1 Source Code.....	358
33.9.2 Documentation.....	358
33.10 Navigation.....	358
34 AMBA Protocol Shims.....	358
34.1 Overview.....	358
34.1.1 Key Features.....	358
34.2 Available Shims.....	359
34.3 Quick Start.....	359
34.3.1 AXI4 to APB Bridge.....	359
34.3.2 PeakRDL Register Integration.....	360
34.4 Design Patterns.....	360
34.4.1 Pattern 1: CPU to Peripherals Bridge.....	360
34.4.2 Pattern 2: High-Speed to Low-Speed Bridge.....	361
34.4.3 Pattern 3: Register Block Integration.....	361
34.5 Performance Characteristics.....	362
34.5.1 AXI4 to APB Bridge.....	362
34.5.2 PeakRDL Adapter.....	362
34.6 Testing.....	362
34.7 Common Use Cases.....	363
34.7.1 Use Case 1: SoC Integration.....	363
34.7.2 Use Case 2: DMA to Peripherals.....	363
34.7.3 Use Case 3: Configuration Registers.....	363
34.8 Parameter Selection Guidelines.....	364

34.8.1 AXI to APB Bridge.....	364
34.8.2 PeakRDL Adapter.....	364
34.9 Design Tradeoffs.....	365
34.9.1 AXI to APB Bridge.....	365
34.9.2 PeakRDL Adapter.....	365
34.10 Synthesis Notes.....	365
34.10.1 Resource Usage.....	365
34.10.2 Timing Closure.....	366
34.11 Related Documentation.....	366
34.11.1 Protocol Specifications.....	366
34.11.2 RTL Design Sherpa Documentation.....	366
34.11.3 External Resources.....	366
34.11.4 Source Code.....	366
34.12 Design Notes.....	367
34.12.1 When to Use Shims.....	367
34.12.2 Clock Domain Crossing.....	367
34.13 Migration Guide.....	367
34.13.1 Upgrading from Legacy AXI-APB Bridges.....	367
34.14 Troubleshooting.....	368
34.14.1 Issue 1: High Latency.....	368
34.14.2 Issue 2: PeakRDL Stalls.....	368
34.14.3 Issue 3: Data Corruption.....	368
34.15 Navigation.....	368
35 axi4_to_apb_convert.....	368

35.1 Overview.....	369
35.1.1 Key Features.....	369
35.2 Module Declaration.....	369
35.3 Parameters.....	370
35.4 Architecture.....	371
35.4.1 Component Structure.....	371
35.4.2 Data Flow.....	371
35.5 Behavior.....	372
35.5.1 Burst Decomposition.....	372
35.5.2 Address Generation.....	372
35.5.3 Width Adaptation.....	373
35.5.4 Error Handling.....	373
35.5.5 Side Information Tracking.....	373
35.6 Packet Formats.....	374
35.6.1 APB Command Packet.....	374
35.6.2 APB Response Packet.....	374
35.7 Usage Examples.....	374
35.7.1 Example 1: Standalone Conversion (Same Width).....	374
35.7.2 Example 2: Width Conversion (64b → 32b).....	375
35.8 State Machines.....	376
35.9 Performance.....	378
35.9.1 Latency (Cycles).....	378
35.9.2 Throughput.....	378
35.10 Testing.....	378

35.11 Constraints and Limitations.....	379
35.12 Synthesis Notes.....	379
35.12.1 Resource Usage.....	379
35.13 Related Modules.....	379
35.14 When to Use.....	380
35.15 Navigation.....	380
36 axi4_to_apb_shim.....	380
36.1 Overview.....	380
36.1.1 Key Features.....	380
36.2 Module Declaration.....	381
36.3 Parameters.....	383
36.3.1 Skid Buffer Depths.....	383
36.3.2 Internal Buffering.....	383
36.3.3 AXI Interface.....	384
36.3.4 APB Interface.....	384
36.4 Architecture.....	384
36.4.1 Component Hierarchy.....	384
36.4.2 Data Flow.....	385
36.5 Behavior.....	385
36.5.1 AXI to APB Conversion.....	385
36.5.2 Clock Domain Crossing.....	386
36.6 Usage Examples.....	386
36.6.1 Example 1: Same Clock, Same Width.....	386
36.6.2 Example 2: Dual-Clock with Width Conversion.....	386

36.6.3 Example 3: CPU-Peripheral Bridge.....	387
36.7 Performance Characteristics.....	388
36.7.1 Latency.....	388
36.7.2 Throughput.....	388
36.8 Testing.....	388
36.9 Constraints and Limitations.....	389
36.10 Synthesis Notes.....	389
36.10.1 Resource Usage (Typical).....	389
36.10.2 Timing Closure.....	389
36.11 Related Modules.....	390
36.12 When to Use.....	390
36.13 Navigation.....	390
37 peakrdl_to_cmdrsp.....	390
37.1 Overview.....	391
37.1.1 Key Features.....	391
37.2 Background: PeakRDL Integration.....	391
37.3 Module Declaration.....	391
37.4 Parameters.....	393
37.5 Ports.....	393
37.5.1 CMD/RSP Interface (RTL Design Sherpa Standard).....	393
37.5.2 PeakRDL Passthrough Interface.....	394
37.6 Behavior.....	395
37.6.1 Command State Machine.....	395
37.6.2 Response State Machine.....	395

37.6.3 Strobe to Bit Enable Conversion.....	396
37.6.4 Stall Handling.....	396
37.7 Usage Examples.....	396
37.7.1 Example 1: Basic Integration with PeakRDL Register Block.....	396
37.7.2 Example 2: APB to PeakRDL Complete Path.....	398
37.8 Timing Diagrams.....	399
37.8.1 Read Transaction (No Stall).....	399
37.8.2 Write Transaction with Stall.....	399
37.9 Testing.....	400
37.10 Assertions.....	400
37.11 Performance.....	400
37.11.1 Latency.....	400
37.11.2 Throughput.....	401
37.12 Synthesis Notes.....	401
37.12.1 Resource Usage.....	401
37.13 Related Modules.....	401
37.14 When to Use.....	401
37.15 PeakRDL Workflow.....	402
37.16 Navigation.....	402

1 GAXI (Generic AXI) Modules

Location: rtl/amba/gaxi/ **Test Location:** val/amba/ **Status:** ✓ Production Ready

1.1 Overview

The GAXI subsystem provides a lightweight valid/ready handshake protocol for streaming data between components. Simpler than full AXI4-Stream while maintaining robust flow control, GAXI is used throughout the RTL Design Sherpa infrastructure for elastic buffering, clock domain crossing, and timing closure.

1.1.1 Key Features

- ✓ **Simple Handshake:** Valid/ready only - minimal overhead
- ✓ **Elastic Buffering:** Skid buffers for zero-latency bypass
- ✓ **Clock Domain Crossing:** Async variants with proper CDC
- ✓ **Type-Safe Structs:** Struct-aware buffers for complex data
- ✓ **Drop Capability:** Special FIFO variant with drop-by-count/drop-all
- ✓ **Flexible Modes:** Mux/flop output modes for area/speed trade-offs

1.2 Module Categories

1.2.1 Skid Buffers (Elastic Buffering)

Module	Description	Documentation	Status
gaxi_skid_buffer	Synchronous elastic buffer with zero-latency bypass	gaxi_skid_buffer.md	✓ Documented
gaxi_skid_buffer_struct	Type-parameterized variant for complex structs	gaxi_skid_buffer_struct.md	✓ Documented
gaxi_skid_buffer_async	Asynchronous elastic buffer for CDC	gaxi_skid_buffer_async.md	✓ Documented
gaxi_skid_buffer_dbldr_n	Double-drain skid buffer variant	gaxi_skid_buffer_dbldr_n.md	✓ Documented

1.2.2 FIFOs (Deep Buffering)

Module	Description	Documentation	Status
gaxi_fifo_sync_nc	Synchronous FIFO with mux/flop modes	gaxi_fifo_sync.md	✓ Documented
gaxi_fifo_async	Asynchronous FIFO	gaxi_fifo_async.md	✓

Module	Description	Documentation	Status
ync	for clock domain crossing		Documented
gaxi_drop_fifo_sync	FIFO with drop-by-count and drop-all capability	gaxi_drop_fifo_sync.md	✓ Documented

1.2.3 Test-Code Multi-Instance Helpers

rtl/amba/testcode/ holds thin multi-instance wrappers used by the DV harnesses — they instantiate a bank of the base GAXI primitive (and, for the _sigmap variant, present a flattened array-signal port style). They are not documented individually; see the base module doc for the full behavior:

Test-code wrapper	Base module	Full documentation
gaxi_fifo_sync_multi	gaxi_fifo_sync	gaxi_fifo_sync.md
gaxi_fifo_async_multi	gaxi_fifo_async	gaxi_fifo_async.md
gaxi_skid_buffer_multi	gaxi_skid_buffer	gaxi_skid_buffer.md
gaxi_skid_buffer_multi_sigmap	gaxi_skid_buffer	gaxi_skid_buffer.md
gaxi_skid_buffer_async_multi	gaxi_skid_buffer_async	gaxi_skid_buffer_async.md

1.3 Protocol Overview

1.3.1 Interface Signals

Write Interface (Producer): - wr_valid - Data valid from source - wr_ready - Ready to accept data (backpressure) - wr_data[N:0] - Data from source

Read Interface (Consumer): - rd_valid - Data valid to sink - rd_ready - Ready to accept data from sink - rd_data[N:0] - Data to sink

Optional Monitoring: - count[3:0] - Current FIFO/buffer occupancy

1.3.2 Handshake Rules

1. **Transfer:** Occurs when valid && ready on rising clock edge
2. **Data Stability:** data must be stable when valid is high
3. **Backpressure:** ready may deassert to apply backpressure
4. **Valid Persistence:** valid must remain high until transfer completes

1.4 Quick Start

1.4.1 Basic Skid Buffer (Zero-Latency)

```
gaxi_skid_buffer #(
    .DATA_WIDTH(32),
    .DEPTH(4)           // 4 entries
) u_skid (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),

    // Write port (from producer)
    .wr_valid      (src_valid),
    .wr_ready      (src_ready),
    .wr_data       (src_data),

    // Read port (to consumer)
    .rd_valid      (dst_valid),
    .rd_ready      (dst_ready),
    .rd_data       (dst_data),

    // Status
    .count         (buf_count),
    .rd_count      ()
);
```

1.4.2 Synchronous FIFO (Deep Buffering)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(64),         // 64 entries
    .REGISTERED(0)      // 0=mux mode (fast), 1=flop mode (more area)
) u_fifo (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),

    .wr_valid      (wr_valid),
    .wr_ready      (wr_ready),
    .wr_data       (wr_data),

    .rd_valid      (rd_valid),
    .rd_ready      (rd_ready),
    .rd_data       (rd_data),

```

```

        .count          (fifo_level)
    );

```

1.4.3 Asynchronous FIFO (Clock Domain Crossing)

Two depth rules in this directory, and they are not a contradiction.

The shallow skid buffers (gaxi_skid_buffer, gaxi_skid_buffer_struct, gaxi_skid_buffer_dbldrn) store entries in a small unpacked array and accept DEPTH in {2, 4, 6, 8}. The FIFOs (gaxi_fifo_sync, gaxi_drop_fifo_sync, gaxi_fifo_async) address memory with a binary pointer and need a power of 2 – except gaxi_fifo_async with USE_JOHNSON=1, where Johnson pointers allow any even depth. Check the module you are instantiating, not the directory.

```

gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),           // Must be power of 2
    .N_FLOP_CROSS(3),    // CDC synchronizer stages
    .REGISTERED(0)
) u_async_fifo (
    // Write domain
    .axi_wr_aclk      (wr_clk),
    .axi_wr_aresetn   (wr_rst_n),
    .wr_valid         (wr_valid),
    .wr_ready         (wr_ready),
    .wr_data          (wr_data),

    // Read domain
    .axi_rd_aclk      (rd_clk),
    .axi_rd_aresetn   (rd_rst_n),
    .rd_valid         (rd_valid),
    .rd_ready         (rd_ready),
    .rd_data          (rd_data)
);

```

1.4.4 Type-Safe Struct Buffer

```

// Define AXI AR channel struct
typedef struct packed {
    logic [7:0]  arid;
    logic [31:0] araddr;
    logic [7:0]  arlen;
    logic [2:0]  arsize;
    logic [1:0]  arburst;
} axi_ar_t;

// Struct-aware buffer

```

```

gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(axi_ar_t),
    .DEPTH(4)
) u_ar_buf (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),

    .wr_valid      (master_arvalid),
    .wr_ready      (master_arready),
    .wr_data       (master_ar),      // Type-safe struct

    .rd_valid      (slave_arvalid),
    .rd_ready      (slave_arready),
    .rd_data       (slave_ar),      // Type-safe struct

    .count         (ar_count)
);

```

1.4.5 Drop FIFO (Packet Processing)

```

gaxi_drop_fifo_sync #(
    .DATA_WIDTH(512),
    .DEPTH(64),
    .REGISTERED(1)      // Flop mode for 512-bit width
) u_drop_fifo (
    .axi_aclk          (clk),
    .axi_aresetn       (rst_n),

    .wr_valid          (pkt_valid),
    .wr_ready          (pkt_ready),
    .wr_data           (pkt_data),

    .rd_valid          (proc_valid),
    .rd_ready          (proc_ready),
    .rd_data           (proc_data),

    // Drop control
    .drop_count        (drop_n_packets), // Drop N oldest entries
    .drop_count_valid  (drop_trigger),
    .drop_all          (flush_fifo),     // Flush entire FIFO

    .count             (pkt_count)
);

```

1.5 Testing

All GAXI modules are verified using CocoTB-based testbenches:

Run all GAXI tests

```
pytest val/amba/test_gaxi*.py -v
```

Run specific module tests

```
pytest val/amba/test_gaxi_buffer_sync.py -v      # Skid buffers + sync  
FIFOs
```

```
pytest val/amba/test_gaxi_buffer_async.py -v     # Async FIFO
```

```
pytest val/amba/test_gaxi_drop_fifo_sync.py -v   # Drop FIFO
```

Run with waveforms

```
pytest val/amba/test_gaxi_buffer_sync.py --vcd=waves.vcd -v
```

Generate WaveDrom timing diagrams

```
pytest val/amba/test_gaxi_wavedrom_example.py -v
```

```
cd val/amba && bash wd_cmd.sh # Generate PNG/SVG
```

1.6 Design Patterns

1.6.1 Pattern 1: Timing Closure (Pipeline Stage)

Use skid buffer to break combinatorial paths:

// Without buffer: long combinatorial path

```
assign downstream_data = upstream_data; // May not meet timing
```

// With skid buffer: registered boundary

```
gaxi_skid_buffer #(.DATA_WIDTH(32), .DEPTH(2)) u_pipeline (  
    .wr_valid(upstream_valid),  
    .wr_ready(upstream_ready),  
    .wr_data(upstream_data),  
    .rd_valid(downstream_valid),  
    .rd_ready(downstream_ready),  
    .rd_data(downstream_data),  
    ...  
);
```

1.6.2 Pattern 2: Rate Adaptation

Use FIFO to handle burst traffic:

```
// Bursty writer, continuous reader
gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(64)) u_rate_adapter (
    .wr_valid(bursty_valid),      // Intermittent bursts
    .wr_ready(bursty_ready),
    .wr_data(bursty_data),
    .rd_valid(continuous_valid), // Always reading
    .rd_ready(continuous_ready),
    .rd_data(continuous_data),
    ...
);
```

1.6.3 Pattern 3: Clock Domain Crossing

Use async FIFO for safe CDC:

```
// Domain A → Domain B
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .N_FLOP_CROSS(3)      // 3-stage synchronizer
) u_cdc (
    .axi_wr_aclk(clk_a),
    .axi_wr_aresetn(rst_a_n),
    .wr_valid(domain_a_valid),
    .wr_ready(domain_a_ready),
    .wr_data(domain_a_data),
    .axi_rd_aclk(clk_b),
    .axi_rd_aresetn(rst_b_n),
    .rd_valid(domain_b_valid),
    .rd_ready(domain_b_ready),
    .rd_data(domain_b_data)
);
```

1.7 Performance Characteristics

1.7.1 Latency

Module	Empty Latency	Full Latency	Notes
gaxi_skid_buffer	0 cycles	1 cycle	Zero-latency bypass when empty
gaxi_fifo_sync	1 cycle	1 cycle	Combinational

Module	Empty Latency	Full Latency	Notes
c (mux)			rial read
gaxi_fifo_sync c (flop)	2 cycles	2 cycles	Registered read
gaxi_fifo_async nc	3-5 cycles	3-5 cycles	CDC synchroni zation overhead
gaxi_drop_fifo_sync	1 cycle	1 cycle	Plus 3 cycles for drop operation

1.7.2 Throughput

All modules support **1 transfer per clock cycle** when not applying backpressure.

1.7.3 Resource Utilization (Typical)

Module	Logic	Memory	Notes
gaxi_skid_buffer (D=4, DW=32)	~50 LUTs	128 FFs	Shift register
gaxi_fifo_sync (D=16, DW=32)	~100 LUTs	512 FFs	+ counters
gaxi_fifo_async (D=16, DW=32)	~150 LUTs	512 FFs	+ CDC logic
gaxi_drop_fifo_sync (D=64, DW=32)	~200 LUTs	2048 FFs	+ drop control

1.8 Common Use Cases

1.8.1 Video Processing (Skid Buffer)

Characteristics: - High throughput (pixel streams) - Minimal latency required - Regular flow control

Recommended:

```
gaxi_skid_buffer #(
    .DATA_WIDTH(96),    // 4 pixels x 24-bit RGB
```



```
        .DEPTH(4)           // Shallow buffer
    )
```

1.8.2 Network Packet Processing (Drop FIFO)

Characteristics: - Variable packet sizes - Need to drop oldest on overflow - High data rate

Recommended:

```
gaxi_drop_fifo_sync #(
    .DATA_WIDTH(512),    // Wide data bus
    .DEPTH(64),          // Deep buffer
    .REGISTERED(1)       // Flop mode for timing
)
```

1.8.3 Clock Domain Crossing (Async FIFO)

Characteristics: - Independent clock domains - Metastability protection required - Variable clock ratios

Recommended:

```
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),          // Must be power of 2
    .N_FLOP_CROSS(3)     // 3-stage synchronizer
)
```

1.9 Parameter Selection Guidelines

1.9.1 Buffer Depth (DEPTH)

Application	Recommended Depth	Rationale
Pipeline stage	2-4	Minimal latency, timing closure
Video line buffer	16-32	Line-sized bursts
Network packet buffer	32-64	Variable packet sizes
CDC buffer	8-16	Handle clock ratio variations

1.9.2 Output Mode (REGISTERED)

Mux Mode (REGISTERED=0): - ✓ Lower latency (1 cycle) - ✓ Smaller area - ✗ May not meet timing with wide data - **Use for:** Narrow data (≤ 64 bits), non-critical paths

Flop Mode (REGISTERED=1): - ✓ Better timing (registered output) - ✓ Supports wide data - ✗ Higher latency (2 cycles) - ✗ More area - **Use for:** Wide data (≥ 128 bits), critical paths

1.9.3 CDC Synchronizer Stages (N_FLOP_CROSS)

Stages	Use Case
2	Slow clocks (<100 MHz), low risk
3	Recommended default
4+	High-speed (>400 MHz) or high-reliability

1.10 Related Documentation

1.10.1 Comprehensive Guide

- [GAXI Index \(Detailed\)](#) - Comprehensive guide with timing diagrams

1.10.2 Protocol Specifications

- ARM AMBA AXI4-Stream Protocol Specification (similar handshake)

1.10.3 RTL Design Sherpa Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem architecture
- [AXI4 Modules](#) - Full AXI4 protocol
- [AXI4-Stream Modules](#) - AXI4-Stream protocol
- [GAXI WaveDrom Tutorial](#) - Timing diagrams

1.10.4 Source Code

- RTL: rtl/amba/gaxi/
 - Tests: val/amba/test_gaxi*.py
 - Framework: bin/TBClasses/components/gaxi/
-

1.11 Design Notes

1.11.1 When to Use GAXI vs AXI4-Stream

✓ **Use GAXI For:** - Internal buffering and timing closure - Simple data streaming - CDC with basic handshake - Infrastructure/glue logic

✓ **Use AXI4-Stream For:** - External interfaces - Need TID/TDEST/TUSER signals - Complex routing requirements - Standards compliance

1.11.2 Buffer vs FIFO Selection

Use Skid Buffer (gaxi_skid_buffer) When: - ✓ Need zero-latency bypass - ✓ Shallow depth (2-8 entries) - ✓ Timing closure only - ✓ Minimal area

Use FIFO (gaxi_fifo_sync) When: - ✓ Need deeper buffering (16+ entries) - ✓ Rate adaptation required - ✓ Burst traffic handling - ✓ More predictable latency

Use Async FIFO (gaxi_fifo_async) When: - ✓ Clock domain crossing required - ✓ Independent clock domains - ✓ Need CDC-safe handshake

Use Drop FIFO (gaxi_drop_fifo_sync) When: - ✓ Need to discard oldest data on overflow - ✓ Packet-based processing - ✓ Flush/reset functionality required

1.12 Common Issues and Debugging

1.12.1 Issue 1: Data Loss

Symptom: Missing data at output

Debug: 1. Check count signal - should not overflow (reach DEPTH) 2. Verify `wr_valid` stays high until `wr_ready` asserts 3. Increase DEPTH if necessary 4. Add backpressure handling upstream

1.12.2 Issue 2: Timing Violation

Symptom: Setup/hold violations in synthesis

Solutions: 1. Use flop mode: `.REGISTERED(1)` 2. Increase DEPTH for better buffering 3. Add pipeline stages with multiple skid buffers

1.12.3 Issue 3: CDC Metastability

Symptom: Incorrect data, simulation/silicon mismatch

Solutions: 1. Increase `N_FLOP_CROSS` (minimum 3 recommended) 2. Verify independent clock domains (no relationship) 3. Check reset synchronization in both domains 4. Use constraints for CDC paths

1.13 Navigation

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

2 GAXI Drop FIFO Sync

2.1 Overview

The `gaxi_drop_fifo_sync` module is a synchronous FIFO with dynamic drop capability. It provides standard GAXI (Generic AXI) valid/ready handshaking for both write and read operations, plus an additional drop interface for removing entries from the FIFO without reading them.

Key Features: - Standard GAXI valid/ready handshaking - Configurable data width and depth - Drop by count (remove N oldest entries) - Drop all (flush entire FIFO) - I/O blocking during drop operations (3-cycle latency) - Registered or mux-based output modes - FIFO count and almost-full/almost-empty flags

Location: `rtl/amba/gaxi/gaxi_drop_fifo_sync.sv`

Testbench: `val/amba/test_gaxi_drop_fifo_sync.py`

2.2 Module Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Width of data bus in bits
DEPTH	int	16	FIFO depth (must be power of 2)
REGISTERED	int	0	0 = mux mode, 1 = registered output
ALMOST_WR_MARGIN	int	1	Threshold for almost_full flag

Parameter	Type	Default	Description
ALMOST_RD_MARGIN	int	1	Threshold for almost_empty flag

2.3 Interface Signals

2.3.1 Clock and Reset

Signal	Direction	Width	Description
axi_aclk	input	1	Clock signal
axi_aresetn	input	1	Active-low asynchronous reset

2.3.2 Write Interface (GAXI Master → FIFO)

Signal	Direction	Width	Description
wr_valid	input	1	Write data valid
wr_ready	output	1	FIFO ready to accept write
wr_data	input	DATA_WIDTH	Write data

Handshake: Write occurs when wr_valid && wr_ready on rising edge of clock.

2.3.3 Read Interface (FIFO → GAXI Slave)

Signal	Direction	Width	Description
rd_valid	output	1	Read data valid
rd_ready	input	1	Downstream ready to accept read
rd_data	output	DATA_WIDTH	Read data

Handshake: Read occurs when rd_valid && rd_ready on rising edge of clock.

2.3.4 Drop Interface

Signal	Direction	Width	Description
drop_valid	input	1	Drop request valid
drop_ready	output	1	Drop operation complete
drop_count	input	8	Number of entries to drop
drop_all	input	1	Drop all entries (ignore count)

Handshake: Drop completes when drop_valid && drop_ready on rising edge of clock.

Drop Latency: 3 clock cycles from drop_valid assertion to drop_ready assertion.

2.3.5 Status Signals

Signal	Direction	Width	Description
count	output	$\lceil \log_2(\text{DEPTH} + 1) \rceil$	Current number of entries in FIFO
almost_full	output	1	High when count $\geq (\text{DEPTH} - \text{ALMOST_WR_MARGIN})$
almost_empty	output	1	High when count $\leq \text{ALMOST_RD_MARGIN}$

2.4 Drop Operation Behavior

2.4.1 Drop by Count

When drop_valid=1 and drop_all=0, the FIFO removes drop_count oldest entries:

1. **Cycle 0:** Assert drop_valid, set drop_count=N
2. **Cycle 1-2:** Drop operation in progress, drop_ready=0, I/O blocked
3. **Cycle 3:** Drop complete, drop_ready=1
4. **Result:** FIFO count decreases by min(N, current_count)

I/O Blocking: During drop operation (cycles 1-2): - wr_ready = 0 (writes blocked) - rd_valid = 0 (reads blocked)

2.4.2 Drop All

When `drop_valid=1` and `drop_all=1`, the FIFO is completely flushed:

1. **Cycle 0:** Assert `drop_valid` and `drop_all=1`
2. **Cycle 1-2:** Drop operation in progress
3. **Cycle 3:** Drop complete, `drop_ready=1`
4. **Result:** FIFO count becomes 0

Note: `drop_count` is ignored when `drop_all=1`.

2.4.3 Simultaneous Operations

Write + Drop: Blocked - writes cannot proceed during drop operation.

Read + Drop: Blocked - reads cannot proceed during drop operation.

Drop + Drop: Not supported - wait for `drop_ready` before issuing next drop.

2.5 Timing Diagrams

2.5.1 Fill FIFO

Generated by `test_gaxi_drop_fifo_wavedrom.py`

This scenario shows writing 4 entries to an empty FIFO: - Write handshakes occur when `wr_valid && wr_ready` - FIFO count increments with each successful write - Data values: `0xA0`, `0xA1`, `0xA2`, `0xA3`

Timing diagram not yet captured. The scenario is described above. `test_gaxi_drop_fifo_wavedrom.py` is intended to emit `gaxi_drop_fifo_fill.json` into `docs/markdown/assets/WAVES/`, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.5.2 Drop by Count

Generated by `test_gaxi_drop_fifo_wavedrom.py`

This scenario demonstrates dropping 2 oldest entries: - `drop_valid` asserted with `drop_count=2` - 3-cycle latency until `drop_ready` assertion - FIFO count decreases by 2 - Normal I/O blocked during drop

Timing diagram not yet captured. The scenario is described above. `test_gaxi_drop_fifo_wavedrom.py` is intended to emit `gaxi_drop_fifo_drop_by_count.json` into

docs/markdown/assets/WAVES/, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.5.3 Drop All

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario shows flushing the entire FIFO: - drop_valid asserted with drop_all=1 - 3-cycle latency until drop_ready assertion - FIFO count becomes 0 - All entries discarded

Timing diagram not yet captured. The scenario is described above.

test_gaxi_drop_fifo_wavedrom.py is intended to emit gaxi_drop_fifo_drop_all.json into docs/markdown/assets/WAVES/, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.5.4 Comprehensive Scenario

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario demonstrates mixed operations: - Read 1 entry from FIFO - Drop remaining entries with drop_all=1 - Shows interaction between read and drop interfaces

Timing diagram not yet captured. The scenario is described above.

test_gaxi_drop_fifo_wavedrom.py is intended to emit gaxi_drop_fifo_comprehensive.json into docs/markdown/assets/WAVES/, but it does not currently write the file; the diagram is a documentation gap, not a missing feature of the module.

2.6 Usage Examples

2.6.1 Basic Write/Read

```
// Instantiate FIFO
gaxi_drop_fifo_sync #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .REGISTERED(0)
) u_fifo (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid      (wr_valid),
    .wr_ready      (wr_ready),
    .wr_data       (wr_data),
    .rd_valid      (rd_valid),
    .rd_ready      (rd_ready),
```



```

        .rd_data      (rd_data),
        .drop_valid   (1'b0),           // No drop
        .drop_ready   (),
        .drop_count   (8'h0),
        .drop_all     (1'b0),
        .count        (fifo_count),
        .almost_full  (almost_full),
        .almost_empty(almost_empty)
    );

    // Write logic
    always_ff @(posedge clk) begin
        if (!rst_n) begin
            wr_valid <= 1'b0;
        end else if (wr_valid && wr_ready) begin
            wr_valid <= 1'b0; // Deassert after handshake
        end else if (have_data_to_write && !almost_full) begin
            wr_valid <= 1'b1;
            wr_data  <= next_data;
        end
    end
end

```

2.6.2 Drop Operation

```

    // Drop state machine
    typedef enum logic [1:0] {
        IDLE,
        DROP_WAIT,
        DROP_DONE
    } drop_state_t;

    drop_state_t state;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state <= IDLE;
            drop_valid <= 1'b0;
        end else begin
            case (state)
                IDLE: begin
                    if (trigger_drop_all) begin
                        drop_valid <= 1'b1;
                        drop_all <= 1'b1;
                        drop_count <= 8'h0; // Ignored when drop_all=1
                        state <= DROP_WAIT;
                    end else if (trigger_drop_count) begin
                        drop_valid <= 1'b1;
                        drop_all <= 1'b0;
                    end
                end
            endcase
        end
    end

```

```

        drop_count <= num_to_drop;
        state <= DROP_WAIT;
    end
end

DROP_WAIT: begin
    if (drop_ready) begin
        drop_valid <= 1'b0;
        state <= DROP_DONE;
    end
end

DROP_DONE: begin
    // Single-cycle state
    state <= IDLE;
end
endcase
end
end

```

2.7 Test Coverage

Comprehensive verification is provided in `val/amba/test_gaxi_drop_fifo_sync.py`:

Test Scenario	Coverage
Basic FIFO Operation	Standard write/read without drops
Drop by Count	Partial FIFO flush with specific count
Drop All	Complete FIFO flush
Drop During I/O	Verify I/O blocking during drop
Wraparound with Drop	Drop across pointer wraparound boundary

Test Configurations: - Data widths: 8, 32, 64 bits - Depths: 8, 16, 64, 256 entries - Modes: Mux (REGISTERED=0) and Flop (REGISTERED=1)

Results: All 8 parameterized test cases pass ✓

2.7.1 Running Tests

Run all drop FIFO tests

`pytest val/amba/test_gaxi_drop_fifo_sync.py -v`

```
# Run specific test configuration
pytest
"val/amba/test_gaxi_drop_fifo_sync.py::test_gaxi_drop_fifo_sync[8-8-0-
minimal-mux]" -V

# Run smoke test (quick verification)
pytest val/amba/test_gaxi_drop_fifo_sync.py::test_gaxi_drop_fifo_smoke
-V

# Generate waveforms for documentation
env ENABLE_WAVEDROM=1 pytest val/amba/test_gaxi_drop_fifo_wavedrom.py
-V
```

2.8 Design Notes

2.8.1 FIFO Depth Requirements

MUST be a power of 2 for efficient address pointer implementation.

Valid: 8, 16, 32, 64, 128, 256, ...

Invalid: 10, 15, 20, 100, ...

2.8.2 Registered vs. Mux Mode

Mux Mode (REGISTERED=0): - Lower latency (combinatorial path from FIFO RAM to rd_data) - Better for timing-critical paths where output is registered downstream - Simpler implementation

Registered Mode (REGISTERED=1): - Extra register stage on rd_data output - Better timing closure for long data paths - Adds 1 cycle of read latency

2.8.3 Drop Latency

The 3-cycle drop latency is inherent to the implementation: 1. **Cycle 1:** Latch drop request 2. **Cycle 2:** Update read pointer 3. **Cycle 3:** Assert drop_ready

This latency ensures clean separation between normal FIFO operations and drop operations.

2.8.4 Almost Full/Empty Margins

Configure ALMOST_WR_MARGIN and ALMOST_RD_MARGIN based on: - Pipeline depth of upstream producer - Pipeline depth of downstream consumer - Latency tolerance requirements

Example: If upstream has 4-cycle latency, set ALMOST_WR_MARGIN >= 4.

2.9 Related Modules

- `fifo_control.sv` - Core FIFO control logic

- counter_bin.sv - Binary counter for address generation
- counter_bin_load.sv - Loadable binary counter for drop pointer updates
- gaxi_drop_fifo_async.sv - Asynchronous clock domain version (future)

2.10 Revision History

Version	Date	Description
1.0	2025-10-17	Initial documentation with WaveDrom examples

3 GAXI Asynchronous FIFO

Module: gaxi_fifo_async.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

3.1 Overview

The GAXI asynchronous FIFO enables safe clock domain crossing (CDC) between independent clock domains. It uses Gray code pointers and multi-flop synchronizers to prevent metastability and ensure data integrity.

3.1.1 Key Features

- ✓ **Clock Domain Crossing:** Safe transfer between independent clocks
- ✓ **Gray Code Pointers:** Prevents multi-bit synchronization issues
- ✓ **Configurable CDC Stages:** 2-3 flop synchronizers (3 recommended)
- ✓ **Arbitrary Clock Ratios:** Works with any write:read clock ratio
- ✓ **Two Read Modes:** Mux or Flop mode

3.2 Module Interface

```

module gaxi_fifo_async #(
    parameter int REGISTERED = 0,           // 0=mux mode, 1=flop mode
    parameter int DATA_WIDTH = 8,
    parameter int DEPTH = 16,               // power of 2 unless
USE_JOHNSON=1
    parameter int USE_JOHNSON = 0,         // 0=Gray pointers, 1=Johnson
pointers

```

```

parameter int N_FLOP_CROSS = 2,           // CDC synchronizer stages
parameter int ALMOST_WR_MARGIN = 1,
parameter int ALMOST_RD_MARGIN = 1,
parameter     INSTANCE_NAME = "DEADF1F0"
) (
    // Write Domain
    input  logic          axi_wr_aclk,
    input  logic          axi_wr_aresetn,
    input  logic          wr_valid,
    output logic          wr_ready,        // not full
    input  logic [DW-1:0] wr_data,

    // Read Domain
    input  logic          axi_rd_aclk,
    input  logic          axi_rd_aresetn,
    input  logic          rd_ready,
    output logic          rd_valid,        // not empty
    output logic [DW-1:0] rd_data
);

```

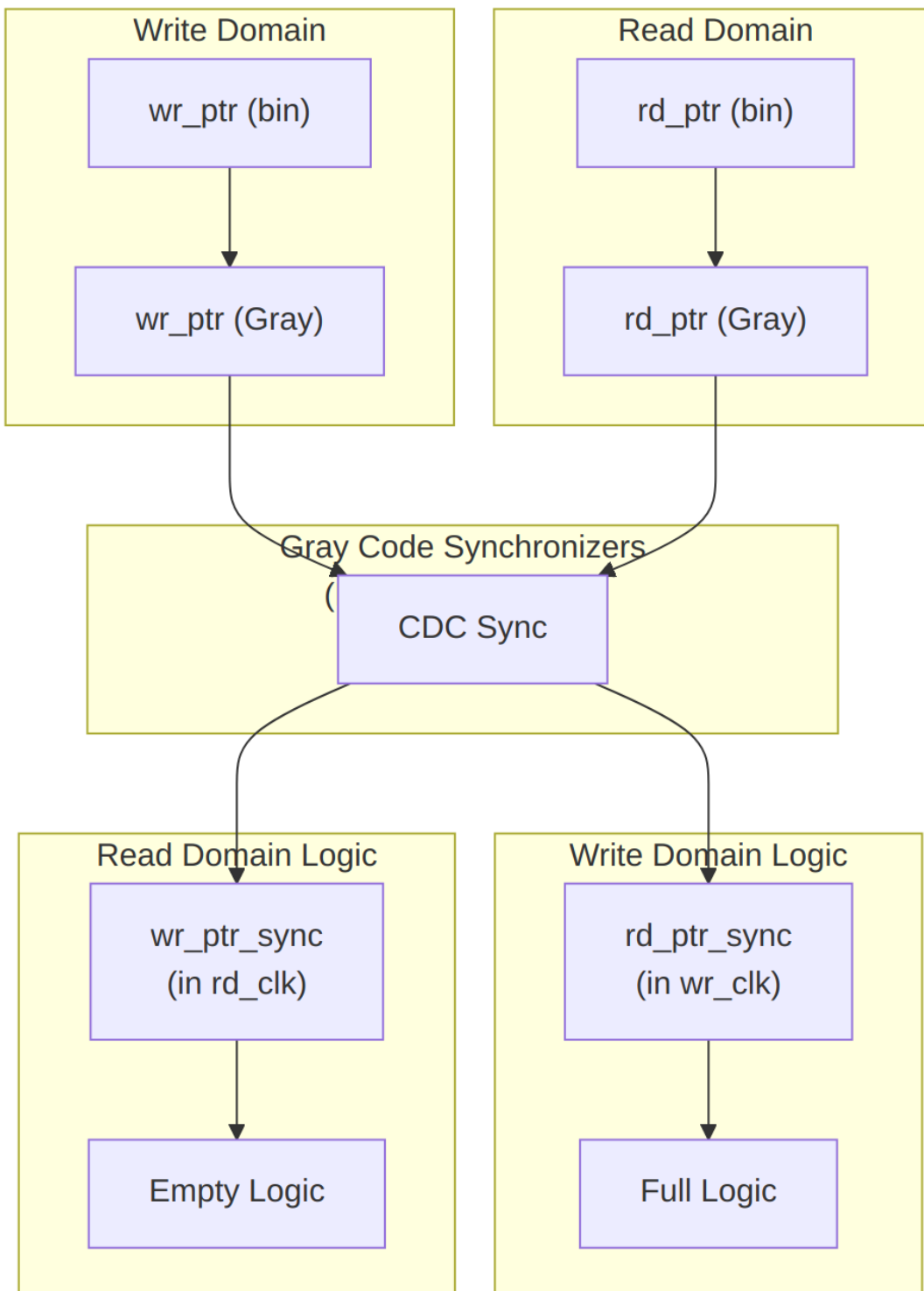
3.3 Parameters

Parameter	Default	Description
REGISTERED	0	0=mux mode, 1=flop mode (read path)
DATA_WIDTH	8	Data bus width
DEPTH	16	FIFO depth. Power of 2 with Gray pointers; any even value with USE_JOHNSON=1
USE_JOHNSON	0	Pointer CDC encoding: 0 = Gray ($\log_2(\text{DEPTH}) + 1$ bits, power-of-2 depth only), 1 = Johnson (DEPTH bits, any even depth). An illegal combination fails at elaboration with an explicit \$error.
N_FLOP_CROSS	2	Synchronizer stages (3 recommended for safety)
ALMOST_WR_MARGIN	1	Almost full threshold

Parameter	Default	Description
ALMOST_RD_MARGIN	1	Almost empty threshold

 **Important:** Set N_FLOP_CROSS=3 for production designs to ensure metastability protection.

3.4 Functional Description



Clock Domain Crossing Architecture

3.4.1 Why Gray Code?

Gray code ensures only one bit changes at a time during pointer updates:

Binary: 011 → 100 (2 bits change - hazard!)
Gray: 010 → 110 (1 bit changes - safe!)

This prevents glitches during synchronization across clock domains.

3.4.2 Dependencies

- `counter_bin.sv` - Binary counters
 - `counter_johnson.sv` - Johnson (twisted-ring) counters, used when `USE_JOHNSON=1`. Johnson code is NOT Gray code – it is a distinct encoding that happens to share the single-bit-change property needed for CDC. Both change exactly one bit per increment, including the wrap.
 - `glitch_free_n_dff_arn.sv` - Multi-flop synchronizers
 - `johnson2bin.sv` - Johnson-to-binary conversion (registered), used when `USE_JOHNSON=1`
 - `counter_bingray.sv` / `gray2bin.sv` - Gray pointer path (combinational decode), used when `USE_JOHNSON=0`
 - `fifo_control.sv` - Full/empty flag generation
-

3.5 Usage Examples

3.5.1 Example 1: Basic CDC FIFO

```
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .N_FLOP_CROSS(3),      // 3-stage synchronizer
    .REGISTERED(0)
) u_cdc_fifo (
    // Write domain @ 100 MHz
    .axi_wr_aclk      (clk_100mhz),
    .axi_wr_aresetn   (rst_100_n),
    .wr_valid         (domain_a_valid),
    .wr_ready         (domain_a_ready),
    .wr_data          (domain_a_data),

    // Read domain @ 50 MHz
    .axi_rd_aclk      (clk_50mhz),
```



```

        .axi_rd_aresetn (rst_50_n),
        .rd_ready      (domain_b_ready),
        .rd_valid      (domain_b_valid),
        .rd_data       (domain_b_data)
    );

```

3.5.2 Example 2: High-Speed to Low-Speed CDC

```

// Fast writer (250 MHz) → Slow reader (62.5 MHz)
// Needs deeper FIFO to handle burst traffic
gaxi_fifo_async #(
    .DATA_WIDTH(64),
    .DEPTH(32),           // Deeper for rate mismatch
    .N_FLOP_CROSS(3),
    .REGISTERED(1)       // Flop mode for timing
) u_rate_converter (
    .axi_wr_aclk   (clk_250mhz),
    .axi_wr_aresetn (rst_fast_n),
    .wr_valid      (fast_valid),
    .wr_ready      (fast_ready),
    .wr_data       (fast_data),

    .axi_rd_aclk   (clk_62p5mhz),
    .axi_rd_aresetn (rst_slow_n),
    .rd_ready      (slow_ready),
    .rd_valid      (slow_valid),
    .rd_data       (slow_data)
);

```

3.6 Timing Characteristics

Clock Ratio (wr:rd)	Latency	Notes
1:1 (same freq)	3-5 cycles	CDC synchronizat ion overhead
2:1 (fast→slow)	4-6 cycles	Additional write-side delay
1:2 (slow→fast)	3-5 cycles	Read-side samples faster
Any ratio	3-7 cycles	Depends on synchronizer

Clock Ratio (wr:rd)	Latency	Notes
		stages + clock relationship

Latency Formula: $\sim(2 \times N_FLOP_CROSS) + 1$ in slower clock domain cycles

3.7 Design Considerations

3.7.1 Depth Sizing for Clock Ratio

When write clock >> read clock, size FIFO to handle burst accumulation:

Required Depth $\geq$ Burst Size $\times$ (Write Freq / Read Freq) $\times$ Safety Margin

Example:

- Write: 100 MHz, Read: 25 MHz
 - Burst: 16 transfers
 - Safety: 1.5x
- Depth $\geq 16 \times (100/25) \times 1.5 = 96$ entries

3.7.2 Reset Synchronization

Critical: Both clock domains must have properly synchronized resets!

// Separate reset synchronizers for each domain

```
reset_sync u_wr_rst_sync (
    .clk(axi_wr_aclk),
    .async_rst_n(global_rst_n),
    .sync_rst_n(axi_wr_aresetn)
);
```

```
reset_sync u_rd_rst_sync (
    .clk(axi_rd_aclk),
    .async_rst_n(global_rst_n),
    .sync_rst_n(axi_rd_aresetn)
);
```

3.7.3 Metastability Protection

N_FLOP_CROSS determines MTBF (Mean Time Between Failures):

Stages	MTBF (typical @ 100 MHz)	Use Case
2	~years	Short-term

Stages	MTBF (typical @ 100 MHz)	Use Case
		prototyping only
3	~centuries	Production standard
4	~millennia	Ultra-critical systems

Recommendation: Always use N_FLOP_CROSS=3 in production.

3.8 Error Checking

Simulation-only assertions catch protocol violations:

```
// Write domain checks
always @(posedge axi_wr_aclk) begin
    if (!axi_wr_aresetn && wr_valid && !wr_ready) begin
        $display("Error: %s write while FIFO full", INSTANCE_NAME);
    end
end

// Read domain checks
always @(posedge axi_rd_aclk) begin
    if (!axi_rd_aresetn && rd_ready && !rd_valid) begin
        $display("Error: %s read while FIFO empty", INSTANCE_NAME);
    end
end
```

3.9 Testing

```
# Async FIFO tests with various clock ratios
pytest val/amba/test_gaxi_buffer_async.py -v

# Test specific clock ratio (wr:rd)
pytest val/amba/test_gaxi_buffer_async.py -k "wr10_rd12" -v # 10ns : 12ns
```

Test matrix includes: - Same clocks (1:1) - 1.25x ratio (10ns : 12ns) - 2x ratio (10ns : 20ns) - 2.5x ratio (8ns : 20ns)

3.10 Common Issues

3.10.1 Issue 1: Metastability

Symptom: Random data corruption, simulation/hardware mismatch

Solution: Increase N_FLOP_CROSS to 3 or 4

3.10.2 Issue 2: Pointer Synchronization Failure

Symptom: FIFO full/empty signals incorrect

Debug: 1. Verify clocks are truly independent (no PLL relationship violations) 2. Check reset synchronization - both domains must reset properly 3. Verify Gray code conversion logic

3.10.3 Issue 3: Underflow/Overflow

Symptom: Data loss or corruption

Debug: 1. Check DEPTH is sufficient for clock ratio 2. Verify flow control respected in both domains 3. Monitor pointer values in both domains

3.11 Related Modules

- [gaxi_fifo_sync](#) - Single clock domain version
 - [gaxi_skid_buffer_async](#) - Async skid buffer
 - [GAXI Index](#) - Overview
-

3.12 References

- **Clifford Cummings:** “Simulation and Synthesis Techniques for Asynchronous FIFO Design” (Sunburst Design)
 - **Source:** rtl/amba/gaxi/gaxi_fifo_async.sv
 - **Tests:** val/amba/test_gaxi_buffer_async.py
-

Version: 1.0 **Last Updated:** 2025-10-06

4 GAXI Synchronous FIFO

Module: gaxi_fifo_sync.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

4.1 Overview

The GAXI synchronous FIFO provides elastic buffering for any depth (power of 2 recommended). Unlike the skid buffer, this module uses a true FIFO architecture with read/write pointers, supporting larger depths efficiently.

4.1.1 Key Features

- ✓ **Arbitrary Depth:** Any depth supported (power of 2 optimal)
 - ✓ **Two Read Modes:** Mux mode (combinatorial) or Flop mode (registered)
 - ✓ **Counter-Based:** Binary counters with wrapping
 - ✓ **Almost Full/Empty:** Configurable thresholds
 - ✓ **Single Clock Domain:** Synchronous design
-

4.2 Module Interface

```
module gaxi_fifo_sync #(
    parameter int REGISTERED = 0,           // 0=mux mode, 1=flop mode
    parameter int DATA_WIDTH = 4,
    parameter int DEPTH = 4,
    parameter int ALMOST_WR_MARGIN = 1,
    parameter int ALMOST_RD_MARGIN = 1,
    parameter string INSTANCE_NAME = "DEADF1F0"
) (
    input logic axi_aclk,
    input logic axi_aresetn,
    input logic wr_valid,
    output logic wr_ready,           // not full
    input logic [DW-1:0] wr_data,
    input logic rd_ready,
    output logic [AW:0] count,
    output logic rd_valid,           // not empty
    output logic [DW-1:0] rd_data
);
```

4.3 Parameters

Parameter	Default	Description
REGISTERED	0	0=mux mode (comb read), 1=flop mode

Parameter	Default	Description
		(reg read)
DATA_WIDTH	4	Data bus width
DEPTH	4	FIFO depth (any value, power-of-2 optimal)
ALMOST_WR_MARGIN	1	Almost full threshold
ALMOST_RD_MARGIN	1	Almost empty threshold
INSTANCE_NAME	“DEADF1F0”	Debug instance name

4.4 Functional Description

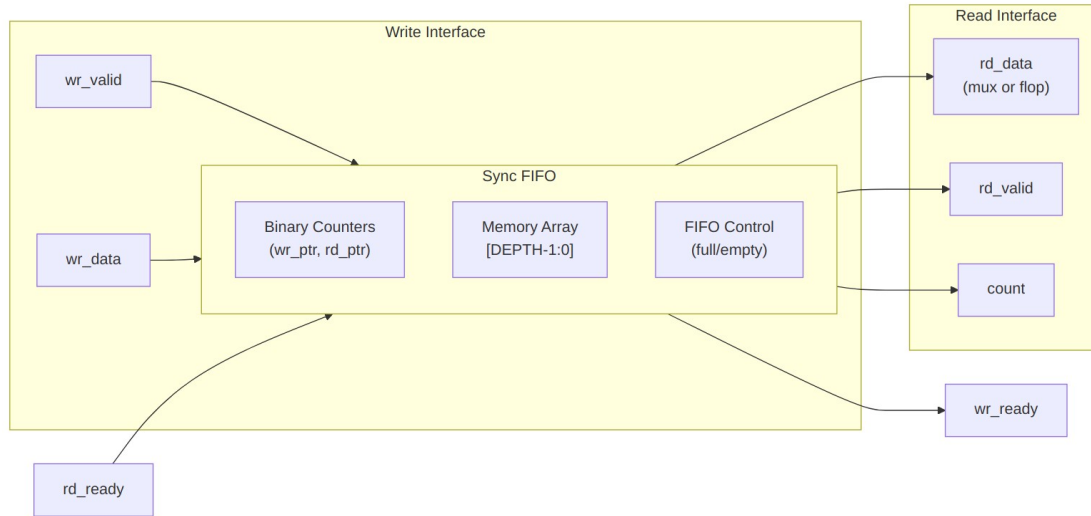
4.4.1 Read Modes

4.4.1.1 Mux Mode (*REGISTERED=0*)

- **Latency:** 1 cycle write → read
- **Read Path:** Combinatorial from memory
- **Use Case:** Low latency applications
- **Timing:** May create combinatorial path

4.4.1.2 Flop Mode (*REGISTERED=1*)

- **Latency:** 2 cycles write → read
- **Read Path:** Registered output
- **Use Case:** Timing closure, deep pipelines
- **Timing:** No combinatorial paths



Architecture

4.4.2 Dependencies

- counter_bin.sv - Binary counters for read/write pointers
- fifo_control.sv - Full/empty flag generation

4.5 Usage Examples

4.5.1 Example 1: Mux Mode (Low Latency)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(32),
    .REGISTERED(0)           // Mux mode: 1-cycle latency
) u_low_latency_fifo (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid      (wr_valid),
    .wr_ready      (wr_ready),
    .wr_data       (wr_data),
    .rd_ready      (rd_ready),
    .rd_valid      (rd_valid),
    .rd_data       (rd_data), // Combinatorial read
    .count         (fifo_level)
);
```

4.5.2 Example 2: Flop Mode (Timing Closure)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(128),
```

```

        .DEPTH(64),
        .REGISTERED(1)           // Flop mode: 2-cycle latency
    ) u_registered_fifo (
        .axi_aclk      (clk),
        .axi_aresetn   (rst_n),
        .wr_valid      (wr_valid),
        .wr_ready      (wr_ready),
        .wr_data       (wr_data),
        .rd_ready      (rd_ready),
        .rd_valid      (rd_valid),
        .rd_data       (rd_data), // Registered output
        .count         (fifo_level)
    );

```

4.6 Timing Characteristics

Mode	Write→Read Latency	Max Throughput	Read Path
Mux (REGISTERED=0)	1 cycle	1/cycle	Combinatorial
Flop (REGISTERED=1)	2 cycles	1/cycle	Registered

4.7 Resource Utilization

DEPTH	Mode	Flops	LUTs	Memory Bits
16	Mux	16×DW + ~20	~80	16×DW
16	Flop	16×DW + DW + ~20	~80	16×DW
64	Mux	64×DW + ~30	~120	64×DW
64	Flop	64×DW + DW + ~30	~120	64×DW

4.8 Testing

Test FIFO modes

```
pytest val/amba/test_gaxi_buffer_sync.py -k "fifo_mux" -v # Mux mode
pytest val/amba/test_gaxi_buffer_sync.py -k "fifo_flop" -v # Flop mode
```

4.9 Related Modules

- [gaxi_skid_buffer](#) - Zero-latency bypass, smaller depths
 - [gaxi_fifo_async](#) - Clock domain crossing version
 - [GAXI Index](#) - Overview of all GAXI modules
-

Version: 1.0 **Last Updated:** 2025-10-06

5 GAXI Register Slice

Module: gaxi_regslice.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

5.1 Overview

The GAXI register slice provides a timing-friendly 1-deep elastic buffer for pipeline timing isolation. It implements the classic register slice pattern with always-registered data transfer, guaranteeing predictable 1-cycle latency and consistent throughput.

5.1.1 Key Features

- ✓ **Fixed 1-Cycle Latency:** Always registered, predictable timing
 - ✓ **Full Throughput:** 1 beat/cycle in steady state
 - ✓ **Elastic Buffering:** Absorbs single-cycle backpressure
 - ✓ **Port Compatible:** Intentionally aligned with gaxi_skid_buffer
 - ✓ **Ready/Valid Handshake:** Industry-standard AXI-like protocol
 - ✓ **Minimal Resources:** Single register stage
-

5.2 Module Interface

```
module gaxi_regslice #(
    parameter int DATA_WIDTH    = 32,
    parameter     INSTANCE_NAME = "REGSL1D",
    parameter int DW             = DATA_WIDTH    // Derived
) (
    // Global Clock and Reset
    input logic      axi_aclk,
    input logic      axi_aresetn,

    // Write Interface (Input Side)
    input logic      wr_valid,
    output logic     wr_ready,
    input logic [DW-1:0] wr_data,

    // Read Interface (Output Side)
    output logic     rd_valid,
    input logic      rd_ready,
    output logic [DW-1:0] rd_data,

    // Status/Monitoring
    output logic [3:0] count,      // 0 or 1
    output logic [3:0] rd_count   // mirror of count
);
```

5.3 Parameters

Parameter	Default	Description
DATA_WIDTH	32	Data bus width (arbitrary)
INSTANCE_NAME	"REGSL1D"	Debug instance name
DW	DATA_WIDTH	Derived parameter (internal use)

5.4 Functional Description

5.4.1 Architecture

The register slice implements a single-entry elastic buffer with simultaneous push/pop capability:

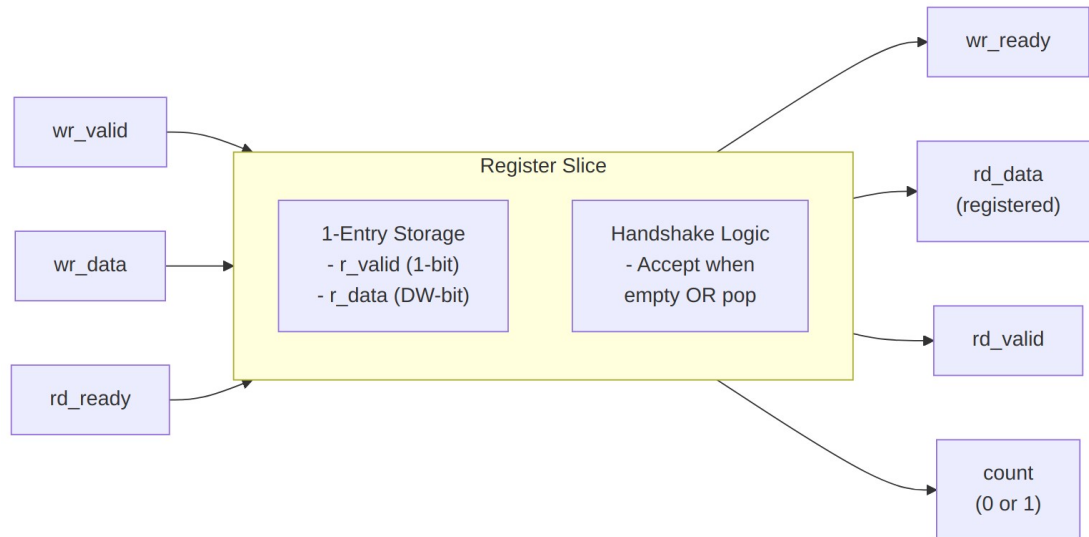


Diagram 3

5.4.2 Ready/Valid Protocol

Write Ready Conditions:

$wr_ready = (!r_valid) \ || \ (r_valid \ \&\& \ rd_ready)$

- Accept when storage is **empty** ($r_valid=0$)
- Accept when storage is **full** AND downstream **consuming** (simultaneous transfer)

Read Valid:

$rd_valid = r_valid$

- Valid when storage is occupied

5.4.3 Transfer Scenarios

Scenario	wr_valid	wr_ready	rd_valid	rd_ready	Action
Idle	0	1 (empty)	0	X	Storage empty, ready to accept
Fill	1	1	0	X	Write data → storage, $r_valid=1$

Scenario	wr_valid	wr_ready	rd_valid	rd_ready	Action
Full	0	0	1	0	Storage occupied, downstream not consuming
Drain	0	1 (will empty)	1	1	Read consumes data, r_valid=0
Pass-through	1	1	1	1	Simultaneous write+read, data passes through

5.4.4 Storage Update Logic

```

unique case ({w_wxfer, w_rxfer})
  2'b10: begin // Write only: fill
    r_valid <= 1'b1;
    r_data  <= wr_data;
  end
  2'b01: begin // Read only: drain
    r_valid <= 1'b0;
  end
  2'b11: begin // Simultaneous: pass through with register
    r_valid <= 1'b1;
    r_data  <= wr_data;
  end
  default: begin // Idle: hold state
    r_valid <= r_valid;
    r_data  <= r_data;
  end
endcase

```

5.5 Usage Examples

5.5.1 Example 1: Pipeline Timing Isolation

// Insert register slice to break long combinatorial path

```
gaxi_regslice #(
    .DATA_WIDTH(64)
) u_pipeline_break (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid      (stage1_valid),
    .wr_ready      (stage1_ready),
    .wr_data       (stage1_data),
    .rd_valid      (stage2_valid),
    .rd_ready      (stage2_ready),
    .rd_data       (stage2_data),
    .count         () // Optional monitoring
);
```

5.5.2 Example 2: AXI Channel Register Slice

// Register slice for AXI AR channel

```
gaxi_regslice #(
    .DATA_WIDTH(AXI_ID_WIDTH + AXI_ADDR_WIDTH + 8 + 3 + 2),
    .INSTANCE_NAME("AR_REGSL")
) u_ar_regslice (
    .axi_aclk      (aclk),
    .axi_aresetn   (aresetn),
    .wr_valid      (m_axi_arvalid),
    .wr_ready      (m_axi_arready),
    .wr_data       ({m_axi_arid, m_axi_araddr, m_axi_arlen,
                     m_axi_arsize, m_axi_arburst}),
    .rd_valid      (s_axi_arvalid),
    .rd_ready      (s_axi_arready),
    .rd_data       ({s_axi_arid, s_axi_araddr, s_axi_arlen,
                     s_axi_arsize, s_axi_arburst}),
    .count         (ar_occupancy)
);
```

5.5.3 Example 3: Multiple Register Slices for Deep Pipelining

// Chain multiple register slices for multi-cycle pipeline

```
gaxi_regslice #(.DATA_WIDTH(32)) u_stage1 (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(in_valid), .wr_ready(in_ready), .wr_data(in_data),
    .rd_valid(pipel_valid), .rd_ready(pipel_ready), .rd_data(pipel_dat
a)
);
```

```

gaxi_regslice #(.DATA_WIDTH(32)) u_stage2 (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(pipe1_valid), .wr_ready(pipe1_ready), .wr_data(pipe1_data),
    .rd_valid(pipe2_valid), .rd_ready(pipe2_ready), .rd_data(pipe2_data)
);

gaxi_regslice #(.DATA_WIDTH(32)) u_stage3 (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(pipe2_valid), .wr_ready(pipe2_ready), .wr_data(pipe2_data),
    .rd_valid(out_valid), .rd_ready(out_ready), .rd_data(out_data)
);
// Total latency: 3 cycles, full throughput maintained

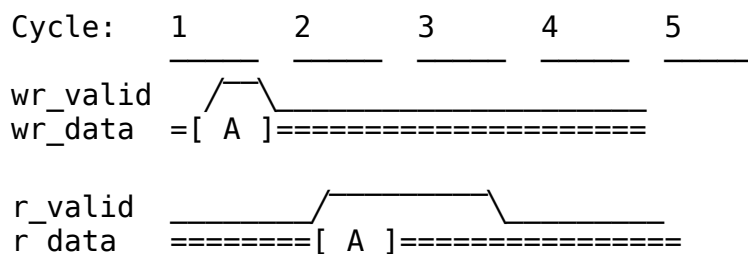
```

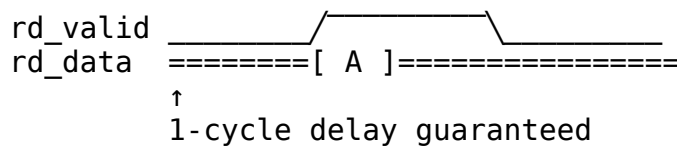
5.6 Timing Characteristics

Characteristic	Value	Notes
Write→Read Latency	1 cycle	Always registered, fixed latency
Max Throughput	1 beat/cycle	Sustained in steady state
Backpressure Absorption	1 beat	Single-entry elastic buffer
Read Path	Registered	No combinatorial paths
Write Path	Combinatorial	wr_ready depends on rd_ready

5.6.1 Latency Guarantee

Unlike skid buffers which can have 0-cycle bypass, the register slice **always** introduces 1-cycle latency:





5.7 Comparison with Related Modules

5.7.1 gaxi_regslice vs gaxi_skid_buffer

Feature	gaxi_regslice	gaxi_skid_buffer
Depth	1 entry	1 entry (+ skid slot)
Latency	1 cycle (fixed)	0-1 cycle (variable)
Bypass Path	✗ No	✓ Yes (when empty)
Throughput	1 beat/cycle	1 beat/cycle
Use Case	Timing isolation	Low latency
Registered Output	✓ Always	⚠ Only when filled
Timing Closure	✓ Better (no bypass)	⚠ Has combinatorial bypass

When to Choose: - **gaxi_regslice:** Timing closure priority, need predictable latency - **gaxi_skid_buffer:** Latency-sensitive paths, tolerate variable timing

5.7.2 gaxi_regslice vs gaxi_fifo_sync

Feature	gaxi_regslice	gaxi_fifo_sync
Depth	1 entry (fixed)	Parameterized (N entries)
Latency	1 cycle	1-2 cycles (mode-dependent)
Resources	Minimal (1 register)	Scales with depth
Use Case	Pipeline breaks	Buffering, rate matching

5.8 Resource Utilization

5.8.1 FPGA Resources (Typical)

DATA_WIDTH	Flops	LUTs	Slice Registers
8	9	~6	9
32	33	~12	33
64	65	~18	65
128	129	~24	129

Scaling: Approximately DATA_WIDTH + 1 flops (data + valid flag)

5.9 Testing

5.9.1 Test Coverage

Test File: val/amba/test_gaxi_regslice.py

Test Methods: - Simple incremental loops (fill/drain cycles) - Back-to-back transfers (sustained throughput) - Comprehensive randomizer sweep (varied timing patterns) - Stress test with random patterns

Test Levels: - **basic:** Quick smoke test (~30s, 4 loops) - **medium:** Moderate coverage (~2min, expanded patterns) - **full:** Comprehensive validation (~5min, 100+ loops)

5.9.2 Run Tests

Basic test (quick validation)

```
TEST_LEVEL=basic pytest val/amba/test_gaxi_regslice.py -v
```

Medium test (normal CI)

```
TEST_LEVEL=medium pytest val/amba/test_gaxi_regslice.py -v
```

Full test (pre-release validation)

```
TEST_LEVEL=full pytest val/amba/test_gaxi_regslice.py -v
```

5.10 Design Considerations

5.10.1 Port Compatibility

Intentional alignment with gaxi_skid_buffer allows drop-in replacement:


```
// Both modules share identical port signatures
gaxi_regslice #(.DATA_WIDTH(64)) u_option1 (...);
gaxi_skid_buffer #(.DATA_WIDTH(64)) u_option2 (...);
```

This enables easy experimentation: - Start with **gaxi_skid_buffer** for low latency - Upgrade to **gaxi_regslice** if timing closure issues arise

5.10.2 Status Signals

count and **rd_count** are 4-bit for interface compatibility: - Value 0: Storage empty - Value 1: Storage occupied - Both signals always equal (redundant, but matches skid buffer interface)

5.10.3 Assertions

Built-in simulation checks (disabled in synthesis):

```
// Detect backpressure hot spots
if (wr_valid && !wr_ready) ...

// Detect invalid reads
if (rd_ready && !rd_valid) ...

// Sanity check
if (count > 4'd1) $error(...)
```

5.11 Related Modules

- [gaxi_skid_buffer](#) - Zero-latency bypass alternative
 - [gaxi_fifo_sync](#) - Multi-entry FIFO version
 - [gaxi_fifo_async](#) - Clock domain crossing version
 - [GAXI Index](#) - Overview of all GAXI modules
-

5.12 Common Use Cases

5.12.1 1. Breaking Long Combinatorial Paths

Problem: Critical path through multiple modules

```
// Before: Long combinatorial path
assign module_b_in = complex_function(module_a_out);
```

Solution: Insert register slice

```
// After: Broken with 1-cycle register
gaxi_regslice #(.DATA_WIDTH(32)) u_break (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(module_a_valid), .wr_data(module_a_out), .wr_ready(modul
e_a_ready),
    .rd_valid(module_b_valid), .rd_data(module_b_in), .rd_ready(modu
le_b_ready)
);
```

5.12.22. AXI Interface Pipelining

Improve timing across AXI channels:

```
// Pipeline all 5 AXI4 channels
gaxi_regslice #(...) u_ar_slice (...); // Address Read
gaxi_regslice #(...) u_r_slice (...); // Read Data
gaxi_regslice #(...) u_aw_slice (...); // Address Write
gaxi_regslice #(...) u_w_slice (...); // Write Data
gaxi_regslice #(...) u_b_slice (...); // Write Response
```

5.12.33. CDC Preparation

Use before async FIFO to ensure registered inputs:

```
// Register slice before CDC FIFO
gaxi_regslice #(.DATA_WIDTH(32)) u_pre_cdc (
    .axi_aclk(clk_a), .axi_aresetn(rst_a_n),
    .wr_valid(src_valid), .wr_data(src_data), .wr_ready(src_ready),
    .rd_valid(fifo_wr_valid), .rd_data(fifo_wr_data), .rd_ready(fifo_w
r_ready)
);

gaxi_fifo_async #(.DATA_WIDTH(32), .DEPTH(16)) u_cdc_fifo (
    .wr_clk(clk_a), .rd_clk(clk_b), ...
    .wr_valid(fifo_wr_valid), .wr_data(fifo_wr_data), .wr_ready(fifo_w
r_ready),
    ...
);
```

Version: 1.0 **Last Updated:** 2025-10-23

6 GAXI Skid Buffer

Module: gaxi_skid_buffer.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

6.1 Overview

The GAXI skid buffer is an elastic buffer that provides **zero-latency bypass** when empty while offering elastic buffering to prevent pipeline stalls. It combines the best of both worlds: low latency for the common case (empty) and elastic storage for handling backpressure.

6.1.1 Key Features

- ✓ **Zero-Latency Bypass:** When empty, data appears at output immediately (0 cycles)
 - ✓ **Elastic Buffering:** Depth 2-8 entries prevent pipeline stalls
 - ✓ **Single Clock Domain:** Synchronous design for simplicity
 - ✓ **Parameterized:** Configurable data width and depth
 - ✓ **Efficient:** Shift register implementation, minimal logic
-

6.2 Module Interface

```
module gaxi_skid_buffer #(
    parameter int DATA_WIDTH = 32,
    parameter int DEPTH = 2,      // Must be one of {2, 4, 6, 8}
    parameter    INSTANCE_NAME = "DEADF1F0",
    parameter int DW = DATA_WIDTH,
    parameter int BUF_WIDTH = DATA_WIDTH * DEPTH,
    parameter int BW = BUF_WIDTH
) (
    // Global Clock and Reset
    input logic      axi_aclk,
    input logic      axi_aresetn,

    // Write Interface (Input Side)
    input logic      wr_valid,
    output logic     wr_ready,
    input logic [DW-1:0] wr_data,

    // Read Interface (Output Side)
    output logic     rd_valid,
    input logic      rd_ready,
    output logic [DW-1:0] rd_data,

    // Status/Monitoring
    output logic [3:0] count,      // Current buffer occupancy
    output logic [3:0] rd_count   // Same as count (for
compatibility)
);
```

6.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Data bus width in bits
DEPTH	int	2	Buffer depth (must be 2, 4, 6, or 8)
INSTANCE_NAME	string	“DEADF1F0”	Instance name for debug messages

Derived Parameters: - DW = DATA_WIDTH - BUF_WIDTH = DATA_WIDTH × DEPTH (total buffer storage) - BW = BUF_WIDTH

6.4 Ports

6.4.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset

6.4.2 Write Interface

Port	Direction	Width	Description
wr_valid	input	1	Write data valid
wr_ready	output	1	Ready to accept write
wr_data	input	DATA_WIDTH	Write data

6.4.3 Read Interface

Port	Direction	Width	Description
rd_valid	output	1	Read data valid

Port	Direction	Width	Description
rd_ready	input	1	Ready to accept read
rd_data	output	DATA_WIDTH	Read data

6.4.4 Status

Port	Direction	Width	Description
count	output	4	Current buffer occupancy (0 to DEPTH)
rd_count	output	4	Same as count (for compatibility)

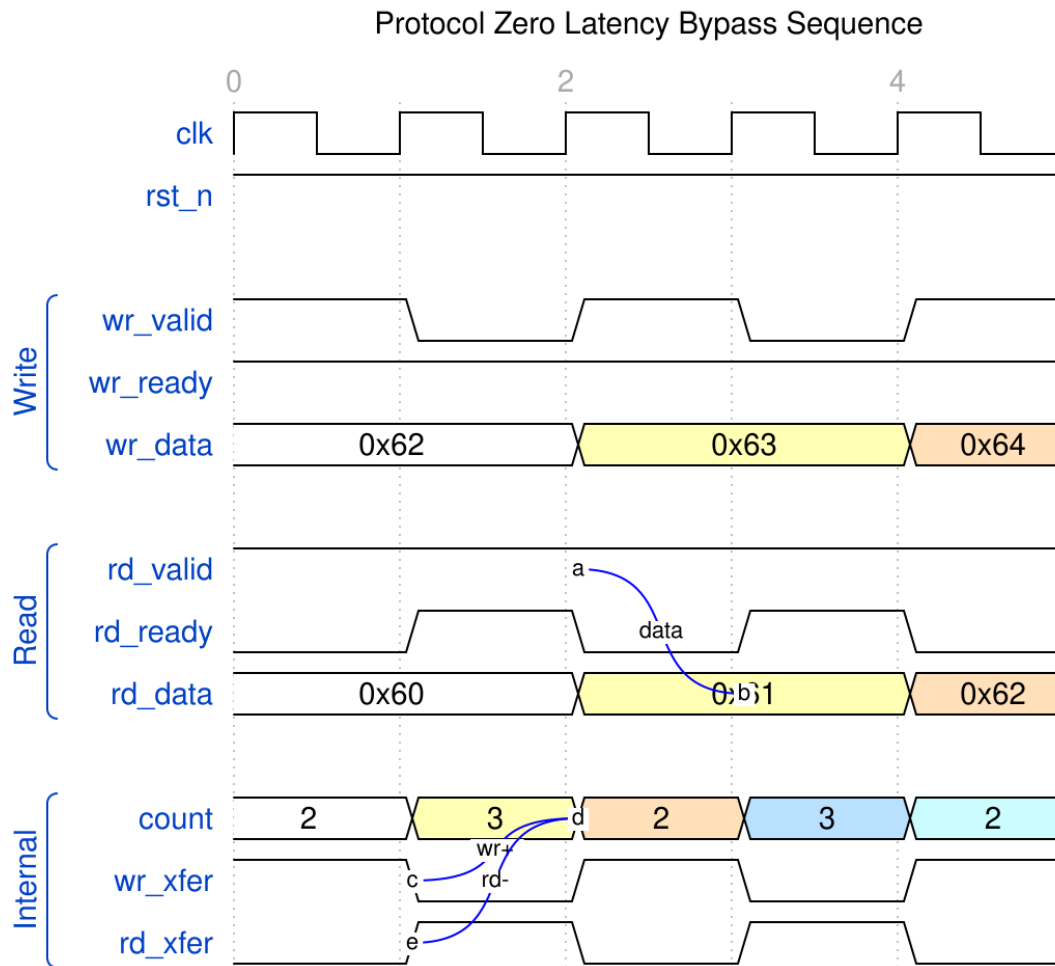
6.5 Functional Description

6.5.1 Operation Modes

The skid buffer operates in different modes based on occupancy:

6.5.1.1 1. Empty Mode (*count* = 0)

Zero-Latency Bypass: - wr_data → rd_data **immediately** (combinatorial path) - rd_valid asserts same cycle as wr_valid - No buffering delay



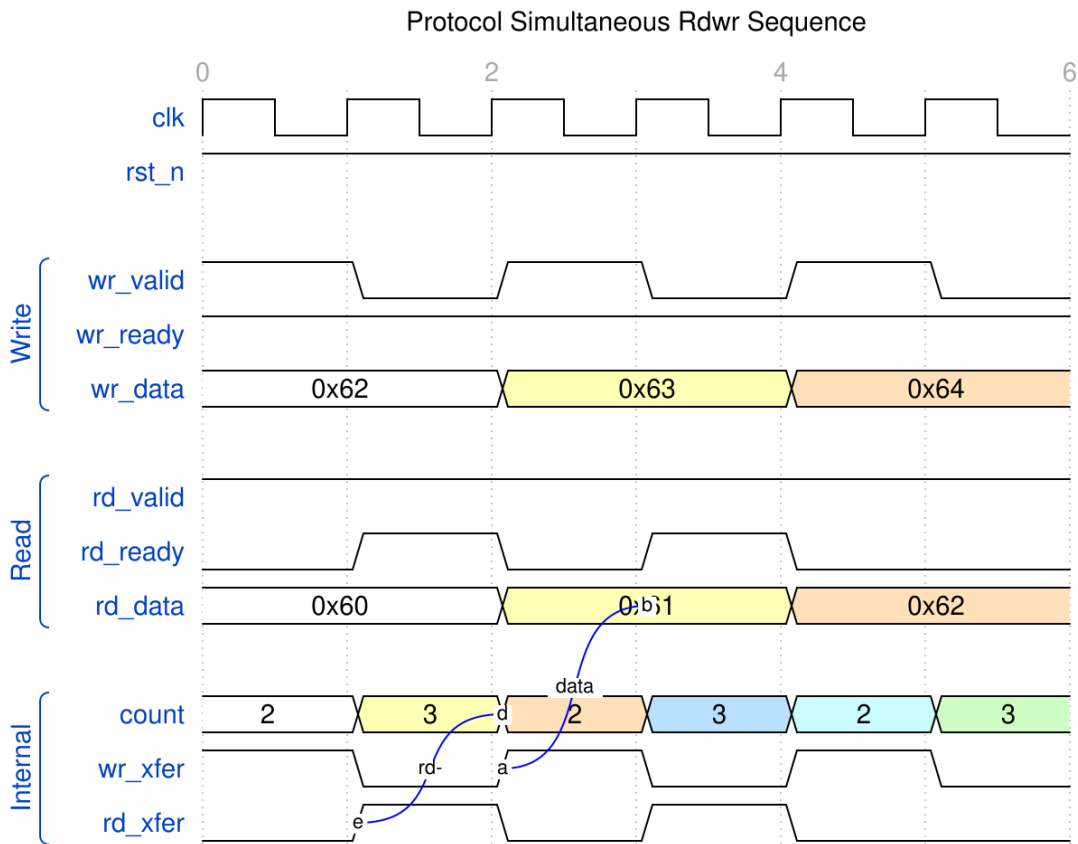
Zero-Latency Bypass

WaveJSON: [zero_latency_bypass_001.json](#)

Key Observations: - Write at cycle N → Read valid at cycle N - Data flows through with 0-cycle latency - count increments when write occurs without simultaneous read

6.5.1.2.2. Partially Full Mode ($0 < \text{count} < \text{DEPTH}$)

Elastic Storage: - Data buffered in shift register - Both write and read can occur simultaneously - count tracks occupancy



Simultaneous Read/Write

WaveJSON: [simultaneous_rdwr_001.json](#)

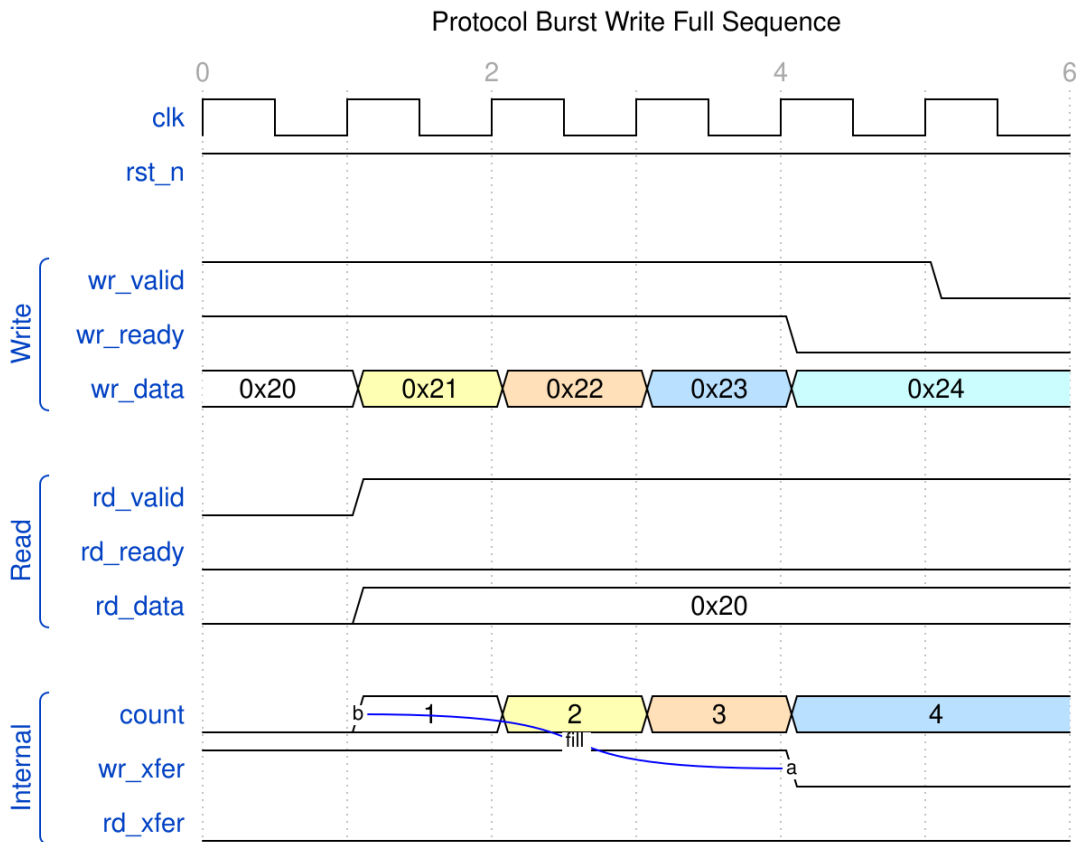
Key Observations: - Simultaneous operations: count stays constant - Write-only: count increments - Read-only: count decrements

6.5.1.3 3. Nearly Full Mode ($count = DEPTH - 1$)

Smart Backpressure: - `wr_ready` stays high if read occurring - Prevents false stalls

6.5.1.4 4. Full Mode ($count = DEPTH$)

Backpressure Applied: - `wr_ready` = 0 (cannot accept more writes) - Must read to make space



Burst Write Until Full

WaveJSON: [burst_write_full_001.json](#)

Key Observations: - wr_ready deasserts when buffer full - Subsequent writes blocked until space available - Reading creates space, wr_ready reasserts

6.5.2 Internal State Machine

The module implements a shift register for data storage:

```
// Data storage: shift register
logic [BUF_WIDTH-1:0] r_data;           // [DEPTH * DATA_WIDTH - 1 : 0]
logic [3:0] r_data_count;             // Current occupancy

// Transfer detection
logic w_wr_xfer = wr_valid & wr_ready;
logic w_rd_xfer = rd_valid & rd_ready;
```

State Transitions:

Condition	wr_xfer	rd_xfer	Action	count Change
Write only	1	0	Shift data in	+1
Read only	0	1	Shift data out	-1
Simultaneous	1	1	Shift data in/out	0
Idle	0	0	No change	0

6.5.3 Ready/Valid Logic

// Write ready conditions

```
wr_ready <= (count <= DEPTH-2) ||
             (count == DEPTH-1 && (~wr_xfer || rd_xfer)) ||
             (count == DEPTH && rd_xfer);
```

// Read valid conditions

```
rd_valid <= (count >= 2) ||
            (count == 1 && (~rd_xfer || wr_xfer)) ||
            (count == 0 && wr_xfer);
```

6.6 Timing Characteristics

6.6.1 Latency

Condition	Latency	Cycles
Empty → Read	0	Immediate (combinatorial)
Write → Read (buffered)	1	One clock cycle
Backpressure response	1	One clock after full response

6.6.2 Throughput

- **Maximum:** 1 transfer per clock cycle
- **Sustained:** 1 transfer per clock (with proper flow control)

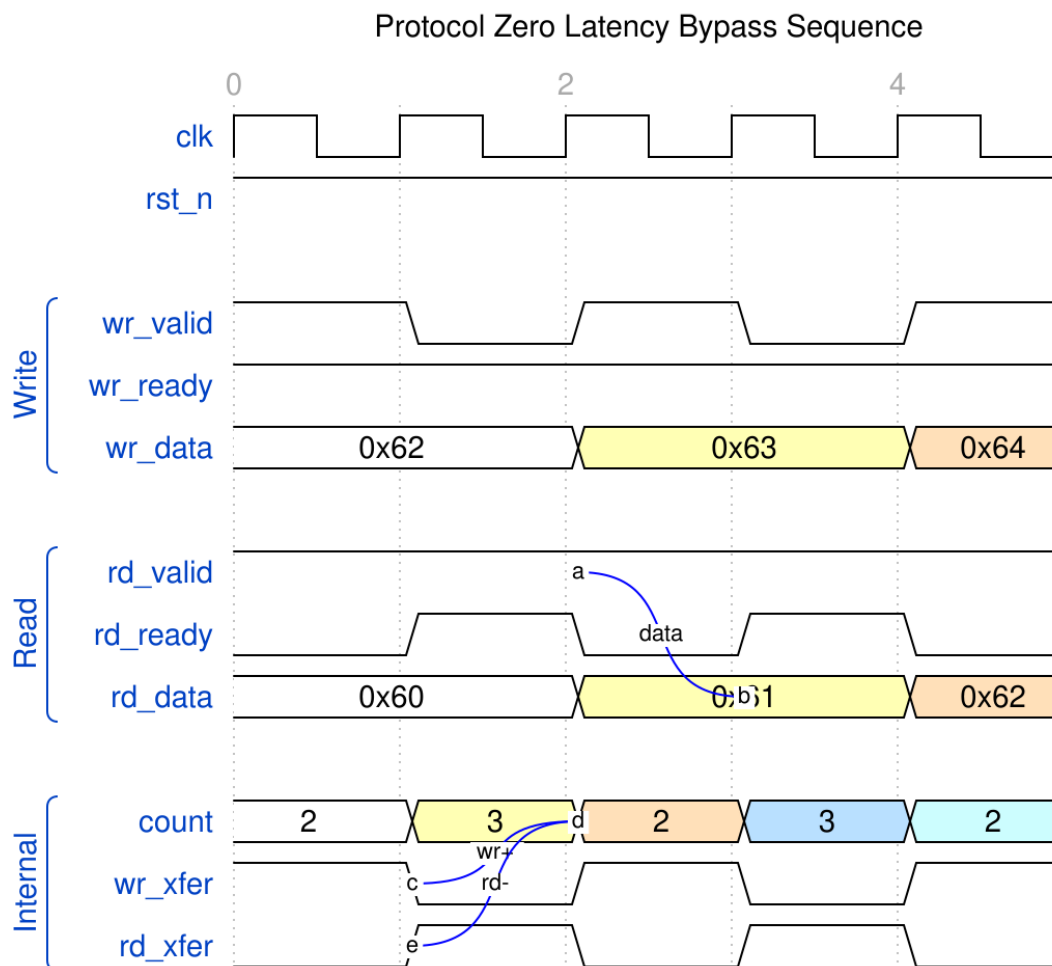
6.6.3 Critical Paths

- **Write → Read (empty):** Combinatorial path (may need timing closure in deep pipelines)
- **Ready/Valid Logic:** Single-cycle registered

6.7 Comprehensive Timing Examples

6.7.1 Scenario 1: Zero-Latency Bypass

Write to empty buffer shows immediate `rd_valid`:

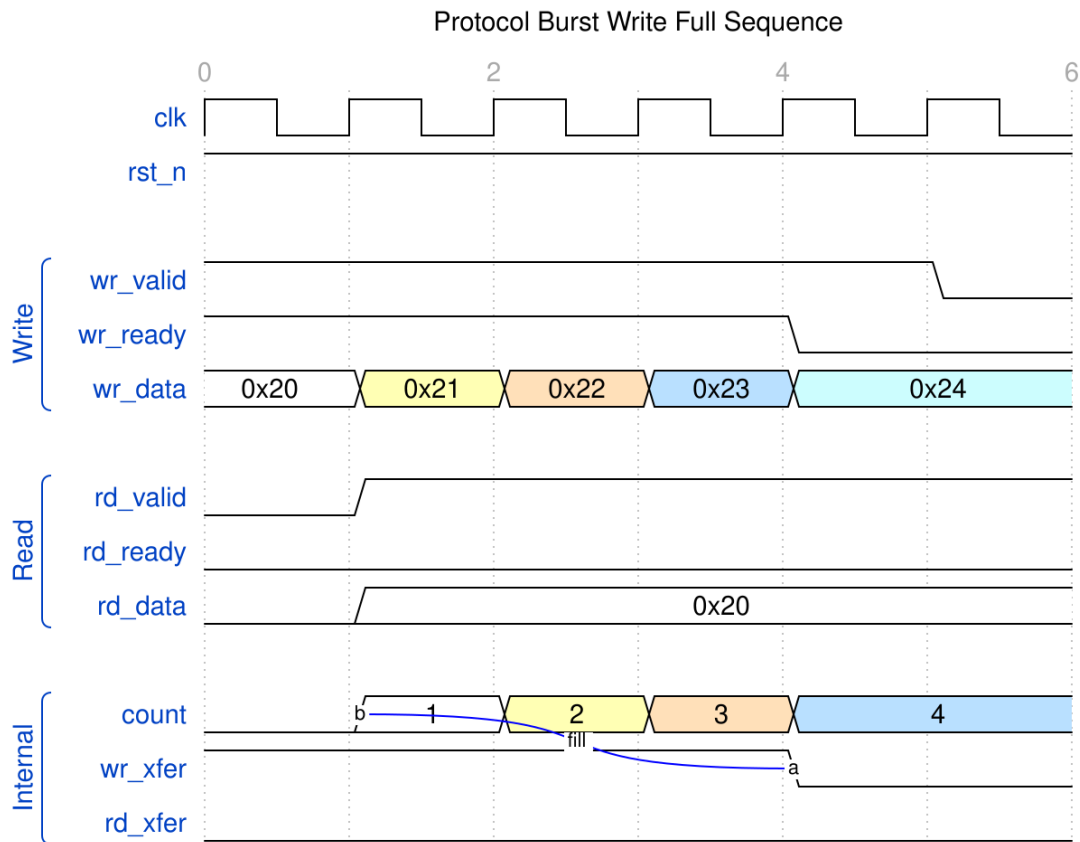


Zero-Latency Bypass

WaveJSON: [zero_latency_bypass_001.json](#)

6.7.2 Scenario 2: Burst Write Until Full

Demonstrates backpressure behavior:

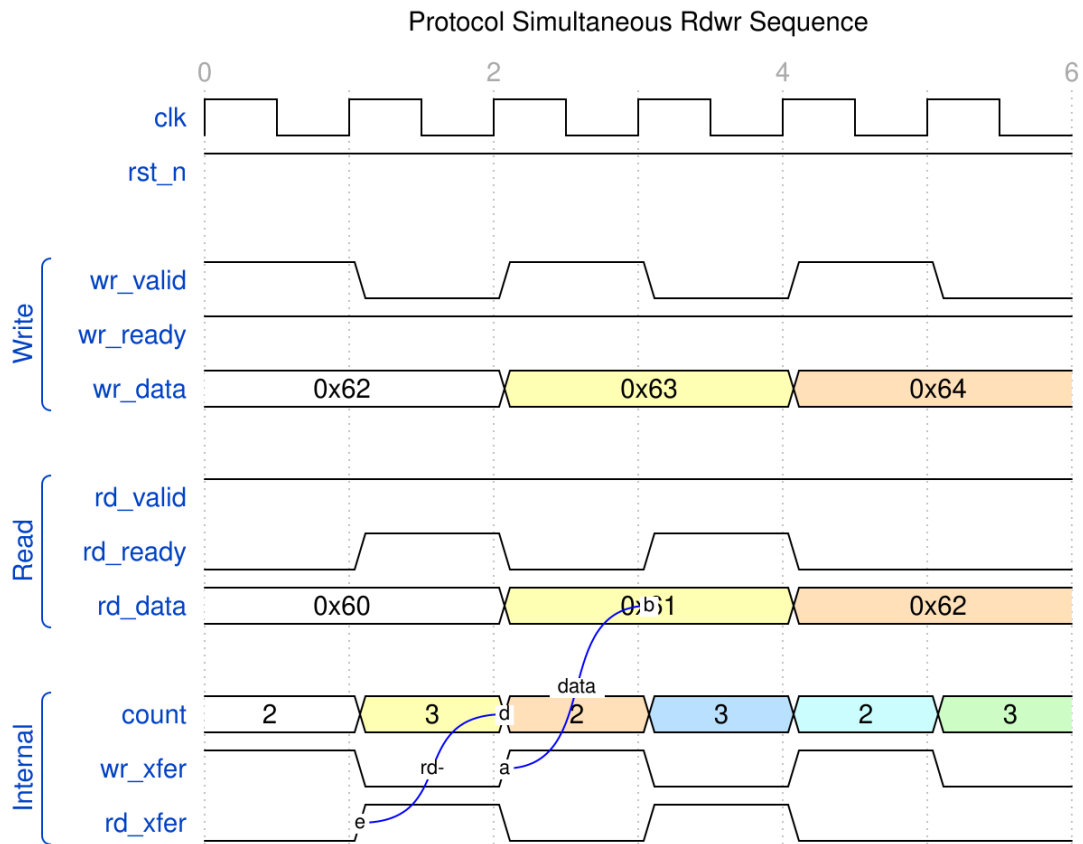


Burst Write Full

WaveJSON: [burst_write_full_001.json](#)

6.7.3 Scenario 3: Simultaneous Read/Write

Pass-through operation with constant occupancy:

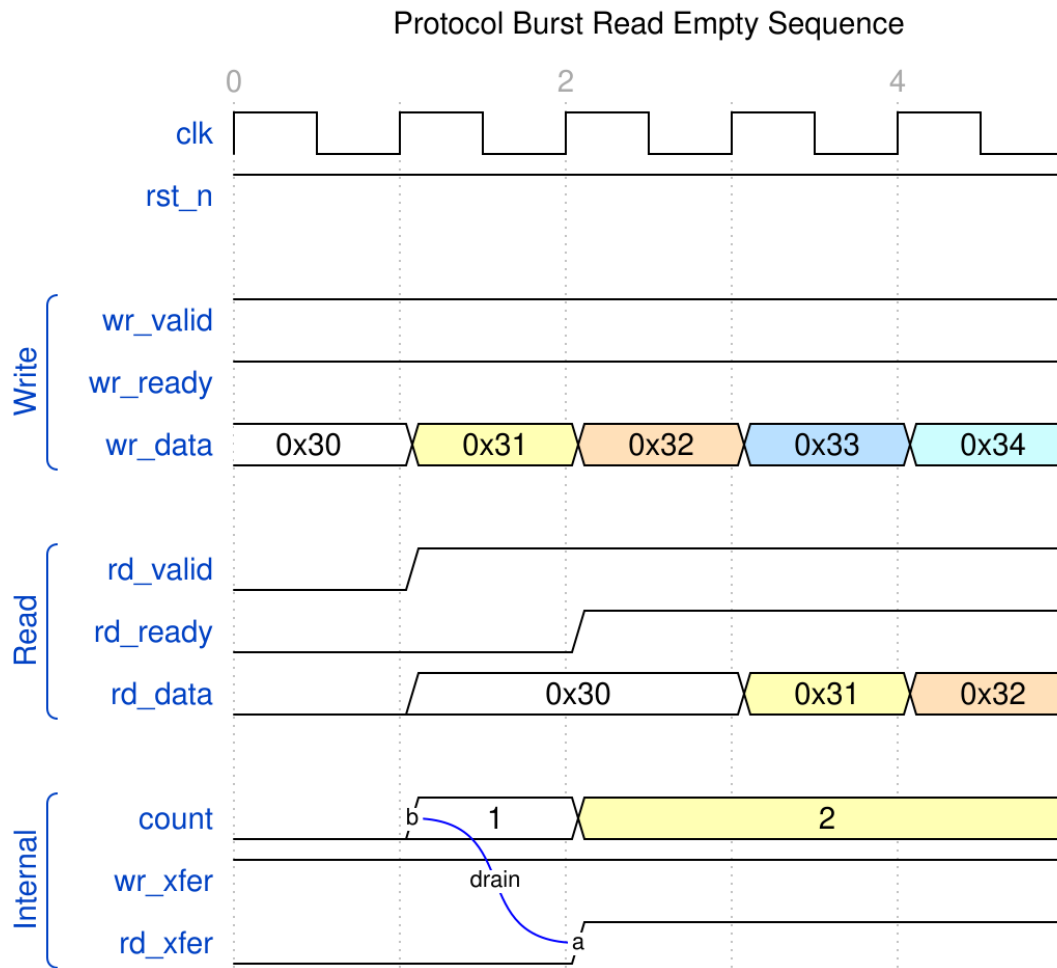


Simultaneous R/W

WaveJSON: [simultaneous_rdwr_001.json](#)

6.7.4 Scenario 4: Burst Read Until Empty

Draining the buffer:

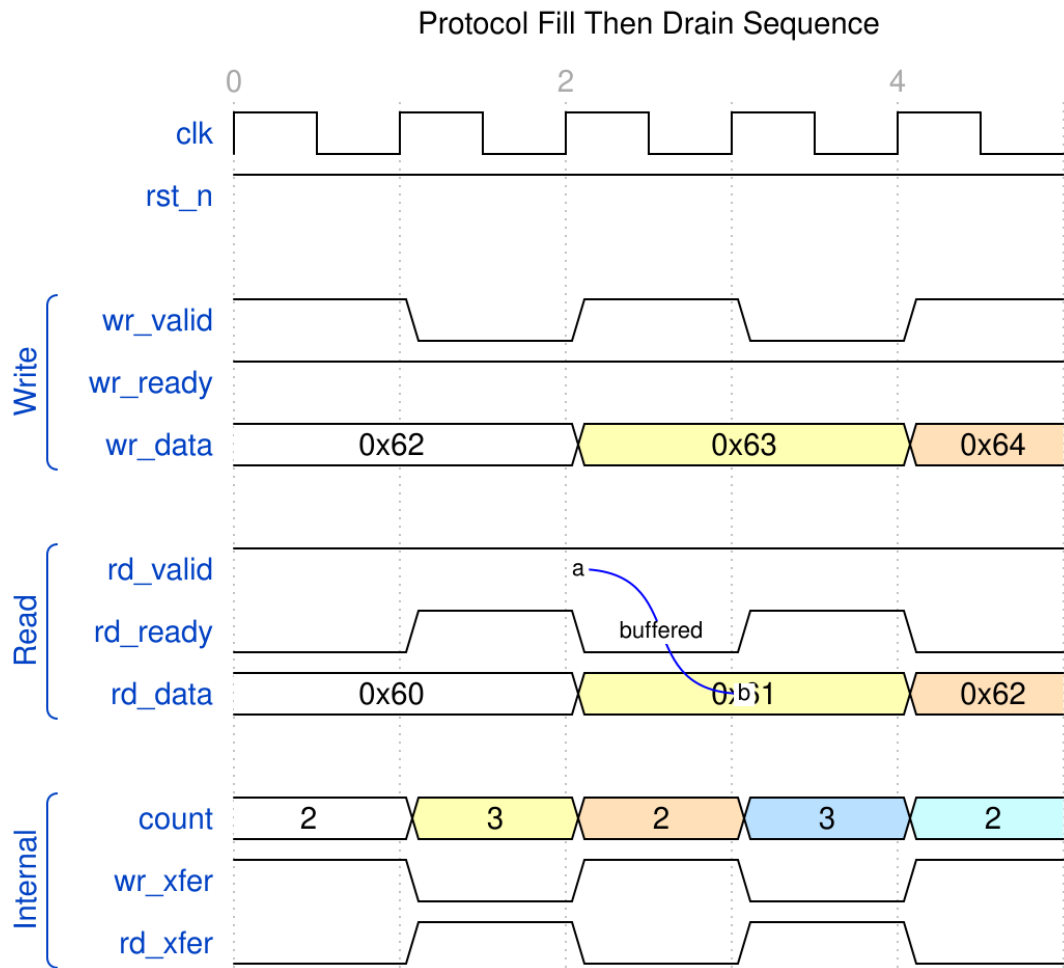


Burst Read Empty

WaveJSON: [burst_read_empty_001.json](#)

6.7.5 Scenario 5: Fill Then Drain Pattern

Complete fill phase followed by drain:

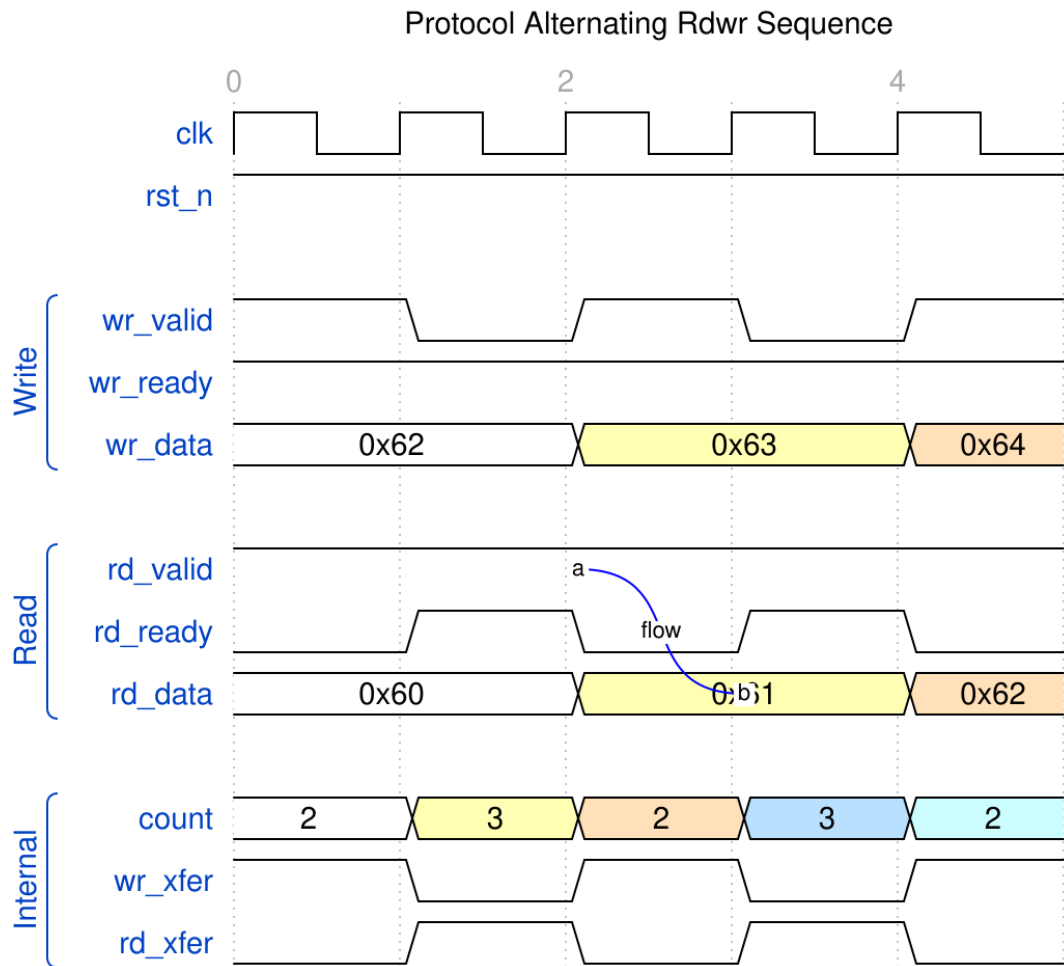


Fill Then Drain

WaveJSON: [fill_then_drain_001.json](#)

6.7.6 Scenario 6: Alternating Read/Write

Continuous interleaved operations:



Alternating R/W

WaveJSON: [alternating_rdwr_001.json](#)

6.8 Usage Examples

6.8.1 Example 1: Basic Pipeline Stage

```
// Break long combinatorial path
gaxi_skid_buffer #(
    .DATA_WIDTH(64),
    .DEPTH(4)
) u_pipeline_stage (
    .axi_aclk      (clk),
    .axi_aresetn  (rst_n),
    .wr_valid      (stage1_valid),
```

```

        .wr_ready      (stage1_ready),
        .wr_data       (stage1_data),
        .rd_valid      (stage2_valid),
        .rd_ready      (stage2_ready),
        .rd_data       (stage2_data),
        .count         (stage_occupancy),
        .rd_count      ()
    );

```

6.8.2 Example 2: Backpressure Absorption

```

// Absorb temporary backpressure from downstream
gaxi_skid_buffer #(
    .DATA_WIDTH(128),
    .DEPTH(8),           // Larger depth for more buffering
    .INSTANCE_NAME("BP_ABSORB")
) u_backpressure_buffer (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid      (continuous_valid), // Always producing
    .wr_ready      (continuous_ready),
    .wr_data       (continuous_data),
    .rd_valid      (bursty_valid),     // Intermittent consumer
    .rd_ready      (bursty_ready),
    .rd_data       (bursty_data),
    .count         (buffer_level),
    .rd_count      ()
);

// Monitor buffer level for flow control
assign almost_full = (buffer_level >= 6); // Threshold

```

6.8.3 Example 3: Monitoring Occupancy

```

gaxi_skid_buffer #(
    .DATA_WIDTH(32),
    .DEPTH(4)
) u_monitored (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid      (wr_valid),
    .wr_ready      (wr_ready),
    .wr_data       (wr_data),
    .rd_valid      (rd_valid),
    .rd_ready      (rd_ready),
    .rd_data       (rd_data),
    .count         (fifo_count),
    .rd_count      ()
);

```



```
// Performance monitoring
always_ff @(posedge clk) begin
    if (fifo_count > max_count_seen)
        max_count_seen <= fifo_count;

    if (fifo_count == 4'h0 && wr_valid && wr_ready)
        bypass_events <= bypass_events + 1; // Track bypass usage
end
```

6.9 Design Considerations

6.9.1 Depth Selection

DEPTH	Use Case	Latency	Buffering
2	Minimal buffering, tight timing	0-1 cycles	Limited backpressure tolerance
4	Standard pipeline stage	0-1 cycles	Moderate backpressure tolerance
6	Heavier buffering needs	0-1 cycles	Good backpressure tolerance
8	Maximum buffering	0-1 cycles	Excellent backpressure tolerance

Recommendation: Use DEPTH=4 for most cases - good balance of buffering and resource usage.

6.9.2 Timing Closure

If zero-latency bypass creates timing violations:

Option 1: Add pipeline stage after skid buffer

```
// Skid buffer (combinatorial read)
gaxi_skid_buffer #(.DATA_WIDTH(32), .DEPTH(4)) u_skid (...);

// Additional register stage
always_ff @(posedge clk) begin
    if (skid_rd_valid && skid_rd_ready) begin
        final_data <= skid_rd_data;
        final_valid <= 1'b1;
    end
end
```

```

        end else if (final_ready) begin
            final_valid <= 1'b0;
        end
    end
end

```

Option 2: Use FIFO with REGISTERED=1 instead (see [gaxi_fifo_sync](#))

6.9.3 Area vs. Performance Trade-offs

Metric	DEPTH=2	DEPTH=4	DEPTH=8
Flops	2×DW	4×DW	8×DW
LUTs	~40	~50	~70
Bypass Latency	0 cycles	0 cycles	0 cycles
Backpressure Tolerance	Low	Moderate	High

6.10 Error Checking

The module includes assertion checks (simulation only):

```

// synopsys translate_off
always @(posedge axi_aclk) begin
    if ((wr_valid && !wr_ready) && (wr_xfer)) begin
        $display("Error: %s write while buffer full, %t",
        INSTANCE_NAME, $time);
    end
end

always @(posedge axi_aclk) begin
    if ((rd_ready && !rd_valid) && (rd_xfer)) begin
        $display("Error: %s read while buffer empty, %t",
        INSTANCE_NAME, $time);
    end
end
// synopsys translate_on

```

These checks help catch: - Protocol violations (writes when not ready) - Invalid reads (reading when empty)

6.11 Testing

6.11.1 Test File

Location: val/amba/test_gaxi_buffer_sync.py

Test Modes: - mode='skid' → Tests this module specifically - mode='fifo_mux' → Tests gaxi_fifo_sync with REGISTERED=0 - mode='fifo_flop' → Tests gaxi_fifo_sync with REGISTERED=1

6.11.2 WaveDrom Test

Location: val/amba/test_gaxi_wavedrom_example.py

Generates comprehensive timing diagrams showing all 6 scenarios.

6.11.3 Running Tests

```
# Functional test (skid buffer mode)
pytest val/amba/test_gaxi_buffer_sync.py -k "skid" -v
```

```
# Generate waveforms
pytest val/amba/test_gaxi_wavedrom_example.py -v
cd val/amba && bash wd_cmd.sh
```

6.12 Common Issues

6.12.1 Issue 1: Timing Violation on Bypass Path

Symptom: Setup time violation from wr_data to rd_data

Solution:

```
// Add register stage after skid buffer
gaxi_skid_buffer #(...) u_skid (...);
```

```
always_ff @(posedge clk) begin
    registered_data <= skid_rd_data;
    registered_valid <= skid_rd_valid;
end
```

6.12.2 Issue 2: Buffer Overflow

Symptom: Data loss, count exceeds DEPTH

Debug: 1. Check write logic respects wr_ready 2. Verify DEPTH parameter is sufficient 3. Monitor count signal - should never exceed DEPTH

6.12.3 Issue 3: Unexpected Latency

Symptom: Data takes multiple cycles to appear at output

Debug: 1. Check buffer was empty when write occurred 2. Verify rd_ready is asserted (prevents data from appearing) 3. Look at count - if non-zero, data is buffered

6.13 Related Modules

- [gaxi_fifo_sync](#) - Larger depth, optional registered output
 - [gaxi_fifo_async](#) - Clock domain crossing
 - [gaxi_skid_buffer_async](#) - Async version
-

6.14 References

- **Tutorial:** [GAXI WaveDrom Tutorial](#)
 - **Overview:** [GAXI Index](#)
 - **Source:** rtl/amba/gaxi/gaxi_skid_buffer.sv
 - **Tests:** val/amba/test_gaxi_buffer_sync.py,
val/amba/test_gaxi_wavedrom_example.py
-

Version: 1.0 **Last Updated:** 2025-10-06 **Maintained By:** RTL Design Sherpa Project

7 GAXI Asynchronous Skid Buffer

Module: gaxi_skid_buffer_async.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

7.1 Overview

The GAXI asynchronous skid buffer combines the zero-latency bypass feature of the skid buffer with clock domain crossing capability. It's a wrapper that instantiates a synchronous skid buffer on the write side followed by an asynchronous FIFO.

7.1.1 Key Features

- ✓ **CDC Capability:** Safe clock domain crossing
- ✓ **Elastic Buffering:** Combines skid buffer + async FIFO

- ✓ **Write-Side Buffering:** Skid buffer in write domain prevents stalls
- ✓ **Configurable CDC:** Adjustable synchronizer stages
- ✓ **Dual Clock Domains:** Independent write/read clocks

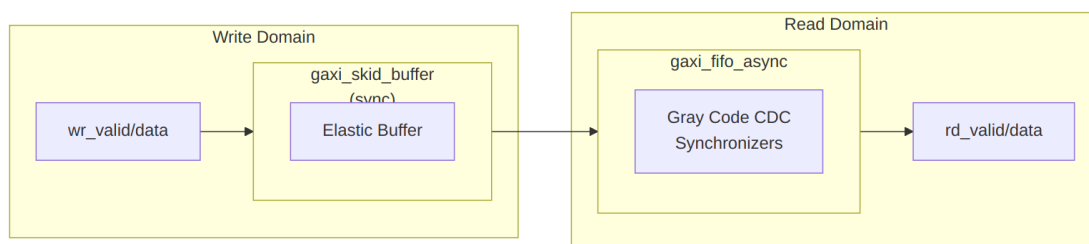
7.2 Module Interface

```

module gaxi_skid_buffer_async #(
    parameter int REGISTERED = 0,           // FIFO read mode
    parameter int DATA_WIDTH = 32,
    parameter int DEPTH = 2,               // Async FIFO depth
    parameter int N_FLOP_CROSS = 2,       // CDC stages
    parameter    INSTANCE_NAME = "DEADF1F0"
) (
    // Write Domain
    input  logic      axi_wr_aclk,
    input  logic      axi_wr_aresetn,
    input  logic      wr_valid,
    output logic      wr_ready,
    input  logic [DW-1:0] wr_data,

    // Read Domain
    input  logic      axi_rd_aclk,
    input  logic      axi_rd_aresetn,
    output logic      rd_valid,
    input  logic      rd_ready,
    output logic [DW-1:0] rd_data
);

```



Architecture

7.2.1 Internal Components

1. **gaxi_skid_buffer (Write Domain):**
 - Provides elastic buffering
 - Prevents write-side stalls

- Fixed depth (typically 2-4 entries)
2. **gaxi_fifo_async (CDC):**
- Handles clock domain crossing
 - Configurable depth (DEPTH parameter)
 - Gray code pointer synchronization
-

7.3 Parameters

Parameter	Default	Description
REGISTERED	0	Async FIFO read mode (0=mux, 1=flop)
DATA_WIDTH	32	Data bus width
DEPTH	2	Async FIFO depth (CDC stage)
N_FLOP_CROSS	2	Synchronizer stages (3 recommended)
INSTANCE_NAME	“DEADF1F0”	Debug instance name

Note: Skid buffer depth is fixed (determined by gaxi_skid_buffer default).

7.4 Usage Examples

7.4.1 Example 1: Basic Async Skid Buffer

```
gaxi_skid_buffer_async #(
    .DATA_WIDTH(64),
    .DEPTH(4),           // Async FIFO depth
    .N_FLOP_CROSS(3),    // CDC stages
    .REGISTERED(0)
) u_async_skid (
    // Write domain @ 100 MHz
    .axi_wr_aclk      (clk_100),
    .axi_wr_aresetn   (rst_100_n),
    .wr_valid         (src_valid),
    .wr_ready         (src_ready),
    .wr_data          (src_data),
```

```

    // Read domain @ 75 MHz
    .axi_rd_aclk      (clk_75),
    .axi_rd_aresetn  (rst_75_n),
    .rd_valid        (dst_valid),
    .rd_ready        (dst_ready),
    .rd_data         (dst_data)
);

```

7.4.2 Example 2: Pipeline Stage with CDC

```

// Break long path AND cross clock domains
gaxi_skid_buffer_async #(
    .DATA_WIDTH(128),
    .DEPTH(8),           // Larger for rate buffering
    .N_FLOP_CROSS(3),
    .REGISTERED(1)       // Registered output for timing
) u_cdc_pipeline (
    .axi_wr_aclk      (source_clk),
    .axi_wr_aresetn  (source_rst_n),
    .wr_valid        (stage_in_valid),
    .wr_ready        (stage_in_ready),
    .wr_data         (stage_in_data),

    .axi_rd_aclk      (dest_clk),
    .axi_rd_aresetn  (dest_rst_n),
    .rd_valid        (stage_out_valid),
    .rd_ready        (stage_out_ready),
    .rd_data         (stage_out_data)
);

```

7.5 Timing Characteristics

7.5.1 Latency Components

Component	Latency	Domain
Skid buffer (empty)	0 cycles	Write
Skid buffer (buffered)	1 cycle	Write
Async FIFO CDC	3-5 cycles	Both
Total (empty path)	3-5 cycles	End-to-end
Total (buffered)	4-6 cycles	End-to-end

7.5.2 Throughput

- **Write Side:** 1 transfer/cycle (when not full)
 - **Read Side:** 1 transfer/cycle (when not empty)
 - **Sustained:** Limited by slower clock domain
-

7.6 Design Considerations

7.6.1 When to Use This Module

Use async skid buffer when: - Need CDC between clock domains - Want elastic buffering on write side - Write-side needs low-latency response

Use plain async FIFO instead when: - Only need CDC, no special write-side buffering - Want deeper buffering (larger DEPTH) - Don't need skid buffer complexity

7.6.2 Depth Selection

Total buffering = Skid buffer entries + DEPTH

Recommendations: - **DEPTH=4:** Minimal CDC buffering, light traffic - **DEPTH=8:** Moderate buffering, balanced - **DEPTH=16:** Heavy buffering, burst traffic

7.6.3 Reset Synchronization

Critical: Both domains need synchronized resets!

```
// Example reset synchronization
reset_sync #(.STAGES(3)) u_wr_sync (
    .clk(axi_wr_aclk),
    .async_rst_n(global_rst_n),
    .sync_rst_n(axi_wr_aresetn)
);

reset_sync #(.STAGES(3)) u_rd_sync (
    .clk(axi_rd_aclk),
    .async_rst_n(global_rst_n),
    .sync_rst_n(axi_rd_aresetn)
);
```

7.7 Resource Utilization

Combines resources of both sub-modules:

Component	Flops	LUTs
Skid buffer (typical)	$\sim 4 \times DW$	~ 50
Async FIFO (DEPTH=8)	$8 \times DW + \sim 40$	~ 120
Total (DEPTH=8)	$\sim 12 \times DW + 40$	~ 170

7.8 Testing

Async skid buffer tests

```
pytest val/amba/test_gaxi_buffer_async.py -k "skid" -v
```

Test specific clock ratio

```
pytest val/amba/test_gaxi_buffer_async.py -k "skid" -k "wr10_rd20" -v
```

7.9 Common Issues

7.9.1 Issue 1: CDC Violations

Symptom: Metastability, data corruption

Solution: - Set `N_FLOP_CROSS=3` (minimum for production) - Verify clocks are truly independent - Check reset synchronization

7.9.2 Issue 2: Write-Side Stalls

Symptom: `wr_ready` deasserts unexpectedly

Debug: 1. Check async FIFO depth (DEPTH parameter) 2. Verify read-side is consuming data 3. Monitor clock ratio - may need deeper FIFO

7.9.3 Issue 3: Unexpected Latency

Symptom: Data takes longer than expected

Explanation: - Total latency = skid buffer + async FIFO CDC - Minimum ~ 3 -5 cycles even when empty - This is inherent to CDC design

7.10 Related Modules

- [gaxi_skid_buffer](#) - Synchronous version

- [gaxi_fifo_async](#) - Async FIFO without skid buffer
 - [GAXI Index](#) - Overview
-

7.11 References

- **Source:** rtl/amba/gaxi/gaxi_skid_buffer_async.sv
 - **Dependencies:**
 - gaxi_skid_buffer.sv
 - gaxi_fifo_async.sv
 - **Tests:** val/amba/test_gaxi_buffer_async.py
-

Version: 1.0 **Last Updated:** 2025-10-06

[← Back to GAXI Index](#) | [← Back to RTLamba Index](#)

8 GAXI Skid Buffer (Double-Drain)

Module: gaxi_skid_buffer_dbldrn.sv **Location:** rtl/amba/gaxi/ **Status:** Production Ready

8.1 Overview

gaxi_skid_buffer_dbldrn is a “double-drain” variant of the gaxi_skid_buffer. It keeps the same elastic write side and single-read output, but adds a **second read output** (rd_data2) and a **second drain request** (rd_ready2) so the consumer can pop **two entries in one clock** when at least two are buffered. This is useful for consumers that can retire two items per cycle (for example a 2-wide unpack or a burst aligner) and would otherwise be throughput-limited by a one-per-cycle drain.

Like the base skid buffer, storage is a shift register of DEPTH entries and the read/write handshakes use the GAXI valid/ready convention.

8.1.1 Key Features

- **Double-drain:** Assert rd_ready2 (legal only when rd_count $\geq$ 2) to pop two entries in a single cycle
- **Two read data outputs:** rd_data (lowest entry) and rd_data2 (next entry) presented simultaneously
- **Single-drain compatible:** With rd_ready2 low it behaves as an ordinary single-drain skid buffer

- **Priority-encoded update:** Write / single-read / double-read combinations handled by a one-hot case, double-drain prioritized over single-drain
- **Backpressure-aware ready/valid:** wr_ready and rd_valid account for both drain widths
- **Legality assertion:** Simulation \$error fires if rd_ready2 is asserted with fewer than two entries buffered

8.2 Module Purpose

The base skid buffer drains at most one entry per cycle, which caps a fast consumer at one item per clock. This variant lets a consumer that can accept two items per cycle drain both the lowest and next-lowest buffered entries together, doubling drain throughput while still honoring GAXI backpressure and preserving in-order (lowest-position-first) delivery.

Use Cases:

- Feeding a consumer that retires two entries per clock (2-wide unpack, dual-issue, burst aligner)
- Draining a buffer faster than one-per-cycle to relieve upstream backpressure
- Any place a gaxi_skid_buffer is used but the sink can opportunistically take a pair

Key Benefit: Doubles peak drain throughput (two entries per clock) when two or more are buffered, while degrading gracefully to ordinary single-drain skid-buffer behavior when only one entry is available.

8.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Data bus width in bits
DEPTH	int	4	Buffer depth in entries (must be one of {2, 4, 6, 8})
DW	int	DATA_WIDTH	Derived alias for DATA_WIDTH (do not override)
BUF_WIDTH	int	DATA_WIDTH * DEPTH	Derived total shift-register width (do not override)

Parameter	Type	Default	Description
BW	int	BUF_WIDTH	Derived alias for BUF_WIDTH (do not override)

8.4 Port Groups

8.4.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	Input	1	Clock
axi_aresetn	Input	1	Active-low asynchronous reset

8.4.2 Write Interface (input side)

Port	Direction	Width	Description
wr_valid	Input	1	Write data valid
wr_ready	Output	1	Ready to accept a write (registered)
wr_data	Input	DATA_WIDTH	Write data

8.4.3 Read Interface (output side)

Port	Direction	Width	Description
rd_valid	Output	1	Read data valid (registered)
rd_ready	Input	1	Consumer ready to accept a read
rd_ready2	Input	1	Double-drain request; legal only when rd_count >= 2
rd_data	Output	DATA_WIDTH	First (lowest-position) entry
rd_data2	Output	DATA_WIDTH	Second (next-position) entry, valid when double-

Port	Direction	Width	Description
			draining

8.4.4 Status

Port	Direction	Width	Description
count	Output	4	Current buffer occupancy
rd_count	Output	4	Same as count (read-side occupancy)

8.5 Functional Description

8.5.1 Transfer Detection

Three transfer conditions are decoded combinationally. Note that a single read is only recognized when `rd_ready2` is **not** asserted, and a double read additionally requires two entries present:

```
assign w_wr_xfer      = wr_valid & wr_ready;
assign w_rd_xfer      = rd_valid & rd_ready & ~rd_ready2;
// single drain
assign w_rd_dbl_xfer = rd_valid & rd_ready & rd_ready2 & (r_data_count
>= 2); // double drain
```

8.5.2 Storage Update

The shift register `r_data` and occupancy `r_data_count` are updated from a one-hot case over `{w_wr_xfer, w_rd_dbl_xfer, w_rd_xfer}`, with double-drain prioritized above single-drain:

Case	Meaning	r_data action	count change
3'b100	Write only	Load <code>wr_data</code> at the top of the occupied region	+1
3'b001	Single read only	Shift down by one entry	-1
3'b010	Double read only	Shift down by two entries	-2
3'b101	Write + single read	Shift down one, then insert <code>wr_data</code> at the new top	0
3'b110	Write + double read	Shift down two, then insert <code>wr_data</code> at the new top	-1

Case	Meaning	r_data action	count change
default	Idle / illegal (000, 011, 111)	No change	0

The 011 and 111 combinations are illegal (a single- and double-drain cannot both fire) and fall through to the no-change default. Reads always drain from the low positions (rd_data = entry 0, rd_data2 = entry 1), preserving in-order delivery.

8.5.3 Ready/Valid Logic

wr_ready and rd_valid are registered and computed one cycle ahead, accounting for both drain widths so that space freed by a double-drain is reflected in wr_ready:

```
wr_ready <= (r_data_count <= DEPTH-2) ||
             (r_data_count == DEPTH-1 && (~w_wr_xfer || w_rd_xfer ||
w_rd_dbl_xfer)) ||
             (r_data_count == DEPTH && (w_rd_xfer || w_rd_dbl_xfer));

rd_valid <= (r_data_count >= 2) ||
             (r_data_count == 1 && (~w_rd_xfer || w_wr_xfer)) ||
             (r_data_count == 0 && w_wr_xfer);
```

8.5.4 Output Assignments

The two lowest shift-register slots are presented on the two data outputs, and the occupancy drives both status ports:

```
assign rd_data  = r_data[DW-1:0];      // first item (lowest position)
assign rd_data2 = r_data[2*DW-1:DW];   // second item (next position)
assign rd_count = r_data_count;
assign count    = r_data_count;
```

8.5.5 Legality Assertion

A simulation-only check enforces the double-drain contract: asserting rd_ready2 (with rd_ready) when fewer than two entries are buffered raises a \$error. This catches consumers that request a double-drain the buffer cannot service.

8.6 Usage Example

```
// Skid buffer feeding a consumer that can retire two entries per
// clock.
gaxi_skid_buffer_dbldrn #(
    .DATA_WIDTH(32),
    .DEPTH(4)
) u_dbldrn (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
```

```

// write side
.wr_valid    (src_valid),
.wr_ready    (src_ready),
.wr_data     (src_data),
// read side
.rd_valid    (dst_valid),
.rd_ready    (dst_ready),
.rd_ready2   (dst_take_two & (rd_occupancy >= 2)), // only when
>= 2 buffered
.rd_data     (dst_data0),
.rd_data2    (dst_data1),
// status
.count       (rd_occupancy),
.rd_count    ()
);

```

8.7 Design Notes

- **rd_ready2 legality.** Only assert rd_ready2 (together with rd_ready) when rd_count >= 2. The RTL guards the transfer with the same condition, and the simulation assertion flags misuse.
- **Priority.** When both a single- and double-drain could be decoded, double-drain wins; the illegal simultaneous cases (011, 111) are ignored by the default branch.
- **In-order delivery.** Entry 0 is always the oldest; rd_data/rd_data2 expose the two oldest entries, and drains shift the register down, so ordering is preserved.
- **Graceful degradation.** With rd_ready2 tied low the module is functionally an ordinary single-drain skid buffer.
- **Reset macros.** The module uses the project reset_defs.svh ALWAYS_FF_RST / RST_ASSERTED macros for its registered state.

8.8 Related Modules

8.8.1 Used By

- Consumers that retire two GAXI entries per clock (2-wide unpack / aligner front-ends)

8.8.2 Uses

- None (self-contained; relies only on the reset_defs.svh reset macros)

8.8.3 See Also

- [gaxi_skid_buffer](#) - The single-drain base skid buffer this variant extends
- [gaxi_skid_buffer_struct](#) - Struct-typed skid buffer variant
- [gaxi_fifo_sync](#) - Larger-depth synchronous FIFO with optional registered output

8.9 References

8.9.1 Source Code

- `rtl/amba/gaxi/gaxi_skid_buffer_dbldrn.sv`
- `rtl/amba/gaxi/gaxi_skid_buffer.sv` (base variant)

8.9.2 Documentation

- `rtl/amba/PRD.md`
-

Last Updated: 2026-07-15

8.10 Navigation

- [← Back to GAXI Index](#)
- [← Back to RTLAmba Index](#)

9 gaxi_skid_buffer_struct

Module: `gaxi_skid_buffer_struct.sv` **Location:** `rtl/amba/gaxi/` **Status:** ✓ Production Ready

9.1 Overview

Struct-aware variant of [gaxi_skid_buffer](#) that accepts SystemVerilog type parameters instead of explicit data width. This enables clean handling of complex data structures (e.g., AXI channels, custom packets) without manual packing/unpacking.

9.1.1 Key Features

- ✓ **Type-Parameterized:** Accepts any SystemVerilog type (struct, union, enum, packed array)

- ✓ **Automatic Width Calculation:** Uses \$bits() to determine buffer width
 - ✓ **Zero-Latency Bypass:** Identical performance to base skid buffer
 - ✓ **Same Architecture:** Shift register with valid/ready flow control
 - ✓ **Debug Support:** Instance naming and transaction logging
-

9.2 Module Declaration

```

module gaxi_skid_buffer_struct #(
    parameter type STRUCT_TYPE = logic [31:0], // Any SystemVerilog
type
    parameter int DEPTH = 2, // Must be {2, 4, 6,
8}
    parameter INSTANCE_NAME = "DEADF1F0", // Debug identifier

    // Automatically derived
    localparam int STRUCT_WIDTH = $bits(STRUCT_TYPE),
    localparam int BUF_WIDTH = STRUCT_WIDTH * DEPTH
) (
    // Global Clock and Reset
    input logic axi_aclk,
    input logic axi_aresetn,

    // Input side
    input logic wr_valid,
    output logic wr_ready,
    input STRUCT_TYPE wr_data, // Type-safe input

    // Output side
    output logic [3:0] count,
    output logic rd_valid,
    input logic rd_ready,
    output logic [3:0] rd_count,
    output STRUCT_TYPE rd_data // Type-safe output
);

```

9.3 Parameters

Parameter	Type	Default	Description
STRUCT_TYPE	type	logic [31:0]	Any SystemVerilog type (struct, union, enum, array)

Parameter	Type	Default	Description
DEPTH	int	2	Buffer depth (must be 2, 4, 6, or 8)
INSTANCE_NAME	string	“DEADF1F0”	Debug identifier for simulation messages

9.4 Ports

9.4.1 Clock and Reset

Port	Width	Direction	Description
axi_aclk	1	Input	Clock
axi_aresetn	1	Input	Active-low asynchronous reset

9.4.2 Write Interface

Port	Width	Direction	Description
wr_valid	1	Input	Write data valid
wr_ready	1	Output	Ready to accept write
wr_data	STRUCT_TYPE	Input	Input data (type-safe)

9.4.3 Read Interface

Port	Width	Direction	Description
rd_valid	1	Output	Read data valid
rd_ready	1	Input	Ready to consume read
rd_data	STRUCT_TYPE	Output	Output data (type-safe)

9.4.4 Status

Port	Width	Direction	Description
count	4	Output	Current buffer occupancy (0 to DEPTH)
rd_count	4	Output	Same as count (for compatibility)

9.5 Usage Examples

9.5.1 Example 1: AXI Read Address Channel

```
// Define AXI AR channel struct
typedef struct packed {
    logic [7:0] arid;
    logic [31:0] araddr;
    logic [7:0] arlen;
    logic [2:0] arsize;
    logic [1:0] arburst;
    logic arlock;
    logic [3:0] arcache;
    logic [2:0] arprot;
    logic [3:0] arqos;
} axi_ar_t;

// Instantiate struct-aware buffer
gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(axi_ar_t),
    .DEPTH(4),
    .INSTANCE_NAME("AXI_AR_BUF")
) u_ar_buffer (
    .axi_aclk (axi_clk),
    .axi_aresetn (axi_resetn),

    // Input: Directly assign struct
    .wr_valid (master_arvalid),
    .wr_ready (master_arready),
    .wr_data ({master_arid, master_araddr, master_arlen,
               master_arsize, master_arburst, master_arlock,
               master_arcache, master_arprot, master_arqos}),

    // Output: Unpack struct
```

```

        .rd_valid      (slave_arvalid),
        .rd_ready      (slave_arready),
        .rd_data       ({slave_arid, slave_araddr, slave_arlen,
                        slave_arsize, slave_arburst, slave_arlock,
                        slave_arcache, slave_arprot, slave_arqos}),

        .count         (ar_buf_count),
        .rd_count      ()
    );

```

9.5.2 Example 2: Custom Packet with Mixed Fields

```

// Define custom packet structure
typedef struct packed {
    logic [15:0] packet_id;
    logic [7:0]  packet_type;
    logic [31:0] timestamp;
    logic [63:0] payload;
    logic [3:0]  priority;
    logic        last;
} custom_pkt_t;

// Buffer for packet pipeline
gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(custom_pkt_t),
    .DEPTH(8),
    .INSTANCE_NAME("PKT_PIPELINE")
) u_pkt_buffer (
    .axi_aclk      (pkt_clk),
    .axi_aresetn   (pkt_resetn),

    .wr_valid      (ingress_pkt_valid),
    .wr_ready      (ingress_pkt_ready),
    .wr_data       (ingress_pkt),          // Direct struct assignment

    .rd_valid      (egress_pkt_valid),
    .rd_ready      (egress_pkt_ready),
    .rd_data       (egress_pkt),          // Direct struct output

    .count         (pkt_buf_depth),
    .rd_count      ()
);

```

9.5.3 Example 3: Simple Data Array

```

// Array of samples
typedef logic [15:0] sample_array_t [4];

```

```

gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(sample_array_t),
    .DEPTH(4),
    .INSTANCE_NAME("SAMPLE_BUF")
) u_sample_buffer (
    .axi_aclk      (dsp_clk),
    .axi_aresetn   (dsp_resetn),

    .wr_valid      (sample_valid),
    .wr_ready      (sample_ready),
    .wr_data       (sample_array),      // 4 x 16-bit samples

    .rd_valid      (proc_valid),
    .rd_ready      (proc_ready),
    .rd_data       (proc_array),

    .count         (sample_count),
    .rd_count      ()
);

```

9.6 Behavior

9.6.1 Identical to gaxi_skid_buffer

The internal behavior is **identical** to [gaxi_skid_buffer](#):

1. **Shift Register:** Data stored in array, shifts on read
2. **Zero-Latency Bypass:** Empty buffer passes data combinatorially
3. **Valid/Ready Handshake:** Transfer occurs when valid && ready
4. **Backpressure:** wr_ready deasserts when full

9.6.2 Key Difference: Type Safety

gaxi_skid_buffer:

```

.DATA_WIDTH(96),      // Manual width calculation
.wr_data(packed_data), // Manual packing required
.rd_data(packed_out)  // Manual unpacking required

```

gaxi_skid_buffer_struct:

```

.STRUCT_TYPE(axi_ar_t), // Type parameter
.wr_data(ar_struct),    // Direct struct assignment
.rd_data(ar_out_struct) // Direct struct output

```

9.7 Advantages Over Base Skid Buffer

9.7.1 Advantage 1: Type Safety

```
// Base skid buffer - error-prone
gaxi_skid_buffer #(.DATA_WIDTH(96)) u_buf (
    .wr_data({arid, araddr, arlen, ...}) // Easy to get field order
    wrong
);
```

```
// Struct buffer - compiler-checked
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_ar_t)) u_buf (
    .wr_data(ar_channel) // Struct ensures correct fields
);
```

9.7.2 Advantage 2: No Manual Width Calculation

```
// Base: Must calculate width manually
localparam AR_WIDTH = 8 + 32 + 8 + 3 + 2 + 1 + 4 + 3 + 4; // Error-prone
```

```
// Struct: Automatic using $bits()
localparam AR_WIDTH = $bits(axi_ar_t); // Always correct
```

9.7.3 Advantage 3: Cleaner Code

```
// Before: Manual packing/unpacking
wire [95:0] packed_ar;
assign packed_ar = {arid, araddr, arlen, arsize, ...};
assign {slave_arid, slave_araddr, ...} = unpacked_ar;
```

```
// After: Direct struct assignment
axi_ar_t master_ar, slave_ar;
assign slave_ar = master_ar; // Clean and readable
```

9.8 Design Patterns

9.8.1 Pattern 1: AXI Channel Buffering

```
// Define all AXI channel structs
typedef struct packed {
    logic [ID_W-1:0] awid;
    logic [ADDR_W-1:0] awaddr;
    // ... all AW signals
};
```

```

} axi_aw_t;

// Buffer each channel with appropriate type
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_aw_t)) u_aw_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_w_t)) u_w_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_b_t)) u_b_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_ar_t)) u_ar_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_r_t)) u_r_buf (...);

```

9.8.2 Pattern 2: Pipeline Stages with Complex Data

```

// Define pipeline stage struct
typedef struct packed {
    logic [31:0] instruction;
    logic [4:0]  rs1, rs2, rd;
    logic [31:0] imm;
    logic [6:0]  opcode;
    logic        valid;
} pipeline_stage_t;

// Each pipeline stage buffered
pipeline_stage_t fetch_stage, decode_stage, execute_stage;

gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(pipeline_stage_t),
    .INSTANCE_NAME("FETCH_DECODE")
) u_fd_buf (
    .wr_data(fetch_stage),
    .rd_data(decode_stage),
    ...
);

```

9.9 Testing

```

# Tests use parameterized structs
pytest val/amba/test_gaxi_skid_buffer_struct.py -v

# Test with different struct types
pytest val/amba/test_gaxi_skid_buffer_struct.py::test_axi_channel -v
pytest val/amba/test_gaxi_skid_buffer_struct.py::test_custom_struct -v

```

9.10 Synthesis Notes

9.10.1 Resource Usage

Same as **gaxi_skid_buffer** with equivalent DATA_WIDTH: - Logic: ~50 LUTs (for DEPTH=4) - Storage: DEPTH × \$bits(STRUCT_TYPE) flip-flops - No additional overhead from type parameter

9.10.2 Type Requirements

SystemVerilog types that work: - ✓ struct packed (recommended) - ✓ Packed arrays: logic [N:0][M:0] - ✓ Simple types: logic [N:0] - ⚠ struct (unpacked) - will be automatically packed by synthesis

Not supported: - ✗ Dynamic types - ✗ Classes - ✗ Queues/associative arrays

9.11 Related Modules

- [gaxi_skid_buffer](#) - Base implementation (DATA_WIDTH parameter)
 - [gaxi_fifo_sync](#) - Synchronous FIFO alternative
 - [gaxi_skid_buffer_async](#) - Clock domain crossing variant
-

9.12 When to Use

✓ **Use gaxi_skid_buffer_struct When:** - Working with complex data structures (AXI channels, custom packets) - Want type safety and compiler checks - Need cleaner, more readable code - Struct definition already exists

✓ **Use gaxi_skid_buffer When:** - Simple data widths (no complex structure) - Need absolute minimum resource usage - Working with legacy code that uses explicit widths

Last Updated: 2025-10-20

9.13 Navigation

- [← Back to GAXI Index](#)
- [← Back to RTLamba Index](#)

10 Shared Infrastructure Documentation Status

Generated: 2025-10-23 **Last updated:** 2026-06-15 (timing-closure refresh: 3-stage burst-writer pipeline, 2-stage compressor, per-template delta_ts, runtime cfg_compress_en, mod_3_compress helper, new axi4_dma_observer doc) **Location:**
/mnt/data/github/RTLDesignSherpa/docs/markdown/RTLAmba/shared/

10.1 Completion Status

10.1.1 Completed Documentation

1. **axi_monitor_base.md** - COMPLETE
 - Comprehensive module documentation with all sections
 - Usage examples provided
 - Design notes included
 - Cross-references added
2. **axi_monitor_trans_mgr.md** - COMPLETE (rewritten 2026-06-08)
 - Reflects CAM-backed revision (delegates to monitor_trans_cam)
 - Documents synthesis properties carried from the 2026-04-23 WNS fix
 - Equivalence test + TRANS_MGR_VARIANT rollback knob covered
3. **monitor_trans_cam.md** - COMPLETE (new 2026-06-08)
 - Multi-port ID CAM with opaque payload
 - 3 lookup ports + 3-way mutex alloc priority encoder
 - Synthesis notes (keep="true" anchors, per-slot generate-loop storage)
4. **monbus_compressor.md** - COMPLETE (new 2026-06-08; refreshed 2026-06-15)
 - Hardware bulk-trace encoder, ~2.6× compression
 - Full coverage of all 5 compression techniques (template extraction, delta-ts, width-tiered Tier-1, differential payload, Tier-0 RAW)
 - Bit layouts, encoder decision tree, pipeline/timing/synthesis notes
 - 2026-06-15 refresh: pipeline is now 2 stages (lookup/commit -> encode/emit); delta_ts is per-template (matches the CAM's per-entry r_ts[24]) rather than against a single global r_last_ts; added compressor input skid + pblock_monbus floorplan note for the Nexys A7 build
5. **monbus_cam.md** - COMPLETE (new 2026-06-08; refreshed 2026-06-15)

- 32-entry true-LRU caching CAM (position-indexed storage)
 - Backs monbus_compressor's template indexing
 - Caller protocol (NONE/TOUCH/INSTALL), eviction semantics
 - 2026-06-15 refresh: added per-entry r_ts[TS_WIDTH=24] storage with access_old_ts / access_new_ts ports (shifts with the LRU move-to-front). CAM is no longer pure opaque-payload – the caller must drive access_new_ts on every TOUCH/INSTALL.
6. **monbus_group.md** - COMPLETE (rewritten 2026-06-11, was monbus_axil_group.md; refreshed same day for the two-stage burst-sizing fix)
- Renamed from monbus_axil_group.md to reflect the new family.
 - Covers the full 5-file family: 1 protocol-agnostic core (monbus_group_core.sv) + 4 protocol-permutation wrappers (axil_axil / axil_axi4 / axi4_axil / axi4_axi4).
 - Documents the beat-granular write FIFO, watermark + timeout burst writer (4KB-boundary aware), and AXI4 burst behavior.
 - Migration recipe from the legacy single monbus_axil_group module (port-surface changes: cfg_flush_watermark added, FIFO counts widened to 16 bits, FIFO_DEPTH_WRITE now in beats, locked 64-bit data width).
 - “Master-Write Behavior” section refreshed for the two-stage drain plan (drain-cycle plans whole-record beats without a MAX_BURST_BEATS cap; per-AW sub-bursts get the cap inside the FSM). The “Test” section now lists all five tests in the suite, including the new master-write coverage that closed the AXIL-master raw-mode deadlock.
 - 2026-06-15 refresh: burst writer geometry is now a 3-stage registered pipeline with a fresh-FIFO cap applied combinationally at the WR_IDLE commit (necessary because the FIFO keeps filling while WR_IDLE waits); mod-3 rounding moved to mod_3_compress (rtl/common/, carry-save-compressor idiom) so synthesis no longer infers a DSP48 or CARRY4 iterative divider for the /3 path; runtime cfg_compress_en input added (USE_COMPRESSION=1 elaborates the compressor, cfg_compress_en selects at runtime); canonical rtl/amba/filelists/monbus_group.f introduced (one place to add new core deps).
7. **sdpram_slave.md** - COMPLETE (new 2026-06-09)

- Covers the full 5-file family: 1 backend + 4 protocol-specific wrappers (axi4_axi4 / axi4_axil / axil_axi4 / axil_axil).
 - Documents why the split exists (SystemVerilog cannot conditionally include/exclude ports in a single module declaration).
 - Migration recipe from bare sdpram_slave to the matching wrapper.
 - Cross-links from monbus_group.md and monbus_compressor.md (the memory-dump ring's canonical backend is sdpram_slave_axil_axil).
8. **axi_monitor_reporter.md** - COMPLETE (rewritten 2026-06-11)
- Reflects the 2026-06-06 sub-block refactor (thin dispatcher + 6 ENABLE_*_LOGIC-gated detection sub-blocks).
 - Lists the six sub-blocks (error / timeout / compl / threshold / perf / debug), their logic shapes, and their gate parameters.
 - Notes the bridge-case savings (ENABLE_ERROR_LOGIC=1, others 0 drops ~70% LUT/FF).
 - The six sub-block files (axi_monitor_reporter_*.sv) are explicitly covered here rather than as individual doc pages, since they are private to the reporter family.
9. **axi4_dma_observer.md** - COMPLETE (new 2026-06-15)
- Standalone, DMA-agnostic observability harness that wraps any AXI4-master DMA from outside the DMA (non-intrusive). Companion to the per-DMA axi_monitor_* family which wraps from inside.
 - Covers the full instantiation pattern: NUM_RD + NUM_WR axi4_master_*_mon taps, monbus_arbiter aggregator, monbus_axil_axi4_group filter+dump, axi_bus_meter, and axi_perf_latency_hist per port.
 - Documents the runtime rid -> channel map (cfg_rd_rid_per_channel) for read-side per-channel attribution and, for writes, either the built-in AW->W awid order tracker (WR_CH_FROM_AWID=1, no DUT sideband) or the optional dma_wr_active_ch_* sideband.
 - The companion modules axi_bus_meter.sv and axi_perf_latency_hist.sv are covered in this single doc rather than their own pages (small, only useful in context).

10.1.2 New shared infrastructure (no dedicated page, covered above)

- rtl/common/mod_3_compress.sv - carry-save-compressor X mod 3 for 16-bit operands; used by monbus_group_core's whole-record rounding. Covered in monbus_group.md "mod-3 rounding" section.
- rtl/amba/filelists/monbus_group.f - canonical filelist enumerating the group core's dependency tree (math_adder_carry_save_nbit + mod_3_compress + monbus_cam + monbus_compressor + monbus_group_core). All consumers - f-include it. Covered in monbus_group.md "Canonical filelist" subsection.

10.1.3 Remaining Documentation (15 modules)

The following modules require documentation following the same pattern as axi_monitor_base.md:

10.1.3.1 *Monitor Infrastructure*

2. axi_monitor_filtered.md - PENDING
3. axi_monitor_reporter.md - **COMPLETE** (rewritten 2026-06-11, see #17)
 - Now describes the dispatcher + 6 sub-blocks (error / timeout / compl / threshold / perf / debug). The six sub-block files (axi_monitor_reporter_*.sv) are covered here, not as separate doc pages.
4. axi_monitor_timeout.md - PENDING
5. axi_monitor_timer.md - PENDING
6. axi_monitor_trans_mgr.md - **COMPLETE** (rewritten 2026-06-08)
 - See #11 in Completed Documentation above

10.1.3.2 *Monitor Bus Delivery + Bulk-Trace Compression (NEW SECTION)*

- monbus_group.md - **COMPLETE** (rewritten 2026-06-11, see #15)
 - Renamed from monbus_axil_group.md; covers the new 5-file family (core + 4 wrappers).
- monbus_compressor.md - **COMPLETE** (new 2026-06-08, see #13)
- monbus_cam.md - **COMPLETE** (new 2026-06-08, see #14)
- monitor_trans_cam.md - **COMPLETE** (new 2026-06-08, see #12)

10.1.3.3 *Memory / BRAM Slave (NEW SECTION)*

- sdpram_slave.md - **COMPLETE** (new 2026-06-09, see #16)

- Covers backend (sdpram_slave.sv) + 4 wrappers (axi4_axi4, axi4_axil, axil_axi4, axil_axil) in a single doc.

10.1.3.4 *Monitor Bus Arbitration (4 modules)*

7. arbiter_monbus_common.md - PENDING
8. arbiter_rr_pwm_monbus.md - PENDING
9. arbiter_wrr_pwm_monbus.md - PENDING
10. monbus_arbiter.md - PENDING

10.1.3.5 *AXI Utilities (4 modules)*

11. axi_gen_addr.md - PENDING
12. axi_master_rd_splitter.md - PENDING
13. axi_master_wr_splitter.md - PENDING
14. axi_split_combi.md - PENDING

10.1.3.6 *Infrastructure (2 modules)*

15. amba_clock_gate_ctrl.md - PENDING
16. cdc_handshake.md - PENDING

10.2 Documentation Template

Each documentation file should follow this structure (see axi_monitor_base.md as reference):

[Module Name]

```

**Module:** ` [module_name].sv `
**Location:** ` rtl/amba/shared/ `
**Status:** Production Ready

```

Overview

[Brief description from RTL file header comments]

Key Features

- Feature 1
- Feature 2
- Feature 3
- Feature 4

Module Purpose

[Detailed purpose - why this module exists, what problem it solves]

Parameters

Parameter	Type	Default	Description
...	...	...	...

Port Groups

[Group 1 Name]

Port	Direction	Width	Description
...	...	...	...

Functional Description

[How the module works - key behavior, FSM states, protocol details]

Usage Example

```
```systemverilog
[Realistic instantiation example]
```

## 10.3 Design Notes

---

### 10.3.1 [Important design aspect 1]

[Details]

---

## 10.4 Related Modules

---

### 10.4.1 Used By

- [List of parent modules]

### 10.4.2 Uses

- [List of child modules]
- 

## 10.5 References

---

### 10.5.1 Specifications

- [ARM specs]
- [Internal references]

### 10.5.2 Source Code

- RTL: rtl/amba/shared/[module\_name].sv
  - Tests: val/amba/test\_[module\_name].py
- 

**Last Updated:** 2025-10-23

---

## 10.6 Navigation

---

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

---

## Key Information Extracted from RTL

### ### Monitor Infrastructure Modules

#### **\*\*axi\_monitor\_filtered.sv\*\***

- Wraps axi\_monitor\_base with configurable packet filtering
- 3-level filtering hierarchy: packet type masking, error routing, event code masking
- AXI protocol specific (protocol 3'b000)
- Optional pipeline stage for timing closure

#### **\*\*axi\_monitor\_reporter.sv\*\***

- Reports events/errors through shared monitor bus
- Detects conditions from transaction table
- Formats 64-bit interrupt packets
- Supports error, completion, timeout, threshold, performance, debug packets
- FIFO buffering with gaxi\_fifo\_sync
- Event reported feedback to trans\_mgr (FIX-001)

#### **\*\*axi\_monitor\_timeout.sv\*\***

- Monitors transaction table for timeout conditions
- Per-phase timeout detection (address, data, response)
- Uses timer tick from frequency invariant timer
- Configurable timeout thresholds per phase

#### **\*\*axi\_monitor\_timer.sv\*\***

- Frequency invariant timer for timeout detection
- Uses counter\_freq\_invariant module
- Generates timing ticks based on frequency selection
- Maintains global timestamp counter

#### **\*\*axi\_monitor\_trans\_mgr.sv\*\***

- Manages transaction tracking table
- Tracks up to MAX\_TRANSACTIONS concurrent transactions
- Handles out-of-order completions
- Supports data-before-address scenarios
- Event reported feedback input (FIX-001)

### ### Monitor Bus Arbitration Modules

#### **\*\*arbiter\_monbus\_common.sv\*\***

- Comprehensive monitoring for RR and WRR arbiters
- Silicon debug monitor with PROTOCOL\_ARB events
- 3-bit protocol field [59:57]
- Event categories: error, timeout, completion, threshold, performance, debug
- Per-client ACK timeout tracking



- Protocol violation detection
- Fairness deviation monitoring
- Grant efficiency tracking

**\*\*arbiter\_rr\_pwm\_monbus.sv\*\***

- Round-robin arbiter with PWM control
- Standardized fixed internal configurations
- PWM\_WIDTH = 16 bits
- MON\_FIFO\_DEPTH = 16
- Uses arbiter\_monbus\_common for monitoring

**\*\*arbiter\_wrr\_pwm\_monbus.sv\*\***

- Weighted round-robin arbiter with PWM control
- Per-client weight thresholds
- Standardized fixed internal configurations
- Enhanced debug outputs for silicon debug

**\*\*monbus\_arbiter.sv\*\***

- Round-robin arbiter for monitor bus interfaces
- Optional input and output skid buffers
- ACK mode operation (grants held until acknowledged)
- 128-bit packet + 64-bit side-band timestamp, carried atomically through a 192-bit skid
- Parameterizable number of clients

### ### AXI Utilities Modules

**\*\*axi\_gen\_addr.sv\*\***

- Address generation for AXI bursts
- Supports FIXED, INCR, WRAP burst types
- Handles data width conversions
- Calculates next address and aligned address

**\*\*axi\_master\_rd\_splitter.sv\*\***

- Splits AXI read transactions across boundary crossings
- Assumptions: aligned addresses, fixed transfer size, incrementing bursts
- No address wraparound handling
- Split information FIFO for tracking
- State machine: IDLE, SPLITTING

**\*\*axi\_master\_wr\_splitter.sv\*\***

- Splits AXI write transactions across boundary crossings
- Same assumptions as read splitter
- WLAST generation for split transactions
- Response consolidation (N split responses -> 1 upstream response)

- Error priority: DECERR > SLVERR > EXOKAY > OKAY

**\*\*axi\_split\_combi.sv\*\***

- Pure combinational split decision logic
- Simplified boundary crossing detection
- No wraparound handling
- Comprehensive assertions for validation
- Used by both read and write splitters

### ### Infrastructure Modules

**\*\*amba\_clock\_gate\_ctrl.sv\*\***

- Wrapper for clock\_gate\_ctrl with AMBA-specific activity monitoring
- Monitors user\_valid and axi\_valid signals
- Configurable idle countdown
- Generates gated clock output

**\*\*cdc\_handshake.sv\*\***

- Clock domain crossing with handshake protocol
- 3-stage synchronizers for metastability protection
- Separate FSMs for source and destination domains
- Parameterizable data width

---

### ## Next Steps

1. Create remaining 15 markdown files following the template
2. Create README.md index file for shared infrastructure directory
3. Update parent RTLamba/README.md to link to shared/ subdirectory
4. Cross-reference all related modules
5. Add usage examples for each module
6. Document integration patterns

---

### ## Notes

- All RTL source files have been read and analyzed
- Module purposes and key features extracted
- Parameter tables ready for documentation
- Port groupings identified
- Design notes captured
- No emojis used in technical documentation

```
... {.pagebreak}
...
```

## # AMBA Shared Infrastructure

```
Location: `rtl/amba/shared/`
Test Location: Tested indirectly via protocol modules
Status: ✓ Production Ready
```

---

## ## Overview

The shared subsystem provides foundational infrastructure modules used throughout the AMBA ecosystem. These modules implement common functionality for monitors, arbiters, clock domain crossing, and utility functions. They are typically not instantiated directly by users but form the backbone of higher-level protocol modules.

## ### Key Features

- ✓ **Monitor Infrastructure:** Complete AXI/APB transaction monitoring system
- ✓ **CDC Utilities:** Clock domain crossing with proper synchronization
- ✓ **Arbitration:** Monitor bus arbitration for multi-agent systems
- ✓ **Address Generation:** AXI burst address calculation
- ✓ **Clock Gating:** Power management infrastructure
- ✓ **Production Ready:** Extensively tested through protocol modules

---

## ## Module Categories

### ### Monitor Infrastructure

Core modules that implement the AXI/APB monitoring functionality. The reporter (`axi\_monitor\_reporter`) is now a thin dispatcher around six packet-type-specific sub-blocks (error / timeout / compl / threshold / perf / debug); each is gated by an `ENABLE\_\*\_LOGIC` parameter so an integrator that only needs error packets pays no LUT/FF for the others.

Module	Purpose	Key Features	Documentation
--------	---------	--------------	---------------

```

|-----|-----|-----|-----|
| **axi_monitor_base** | Core monitor logic | Transaction tracking,
timeout detection, event generation |
[axi_monitor_base.md](../monitor/axi_monitor_base.md) |
| **axi_monitor_filtered** | Monitor wrapper with filtering | 3-level
packet filtering, configuration validation | [axi_monitor_filtered.md]
(../monitor/axi_monitor_filtered.md) |
| **axi_monitor_trans_mgr** | Transaction table management (CAM-
backed) | Out-of-order tracking, ID management |
[axi_monitor_trans_mgr.md](../monitor/axi_monitor_trans_mgr.md) |
| **monitor_trans_cam** | Multi-port ID CAM with payload | 3 lookup
ports, 3-way alloc mutex, used by trans_mgr | [monitor_trans_cam.md]
(../monitor/monitor_trans_cam.md) |
| **axi_monitor_reporter** | Event packet dispatcher | Top-level mux
over 6 detection sub-blocks (error / timeout / compl / threshold /
perf / debug), each gated by `ENABLE_*_LOGIC` |
[axi_monitor_reporter.md](../monitor/axi_monitor_reporter.md) |
| `axi_monitor_reporter_{error,timeout,compl,threshold,perf,debug}` |
Per-packet-type detection sub-blocks | One MonBus packet type each,
muxed by axi_monitor_reporter; each gated by `ENABLE_*_LOGIC` |
[error](../monitor/axi_monitor_reporter_error.md) ·
[timeout](../monitor/axi_monitor_reporter_timeout.md) ·
[compl](../monitor/axi_monitor_reporter_compl.md) ·
[threshold](../monitor/axi_monitor_reporter_threshold.md) · [perf]
(../monitor/axi_monitor_reporter_perf.md) ·
[debug](../monitor/axi_monitor_reporter_debug.md) |
| **axi_monitor_timeout** | Timeout detection | Configurable
thresholds, multi-channel monitoring |
[axi_monitor_timeout.md](../monitor/axi_monitor_timeout.md) |
| **axi_monitor_timer** | Frequency-invariant timer | Configurable
tick generation |
[axi_monitor_timer.md](../monitor/axi_monitor_timer.md) |
| **axi_monitor_addr_check** | Address-match watchpoints | Per-channel
address compare and event emission |
[axi_monitor_addr_check.md](../monitor/axi_monitor_addr_check.md) |
| **apb_monitor_addr_check** | APB address-range filter | N
configurable ranges; emits APB address-range error packets |
[apb_monitor_addr_check.md](../monitor/apb_monitor_addr_check.md) |
| **amba_clock_gate_ctrl** | Clock gating control | Dynamic gating,
idle detection | amba_clock_gate_ctrl.md |

```

### ### Monitor Bus Delivery + Bulk-Trace Compression

The path from in-flight monitor packets to host-visible memory and CPU IRQs.

The monbus\_group family pattern matches sdpram\_slave – one protocol-agnostic core plus four wrappers that adapt the slave-read / master-write

surfaces to  
the matching fabric without spurious AXI4-only fields on AXIL sides.

Module	Purpose	Key Features	Documentation
<b>monbus_group_core</b>	Protocol-agnostic core	Per-protocol filter, err FIFO (record-granular), write FIFO (beat-granular), watermark+timeout burst writer, 4KB-boundary aware, optional bulk-trace compression	[monbus_group_core.md](../monitor/monbus_group_core.md) (family: [monbus_group.md](../monitor/monbus_group.md))
<b>monbus_axil_axil_group</b>	Wrapper – AXIL/AXIL	Slave-read + master-write both AXIL (single-beat). Replaces legacy monbus_axil_group.	[monbus_axil_axil_group.md](../monitor/monbus_axil_axil_group.md)
<b>monbus_axil_axi4_group</b>	Wrapper – AXIL/AXI4	AXIL slave-read + AXI4 burst master-write (up to 256 beats/burst)	[monbus_axil_axi4_group.md](../monitor/monbus_axil_axi4_group.md)
<b>monbus_axi4_axil_group</b>	Wrapper – AXI4/AXIL	AXI4 burst slave-read + AXIL master-write	[monbus_axi4_axil_group.md](../monitor/monbus_axi4_axil_group.md)
<b>monbus_axi4_axi4_group</b>	Wrapper – AXI4/AXI4	Burst on both sides	[monbus_axi4_axi4_group.md](../monitor/monbus_axi4_axi4_group.md)
<b>monbus_halfbeat_packer</b>	Half-beat packer	Packs MonBus records into half-beats to push past the compressor's ~66.7% ceiling; bit-exact to Python golden	[monbus_halfbeat_packer.md](../monitor/monbus_halfbeat_packer.md)
<b>monbus_compressor</b>	Bulk-trace encoder	32-entry LRU CAM, 4-tag slot format, ~2.6× ratio, 2-stage pipeline, per-template `delta_ts`, optional half-beat packing, bit-exact to Python golden	[monbus_compressor.md](../monitor/monbus_compressor.md)
<b>monbus_cam_pipe</b>	Pipelined LRU CAM (production)	2-cycle compare + commit, depth-1 forwarding, sync `clear`, self-derived TOUCH/INSTALL – used in `monbus_compressor`	[monbus_cam_pipe.md](../monitor/monbus_cam_pipe.md)
<b>monbus_cam</b>	LRU CAM (reference)	Single-cycle reference design – same LRU semantics, superseded by `monbus_cam_pipe` in production	[monbus_cam.md](../monitor/monbus_cam.md)

The group family uses a small carry-save-compressor helper (`rtl/common/mod\_3\_compress.sv`) for whole-record  $X - (X \bmod 3)$  rounding. A single canonical filelist `rtl/amba/filelists/monbus\_group.f` enumerates the core's dependency tree so a new core dep lands in one place; every consumer `-f`-includes it.

### ### Standalone DMA Observability

The drop-in observer wraps any AXI4-master DMA's external pins from outside the DMA (non-intrusive), produces a monbus stream + IRQ FIFO + bulk-trace dump, and includes per-port valid/ready bucket counters. Companion piece to the per-DMA `axi\_monitor\_\*` family.

Module	Purpose	Key Features	Documentation
<b>**axi4_dma_observer**</b>	Standalone DMA observability harness	NUM_RD + NUM_WR tap pairs, monbus_arbiter + monbus_axil_axi4_group, axi_bus_meter + axi_perf_latency_hist per tap; reads via rid map, writes via AW→W awid tracker	[axi4_dma_observer.md]
<b>**axi4_dma_observer**</b>	(axi4_dma_observer.md)		
<b>**axi_bus_meter**</b>	Per-cycle valid/ready bucket counter	4 buckets (productive / backpressure / starvation / idle), aggregate + per-channel	[axi_bus_meter.md]
<b>**axis_bus_meter**</b>	AXIS per-cycle bucket counter + throughput	Same 4 buckets plus window-independent byte (tstrb) / beat / packet (tlast) counters, per-tid bins	[axis_bus_meter.md]
<b>**axis_bus_meter**</b>	(axis_bus_meter.md)		
<b>**axi_perf_latency_hist**</b>	Per-transaction latency histogram	16 log2 bins; read AR→first-R + AR→RLAST, write AW→B; per-channel timestamp FIFOs; indexed readout	[axi_perf_latency_hist.md]
<b>**axi_perf_latency_hist**</b>	(axi_perf_latency_hist.md)		

### ### Memory / BRAM Slave (sdpram\_slave family)

Simple dual-port BRAM slave with independent protocol choice on each side. Most commonly used as the memory-dump ring backend for `monbus\_axil\_axil\_group`. One common backend + four protocol-specific wrappers (since SystemVerilog can't conditionally include/exclude ports in a single module's port list, each combination gets its own wrapper module name).

Module	Purpose	Wr / Rd ports	Documentation
<b>**sdpram_core**</b>	Common backend (BRAM glue, clear FSM, protocol skids)	AXI4 superset (parameterized)	[sdpram_core.md]
<b>**sdpram_core**</b>	(sdpram_core.md)		
<b>**sdpram_slave_axi4_axi4**</b>	Wrapper – pure AXI4	`s_axi_*` / `s_axi_*`	[sdpram_slave_axi4_axi4.md]
<b>**sdpram_slave_axi4_axi4**</b>	(sdpram_slave_axi4_axi4.md)		
<b>**sdpram_slave_axi4_axil**</b>	Wrapper – AXI4 wr + AXIL rd	`s_axi_*` / `s_axil_*`	[sdpram_slave_axi4_axil.md]
<b>**sdpram_slave_axi4_axil**</b>	(sdpram_slave_axi4_axil.md)		
<b>**sdpram_slave_axil_axi4**</b>	Wrapper – AXIL wr + AXI4 rd	`s_axil_*` / `s_axi_*`	[sdpram_slave_axil_axi4.md]
<b>**sdpram_slave_axil_axi4**</b>	(sdpram_slave_axil_axi4.md)		

```
| **sdpram_slave_axil_axil** | Wrapper – pure AXIL (canonical SRAM-
ring backend) | `s_axil_*` / `s_axil_*` | [sdpram_slave_axil_axil.md]
(sdpram_slave_axil_axil.md) |
```

### ### Characterization Data-Plane (on-chip pattern gen / check)

Synthesizable, self-checking traffic generators and checkers used by the  
FPGA characterization harnesses (``stream_char_harness``,  
``rapids_char_harness``,  
``ddr2_char_harness``) as on-chip stimulus + verification – no host  
memory model  
needed. All share one LFSR (``0xDEADBEEF``, taps {32,22,2,1}) + CRC-32  
config so  
generated and checked data stay bit-consistent across every block.

Module	Purpose	Interface	Documentation
<b>**axi4_slave_rd_pattern_gen**</b>	Read-data source (LFSR)	AXI4 slave (R)	[axi4_slave_rd_pattern_gen.md](axi4_slave_rd_pattern_gen.md)
<b>**axi4_slave_wr_crc_check**</b>	Write-data verify (per-channel CRC)	AXI4 slave (W/B)	[axi4_slave_wr_crc_check.md](axi4_slave_wr_crc_check.md)
<b>**axi4_master_rd_crc_check**</b>	CSR-driven read master + CRC compare	AXI4 master (AR/R)	[axi4_master_rd_crc_check.md](axi4_master_rd_crc_check.md)
<b>**axi4_master_wr_pattern_gen**</b>	CSR-driven write master (LFSR/hash)	AXI4 master (AW/W/B)	[axi4_master_wr_pattern_gen.md](axi4_master_wr_pattern_gen.md)
<b>**axis4_master_pattern_gen**</b>	AXIS stream source	AXIS master	[axis4_master_pattern_gen.md](axis4_master_pattern_gen.md)
<b>**axis4_slave_pattern_check**</b>	AXIS stream verify	AXIS slave	[axis4_slave_pattern_check.md](axis4_slave_pattern_check.md)
<b>**axi4_dma_slaves**</b>	Source/sink bundle (rd gen + wr check)	AXI4 slaves	[axi4_dma_slaves.md](axi4_dma_slaves.md)

### ### Arbitration (4 modules)

Monitor bus arbiters for aggregating packets from multiple monitors:

Module	Purpose	Algorithm	Documentation
<b>**monbus_arbiter**</b>	Basic monitor bus arbiter	Round-robin with priority	[monbus_arbiter.md](../monitor/monbus_arbiter.md)
<b>**arbiter_monbus_common**</b>	Common arbiter infrastructure	Shared logic for all arbiters	[arbiter_monbus_common.md](../monitor/arbiter_monbus_common.md)

```
| **arbiter_rr_pwm_monbus** | Round-robin PWM arbiter | Equal priority
rotation |
[arbiter_rr_pwm_monbus.md](../monitor/arbiter_rr_pwm_monbus.md) |
| **arbiter_wrr_pwm_monbus** | Weighted round-robin arbiter |
Configurable priority weights |
[arbiter_wrr_pwm_monbus.md](../monitor/arbiter_wrr_pwm_monbus.md) |
```

### ### AXI Utilities (5 modules)

Utility modules for AXI protocol handling:

Module	Purpose	Use Case	Documentation
<b>**axi_gen_addr**</b>	Burst address generation	Calculate next address for INCR/WRAP bursts	[axi_gen_addr.md](axi_gen_addr.md)
<b>**axi_master_rd_splitter**</b>	Read channel splitter	Split AXI read across boundaries	[axi_master_rd_splitter.md](axi_master_rd_splitter.md)
<b>**axi_master_wr_splitter**</b>	Write channel splitter	Split AXI write with response consolidation	[axi_master_wr_splitter.md](axi_master_wr_splitter.md)
<b>**axi_split_combi**</b>	Combined splitter	Bidirectional split (read + write)	[axi_split_combi.md](axi_split_combi.md)

```
> **Clock domain crossing modules have moved.**
`cdc_2_phase_handshake`,
> `cdc_4_phase_handshake`, `cdc_open_loop` and `cdc_synchronizer` now
live in
> `rtl/amba/cdc/`, documented under [RTLamba/cdc](../cdc/README.md).
Start with the
> [CDC Primer](../cdc/cdc.md) for the decision guide.
```

---

## ## Monitor Infrastructure

### ### axi\_monitor\_base

**\*\*Purpose:\*\*** Core AXI/AXI-Lite transaction monitoring engine

**\*\*Key Features:\*\***

- Transaction table tracking (up to MAX\_TRANSACTIONS concurrent)
- Out-of-order completion support
- Timeout detection with configurable thresholds
- Multiple packet types: error, completion, timeout, performance, debug



- Integrated with monitor\_pkg for standardized events

```
Parameters:
```systemverilog
module axi_monitor_base #(
    parameter int UNIT_ID          = 9,      // 4-bit unit
    identifier
    parameter int AGENT_ID         = 99,     // 8-bit agent
    identifier
    parameter int MAX_TRANSACTIONS = 16,     // Transaction table
    size
    parameter int ADDR_WIDTH       = 32,
    parameter int ID_WIDTH         = 8,      // 0 for AXI-Lite
    parameter bit IS_READ          = 1,      // 1=read, 0=write
    parameter bit IS_AXI           = 1,      // 1=AXI, 0=AXI-Lite
    parameter bit ENABLE_PERF_PACKETS = 0,
    parameter bit ENABLE_DEBUG_MODULE = 0
)
)
```

Interfaces: - Command phase (AW/AR): addr, id, len, size, burst, valid, ready - Data channel (W/R): id, last, resp, valid, ready - Response channel (B): id, resp, valid, ready - Monitor bus output: 64-bit packets

Configuration: - Packet type enables: error, completion, timeout, threshold, performance, debug - Timeout counters: addr, data, resp (0-15 ticks) - Frequency selection for timer (0-15) - Threshold config: active transactions, latency

Generated Packets: - **ERROR:** Protocol violations, response errors - **COMPL:** Transaction completion notifications - **TIMEOUT:** Channel timeout events - **THRESH:** Threshold crossings (active transactions, latency) - **PERF:** Performance metrics - **DEBUG:** Detailed transaction traces

Usage: Used internally by all *_mon wrapper modules. Not typically instantiated directly.

10.6.1 axi_monitor_filtered

Purpose: Wrapper for axi_monitor_base with configurable packet filtering

Key Features: - 3-level filtering hierarchy: - Level 1: Packet type masking (drop entire types) - Level 2: Error routing (separate error handling) - Level 3: Event code masking (fine-grained filtering) - Optional pipeline stage for timing closure - Configuration validation with error detection

Parameters:

```
module axi_monitor_filtered #(
    // All axi_monitor_base parameters, plus:
    parameter bit ENABLE_FILTERING = 1,
```

```

    parameter bit ADD_PIPELINE_STAGE = 0
)

```

Filtering Configuration:

```

input logic [15:0] cfg_axi_pkt_mask,           // Drop packets by type
input logic [15:0] cfg_axi_error_mask,        // Mask specific error
events
input logic [15:0] cfg_axi_timeout_mask,      // Mask specific timeout
events
input logic [15:0] cfg_axi_compl_mask,        // Mask specific completion
events
input logic [15:0] cfg_axi_thresh_mask,       // Mask specific threshold
events
input logic [15:0] cfg_axi_perf_mask,         // Mask specific
performance events
input logic [15:0] cfg_axi_debug_mask        // Mask specific debug
events

```

Filtering Logic:

Unfiltered packet → Packet type check (pkt_mask)
 → Event code check (event_mask)
 → Optional pipeline stage
 → Filtered packet output

Usage: Used by all axi4*_mon and axil4*_mon modules. Provides fine-grained control over which events are reported.

10.6.2 axi_monitor_trans_mgr

Purpose: Transaction table management for monitor

Key Features: - Tracks up to MAX_TRANSACTIONS concurrent transactions - Maintains transaction state (idle, addr_received, data_in_progress, waiting_resp, completed) - Out-of-order completion support - Data-before-address handling - Timeout detection integration

Transaction Table Entry:

```

typedef struct packed {
    logic [ADDR_WIDTH-1:0] addr;
    logic [ID_WIDTH-1:0] id;
    logic [7:0] len;
    logic [2:0] size;
    logic [1:0] burst;
    logic [31:0] timestamp;
    bus_trans_state_t state;
}

```

```

    logic [7:0]          beat_count;
} bus_transaction_t;

```

Operations: - **Allocate:** Find free slot for new transaction - **Update:** Modify transaction state/beat count - **Complete:** Mark transaction done, free slot - **Lookup:** Find transaction by ID - **Timeout Check:** Detect stalled transactions

Usage: Instantiated within axi_monitor_base. Handles all transaction lifecycle management.

10.6.3 axi_monitor_reporter

Purpose: Generate standardized 128-bit monitor_packet_t records (paired with a 64-bit side-band timestamp)

Key Features: - Consolidates events from trans_mgr and timeout modules - Formats packets according to monitor_common_pkg::monitor_packet_t - Handles packet priorities and arbitration - Integrates error flags, completion notifications, performance metrics

Packet Format (128 bits):

```

[127:124] Packet Type (4 bits)
[123:109] Reserved (15 bits, forward-compat slack)
[108:105] Protocol (4 bits)
[104:97]  Event Code (8 bits)
[96:88]   Channel ID (9 bits)
[87:72]   Agent ID (16 bits)
[71:64]   Unit ID (8 bits)
[63:0]    Event Data (64 bits)

```

Event Data Content (depends on packet type): - **ERROR:** Address[34:0] or error code - **COMPL:** Transaction ID + beat count - **TIMEOUT:** Channel identifier + timeout count - **THRESH:** Threshold value + current value - **PERF:** Latency or throughput metric - **DEBUG:** State machine info + transaction data

Usage: Instantiated within axi_monitor_base. Converts internal events to standardized packets.

10.6.4 axi_monitor_timeout

Purpose: Detect timeout conditions on AXI channels

Key Features: - Separate counters for AR/AW, R/W, B channels - Configurable timeout thresholds (0-15 ticks) - Integration with timer tick for time base - Per-transaction timeout tracking

Configuration:

```

input logic [3:0] cfg_addr_cnt, // Address channel timeout (ticks)
input logic [3:0] cfg_data_cnt, // Data channel timeout (ticks)
input logic [3:0] cfg_resp_cnt // Response channel timeout (ticks)

```

Timeout Detection:

Timer tick → Increment active channel counters

- Compare with threshold
- Generate timeout event if exceeded
- Reset counter on channel activity

Usage: Instantiated within axi_monitor_base. Generates timeout packets for reporter.

10.6.5 axi_monitor_timer

Purpose: Frequency-invariant timer for monitor infrastructure

Key Features: - Configurable frequency divider (0-15 selection) - Generates periodic tick for timeout counters - Independent of system clock frequency - Low overhead implementation

Configuration:

```

input logic [3:0] cfg_freq_sel // Frequency selection (0-15)

```

Frequency Table:

```

cfg_freq_sel = 0 → Tick every 1 cycle
cfg_freq_sel = 1 → Tick every 2 cycles
cfg_freq_sel = 2 → Tick every 4 cycles
...
cfg_freq_sel = 15 → Tick every 32768 cycles

```

Usage: Instantiated within axi_monitor_base. Provides time base for timeout detection.

10.6.6 amba_clock_gate_ctrl

Purpose: Dynamic clock gating controller for AMBA modules

Key Features: - Idle detection with configurable threshold - Automatic gating after idle period - Fast ungating on activity: 1 register stage, first usable gated-clock edge 2 cycles after activity (2 stages / 3 cycles on APB, APB5, AXI5-Stream) - Safe clock gating (glitch-free) - Clock cycle counting for power analysis

Parameters:

```

module amba_clock_gate_ctrl #(
    parameter int CG_IDLE_COUNT_WIDTH = 4 // Idle counter width
)

```

Configuration:

```

input logic                cfg_cg_enable,        // Enable gating
input logic [CG_IDLE_COUNT_WIDTH-1:0] cfg_cg_idle_count // Idle
cycles before gate

```

Status:

```

output logic cg_gating,        // Currently gated
output logic [31:0] cg_clk_count // Cumulative gated cycles

```

Gating Logic:

Activity detection → Reset idle counter
 No activity → Increment idle counter
 Idle >= threshold && enable → Gate clock
 Activity detected → Ungate without waiting for the idle counter
 (1 register stage; first usable gated edge 2
 clocks later,
 2 stages / 3 clocks on APB, APB5, AXI5-Stream)

Usage: Used by all *_cg clock-gated variants (axi4, axil4, axis, apb monitors).

10.7 Arbitration

10.7.1 monbus_arbiter

Purpose: Round-robin arbiter for monitor bus aggregation

Key Features: - N-way arbitration (up to 16 agents) - Fair round-robin scheduling - Zero bubble overhead (back-to-back grants) - Configurable arbiter width

Parameters:

```

module monbus_arbiter #(
    parameter int N = 4,        // Number of requesters
    parameter int DATA_WIDTH = 64 // Monitor bus width
)

```

Interfaces:

```

// Per-agent inputs
input logic [N-1:0] i_request,
input logic [N-1:0] [DATA_WIDTH-1:0] i_data,
output logic [N-1:0] o_grant,

```

```
// Aggregated output
output logic          o_valid,
output logic [DATA_WIDTH-1:0] o_data
```

Arbitration:

Request[0..N-1] → Round-robin priority rotation
 → Grant highest priority requester
 → Output granted packet
 → Rotate priority for fairness

Usage: Aggregate monitor packets from multiple AXI/APB monitors into single stream.

Example:

```
// 4 AXI monitors → 1 monitor bus
monbus_arbiter #(.N(4), .DATA_WIDTH(64)) u_arb (
    .i_request({mon3_valid, mon2_valid, mon1_valid, mon0_valid}),
    .i_data({mon3_pkt, mon2_pkt, mon1_pkt, mon0_pkt}),
    .o_grant({mon3_ready, mon2_ready, mon1_ready, mon0_ready}),
    .o_valid(agg_valid),
    .o_data(agg_pkt)
);
```

10.7.2 arbiter_monbus_common

Purpose: Shared arbitration logic for all monitor bus arbiters

Key Features: - Reusable priority encoding - Mask-based round-robin - Optimized for monitor bus (64-bit packets)

Usage: Base module for arbiter_rr_pwm_monbus and arbiter_wrr_pwm_monbus.

10.7.3 arbiter_rr_pwm_monbus

Purpose: Round-robin arbiter with PWM output for monitor bus

Key Features: - Equal priority round-robin - PWM output mode for power management - Optimized for packet streams

Usage: Alternative to monbus_arbiter with PWM support.

10.7.4 arbiter_wrr_pwm_monbus

Purpose: Weighted round-robin arbiter for monitor bus

Key Features: - Configurable per-agent weights - Weighted priority allocation - Fair service guarantees

Configuration:

```
input logic [N-1:0][7:0] cfg_weights // Per-agent weight (0-255)
```

Scheduling:

High-weight agents get more grants
Low-weight agents get fewer grants
Maintains fairness across epochs

Usage: Use when different monitors have different importance levels.

10.8 AXI Utilities

10.8.1 axi_gen_addr

Purpose: Generate next address for AXI burst transactions

Key Features: - Supports INCR, WRAP, FIXED burst types - Handles all AXI data widths (32-512 bits) - Proper address alignment - Boundary wrapping for WRAP bursts

Parameters:

```
module axi_gen_addr #(
    parameter int AW = 32, // Address width
    parameter int DW = 32, // Data width
    parameter int ODW = 32, // Output data width (for width
conversion)
    parameter int LEN = 8 // Length field width
)
```

Inputs:

```
input logic [AW-1:0] curr_addr, // Current address
input logic [2:0] size, // axsize (0-7)
input logic [1:0] burst, // axburst (FIXED/INCR/WRAP)
input logic [LEN-1:0] len // axlen (burst length)
```

Outputs:

```
output logic [AW-1:0] next_addr, // Next address in burst
output logic [AW-1:0] next_addr_align // Aligned next address
```

Address Calculation:

INCR (0x01):

```
next_addr = curr_addr + (1 << size)
```

WRAP (0x10):

```
// Wrap at boundary = (len + 1) * (1 << size)
boundary = (len + 1) << size;
next_addr = (curr_addr + (1 << size)) & ~(boundary - 1);
```

FIXED (0x00):

```
next_addr = curr_addr // Same address
```

Usage: Used by axi4_to_apb_convert for burst decomposition. Also useful for any AXI address calculation.

Example:

```
axi_gen_addr #(.AW(32), .DW(64), .ODW(32)) u_addr_gen (
    .curr_addr(32'h1000),
    .size(3'b010),           // 4 bytes
    .burst(2'b01),           // INCR
    .len(8'd3),               // 4 beats
    .next_addr(next_addr) // → 32'h1004, 32'h1008, 32'h100C
);
```

10.8.2 axi_master_rd_splitter

Purpose: Split AXI read into separate AR and R channels

Key Features: - Independent AR and R skid buffers - Configurable buffer depths - Zero-latency path option

Usage: Decouple AR and R channels for timing closure or protocol adaptation.

10.8.3 axi_master_wr_splitter

Purpose: Split AXI write into separate AW, W, and B channels

Key Features: - Independent channel buffering - AW-W synchronization - B channel response routing

Usage: Decouple write channels for complex write paths.

10.8.4 axi_split_combi

Purpose: Combined read + write splitter

Key Features: - Bidirectional split (all 5 AXI channels) - Shared configuration - Compact implementation

Usage: Full AXI split for complex bridges or adapters.

10.8.5 cdc_4_phase_handshake

Purpose: Clock domain crossing with valid/ready handshake

Key Features: - Dual-clock asynchronous CDC - Data + control crossing - Gray-code free pointer-based - Safe for arbitrary clock ratios - Zero data loss

Parameters:

```
module cdc_4_phase_handshake #(
    parameter int DATA_WIDTH = 8
)
```

Interfaces:

```
// Source clock domain
input logic          clk_src,
input logic          rst_src_n,
input logic          src_valid,
output logic         src_ready,
input logic [DATA_WIDTH-1:0] src_data,

// Destination clock domain
input logic          clk_dst,
input logic          rst_dst_n,
output logic         dst_valid,
input logic          dst_ready,
output logic [DATA_WIDTH-1:0] dst_data
```

FSM States:

Source Domain:

S_IDLE → Ready for new data
S_WAIT_ACK → Waiting for destination ack
S_WAIT_ACK_CLR → Waiting for ack to clear

Destination Domain:

D_IDLE → Waiting for request
D_WAIT_READY → Request received, waiting for ready
D_WAIT_REQ_CLR → Ack sent, waiting for request clear

Timing: - Latency: 3-5 cycles (depends on clock ratio) - Synchronization: 3-stage shift register -
Handshake: 4-phase (req → ack → req_clr → ack_clr)

Usage: Used extensively in axi4_to_apb_shim for cmd/rsp CDC. General-purpose CDC utility.

Example:

```
cdc_4_phase_handshake #(.DATA_WIDTH(32)) u_cdc (  
    .clk_src(clk_100mhz),  
    .rst_src_n(rst_n),  
    .src_valid(cmd_valid),  
    .src_ready(cmd_ready),  
    .src_data(cmd_data),  
  
    .clk_dst(clk_200mhz),  
    .rst_dst_n(rst_n),  
    .dst_valid(req_valid),  
    .dst_ready(req_ready),  
    .dst_data(req_data)  
);
```

10.9 Testing

Shared modules are tested indirectly through higher-level protocol modules:

```
# Monitor infrastructure tested via monitor tests  
pytest val/amba/test_axi4*_mon.py -v  
pytest val/amba/test_axil4*_mon.py -v  
pytest val/amba/test_apb_monitor.py -v
```

```
# CDC tested via shims  
pytest val/integ_amba/test_axi2apb_shim.py -v
```

```
# Address generation tested via conversion  
pytest val/integ_amba/test_axi2apb_shim.py -v
```

10.10 Design Notes

10.10.1 Monitor Infrastructure

Architecture:

Transaction → axi_monitor_trans_mgr (track state)
→ axi_monitor_timeout (detect stalls)
→ axi_monitor_reporter (generate packets)
→ axi_monitor_filtered (apply filters)
→ Monitor bus output

Packet Flow:

axi_monitor_base → Unfiltered packets
→ axi_monitor_filtered (if enabled)
→ Filtered packets
→ FIFO/arbiter
→ System monitor bus

10.10.2 When to Use Each Module

✓ **Use axi_monitor_base When:** - Creating custom monitor variants - Need full control over filtering - Integrating with custom infrastructure

✓ **Use axi_monitor_filtered When:** - Standard monitor usage - Need packet filtering - Using with monitor bus aggregation

✓ **Use cdc_4_phase_handshake When:** - Clock domain crossing required - Valid/ready protocol - Arbitrary clock ratios - Safe data transfer needed

✓ **Use axi_gen_addr When:** - Implementing AXI bridges - Burst address calculation - Protocol conversion

✓ **Use monbus_arbiter When:** - Multiple monitors on same bus - Need fair arbitration - Aggregating monitor streams

10.11 Configuration Best Practices

10.11.1 Monitor Configuration

Functional Debug Mode (Recommended Default):

```
.cfg_error_enable(1'b1),           // Catch errors
.cfg_compl_enable(1'b1),           // Track completions
.cfg_timeout_enable(1'b1),         // Detect hangs
```

```
.cfg_perf_enable(1'b0),      // ⚠️ DISABLE (high traffic)
.cfg_debug_enable(1'b0)     // Only if needed
```

Performance Analysis Mode:

```
.cfg_error_enable(1'b1),    // Still catch errors
.cfg_compl_enable(1'b0),    // ⚠️ DISABLE
.cfg_timeout_enable(1'b0),
.cfg_perf_enable(1'b1),     // Enable metrics
.cfg_debug_enable(1'b0)
```

⚠️ NEVER enable `cfg_compl_enable` and `cfg_perf_enable` together! This causes packet congestion.

10.11.2 Timeout Configuration

Conservative (Slow Peripherals):

```
.cfg_freq_sel(4'd8),        // Slower tick rate
.cfg_addr_cnt(4'd15),       // Long timeout
.cfg_data_cnt(4'd15),
.cfg_resp_cnt(4'd15)
```

Aggressive (Fast Detection):

```
.cfg_freq_sel(4'd0),        // Fast tick rate
.cfg_addr_cnt(4'd3),        // Short timeout
.cfg_data_cnt(4'd3),
.cfg_resp_cnt(4'd3)
```

10.11.3 Clock Gating Configuration

Aggressive Gating:

```
.cfg_cg_enable(1'b1),
.cfg_cg_idle_count(4'd1) // Gate after 1 idle cycle
```

Conservative Gating:

```
.cfg_cg_enable(1'b1),
.cfg_cg_idle_count(4'd10) // Gate after 10 idle cycles
```

10.12 Performance Characteristics

10.12.1 Monitor Overhead

Resource Usage (per monitor): - axi_monitor_base: ~500 LUTs, ~400 FFs - axi_monitor_filtered: +50 LUTs, +30 FFs - Total: ~550 LUTs, ~430 FFs per monitor

Packet Generation Rate: - Completion packets: 1 per transaction - Error packets: As needed (protocol violations) - Timeout packets: Low frequency (cfg_freq_sel dependent) - Performance packets: 1 per N transactions (configurable)

10.12.2 CDC Latency

cdc_4_phase_handshake: - Minimum: 6 cycles (src → dst → src) - Typical: 8-12 cycles - Maximum: Depends on clock ratio

10.12.3 Arbitration

monbus_arbiter: - Latency: 1 cycle - Throughput: 1 packet/cycle (no bubbles) - Fairness: Perfect round-robin

10.13 Synthesis Notes

10.13.1 Clock Gating

amba_clock_gate_ctrl: - Implements glitch-free clock gating - Use vendor clock gating cells for best results - Requires proper clock gating constraints

Constraints (example):

```
set_clock_gating_check -setup 0.1 [get_clocks aclk]
set_dont_touch [get_cells */u_clock_gate_ctrl/*]
```

10.13.2 CDC Constraints

cdc_4_phase_handshake:

```
set_false_path -from [get_clocks clk_src] -to [get_clocks clk_dst]
set_false_path -from [get_clocks clk_dst] -to [get_clocks clk_src]
```

Constrain synchronizer paths

```
set_max_delay -from */r_req_src -to */r_req_sync[0] 10.0
set_max_delay -from */r_ack_dst -to */r_ack_sync[0] 10.0
```

10.14 Related Documentation

10.14.1 RTL Design Sherpa Documentation

- [RTLAmba Overview](#) - Complete AMBA architecture
- [AXI4 Monitors](#) - Uses monitor infrastructure

- [APB Monitor](#) - Uses monitor infrastructure
- [Shims](#) - Uses CDC and address generation

10.14.2 Monitor Package

- [Monitor Package Spec](#) - Event packet format

10.14.3 Source Code

- RTL: rtl/amba/shared/
 - Monitor package: rtl/amba/monitor_pkg.sv
 - Tests: Indirect via val/amba/test_*_mon.py
-

10.15 Summary

The shared subsystem provides essential infrastructure:

Monitor Infrastructure: - Complete transaction tracking and event generation - Configurable filtering and timeout detection - Standardized 64-bit event packets - Multi-protocol support (AXI, APB, AXIS)

Arbitration: - Fair round-robin scheduling - Weighted priority support - Zero-overhead aggregation

Utilities: - Clock domain crossing (cdc_4_phase_handshake) - AXI address generation (axi_gen_addr) - Channel splitting (axi*_splitter) - Clock gating (amba_clock_gate_ctrl)

Usage Pattern: Most users interact with higher-level protocol modules (axi4_master_rd_mon, etc.) which internally use these shared components. Direct instantiation is rare except for custom implementations.

Last Updated: 2025-10-24

10.16 Navigation

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

11 AMBA Clock Gate Controller

Module: amba_clock_gate_ctrl.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

11.1 Overview

The AMBA Clock Gate Controller provides dynamic clock gating capability tailored for AMBA protocol interfaces. It monitors transaction activity on both user-side and AXI-side interfaces to automatically gate the clock during idle periods, reducing dynamic power consumption while maintaining protocol correctness.

11.1.1 Key Features

- Automatic activity detection from user and AXI valid signals
 - Configurable idle threshold before gating activation
 - Integration with standard ICG (Integrated Clock Gate) cells
 - Registered wakeup signal for metastability protection
 - Global enable/disable control
 - Idle status monitoring
 - Zero added latency while the clock is already ungated (wake-up from gated costs 1 register stage here; see Performance Considerations)
-

11.2 Module Purpose

AMBA protocol interfaces often experience periods of inactivity during system operation. Traditional always-on clocking wastes power during these idle periods. The AMBA Clock Gate Controller solves this by:

1. Monitoring transaction activity on both interfaces
2. Detecting extended idle periods
3. Automatically gating the clock to save power
4. Ungating on new activity without waiting for the idle counter

Use Cases: - Power-critical AMBA interface implementations - Battery-powered systems with intermittent bus activity - Multi-master systems where individual masters idle frequently - ASIC designs requiring aggressive dynamic power reduction

Key Benefit: Transparent power saving with no protocol impact (activity detection ensures clock availability when needed).

11.3 Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (determines maximum idle threshold)
ICW	int	CG_IDLE_COUNT_WIDTH	Shorthand alias for idle count width

Idle Count Range: - Minimum: 0 (immediate gating when idle) - Maximum: $2^{\text{CG_IDLE_COUNT_WIDTH}} - 1$ - Default (4-bit): 0 to 15 cycles

11.4 Port Groups

11.4.1 Clock and Reset

Port	Direction	Width	Description
clk_in	input	1	Input clock (always running, feeds ICG cell)
aresetn	input	1	Active-low asynchronous reset

11.4.2 Configuration Interface

Port	Direction	Width	Description
cfg_cg_enable	input	1	Global clock gating enable (1=allow gating, 0=always ungated)
cfg_cg_idle_count	input	ICW	Idle cycle threshold before gating activates

11.4.3 Activity Monitoring

Port	Direction	Width	Description
user_valid	input	1	User-side activity indicator (any valid signal)

Port	Direction	Width	Description
axi_valid	input	1	from user interface) AXI-side activity indicator (any valid signal from AXI interface)

11.4.4 Clock Gating Outputs

Port	Direction	Width	Description
clk_out	output	1	Gated output clock (drives clocked logic)
gating	output	1	Gating active indicator (1=clock currently gated)
idle	output	1	Idle status (1=no activity detected, 0=activity present)

11.5 Functional Description

11.5.1 Activity Detection Logic

The module combines activity signals from both user and AXI interfaces to form a unified wakeup signal:

wakeup = user_valid OR axi_valid

Why Both Interfaces? - User interface: Detects upstream master activity - AXI interface: Detects downstream slave activity - Combined: Ensures clock active for bidirectional traffic

Registered Wakeup:

```
always_ff @(posedge clk_in or negedge aresetn) begin
    if (!aresetn)
        r_wakeup <= 1'b1; // Default: Active (clock ungated at reset)
    else
        r_wakeup <= user_valid || axi_valid;
end
```

Key Property: Wakeup signal is registered to: 1. Prevent metastability (if activity signals cross clock domains) 2. Provide stable input to downstream clock gate control 3. Default to active state during reset (ensures clock available)

11.5.2 Idle Signal Generation

Idle status is the inverse of wakeup:

`idle = ~r_wakeup`

Interpretation: - idle = 1: No activity for at least 1 cycle (eligible for gating) - idle = 0: Recent activity detected (clock must remain ungated)

Usage: Exported for monitoring/debug (system can observe interface idle status).

11.5.3 Clock Gating Control Integration

The module instantiates a base clock gate controller:

```
clock_gate_ctrl #(
    .IDLE_CNTR_WIDTH    (ICW)
) u_clock_gate_ctrl (
    .clk_in              (clk_in),
    .aresetn             (aresetn),
    .cfg_cg_enable       (cfg_cg_enable),
    .cfg_cg_idle_count   (cfg_cg_idle_count),
    .wakeup              (r_wakeup),
    .clk_out             (clk_out),
    .gating              (gating)
);
```

Base Controller Operation: 1. Monitors r_wakeup signal 2. Increments idle counter when wakeup=0 (idle) 3. Gates clock when counter >= cfg_cg_idle_count 4. Ungates when wakeup=1 (activity), without waiting for the idle counter

11.5.4 Gating Threshold Behavior

cfg_cg_idle_count Examples (4-bit counter):

Value	Behavior
0	Aggressive: gate 1 clock after the internal wakeup deasserts (2 clocks after the last bus activity on single-stage families, 3 on two-stage families)
1	Gate after 1 idle cycle
4	Conservative: Gate after 4 consecutive idle cycles
15	Maximum delay: Gate after 15 consecutive idle cycles

Trade-off: - Low threshold: Maximum power saving, higher gating/ungating frequency - High threshold: Reduced gating overhead, less power saving

Recommendation: Start with moderate value (4-8) and tune based on measured activity patterns.

11.5.5 Gating/Ungating Timing

Gating Sequence (Activity → Idle): 1. Cycle N: user_valid=1, axi_valid=0 → r_wakeup=1 (active) 2. Cycle N+1: user_valid=0, axi_valid=0 → r_wakeup=0 (idle) 3. Idle counter starts incrementing 4. Cycle N+1+cfg_cg_idle_count: Counter threshold reached 5. Clock gates on next cycle edge (gating=1)

Ungating Sequence (Idle → Activity): 1. Clock currently gated (gating=1) 2. Cycle M: user_valid=1 (new activity arrives) 3. Cycle M+1: r_wakeup=1 (registered); gating=0 combinationally 4. Cycle M+2: first usable gated-clock rising edge 5. Normal operation resumes with no lost cycles

Critical Property: Ungating does not wait on the idle counter. It still costs 1 register stage here, so the first usable gated-clock edge arrives 2 clocks after activity asserts (3 on the two-stage APB, APB5, and AXI5-Stream wrappers).

11.6 Usage Example

11.6.1 Basic Integration with AXI Interface

```
// Instantiate AMBA clock gate controller
amba_clock_gate_ctrl #(
    .CG_IDLE_COUNT_WIDTH      (4) // 0-15 cycle threshold
) u_clock_gate (
    // Input clock (always running)
    .clk_in                    (axi_aclk_raw),
    .aresetn                   (axi_aresetn),

    // Configuration
    .cfg_cg_enable             (power_save_enable), // From system
    .cfg_cg_idle_count         (4'd4),              // Gate after 4
    idle cycles

    // Activity monitoring (OR together all valid signals)
    .user_valid                (s_axi_awvalid | s_axi_wvalid |
s_axi_arvalid),
    .axi_valid                  (m_axi_awvalid | m_axi_wvalid |
m_axi_arvalid |
                                m_axi_rvalid | m_axi_bvalid),

    // Gated clock output
```

```

        .clk_out                (axi_aclk_gated),
        .gating                 (clock_gated_status),
        .idle                   (interface_idle)
    );

    // Use gated clock for all clocked logic
    always_ff @(posedge axi_aclk_gated or negedge axi_aresetn) begin
        if (!axi_aresetn) begin
            // Reset logic
        end else begin
            // Normal operation with gated clock
        end
    end
end

```

```

// Monitor gating status for debug/power measurement
assign power_gating_active = clock_gated_status;
assign bus_idle_indicator = interface_idle;

```

11.6.2 Multi-Master Arbitration with Clock Gating

```

// Individual clock gating per master interface
genvar i;
generate
    for (i = 0; i < NUM_MASTERS; i++) begin : gen_master_cg
        amba_clock_gate_ctrl #(
            .CG_IDLE_COUNT_WIDTH    (4)
        ) u_master_cg (
            .clk_in                  (system_clk),
            .aresetn                 (aresetn),
            .cfg_cg_enable           (cg_enable[i]),
            .cfg_cg_idle_count       (cg_threshold[i]),

            // Monitor master i activity
            .user_valid              (master_valid[i]),
            .axi_valid               (1'b0), // No AXI side for master

            .clk_out                 (master_clk_gated[i]),
            .gating                  (master_gating[i]),
            .idle                    (master_idle[i])
        );
    end
endgenerate

// Power monitoring: Count active masters
assign num_masters_active = NUM_MASTERS - $countones(master_idle);

```

11.6.3 Dynamic Threshold Adjustment

```
// Adjust gating threshold based on system load
logic [3:0] dynamic_threshold;

always_comb begin
    case (system_load_level)
        2'b00: dynamic_threshold = 4'd2;    // Light load: Aggressive
gating
        2'b01: dynamic_threshold = 4'd4;    // Medium load: Moderate
gating
        2'b10: dynamic_threshold = 4'd8;    // High load: Conservative
gating
        2'b11: dynamic_threshold = 4'd15;   // Critical load: Minimal
gating
    endcase
end

amba_clock_gate_ctrl #(
    .CG_IDLE_COUNT_WIDTH    (4)
) u_adaptive_cg (
    .clk_in                  (clk),
    .aresetn                 (rst_n),
    .cfg_cg_enable           (1'b1),
    .cfg_cg_idle_count       (dynamic_threshold), // Runtime
adjustable
    .user_valid              (activity),
    .axi_valid               (1'b0),
    .clk_out                 (clk_gated),
    .gating                  (gating),
    .idle                    (idle)
);
```

11.7 Design Notes

11.7.1 Wakeup Signal Registration

Design Decision: The wakeup signal is registered before passing to clock gate controller.

Rationale: 1. **Metastability Protection:** Activity signals may originate from different clock domains 2. **Stable Input:** Clock gate controller receives glitch-free wakeup signal 3. **Reset Safety:** Defaults to active (1'b1) ensuring clock availability during reset

Trade-off: Adds 1 register stage to activity detection, so the first usable gated-clock edge arrives 2 clocks after activity asserts (3 on the two-stage APB, APB5, and AXI5-Stream wrappers, which register activity again before this module). Robust operation is worth the cost.

11.7.2 Activity Signal Selection

Best Practices:

For User Interface (Upstream):

```
// AXI Master Interface
user_valid = awvalid | wvalid | arvalid

// AXI Slave Interface
user_valid = awready | wready | arready | rvalid | bvalid
```

For AXI Interface (Downstream):

```
// AXI Master Interface
axi_valid = awready | wready | arready | rvalid | bvalid

// AXI Slave Interface
axi_valid = awvalid | wvalid | arvalid
```

Why Not Ready Signals? - Valid signals indicate actual transaction progress - Ready signals may assert speculatively - Valid-based detection prevents premature gating

11.7.3 Integration with ICG Cells

The base clock_gate_ctrl module expects integration with standard ICG cell:

Typical ICG Cell Interface:

```
// Standard ICG cell instantiation (inside clock_gate_ctrl)
ICG u_icg (
    .CLK      (clk_in),
    .EN       (gate_enable), // From controller FSM
    .CLK_OUT  (clk_out)
);
```

ASIC Integration: - Use library-specific ICG cell (vendor-provided) - Ensure glitch-free enable signal (provided by controller) - Consider test mode bypass for scan insertion

FPGA Integration: - Use global clock mux primitives (BUFGMUX, BUFGCE) - Some FPGAs have dedicated clock gating resources - Alternative: Fine-grained clock enable on flip-flops

11.7.4 Power Measurement Methodology

Estimating Power Savings:

1. Measure Idle Time:

```
logic [31:0] idle_cycle_count;
always_ff @(posedge clk_in) begin
    if (idle)
```

```
        idle_cycle_count <= idle_cycle_count + 1;  
end
```

2. Calculate Gating Efficiency:

Gating Efficiency = (gating_active_cycles / total_cycles) × 100%

3. Estimate Power Reduction:

Dynamic Power Saving ≈ Gating Efficiency × Clock Tree Power

Typical Results: - Low-activity interfaces: 60-80% gating efficiency - Burst-heavy traffic: 20-40% gating efficiency - Clock tree power: 20-30% of total dynamic power - Overall savings: 5-20% total chip dynamic power

11.7.5 Performance Considerations

Latency Impact: - No added latency when the clock is already ungated (activity present) - Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. This module contributes the single r_wakeup stage; two-stage families add one more in the wrapper. - First usable gated-clock rising edge: 2 clocks (single-stage families) or 3 clocks (two-stage families) after activity asserts - Ungating does not wait on the idle counter

Throughput Impact: - Zero impact: Clock always available for valid transactions - Activity detection ensures ungating before transaction arrival

Area Overhead: - Minimal: Single register + base controller - Base controller: Small FSM + counter - Typical: <100 gates total

11.8 Related Modules

11.8.1 Used By

- AXI interface wrappers requiring power optimization
- AMBA protocol bridges with intermittent activity
- Multi-master interconnect fabrics
- Power-critical peripheral interfaces

11.8.2 Uses

- **clock_gate_ctrl.sv** - Base clock gating controller (idle counter, FSM)
- **reset_defs.svh** - Standard reset macro definitions

11.8.3 See Also

- **clock_gate_ctrl.sv** - Generic clock gating controller (rtl/common/)

- **icg.sv** - Integrated clock gate cell wrapper
-

11.9 References

11.9.1 Source Code

- RTL: rtl/amba/shared/amba_clock_gate_ctrl.sv
- Base Controller: rtl/common/clock_gate_ctrl.sv
- Tests: val/common/test_clock_gate_ctrl.py

11.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
- Common Module: docs/markdown/RTLCommon/clock_gate_ctrl.md
- Design Guide: rtl/amba/PRD.md

11.9.3 Industry Standards

- IEEE 1801 (UPF) - Unified Power Format for power-aware design
 - ARM AMBA specifications - Clock gating recommendations
-

Last Updated: 2025-10-24

11.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

12 AXI4 DMA Observer

Modules: - axi4_dma_observer.sv — standalone, DMA-agnostic observability wrapper - axi_bus_meter.sv — per-cycle valid/ready bucket counter (used inside the observer; also instantiable standalone)

Location: rtl/amba/shared/ **Category:** Observability / Characterization **Status:** Production Ready

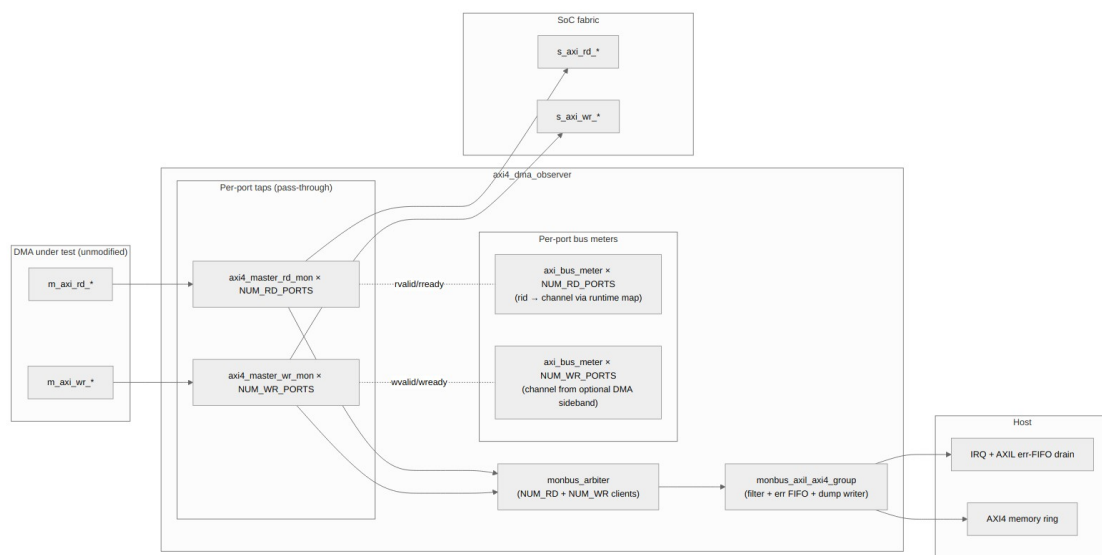
12.1 Overview

axi4_dma_observer is a **drop-in, DMA-agnostic** observability harness that snaps onto any AXI4-master DMA's external pins and produces:

- an **error / interrupt FIFO** drained over an AXI-Lite slave-read port
- a **bulk-trace memory dump** over an AXI4-burst master-write port (raw or compressed via monbus_compressor)
- per-port **valid/ready bucket counters** (productive / backpressure / starvation / idle) via axi_bus_meter, with optional per-channel attribution

The DMA under test is **not modified**. The observer sits inline between the DMA's AXI master pins and the fabric; from the DMA's perspective it's a transparent skid stage. From the fabric's perspective it's the DMA's master port, one register stage later.

This is the **external / non-intrusive** counterpart to the axi_monitor_* family — those wrap a specific DMA's interfaces from inside the DMA. The observer wraps from outside, so the same shape works for STREAM, RAPIDS, third-party DMAs, or anything else with an AXI4 master port.



Architecture

Pass-through latency = the existing `_mon` wrapper's skid stage. The observer adds **no new combinational depth** in the AXI path.

12.2 What's inside

Sub-module	Role
<code>axi4_master_rd_mon × NUM_RD_PORTS</code>	Read-side tap: pass-through skid + transaction monitor that emits a monbus packet on each event
<code>axi4_master_wr_mon × NUM_WR_PORTS</code>	Write-side tap (same pattern)
<code>monbus_arbiter (CLIENTS = NUM_RD + NUM_WR)</code>	Merges the per-port monbus streams into one
<code>monbus_axil_axi4_group</code>	Central filter + err FIFO (AXIL slave-read) + bulk-trace dump (AXI4 burst master-write) — see <code>monbus_group.md</code>
<code>axi_bus_meter × NUM_RD_PORTS + NUM_WR_PORTS</code>	Per-cycle valid/ready bucket counters (productive / backpressure / starvation / idle) — gated by <code>ENABLE_BUS_METER</code>
<code>axi_perf_latency_hist × NUM_RD_PORTS + NUM_WR_PORTS</code>	Per-transaction latency histograms (read: AR→first-R + AR→RLAST; write: AW→B) into <code>HIST_NUM_BINS</code> log2 bins — gated by <code>ENABLE_LATENCY_HIST</code> (RFC Stage E.3)

12.3 Parameters

Parameter	Default	Notes
<code>NUM_RD_PORTS</code>	1	Number of AXI4 read master ports to tap
<code>NUM_WR_PORTS</code>	1	Number of AXI4 write master ports to tap
<code>ADDR_WIDTH</code>	32	Address width on all tap ports + the observer's own dump port
<code>DATA_WIDTH</code>	128	DMA data-bus width
<code>AXI_ID_WIDTH</code>	8	DMA AXI ID width
<code>AXI_USER_WIDTH</code>	1	DMA AXI USER width
<code>OBS_AXI_ID_WIDTH</code>	4	Observer's own dump-

Parameter	Default	Notes
		port (m_axi_*) AXI ID width
MAX_BURST_BEATS	64	Maximum beats per dump-port AW (AXI4 max is 256)
FIFO_DEPTH_ERR	64	Err FIFO depth (records)
FIFO_DEPTH_WRITE	96	Write FIFO depth (beats)
FLUSH_TIMEOUT_CYCLES	1024	Cycles before timeout-driven flush
USE_COMPRESSION	0	0 omits the compressor; 1 elaborates it for runtime selection via <code>cfg_compress_en</code>
MAX_TRANSACTIONS	16	Per-tap monitor transaction-table depth
UNIT_ID	8'h10	UNIT_ID stamped into emitted monbus packets
ENABLE_BUS_METER	1	0 ties all bus-meter outputs to 0
WR_CH_FROM_AWID	0	1 = derive write per-channel attribution from awid via an internal AW→W order tracker (no <code>dma_wr_active_ch_*</code> sideband needed); 0 = use the sideband. Reads always attribute via <code>rid</code>
NUM_CHANNELS	1	1 = aggregate-only buckets; > 1 = per-channel attribution
ENABLE_LATENCY_HIST	1	0 ties all latency-histogram outputs to 0 (RFC Stage E.3)
HIST_NUM_BINS	16	log2 latency bins: bin b

Parameter	Default	Notes
HIST_MAX_OUTSTANDING	8	counts $[2^b, 2^{(b+1)})$ cycles Per-channel timestamp-FIFO depth in each histogram

Latency-histogram readout (indexed, shared across ports): drive `i_hist_metric` (0 = AR→first-R, 1 = AR→RLAST for reads; ignored for writes) and `i_hist_bin`, then read each port's `{rd,wr}_hist_count` (selected bin) and `{rd,wr}_hist_total` (per-metric transaction count = burst count). Frozen and cleared in lockstep with the meters via `i_meter_freeze / i_meter_clear`.

12.4 Port surface

The observer's port list splits into four groups:

1. **Per-port read taps** (`dma_rd_*` from DMA, `fab_rd_*` to fabric). Sized `[NUM_RD_PORTS-1:0]` on every signal so a single instance can tap multiple AXI4 read masters.
2. **Per-port write taps** (`dma_wr_*` / `fab_wr_*`), shape mirrored.
3. **Observer outputs** (`s_axil_*` for the IRQ-drain port, `m_axi_*` for the bulk-dump port, `irq_out`).
4. **Configuration** (address window, flush watermark, per-protocol filter masks — see `monbus_group.md` for the full filter description).
5. **Bus-meter controls and outputs** (clear/freeze inputs, `rid` → channel map, per-port bucket counter outputs).

The leaf-level `cfg_axi_*_mask` inputs on each `axi4_master_*_mon` are tied to 0 inside the observer — leaves do not pre-filter; all filtering happens centrally in `monbus_axil_axi4_group`. The 25 `cfg_<proto>_*` inputs at the observer top are the single point of control.

12.5 axi_bus_meter integration

Each tapped port instantiates an `axi_bus_meter` that classifies each cycle on the data channel (R for reads, W for writes) into one of four buckets:

valid	ready	bucket	meaning
1	1	productive	beat delivered
1	0	backpressure	master wants to send, slave stalls
0	1	starvation	slave ready, master not producing
0	0	idle	both quiet

Counters: 32-bit aggregate per port (4 buckets), plus 16-bit per-channel $\times$ NUM_CHANNELS $\times$ 4 buckets, with per-channel overflow stickies.

12.5.1 Read-side channel attribution: runtime rid $\rightarrow$ channel map

AXI4 R beats carry rid. For each rd port, the observer takes a runtime signal-list mapping:

```
input cfg_rd_rid_per_channel      [NUM_RD_PORTS][NUM_CHANNELS] //
AXI_ID_WIDTH each
input cfg_rd_rid_per_channel_valid [NUM_RD_PORTS][NUM_CHANNELS] // 1
bit each
```

For channel index N on port P, the user writes the expected rid into `cfg_rd_rid_per_channel[P][N]` and asserts the matching `_valid[P][N]`. When an R beat arrives with `rvalid=1`, the observer priority-encodes the lookup (first match wins) and feeds the result to the bus_meter as `i_channel_id + i_channel_valid`. No match $\rightarrow$ the cycle still counts in aggregate but not in any per-channel bucket.

12.5.2 Write-side channel attribution: optional sideband

AXI4 W beats **carry no AXI ID**. Per-channel attribution requires a sideband from the DMA's W-phase FSM. The observer takes optional inputs:

```
input dma_wr_active_ch_id      [NUM_WR_PORTS][CW] // channel index
input dma_wr_active_ch_valid  [NUM_WR_PORTS]      // sideband valid
```

DMA's that expose this (e.g. STREAM's `axi_write_engine` drives `o_active_channel_id / o_active_channel_valid` from its W-phase FSM) wire directly. DMA's without it: tie both to 0; only the aggregate counters tick.

12.5.3 Outputs

Per port (rd or wr): - `*_meter_agg_productive / backpressure / starvation / idle` — 32-bit aggregate counters - `*_meter_ch_productive / backpressure / starvation / idle` — 16-bit per-channel arrays of size NUM_CHANNELS - `*_meter_ch_overflow` — NUM_CHANNELS $\times$ 4 sticky bits (any per-channel counter wrapped)

The caller exposes these to software either by direct probe or via a custom register block — the observer doesn't include its own register-map decoder.

12.6 Pass-through correctness

For each tapped port pair, the observer's job on the AXI path is to forward every channel signal end-to-end with one register stage of skid latency. The test `val/amba/test_axi4_dma_observer.py` covers this directly in Phases 1 and 2:

- Phase 1 issues a fixed sequence of single-beat reads from the DMA side; the fabric-side responder captures every AR and the DMA-side receiver captures every R. Addresses, IDs, and data round-trip byte-exact.
- Phase 2 mirrors the check for the write path: $AW + W(\text{data} + \text{strb} + \text{last}) + B$ all match end-to-end.

Phase 3 then asserts that monbus dump beats actually appear on the observer's own `m_axi_w*` port after real DMA traffic has been observed (i.e. the filter + arbiter + group + writer chain is actually plumbed correctly).

Phase 4 asserts the bus-meter aggregate counters are non-zero after the smoke-test traffic.

12.7 Test

```
pytest val/amba/test_axi4_dma_observer.py -v
```

Default smoke-test parameters: `NUM_RD_PORTS=1`, `NUM_WR_PORTS=1`, `AXI_ID_WIDTH=4`, `NUM_CHANNELS=8`, `MAX_BURST_BEATS=64`, `USE_COMPRESSION=0`. Bus-meter inputs are exercised (`cfg_rd_rid_per_channel` set up as identity for the first 8 channels; write-side sideband tied to 0 — aggregate-only on the W side).

Multi-port and protocol-variant coverage is future work.

12.8 Related Modules

Module	Role
<code>axi_monitor_base</code>	Per-transaction monitor used inside each <code>axi4_master*_mon</code> tap
<code>axi_monitor_reporter</code>	The dispatcher + 6 packet-type sub-blocks that emit monbus packets

Module	Role
monbus_arbiter	Merges per-tap monbus streams
monbus_group family	Central filter + dump writer (this observer uses the AXIL/AXI4 variant)
monbus_compressor	Bulk-trace encoder (instantiated when USE_COMPRESSION=1)
axi4_master_rd_mon / axi4_master_wr_mon	The per-port skid + monitor wrappers the observer instantiates

13 AXI4 DMA Slaves

Module: axi4_dma_slaves.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

13.1 Overview

axi4_dma_slaves bundles a synthetic AXI4 read *source* and write *sink* into one block that a DMA or streaming engine can plug into for source/sink characterization without a real memory backend. The read side (axi4_slave_rd_pattern_gen) answers AR bursts with LFSR-generated data; the write side (axi4_slave_wr_crc_check) accepts AW/W bursts and CRCs the data. Both compute CRC-32 with the same configuration, so a master that reads the pattern and writes it straight back produces matching read/write CRCs when the datapath is clean.

13.1.1 Key Features

- One-instance source + sink slave pair for DMA / streaming-engine characterization
- Read side: LFSR-driven synthetic data source with per-channel CRC-32
- Write side: CRC-32-checking sink with per-channel accounting
- Shared CRC configuration across both sides — read and write CRCs are directly comparable
- LFSR configuration affects only the read (pattern-generation) side
- Independent read-LFSR and write-CRC resets so either side can be re-armed alone
- Per-channel and aggregate CRC / beat-count telemetry from both sides
- Per-side busy_rd / busy_wr status for harness-level stop triggers

13.2 Module Purpose

Characterizing a DMA needs a memory model on both its read and write ports. Rather than instantiate a real RAM (and its checking logic) twice, this block wraps the two purpose-built synthetic slaves and exposes their combined port surface as a single AXI4 read+write slave. The read side manufactures a deterministic pattern; the write side verifies one. Because the CRC polynomial, init, xorout, and reflection settings are threaded identically into both children, the two CRCs are on the same footing — the master writes back the same LFSR data it read, and both sides compute against the same CRC.

Use Cases: - `stream_char_harness` attaches this to STREAM's `m_axi_rd / m_axi_wr` to characterize DMA throughput, response-delay sweeps, and end-to-end CRC integrity - RAPIDS characterization: source/sink termination for an engine's AXI4 master ports - Any AXI4 master needing a fast, checkable, memory-free source/sink pair

Key Benefit: A drop-in memory replacement that is both non-blocking (never the bottleneck) and self-checking (read and write CRCs must agree), all from a single instantiation.

13.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts on each side (channel = id low bits)
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user signal width
SKID_DEPTH_AR	int	2	Read AR skid depth
SKID_DEPTH_R	int	4	Read R skid depth
SKID_DEPTH_AW	int	2	Write AW skid depth
SKID_DEPTH_W	int	4	Write W skid depth
SKID_DEPTH_B	int	2	Write B skid depth
LFSR_WIDTH	int	32	LFSR width (read side only)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed (read side only)

Parameter	Type	Default	Description
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci taps (read side only)
CRC_WIDTH	int	32	CRC width (shared)
CRC_DATA_WIDTH	int	32	Bits per CRC update (shared)
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial (shared)
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value (shared)
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR (shared)
CRC_REFIN	int	1	Reflect input (shared)
CRC_REFOUT	int	1	Reflect output (shared)
REPLICATION_FACTOR	int	(AXI_DATA_WIDTH+31)/32	Read-side pattern replication factor
CRC_SLICE_OFFSET	int	0	Write-side 32-bit CRC slice offset
CIW	int	(NUM_CHANNELS LS>1) ? \$clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: the CRC parameters are passed unchanged to both children on purpose — that shared configuration is what makes read and write CRCs comparable.

13.4 Port Groups

13.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

Port	Direction	Width	Description
read_lfsr_r reset	input	1	Re-arm the read-side LFSR + CRC (independent of write side)
write_crc_r reset	input	1	Re-arm the write-side CRC (independent of read side)

13.4.2 Read-Side Telemetry

Port	Direction	Width	Description
read_crc_v value	output	NUM_CHAN NELS × 32	Per-channel read CRC-32
read_crc_v valid	output	NUM_CHAN NELS	Per-channel read CRC valid
read_beat_ count	output	NUM_CHAN NELS × 32	Per-channel read beats
read_beat_ count_total	output	32	Aggregate read beat count

13.4.3 Write-Side Telemetry

Port	Direction	Width	Description
write_crc_ value	output	NUM_CHAN NELS × 32	Per-channel write CRC-32
write_crc_ valid	output	NUM_CHAN NELS	Per-channel write CRC valid
write_beat_ count	output	NUM_CHAN NELS × 32	Per-channel write beats
write_beat_ count_total	output	32	Aggregate write beat count

13.4.4 AXI4 Read Slave (AR + R)

Port	Direction	Width	Description
s_axi_arid ...	input	—	AR channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
s_axi_arval id			
s_axi_arre ady	output	1	AR ready
s_axi_rid, s_axi_rdata , s_axi_rresp , s_axi_rlast, s_axi_ruser , s_axi_rvali d	output	—	R channel (LFSR pattern data, OKAY resp)
s_axi_rrea dy	input	1	R ready

13.4.5 AXI4 Write Slave (AW + W + B)

Port	Direction	Width	Description
s_axi_awid ...	input	—	AW channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
s_axi_awval lid			
s_axi_awre ady	output	1	AW ready
s_axi_wdat a, s_axi_wstr b, s_axi_wlast , s_axi_wuse	input	—	W channel

Port	Direction	Width	Description
r, s_axi_wvalid			
s_axi_wready	output	1	W ready
s_axi_bid, s_axi_bresp,	output	—	B channel (OKAY resp)
s_axi_bvalid			
s_axi_bready	input	1	B ready

13.4.6 Status

Port	Direction	Width	Description
busy_rd	output	1	Read-side busy (from the read pattern generator)
busy_wr	output	1	Write-side busy (from the write CRC checker)

13.5 Functional Description

13.5.1 Structural Composition

The module is a thin structural wrapper: it instantiates one `axi4_slave_rd_pattern_gen` (`u_rd_pattern_gen`) and one `axi4_slave_wr_crc_check` (`u_wr_crc_check`) and fans the top-level ports out to them. There is no additional logic — all behavior lives in the two children, documented separately.

13.5.2 Read Side (LFSR Source)

`u_rd_pattern_gen` handles the AR/R channels. It returns per-channel LFSR-generated data (32-bit LFSR replicated to the bus width) and accumulates a per-channel CRC-32 over the emitted stream. Its `crc_lfsr_reset` is driven from the top-level `read_lfsr_reset`. LFSR configuration (`LFSR_WIDTH`, `LFSR_SEED`, `LFSR_TAPS`, `REPLICATION_FACTOR`) reaches only this side.

13.5.3 Write Side (CRC Sink)

`u_wr_crc_check` handles the AW/W/B channels. It CRCs the selected 32-bit slice of each accepted W beat per channel and returns OKAY B responses. Its `crc_reset` is driven from the top-level `write_crc_reset`, and `CRC_SLICE_OFFSET` selects the CRC'd lane.

13.5.4 Shared CRC vs Independent Resets

The CRC configuration (`CRC_POLY`, `CRC_INIT`, `CRC_XOROUT`, `CRC_REFIN`, `CRC_REFOUT`, widths) is threaded identically into both children so the read and write CRCs compute over the same algorithm — the intended usage is that the master writes back the LFSR data it read, and the two CRCs must then agree. The two reset inputs are kept separate so the harness can re-arm one side without disturbing the other; in practice both are usually driven from the same CSR clear pulse.

13.6 Usage Example

```
// Source/sink pair on a DMA's read and write master ports.
axi4_dma_slaves #(
    .NUM_CHANNELS    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64)
) u_dma_slaves (
    .aclk             (aclk),
    .aresetn          (aresetn),
    .read_lfsr_reset  (csr_clear_pulse),
    .write_crc_reset  (csr_clear_pulse),

    // Read telemetry
    .read_crc_value   (rd_crc),    .read_crc_valid
(rd_crc_valid),
    .read_beat_count   (rd_beats), .read_beat_count_total
(rd_beats_total),
    // Write telemetry
    .write_crc_value   (wr_crc),    .write_crc_valid
(wr_crc_valid),
    .write_beat_count   (wr_beats), .write_beat_count_total(wr_beats_total),

    // Read slave <- DUT m_axi_rd
    .s_axi_arid        (m_rd_arid), /* ... AR ...
*/ .s_axi_arready(m_rd_arready),
    .s_axi_rid         (m_rd_rid),  /* ... R ... */ .s_axi_rready
(m_rd_rready),
```

```

    // Write slave <- DUT m_axi_wr
    .s_axi_awid (m_wr_awid), /* ... AW ...
*/ .s_axi_awready(m_wr_awready),
    .s_axi_wdata(m_wr_wdata), /* ... W ... */ .s_axi_wready
(m_wr_wready),
    .s_axi_bid (m_wr_bid), /* ... B ... */ .s_axi_bready
(m_wr_bready),

    .busy_rd (rd_busy),
    .busy_wr (wr_busy)
);

// End-to-end integrity: after a loopback run, compare per channel.
assign integrity_ok = (rd_crc[ch] == wr_crc[ch]);

```

13.7 Design Notes

- **Pure composition:** the block adds no logic of its own — it exists to give the harness one instance and one port list instead of two, and to enforce the shared-CRC contract at the parameter level.
 - **Shared CRC config is intentional:** identical CRC parameters on both children are what let a loopback test compare read vs write CRCs directly.
 - **LFSR is read-only:** the write side has no pattern generator; it only checks. LFSR parameters therefore reach only the read child.
 - **Independent re-arm:** separate read_lfsr_reset / write_crc_reset allow re-seeding one side mid-experiment; typical harnesses drive both from one CSR pulse.
 - **No memory backend:** addresses are ignored for data — the read side derives data from the LFSR phase (or, in the master-side blocks, from an address hash), not from stored contents.
-

13.8 Related Modules

13.8.1 Used By

- projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv — attaches to STREAM's m_axi_rd / m_axi_wr
- projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv — source/sink termination

13.8.2 Uses

- **axi4_slave_rd_pattern_gen.sv** — the read-side LFSR data source + CRC
- **axi4_slave_wr_crc_check.sv** — the write-side CRC-checking sink

13.8.3 See Also

- **axi4_master_wr_pattern_gen.sv** / **axi4_master_rd_crc_check.sv** — the master-side counterparts (drive traffic instead of terminating it)
 - **axi4_dma_observer.sv** — non-intrusive observability wrapper for a DMA's master ports
-

13.9 References

13.9.1 Source Code

- RTL: rtl/amba/shared/axi4_dma_slaves.sv

13.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Read child:
docs/markdown/RTLAmba/shared/axi4_slave_rd_pattern_gen.md
 - Write child:
docs/markdown/RTLAmba/shared/axi4_slave_wr_crc_check.md
-

Last Updated: 2026-07-15

13.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

14 AXI4 Master Read CRC Checker

Module: axi4_master_rd_crc_check.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

14.1 Overview

`axi4_master_rd_crc_check` is a CSR-programmed AXI4 read *master* and integrity checker for memory-controller characterization. It walks the *same* algorithmic address mix (via `dma_address_gen`) and the *same* LFSR / hash schedule as `axi4_master_wr_pattern_gen`, so each returned R beat can be compared bit-for-bit against a locally regenerated pattern. It also accumulates a CRC-32 over the regenerated stream, so the harness can compare `o_actual_crc` against the writer's `o_expected_crc`.

14.1.1 Key Features

- CSR-programmed read workload with the *same descriptor shape* as the write pattern generator (one CSR word drives both)
 - Fully decoupled AR and R paths — two independent `dma_address_gen` instances walk the same descriptor in parallel
 - Two expected-data sources: 32-bit Fibonacci LFSR (phase counter) or address-derived Murmur3-fmix hash (out-of-order-safe)
 - Pipelined compare (two isolated 32-bit multiplies) so the mode-1 hash closes 100 MHz; returned data rides the same stages to align the compare
 - Three AR-ID generation modes: FIXED, COUNTER, LFSR
 - Sticky `o_data_error` on any per-beat data mismatch, mismatch counter, and sticky `o_rresp_error` on non-OKAY R beats
 - CRC-32 over the regenerated LFSR stream (`o_actual_crc`) comparable to the writer's expected CRC
 - Optional debug FIFO capturing (`actual`, `expected`, `mismatch`) per R beat for ground-truth disagreement logging
-

14.2 Module Purpose

The read half of a memory-controller integrity loop must know what it *should* receive. This block regenerates the writer's exact data locally — either from the same LFSR phase counter or from the same address hash — and compares every returned R beat against it, latching any mismatch. It also rolls the regenerated data into a CRC-32 that should equal the writer's expected CRC when the wire is clean. Together these give both a per-beat pinpoint (which beat disagreed) and a whole-run summary (CRC compare).

Use Cases: - Reading back a DDR / memory controller and verifying the data written by `axi4_master_wr_pattern_gen` - Address-pattern integrity sweeps sharing one descriptor with the write generator - Multi-id / out-of-order read validation using hash (address-derived) expected data - On-chip (FPGA) read checker in the DDR2 characterization harness

Key Benefit: End-to-end integrity in one block — per-beat compare localizes corruption while the CRC-32 gives a single comparable summary, both derived from exactly the writer’s algorithm.

14.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR skid depth
SKID_DEPTH_R	int	4	R skid depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user width
LFSR_WIDTH	int	32	LFSR width (must match writer)
LFSR_SEED	logic [31:0]	32’hDEADBEEF	LFSR seed default (must match writer)
LFSR_TAPS	logic [47:0]	{12’d32, 12’d22, 12’d2, 12’d1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width (must match writer)
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [CRC_WIDTH-1:0]	32’h04C11DB7	CRC polynomial
CRC_POLY_INIT	logic [CRC_WIDTH-1:0]	’1	CRC initial value
CRC_XOROUT	logic [CRC_WIDTH-1:0]	’1	CRC final XOR
TXN_COUNT_WIDTH	int	16	Width of cfg_txn_count
INDEX_WIDTH	int	16	dma_address_gen index width

Parameter	Type	Default	Description
STRIDE_WIDTH	int	24	dma_address_gen signed stride width
DBG_FIFO_DEPTH	int	0	>0 elaborates a debug FIFO capturing per-beat records; 0 ties dbg_* off
IW / AW / DW / UW	int	—	Aliases for id/addr/data/user widths

Note: LFSR + CRC parameters must match axi4_master_wr_pattern_gen or the compare and CRC roll-up are meaningless. Internally $REPLICATION_FACTOR = (DW+31)/32$ and $HSTAGES = 4$ compare-pipeline stages.

14.4 Port Groups

14.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

14.4.2 Configuration (CSR) — same shape as the write generator

Port	Direction	Width	Description
cfg_start_a ddr	input	AW	Base address for dma_address_gen
cfg_addr_st ride_0	input	signed STRIDE_WID TH	Address stride, dimension 0
cfg_addr_st ride_1	input	signed STRIDE_WID TH	Address stride, dimension 1
cfg_addr_ wrap_mas k_0	input	AW	Address wrap mask, dimension 0

Port	Direction	Width	Description
cfg_addr_wrap_mask_1	input	AW	Address wrap mask, dimension 1
cfg_burst_length	input	8	Beats per burst (1..256); arlen = len - 1
cfg_txn_count	input	TXN_COUNT_WIDTH	Total bursts to issue
cfg_axi_id	input	IW	FIXED-mode id / seed for COUNTER+LFSR modes
cfg_id_mode	input	2	AR-ID mode: 0=FIXED, 1=COUNTER, 2=LFSR
cfg_axi_size	input	3	arsize
cfg_axi_burst	input	2	arburst
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use param)
cfg_data_mode	input	1	0 = phase-counter LFSR; 1 = address hash (OOO-safe)
cfg_hash_seed0/1/2	input	32 each	Murmur3-fmix mixer seeds
cfg_rd_gap	input	4	Inter-burst idle cycles (0..15), independent of the writer's gap
cfg_start	input	1	Pulse to begin the workload
cfg_done	output	1	High when all bursts complete and the compare pipeline drains

14.4.3 Telemetry

Port	Direction	Width	Description
o_actual_crc	output	CRC_WIDTH	Running CRC-32 over the regenerated LFSR stream

Port	Direction	Width	Description
o_actual_cr c_valid	output	1	High with cfg_done (LFSR mode)
o_data_err or	output	1	Sticky on any per-beat data mismatch
o_rresp_err ror	output	1	Sticky on any non-OKAY R beat
o_beats_mismatched	output	TXN_COUNT_WIDTH	Count of mismatching R beats

14.4.4 M-Side AXI4 Read (out to fabric/controller)

Port	Direction	Width	Description
m_axi_arid ...	output	—	AR channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
m_axi_arv alid			
m_axi_arready	input	1	AR ready
m_axi_rid, m_axi_rdata, m_axi_rresp, m_axi_rlast, m_axi_ruser, m_axi_rvalid	input	—	R channel
m_axi_rready	output	1	R ready

14.4.5 Debug Observability (active when DBG_FIFO_DEPTH > 0)

Port	Direction	Width	Description
dbg_valid	output	1	A captured record is

Port	Direction	Width	Description
			available
dbg_ready	input	1	Bench pops a record
dbg_actual	output	DW	Returned data for this beat
dbg_expected	output	DW	Locally regenerated expected data
dbg_mismatch	output	1	This beat disagreed

14.5 Functional Description

14.5.1 Workload FSM

A four-state FSM (S_IDLE, S_RUN, S_GAP, S_DONE) mirrors the write generator. `cfg_start` latches the CSR program and enters S_RUN (or S_DONE if `cfg_txn_count == 0`). S_RUN runs the AR and R paths concurrently; S_GAP inserts `cfg_rd_gap` idle cycles between bursts (pausing both AR and R together); S_DONE holds until re-armed. Because the config shape is identical to the writer, one harness CSR word drives both blocks.

14.5.2 Decoupled AR and R Address Generation

Two independent `dma_address_gen` instances (`u_addr_gen_ar`, `u_addr_gen_r`) walk the same descriptor. The AR path issues one AR per index as fast as `ar_ready` and the address pipeline allow; `arvalid` stays asserted from its first cycle to the last AR handshake at `cfg_rd_gap = 0`. The R path produces the current burst's base address (popped on `rlast`) so the hash-mode expected-data regen has the right anchor for the following beat. R beats are absorbed only after the AR for that burst is on the wire and the R address generator has produced the base.

14.5.3 Expected Data: LFSR vs Address Hash

Muxed by `cfg_data_mode`:

- **Mode 0 (LFSR):** a 32-bit Fibonacci LFSR advances on every accepted R beat — the same logic as the writer, so with the same total beat count the two streams stay phase-aligned and match bit-for-bit. Replicated across the bus for the compare.

- **Mode 1 (address hash):** each 32-bit slice is a Murmur3-fmix function of the beat's byte address and the three hash seeds — a pure function of the address, so out-of-order returns still validate.

14.5.4 Pipelined Compare

The mode-1 hash chains two 32-bit multiplies — the same ~25 ns / 4-DSP cone as the writer, too slow to feed the per-beat compare combinationally at 100 MHz. Each multiply is isolated in its own register stage (HSTAGES = 4); the returned `rdata` rides through the same stages so the compare happens at the pipeline output, aligned with the delayed expected value. Beat/burst accounting stays keyed on the R-beat handshake — only the data compare is delayed. A per-beat mismatch at the pipeline output sets `o_data_error` and increments `o_beats_mismatched`; `rresp` is checked immediately at the R beat (no hash dependency) into `o_rresp_error`. `cfg_done` waits for the compare pipeline to drain (! (| `r_cp_valid`)) so no trailing mismatch is missed.

14.5.5 CRC Accumulation

A `dataint_crc` accumulates over the regenerated LFSR stream (not the returned `rdata`) — matching the writer's accounting so `o_actual_crc` equals the writer's `o_expected_crc` on a clean wire. `o_actual_crc_valid` sets with `cfg_done` in LFSR mode; hash mode relies on the per-beat compare (`o_data_error`).

14.5.6 AR-ID Generation

`arid` is muxed by `cfg_id_mode` exactly like the writer's `awid`: FIXED, an 8-bit COUNTER, or an 8-bit Fibonacci LFSR (taps {8,6,5,4}, seeded `cfg_axi_id[7:0] | 1`).

14.5.7 Out-of-Order Completion — Known Limitation

The v1 LFSR mirror advances on *arrival* index, so it is only correct with a single outstanding AR, or when all ARs share one ID (AXI4 mandates in-order R per id). With multiple outstanding ARs at distinct IDs the controller may return bursts interleaved / fully OOO, which reorders R beats versus the writer's W phase and breaks both the per-beat compare and the CRC roll-up. The header documents the v2 plan (per-address hash expected function + commutative CRC roll-up). Until then, the harness CSR must keep the read block single-outstanding (or all-same-id) when the controller has OOO enabled — or use hash data mode, which is inherently OOO-safe for the per-beat compare.

14.5.8 Optional Debug FIFO

When `DBG_FIFO_DEPTH > 0` a `gaxi_fifo_sync` captures (`actual`, `expected`, `mismatch`) at each compare-pipeline output so the bench can walk the exact disagreements instead of inferring from `o_data_error`. When depth is 0 the generate else arm ties the `dbg_*` outputs off and the FIFO is not built.

14.5.9 Standard Protocol Handler

AR/R skid buffering and AXI compliance are delegated to an `axi4_master_rd` instance; the FSM/LFSR/hash/compare logic drives its FUB side and the `m_axi_*` ports pass straight out to the fabric.

14.6 Usage Example

```
// Read checker paired with the write generator on a DDR2 sweep.
axi4_master_rd_crc_check #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH     (64),
    .DBG_FIFO_DEPTH    (0)
) u_rd_chk (
    .aclk      (aclk),
    .aresetn   (aresetn),

    // SAME descriptor word as the write generator
    .cfg_start_addr      (csr_base_addr),
    .cfg_addr_stride_0    (csr_stride_0),
    .cfg_addr_stride_1    (24'sd0),
    .cfg_addr_wrap_mask_0 (csr_wrap_0),
    .cfg_addr_wrap_mask_1 ('0),
    .cfg_burst_len        (8'd16),
    .cfg_txn_count        (16'd1024),
    .cfg_axi_id           (8'd0),
    .cfg_id_mode          (2'd0),          // FIXED / single-outstanding-
safe
    .cfg_axi_size         (3'd3),
    .cfg_axi_burst        (2'd1),
    .cfg_lfsr_seed        (32'd0),
    .cfg_data_mode        (1'b0),          // LFSR (CRC comparable to
writer)
    .cfg_hash_seed0       (32'd0),
    .cfg_hash_seed1       (32'd0),
    .cfg_hash_seed2       (32'd0),
    .cfg_rd_gap           (4'd0),
    .cfg_start            (csr_start_pulse),
    .cfg_done             (rd_done),

    .o_actual_crc         (rd_actual_crc),
    .o_actual_crc_valid   (rd_actual_crc_valid),
    .o_data_error         (rd_data_error),
    .o_rresp_error        (rd_rresp_error),
    .o_beats_mismatched   (rd_beats_bad),
```

```

// M-side AXI from the controller
.m_axi_arid (arid), /* ... AR ... */ .m_axi_arready(arready),
.m_axi_rid  (rid),  /* ... R ... */ .m_axi_rready (rready),

// Debug FIFO tied off (DBG_FIFO_DEPTH == 0)
.dbg_valid (), .dbg_ready (1'b0),
.dbg_actual(), .dbg_expected(), .dbg_mismatch()
);

// Clean-wire check: CRCs agree and no per-beat mismatch.
assign integrity_ok = (rd_actual_crc == wr_expected_crc) && !
rd_data_error;

```

14.7 Design Notes

- **Mirror the writer exactly:** LFSR seed schedule, CRC config, address descriptor, and ID modes are all mirror images of `axi4_master_wr_pattern_gen` — that symmetry is what makes the CRCs and per-beat expected values comparable.
 - **Per-beat compare + CRC are complementary:** the compare localizes the failing beat; the CRC gives a single-register whole-run summary. Both are provided so a failure can be both detected and pinpointed.
 - **Compare pipeline aligns delayed expected with delayed rdata:** the returned data is intentionally delayed the same HSTAGES as the hash so the compare is apples-to-apples, and `cfg_done` waits for the pipeline to drain.
 - **OOO is a real v1 limitation:** with multi-id OOO completion, use hash data mode (`cfg_data_mode = 1`) or hold the read block single-outstanding — the header spells out the v2 per-address-hash fix.
 - **Debug FIFO is opt-in:** `DBG_FIFO_DEPTH > 0` adds ground-truth per-beat capture at essentially no cost when disabled.
-

14.8 Related Modules

14.8.1 Used By

- `projects/NexysA7/ddr2-characterization/flows-ours-uart/rtl/ddr2_char_harness.sv` — on-chip read checker

- DDR2 characterization macro / harness CSR blocks under `projects/NexysA7/ddr2-characterization/`

14.8.2 Uses

- **`axi4_master_rd.sv`** — standard AXI4 read master protocol handler (AR/R skid + compliance)
- **`dma_address_gen.sv`** — algorithmic address sequence generator (×2, decoupled AR/R)
- **`shifter_lfsr_fibonacci.sv`** — expected-data LFSR and 8-bit AR-ID LFSR
- **`dataint_crc.sv`** — CRC-32 accumulator
- **`gaxi_fifo_sync.sv`** — optional debug capture FIFO

14.8.3 See Also

- **`axi4_master_wr_pattern_gen.sv`** — the matching write-side driver (same LFSR/CRC/hash/descriptor config)
 - **`axi4_slave_rd_pattern_gen.sv`** — the slave-side read pattern source
-

14.9 References

14.9.1 Source Code

- RTL: `rtl/amba/shared/axi4_master_rd_crc_check.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_master_rd.sv`

14.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`
 - Index: `docs/markdown/RTLAmba/index.md`
 - Harness: `projects/NexysA7/ddr2-characterization/README.md`
-

Last Updated: 2026-07-15

14.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

15 AXI4 Master Write Pattern Generator

Module: `axi4_master_wr_pattern_gen.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

15.1 Overview

`axi4_master_wr_pattern_gen` is a CSR-programmed AXI4 write *master* for memory-controller characterization. On a start pulse it walks an algorithmic address mix (via `dma_address_gen`) and streams LFSR-pattern data through `axi4_master_wr`, accumulating a CRC-32 over the data it writes. It pairs with `axi4_master_rd_crc_check`, which regenerates the same pattern on the read side so the two CRCs (and per-beat compares) validate end-to-end data integrity through a real DRAM controller.

15.1.1 Key Features

- CSR-programmed write workload: start address, address-generator strides/wrap masks, burst length, transaction count, AXI id/size/burst attributes
 - Fully decoupled AW and W paths — two independent `dma_address_gen` instances walk the same descriptor in parallel
 - Two data sources: a 32-bit Fibonacci LFSR (phase-counter) or an address-derived Murmur3-fmix hash (out-of-order-safe)
 - Pipelined hash datapath (two isolated 32-bit multiplies) to close 100 MHz, decoupled from the AXI W handshake by a staging FIFO
 - Three AW-ID generation modes: FIXED, COUNTER, LFSR
 - CRC-32 accumulator over the written LFSR stream (`o_expected_crc`) for the read side to compare against
 - Configurable inter-burst idle gap (`cfg_wr_gap`) for throughput-stress sweeps
 - Sticky `o_bresp_error` on any non-OKAY write response; direct re-arm from the done state
-

15.2 Module Purpose

Bringing up and characterizing a memory controller means driving it with realistic, deterministic write traffic and proving the data survives the round trip. This block drives the writes: it walks a programmable address pattern and emits reproducible data, folding that data into a CRC-32 that

becomes the “expected” value for the read-side checker. The write data can be a simple LFSR phase counter (fast, but order-sensitive) or an address-derived hash (each beat’s data is a pure function of its byte address, so multi-id / out-of-order completion still validates).

Use Cases: - Driving a DDR / memory-controller’s AXI4 write port during characterization sweeps
 - Generating the “golden” CRC / data that axi4_master_rd_crc_check compares against - Address-pattern stress (incremental, row-major, column-major page attacks) via the dma_address_gen descriptor - On-chip (FPGA) write stimulus in the DDR2 characterization harness

Key Benefit: A CSR-driven, deterministic, self-documenting write generator whose data can be regenerated bit-for-bit anywhere — turning a memory-controller bring-up into a single expected-vs-actual CRC comparison.

15.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW skid depth
SKID_DEPTH_W	int	4	W skid depth
SKID_DEPTH_B	int	2	B skid depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (derived)
LFSR_WIDTH	int	32	LFSR width; matches slave side so CRCs interchange
LFSR_SEED	logic [31:0]	32’hDEADBEEF	LFSR seed default
LFSR_TAPS	logic [47:0]	{12’d32, 12’d22, 12’d2, 12’d1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update (fixed 32 to match slave side)

Parameter	Type	Default	Description
CRC_POLY	logic [CRC_WID TH-1:0]	32'h04C11DB7	CRC polynomial
CRC_POLY_INIT	logic [CRC_WID TH-1:0]	'1	CRC initial value
CRC_XOROUT	logic [CRC_WID TH-1:0]	'1	CRC final XOR
TXN_COUNT_WIDTH	int	16	Width of cfg_txn_count (up to 64K bursts)
INDEX_WIDTH	int	16	dma_address_gen index width
STRIDE_WIDTH	int	24	dma_address_gen signed stride width
IW / AW / DW / UW / SW	int	—	Aliases for id/addr/data/user/strobe widths

Note: LFSR + CRC parameters mirror axi4_master_rd_crc_check exactly so the two blocks' CRC values are interchangeable. Internally $REP = (DW+31)/32$ 32-bit slices per beat and $HSTAGES = 4$ hash-pipeline stages.

15.4 Port Groups

15.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

15.4.2 Configuration (CSR)

Port	Direction	Width	Description
cfg_start_addr	input	AW	Base address handed to dma_address_gen
cfg_addr_stride_0	input	signed STRIDE_WIDTH	Address stride, dimension 0
cfg_addr_stride_1	input	signed STRIDE_WIDTH	Address stride, dimension 1 (held at index 0 here)
cfg_addr_wrap_mask_0	input	AW	Address wrap mask, dimension 0
cfg_addr_wrap_mask_1	input	AW	Address wrap mask, dimension 1
cfg_burst_length	input	8	Beats per burst (1..256); awlen = len - 1
cfg_txn_count	input	TXN_COUNT_WIDTH	Total bursts to issue
cfg_axi_id	input	IW	FIXED-mode id / seed for COUNTER+LFSR modes
cfg_id_mode	input	2	AW-ID mode: 0=FIXED, 1=COUNTER, 2=LFSR
cfg_axi_size	input	3	awsize
cfg_axi_burst	input	2	awburst (INCR=1)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use LFSR_SEED param)
cfg_data_mode	input	1	0 = phase-counter LFSR; 1 = address-derived hash (OOO-safe)
cfg_hash_s	input	32 each	Murmur3-fmix mixer

Port	Direction	Width	Description
eed0/1/2			seeds (hash mode)
cfg_wr_gap	input	4	Inter-burst idle cycles (0..15)
cfg_start	input	1	Pulse to begin the workload
cfg_done	output	1	High once all B responses received

15.4.3 Telemetry

Port	Direction	Width	Description
o_expected_crc	output	CRC_WIDTH	Running CRC-32 over the written LFSR stream
o_expected_crc_valid	output	1	High with cfg_done (LFSR mode only)
o_bresp_err	output	1	Sticky on any non-OKAY BRESP

15.4.4 M-Side AXI4 Write (out to fabric/controller)

Port	Direction	Width	Description
m_axi_awi d ... m_axi_awv alid	output	—	AW channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
m_axi_awr eady	input	1	AW ready
m_axi_wda ta, m_axi_wst rb, m_axi_wla st, m_axi_wus er, m_axi_wva lid	output	—	W channel (strokes tied full-beat)

Port	Direction	Width	Description
m_axi_wready	input	1	W ready
m_axi_bid, m_axi_bresp, m_axi_buser, m_axi_bvalid	input	—	B channel
m_axi_bready	output	1	B ready

15.5 Functional Description

15.5.1 Workload FSM

A four-state FSM (S_IDLE, S_RUN, S_GAP, S_DONE) sequences a run. On `cfg_start` the CSR program is latched (so software can change the `cfg_*` inputs on the next cycle without disturbing the run) and the block enters S_RUN (or S_DONE immediately if `cfg_txn_count == 0`). In S_RUN the AW and W paths run concurrently; S_GAP inserts `cfg_wr_gap` idle cycles between bursts; S_DONE awaits the final B responses. S_DONE also accepts a fresh `cfg_start` for a direct re-arm without passing back through S_IDLE.

15.5.2 Decoupled AW and W Address Generation

Two independent `dma_address_gen` instances (`u_addr_gen_aw`, `u_addr_gen_w`) walk the *same* descriptor with the same indices, so both produce the identical address sequence. The AW path issues one AW per index at `awready` rate; the W path produces the current burst's base address for the hash datapath, popped on `WLAST`. `awvalid` stays continuously asserted from its first cycle to the last AW handshake when `cfg_wr_gap = 0` — the address generator produces one result per cycle once warmed. Outstanding depth is bounded by the slave's `awready` throttling, not by this block. Only `index_0` is walked (`index_1` held at 0); a second instance with a different `cfg_addr_stride_1` would exercise the 2D path.

15.5.3 Data Path: LFSR vs Address Hash

Two data sources are muxed by `cfg_data_mode`:

- **Mode 0 (LFSR):** a 32-bit Fibonacci LFSR (`shifter_lfsr_fibonacci`, taps {32, 22, 2, 1}) advances on every accepted W beat, replicated across the

data bus (REP copies). The data stream is a deterministic function of (seed, total_beats_so_far). This is order-sensitive and breaks under multi-id / out-of-order completion.

- **Mode 1 (address hash):** each 32-bit slice is a Murmur3-fmix-style function of its byte address and the three `cfg_hash_seed` values (xor-shift + odd multiplies). Because each beat's data depends only on its address, reorder does not perturb the per-beat compare on the read side.

The per-beat byte address is anchored on the W address-generator output and stepped by $2 \times \text{awsz}$ bytes per beat. Full-beat writes only — `wstrb` is tied all-ones.

15.5.4 Hash Pipeline and Staging FIFO

The mode-1 hash chains two 32-bit multiplies — combinationally a ~ 25 ns / 4-DSP cone that misses 100 MHz. Each multiply is isolated in its own register stage (`HSTAGES = 4`), with the `WLAST/mode/LFSR`-data riding alongside so the output stays beat-aligned. The pipeline is latency-insensitive: per-beat data values and order are unchanged, so the CRC / per-beat-compare contract still holds. A show-ahead staging FIFO (`gaxi_fifo_sync`, depth 16) between the pipeline output and the AXI W handshake lets W stream back-to-back independent of pipeline fill or master backpressure; a beat is admitted only when the generator has one and the FIFO has room for it plus the `HSTAGES` beats already in flight.

15.5.5 AW-ID Generation

`awid` is muxed by `cfg_id_mode`: `FIXED` passes `cfg_axi_id` through; `COUNTER` is an 8-bit counter seeded at `cfg_axi_id[7:0]`, +1 per AW; `LFSR` is an 8-bit maximal-length Fibonacci LFSR (taps {8,6,5,4}) seeded with `cfg_axi_id[7:0] | 1` to avoid the all-zero seed. Internal counter/LFSR are 8-bit and zero-extended/truncated to `IW`.

15.5.6 CRC Accumulation and Completion

A `dataint_crc` instance accumulates over the same LFSR stream sent on W (not over the hash), latched into `o_expected_crc` at the last W beat of the last burst — meaningful only in LFSR mode, where `o_expected_crc_valid` sets. In hash mode the CRC is not load-bearing and validity is gated low; the harness uses the read side's per-beat compare instead. B responses are counted independently of FSM phase (except never accepted pre-start); `o_bresp_error` sticks on any non-OKAY BRESP. `cfg_done` asserts once in `S_DONE` with all `cfg_txn_count` B's received.

15.5.7 Standard Protocol Handler

AW/W/B skid buffering and AXI compliance are delegated to an `axi4_master_wr` instance; the FSM/LFSR/hash logic drives its FUB side and the `m_axi_*` ports pass straight out to the fabric.

15.6 Usage Example

```
// Write driver for a DDR2 characterization sweep (LFSR data mode).
axi4_master_wr_pattern_gen #(
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64)
) u_wr_gen (
    .aclk      (aclk),
    .aresetn   (aresetn),

    // CSR program (from the harness register block)
    .cfg_start_addr      (csr_base_addr),
    .cfg_addr_stride_0    (csr_stride_0),
    .cfg_addr_stride_1    (24'sd0),
    .cfg_addr_wrap_mask_0 (csr_wrap_0),
    .cfg_addr_wrap_mask_1 ('0),
    .cfg_burst_len        (8'd16),
    .cfg_txn_count        (16'd1024),
    .cfg_axi_id           (8'd0),
    .cfg_id_mode          (2'd0),          // FIXED
    .cfg_axi_size         (3'd3),          // 8 bytes/beat
    .cfg_axi_burst        (2'd1),          // INCR
    .cfg_lfsr_seed        (32'd0),         // use param default
    .cfg_data_mode        (1'b0),         // LFSR (golden CRC valid)
    .cfg_hash_seed0       (32'd0),
    .cfg_hash_seed1       (32'd0),
    .cfg_hash_seed2       (32'd0),
    .cfg_wr_gap           (4'd0),
    .cfg_start            (csr_start_pulse),
    .cfg_done             (wr_done),

    // Golden CRC for the read side to compare against
    .o_expected_crc       (wr_expected_crc),
    .o_expected_crc_valid (wr_expected_crc_valid),
    .o_bresp_error        (wr_bresp_error),

    // M-side AXI to the controller
    .m_axi_awid (awid), /* ... AW ... */ .m_axi_awready(awready),
    .m_axi_wdata(wdata), /* ... W ... */ .m_axi_wready (wready),
    .m_axi_bid  (bid),  /* ... B ... */ .m_axi_bready (bready)
);
```

15.7 Design Notes

- **Two data modes for two threat models:** LFSR mode is the fast default with a single golden CRC; hash mode trades the CRC for OOO-safe per-beat verification when multi-id traffic reorders completions.
 - **Hash pipelining was a timing fix:** the two-multiply Murmur cone was isolated into DSP-register stages specifically to close 100 MHz on FPGA; the staging FIFO then hides the added latency from the AXI handshake.
 - **CRC follows the LFSR, not the wire:** the CRC absorbs the LFSR words (32-bit), keeping it interchangeable with the read checker and independent of bus width; it is only latched valid in LFSR mode.
 - **Decoupled AW/W:** separate address generators mean AW can race ahead at a ready rate while W streams at a ready rate, exposing controller reorder/backpressure behavior a lock-stepped generator would mask.
 - **Direct re-arm:** S_DONE re-latches on cfg_start so back-to-back sweeps don't require a return to idle.
-

15.8 Related Modules

15.8.1 Used By

- projects/NexysA7/ddr2-characterization/flows-ours-uart/rtl/ddr2_char_harness.sv — on-chip write driver
- DDR2 characterization macro / harness CSR blocks under projects/NexysA7/ddr2-characterization/

15.8.2 Uses

- **axi4_master_wr.sv** — standard AXI4 write master protocol handler (AW/W/B skid + compliance)
- **dma_address_gen.sv** — algorithmic address sequence generator (×2, decoupled AW/W)
- **shifter_lfsr_fibonacci.sv** — LFSR data source and 8-bit AW-ID LFSR
- **dataint_crc.sv** — CRC-32 accumulator
- **gaxi_fifo_sync.sv** — show-ahead staging FIFO decoupling the hash pipeline from W

15.8.3 See Also

- **axi4_master_rd_crc_check.sv** — the matching read-side driver + checker (same LFSR/CRC/hash config)
 - **axi4_slave_wr_crc_check.sv** — the slave-side write CRC sink
-

15.9 References

15.9.1 Source Code

- RTL: rtl/amba/shared/axi4_master_wr_pattern_gen.sv
- Protocol Handler: rtl/amba/axi4/axi4_master_wr.sv

15.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Harness: projects/NexysA7/ddr2-characterization/README.md
-

Last Updated: 2026-07-15

15.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

16 AXI4 Slave Read Pattern Generator

Module: axi4_slave_rd_pattern_gen.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

16.1 Overview

axi4_slave_rd_pattern_gen is a read-only AXI4 slave that answers AR bursts with deterministic, LFSR-generated data and accumulates a running CRC-32 over the stream it emits. It exists to serve as a synthetic data *source* for DMA / streaming-engine characterization: a master under test issues read bursts, the slave returns a reproducible pseudo-random pattern, and the

same pattern (and CRC) can be regenerated anywhere else in the system to prove end-to-end data integrity.

16.1.1 Key Features

- Read-only AXI4 slave (AR + R channels) built on the standard `axi4_slave_rd` protocol handler
 - Per-channel independent LFSR + CRC-32 state, demuxed off the low bits of `arid`
 - 32-bit maximal-length Fibonacci LFSR (seed `0xDEADBEEF`, taps `{32, 22, 2, 1}`) replicated to fill the data bus
 - CRC-32/Ethernet accumulator per channel (poly `0x04C11DB7`, reflected in/out) matching the write-side checker bit-for-bit
 - Gapless back-to-back bursts (AR accepted on the last beat) so `rvalid` never drops mid-stream
 - Per-channel CRC / beat-count telemetry plus an aggregate beat-count total for harness stop triggers
 - Single `crc_lfsr_reset` pulse re-arms all channel LFSRs and CRCs together
-

16.2 Module Purpose

Characterizing a DMA read path needs a slave that is both *fast* (never the bottleneck) and *checkable* (the returned data is a known function of the request). A real memory backend gives neither cheaply. This module instead generates its read data from a Fibonacci LFSR, so every beat is a deterministic function of (`seed`, `beat_index`), and folds that same data into a CRC-32 the harness can compare against an independently computed golden value.

Use Cases: - Feeding a DMA / streaming engine's read (`m_axi_rd`) port during throughput characterization - End-to-end integrity checks where read data is looped back to a write-CRC sink - Per-channel multi-context validation, where each channel's data must be independent of another channel's interleave on the shared AXI port - On-chip (FPGA) stimulus in the STREAM / RAPIDS characterization harnesses

Key Benefit: A memory-free, deterministic read slave whose per-channel CRC-32 is bit-identical to the write-side checker — integrity can be proven with a single register compare, and the slave never starves the master.

16.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Number of independent LFSR/CRC contexts; channel index is <code>arid[CIW-1:0]</code>
SKID_DEPTH_AR	int	2	AR channel skid buffer depth
SKID_DEPTH_R	int	4	R channel skid buffer depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width (may be wider; must be handled by replication)
AXI_USER_WIDTH	int	1	AXI user signal width
LFSR_WIDTH	int	32	LFSR width (fixed at 32 for timing/simplicity)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses $\text{LFSR_SEED} \wedge N$
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci tap indices
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits processed per CRC update (the 32-bit LFSR output)
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input bytes
CRC_REFOUT	int	1	Reflect output

Parameter	Type	Default	Description
REPLICATION_FACTOR	int	(AXI_DATA_WIDTH+31)/32	Derived: number of 32-bit LFSR copies per beat
CIW	int	(NUM_CHANNELS > 1) ? \$clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: CRC_* must match axi4_slave_wr_crc_check (and the AXIS pattern blocks) exactly, or cross-block CRC comparison is meaningless.

16.4 Port Groups

16.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

16.4.2 Test Control

Port	Direction	Width	Description
crc_lfsr_reset	input	1	Pulse to reload all channel LFSR seeds and clear all CRCs / beat counts

16.4.3 Per-Channel Telemetry

Port	Direction	Width	Description
read_crc_value	output	NUM_CHANNELS × 32	Running CRC-32 per channel
read_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid (set after first accepted beat)
read_beat_count	output	NUM_CHANNELS × 32	Per-channel beats emitted
read_beat_sum	output	32	Sum of per-channel beat

Port	Direction	Width	Description
count_total			counts (harness stop trigger)

16.4.4 AXI4 Read Address Channel (AR)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read address ID (low bits select the channel context)
s_axi_araddr	input	AXI_ADDR_WIDTH	Read address (unused for data generation)
s_axi_arlen	input	8	Burst length minus 1 (beats – 1)
s_axi_arsize	input	3	Burst size
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock type
s_axi_arcache	input	4	Cache attributes
s_axi_arprot	input	3	Protection attributes
s_axi_arqos	input	4	Quality of service
s_axi_arregion	input	4	Region identifier
s_axi_aruser	input	AXI_USER_WIDTH	User sideband (echoed on R)
s_axi_arvalid	input	1	AR valid
s_axi_arready	output	1	AR ready

16.4.5 AXI4 Read Data Channel (R)

Port	Direction	Width	Description
s_axi_rid	output	AXI_ID_WIDTH	Read data ID (echoes captured arid)
s_axi_rdata	output	AXI_DATA_WIDTH	LFSR pattern data (32-bit LFSR replicated to fill the bus)
s_axi_rresp	output	2	Response, always OKAY
s_axi_rlast	output	1	Last beat of burst
s_axi_ruser	output	AXI_USER_WIDTH	User sideband (echoes captured aruser)
s_axi_rvalid	output	1	R valid
s_axi_rready	input	1	R ready

16.4.6 Status

Port	Direction	Width	Description
busy	output	1	Protocol-handler busy indicator (from axi4_slave_rd)

16.5 Functional Description

16.5.1 LFSR Pattern Generation

Each channel owns a `shifter_lfsr_fibonacci` instance (32-bit, taps {32, 22, 2, 1}) seeded with `LFSR_SEED ^ channel_index`. XOR-ing the channel index into the seed guarantees each channel produces a distinct stream, so channel interleave at the shared AXI port cannot corrupt any one channel's sequence. Only the channel currently being served advances on a given R-beat handshake; all other channel LFSRs hold their state. On the global `crc_lfsr_reset` pulse every channel reloads its seed simultaneously.

16.5.2 Data Replication to the Bus Width

The 32-bit LFSR output is expanded to `AXI_DATA_WIDTH` by replication. A generate block picks the right path: a direct assignment when the bus is exactly 32 bits, a whole-number replication when the width is a multiple of 32, or a replicate-and-truncate for non-aligned widths

(`REPLICATION_FACTOR = (AXI_DATA_WIDTH+31)/32` copies, sliced down). The active channel's replicated word drives `s_axi_rdata`.

16.5.3 Per-Channel CRC-32

Each channel has its own `dataint_crc` instance (CRC-32/Ethernet, `REFIN/REFOUT = 1`). The CRC absorbs one 32-bit LFSR word per accepted beat for that channel (`load_from_cascade` gated by “R beat AND active channel == this channel”, `cascade_sel = 4'b1000`). A per-channel valid flag sets on the first accepted beat and a per-channel beat counter increments alongside.

`crc_lfsr_reset` clears the CRC (via `load_crc_start`), the valid flag, and the counter. The `read_beat_count_total` output is a combinational sum across all channels, used by the harness as a workload-completion trigger.

16.5.4 Burst FSM and Gapless Back-to-Back Bursts

A small two-state FSM (`RD_IDLE`, `RD_BURST`) drives the FUB-side AR/R handshake. AR is accepted when idle *or* on the last beat of the current burst — the latter (`w_rd_last_beat`) lets the next burst reload `r_rd_id / r_rd_beats_remaining` on the same cycle the current one finishes, keeping `rvalid` continuously asserted. The original idle-only accept forced a one-cycle dead gap per burst, which surfaced as ~1 starvation cycle per burst on the master's R channel (a slave-model artifact, not a DUT limitation); accepting the AR on `rlast` removes it. `rresp` is hardwired to `OKAY` and `rlast` asserts when `r_rd_beats_remaining` reaches 0.

16.5.5 Standard Protocol Handler

All AR/R skid buffering and AXI protocol compliance are delegated to a `axi4_slave_rd` instance. The pattern-gen FSM and LFSR/CRC logic ride on that module's FUB-side (`fub_axi_ar* / fub_axi_r*`) interface, so the external `s_axi_*` ports are fully AXI4-compliant. (A header TODO notes that hand-rolled FUB logic should eventually wrap `axi4_slave_rd_mon` instead — tracked as task #78 — but the current handler is the standard `axi4_slave_rd`.)

16.6 Usage Example

```
// Synthetic read source for a 4-channel DMA characterization run.
axi4_slave_rd_pattern_gen #(
    .NUM_CHANNELS    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .LFSR_SEED       (32'hDEADBEEF)
) u_rd_src (
    .aclk              (aclk),
    .aresetn           (aresetn),
    .crc_lfsr_reset    (csr_clear_pulse), // re-arm all channels
```

```

// Telemetry to the harness CSR block
.read_crc_value      (rd_crc_value),
.read_crc_valid      (rd_crc_valid),
.read_beat_count      (rd_beat_count),
.read_beat_count_total (rd_beats_total),

// AR/R wired to the DUT's read master port
.s_axi_arid      (dut_arid),   .s_axi_araddr (dut_araddr),
.s_axi_arlen      (dut_arlen), .s_axi_arsize (dut_arsize),
.s_axi_arburst    (dut_arburst),.s_axi_arlock  (dut_arlock),
.s_axi_arcache    (dut_arcache),.s_axi_arprot  (dut_arprot),
.s_axi_arqos      (dut_arqos), .s_axi_arregion(dut_arregion),
.s_axi_aruser     (dut_aruser), .s_axi_arvalid  (dut_arvalid),
.s_axi_arready    (dut_arready),

.s_axi_rid      (dut_rid),   .s_axi_rdata  (dut_rdata),
.s_axi_rresp    (dut_rresp), .s_axi_rlast  (dut_rlast),
.s_axi_ruser    (dut_ruser), .s_axi_rvalid  (dut_rvalid),
.s_axi_rready    (dut_rready),

.busy (rd_busy)
);

```

16.7 Design Notes

- **Seed-per-channel independence:** LFSR_SEED ^ N is deliberately simple but sufficient — a maximal-length LFSR seeded at distinct points produces streams that stay decorrelated over the characterization window, so per-channel CRCs are independently checkable.
- **CRC over the LFSR word, not the replicated bus:** only the 32-bit LFSR output feeds the CRC (cascade_sel = 4'b1000), so the CRC value is bus-width independent and interchangeable with the 32-bit-wide write checker and AXIS blocks.
- **Gapless accept is a measurement fix:** the back-to-back AR accept was added specifically so the read model wouldn't inject a false ~6% starvation artifact into the master's utilization numbers.
- **Reset separation:** crc_lfsr_reset is distinct from aresetn so the harness can re-arm the pattern between runs without a full block reset.

- **Standards note:** the AR/R glue is hand-rolled on top of `axi4_slave_rd`; the header flags a future refactor to the monitored `axi4_slave_rd_mon` wrapper (task #78).
-

16.8 Related Modules

16.8.1 Used By

- `axi4_dma_slaves` — bundles this read source with `axi4_slave_wr_crc_check` into a single source/sink slave pair
- `projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv` — on-chip read stimulus
- `projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip read stimulus

16.8.2 Uses

- `axi4_slave_rd.sv` — standard AXI4 read slave protocol handler (AR/R skid + compliance)
- `shifter_lfsr_fibonacci.sv` — per-channel maximal-length Fibonacci LFSR
- `dataint_crc.sv` — per-channel CRC-32 accumulator

16.8.3 See Also

- `axi4_slave_wr_crc_check.sv` — the matching write-side CRC sink (same CRC config)
 - `axi4_dma_slaves.sv` — the source/sink bundle
 - `axis4_master_pattern_gen.sv` — AXIS equivalent of this generator
-

16.9 References

16.9.1 Source Code

- RTL: `rtl/amba/shared/axi4_slave_rd_pattern_gen.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_slave_rd.sv`

16.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`
- Index: `docs/markdown/RTLAmba/index.md`

- [Harness: projects/NexysA7/ddr2-characterization/README.md](#)
-

Last Updated: 2026-07-15

16.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

17 AXI4 Slave Write CRC Checker

Module: `axi4_slave_wr_crc_check.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

17.1 Overview

`axi4_slave_wr_crc_check` is a write-only AXI4 slave that accepts AW/W bursts, folds the written data into a per-channel CRC-32, and returns B responses. It is the *sink* counterpart to `axi4_slave_rd_pattern_gen`: a master under test writes data, this slave CRCs it, and the resulting CRC can be compared against an independently computed golden value to prove the write path carried the data intact.

17.1.1 Key Features

- Write-only AXI4 slave (AW + W + B channels) built on the standard `axi4_slave_wr` protocol handler
- Per-channel independent CRC-32 state, demuxed off the low bits of the captured `awid`
- CRC-32/Ethernet (poly 0x04C11DB7, reflected in/out) — bit-identical to `axi4_slave_rd_pattern_gen`
- Configurable 32-bit data slice (`CRC_SLICE_OFFSET`) selects which 32-bit lane of a wide bus is CRC'd
- Gapless back-to-back bursts (AW accepted on the last W beat) so `wready` never drops mid-stream
- Separately-latched B `id/user` so back-to-back bursts return the correct response `id`

- Per-channel CRC / beat-count telemetry plus an aggregate beat-count total for harness stop triggers
-

17.2 Module Purpose

The write half of a DMA characterization loop needs a slave that accepts write traffic at full rate and reports whether the data arrived correctly. This module CRCs the incoming write data per channel using exactly the same CRC-32 configuration as the read pattern generator, so if a master reads the LFSR pattern and writes it straight back, the write-side CRC must equal the read-side CRC. Any mismatch localizes a data-path corruption to the write path.

Use Cases: - Terminating a DMA / streaming engine's write (`m_axi_wr`) port during characterization - End-to-end integrity checks where read data is looped back and re-verified on write - Per-channel multi-context validation on a shared write port - On-chip (FPGA) write sink in the STREAM / RAPIDS characterization harnesses

Key Benefit: A memory-free write slave that produces a per-channel CRC-32 directly comparable to the read generator — a single register compare confirms the write path is clean, and the sink never backpressures the master unnecessarily.

17.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Number of independent CRC contexts; channel index is <code>awid[CIW-1:0]</code>
SKID_DEPTH_AW	int	2	AW channel skid buffer depth
SKID_DEPTH_W	int	4	W channel skid buffer depth
SKID_DEPTH_B	int	2	B channel skid buffer depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user signal width
CRC_WIDTH	int	32	CRC width

Parameter	Type	Default	Description
CRC_DATA_WIDTH	int	32	Bits processed per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input bytes
CRC_REFOUT	int	1	Reflect output
CRC_SLICE_OFFSET	int	0	Which 32-bit slice of wdata to CRC (in 32-bit units)
CIW	int	(NUM_CHANNE LS>1) ? \$clog2(NUM_CH ANNELS) : 1	Derived: channel-index width

Note: CRC_* must match axi4_slave_rd_pattern_gen exactly. A compile-time \$error fires if CRC_SLICE_OFFSET selects a slice beyond AXI_DATA_WIDTH.

17.4 Port Groups

17.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

17.4.2 Test Control

Port	Direction	Width	Description
crc_reset	input	1	Pulse to clear all channel CRCs, valid flags, and beat

Port	Direction	Width	Description
			counts

17.4.3 Per-Channel Telemetry

Port	Direction	Width	Description
write_crc_value	output	NUM_CHAN NELS × 32	Running CRC-32 per channel
write_crc_valid	output	NUM_CHAN NELS	Per-channel CRC valid (set after first accepted W beat)
write_beat_count	output	NUM_CHAN NELS × 32	Per-channel W beats absorbed
write_beat_count_total	output	32	Sum of per-channel beat counts (harness stop trigger)

17.4.4 AXI4 Write Address Channel (AW)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write address ID (low bits select the channel context)
s_axi_awa_ddr	input	AXI_ADDR_WIDTH	Write address (unused for CRC)
s_axi_awlen	input	8	Burst length minus 1
s_axi_awsize	input	3	Burst size
s_axi_awburst	input	2	Burst type
s_axi_awlock	input	1	Lock type
s_axi_awcache	input	4	Cache attributes
s_axi_awprot	input	3	Protection attributes
s_axi_awqos	input	4	Quality of service

Port	Direction	Width	Description
os			
s_axi_awregion	input	4	Region identifier
s_axi_awuser	input	AXI_USER_WIDTH	User sideband (echoed on B)
s_axi_awvalid	input	1	AW valid
s_axi_awready	output	1	AW ready

17.4.5 AXI4 Write Data Channel (W)

Port	Direction	Width	Description
s_axi_wdata	input	AXI_DATA_WIDTH	Write data (the selected 32-bit slice is CRC'd)
s_axi_wstrb	input	AXI_DATA_WIDTH/8	Write byte strobes (not used to gate the CRC)
s_axi_wlast	input	1	Last beat of burst
s_axi_wuser	input	AXI_USER_WIDTH	User sideband
s_axi_wvalid	input	1	W valid
s_axi_wready	output	1	W ready

17.4.6 AXI4 Write Response Channel (B)

Port	Direction	Width	Description
s_axi_bid	output	AXI_ID_WIDTH	Response ID (separately latched at WLAST)
s_axi_bresp	output	2	Response, always OKAY
s_axi_buser	output	AXI_USER_WIDTH	User sideband
s_axi_bvalid	output	1	B valid

Port	Direction	Width	Description
s_axi_bready	input	1	B ready

17.4.7 Status

Port	Direction	Width	Description
busy	output	1	Protocol-handler busy indicator (from axi4_slave_wr)

17.5 Functional Description

17.5.1 Data Slice Extraction

A wide AXI bus carries several 32-bit lanes; only one is CRC'd so the CRC stays interchangeable with the 32-bit read generator. A generate block assigns `data_slice` directly when the bus is 32 bits wide, and otherwise selects `wdata[CRC_SLICE_OFFSET*32 +: 32]`. A compile-time assertion rejects a `CRC_SLICE_OFFSET` that would read past the end of the bus.

17.5.2 Per-Channel CRC-32

Each channel owns a `dataint_crc` instance (CRC-32/Ethernet, `REFIN/REFOUT = 1`, `cascade_sel = 4'b1000`) fed the 32-bit `data_slice`. The channel selector `w_active_ch` is the low CIW bits of the captured `awid` (`r_wr_id`) — because the AW FSM accepts one burst at a time and W is in-order with AW, the active burst's id identifies the channel during its W beats. A CRC absorbs one word per accepted W beat for its channel (`ch_load_from_cascade = "accepted W beat AND active channel == this channel"`). Per-channel valid flags and beat counters track alongside; `crc_reset` clears them all. `write_beat_count_total` is a combinational sum across channels for the harness stop trigger.

17.5.3 Write Burst FSM and Gapless Bursts

A compact FSM tracks a single active burst via `r_wr_active`. `awready` asserts when not active *or* on the last W beat of the current burst (`w_wr_last_beat`), letting the next burst's AW be accepted back-to-back so `wready` (`= r_wr_active`) never drops between bursts. This mirrors the read-slave gapless fix — without it, a one-cycle !active gap per burst would become the write-side throughput limiter once the read slave feeds data gaplessly.

17.5.4 Back-to-Back B Response Id Latching

When bursts run back-to-back, `r_wr_id` is reloaded with the *next* burst's id on the same cycle the current burst's WLAST lands. The completing burst's B response would then carry the wrong id. To avoid that, the id/user are separately latched into `r_b_id` / `r_b_user` at WLAST and those

drive the B channel. B is single-outstanding (r_b_pending), which is safe because the STREAM master drains B within the burst period; a higher-rate multi-channel sink would need a B FIFO (noted in the RTL).

17.5.5 Standard Protocol Handler

AW/W/B skid buffering and AXI compliance are delegated to a `axi4_slave_wr` instance; the CRC and FSM logic ride on its FUB-side (`fub_axi_aw*` / `fub_axi_w*` / `fub_axi_b*`) interface. (A header TODO tracks a future refactor to wrap the monitored `axi4_slave_wr_mon` — task #79.)

17.6 Usage Example

// Synthetic write sink for a 4-channel DMA characterization run.

```
axi4_slave_wr_crc_check #(
    .NUM_CHANNELS    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .CRC_SLICE_OFFSET(0)
) u_wr_sink (
    .aclk              (aclk),
    .aresetn           (aresetn),
    .crc_reset         (csr_clear_pulse),
```

// Telemetry to the harness CSR block

```
.write_crc_value      (wr_crc_value),
.write_crc_valid       (wr_crc_valid),
.write_beat_count      (wr_beat_count),
.write_beat_count_total (wr_beats_total),
```

// AW/W/B wired to the DUT's write master port

```
.s_axi_awid    (dut_awid),  .s_axi_awaddr  (dut_awaddr),
.s_axi_awlen   (dut_awlen), .s_axi_awsz   (dut_awsz),
.s_axi_awburst (dut_awburst), .s_axi_awlock  (dut_awlock),
.s_axi_awcache (dut_awcache), .s_axi_awprot  (dut_awprot),
.s_axi_awqos   (dut_awqos), .s_axi_awregion (dut_awregion),
.s_axi_awuser  (dut_awuser), .s_axi_awvalid  (dut_awvalid),
.s_axi_awready (dut_awready),
```

```
.s_axi_wdata (dut_wdata), .s_axi_wstrb (dut_wstrb),
.s_axi_wlast (dut_wlast), .s_axi_wuser  (dut_wuser),
.s_axi_wvalid (dut_wvalid), .s_axi_wready (dut_wready),
```

```
.s_axi_bid    (dut_bid),  .s_axi_bresp  (dut_bresp),
.s_axi_buser  (dut_buser), .s_axi_bvalid  (dut_bvalid),
```

```
.s_axi_bready(dut_bready),  
  
.busy (wr_busy)  
);
```

17.7 Design Notes

- **CRC config must match the source:** the whole point is a comparable value. CRC_POLY, CRC_INIT, CRC_XOROUT, CRC_REFIN, CRC_REFOUT, and the 32-bit slice width are all fixed to mirror axi4_slave_rd_pattern_gen.
 - **Channel demux by AW id, not W sideband:** because W is in-order behind AW and only one burst is active at a time, the captured awid unambiguously names the channel for that burst's W beats.
 - **Single-outstanding B is deliberate but bounded:** it assumes the master drains B within a burst period. The RTL explicitly flags that a faster multi-channel sink needs a B FIFO.
 - **Gapless accept is a measurement fix:** the back-to-back AW accept prevents the sink from injecting a false ~1-cycle-per-burst starvation into write-side utilization numbers.
 - **Standards note:** the AW/W/B glue is hand-rolled on top of axi4_slave_wr; a future refactor to axi4_slave_wr_mon is tracked as task #79.
-

17.8 Related Modules

17.8.1 Used By

- axi4_dma_slaves — bundles this write sink with axi4_slave_rd_pattern_gen into a single source/sink slave pair
- projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv — on-chip write sink
- projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv — on-chip write sink

17.8.2 Uses

- **axi4_slave_wr.sv** — standard AXI4 write slave protocol handler (AW/W/B skid + compliance)
- **dataint_crc.sv** — per-channel CRC-32 accumulator

17.8.3 See Also

- **axi4_slave_rd_pattern_gen.sv** — the matching read-side pattern source (same CRC config)
 - **axi4_dma_slaves.sv** — the source/sink bundle
 - **axis4_slave_pattern_check.sv** — AXIS equivalent checker
-

17.9 References

17.9.1 Source Code

- RTL: rtl/amba/shared/axi4_slave_wr_crc_check.sv
- Protocol Handler: rtl/amba/axi4/axi4_slave_wr.sv

17.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Harness: projects/NexysA7/ddr2-characterization/README.md
-

Last Updated: 2026-07-15

17.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

18 AXI Bus Meter

Module: axi_bus_meter.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

18.1 Overview

The AXI Bus Meter is a per-cycle valid/ready classifier for a single AXI data channel. Every clock cycle it inspects the valid and ready handshake pair and attributes that cycle to one of four buckets — productive, backpressure, starvation, or idle — accumulating both aggregate (32-bit)

and per-channel (16-bit) counts. It is a pure passive observer that drives nothing back onto the bus, making it safe to drop onto any live AXI R or W channel to characterize datapath utilization.

18.1.1 Key Features

- Four-bucket per-cycle cycle classification (productive / backpressure / starvation / idle)
 - Aggregate 32-bit counters (~42.9 s at 100 MHz before wrap) — always attributed based on bus state
 - Per-channel 16-bit counters, NUM_CHANNELS deep, binned by a caller-supplied channel id
 - Per-channel 4-bit sticky overflow mask flags counters that wrapped
 - Synchronous one-cycle `i_clear` for atomic reset with the surrounding measurement substrate
 - `i_freeze` window control to close the measurement window the instant the workload finishes
 - Passive snoop — no protocol interaction, zero backpressure onto the metered bus
-

18.2 Module Purpose

AXI datapath performance is dominated by how often data actually moves versus how often a handshake stalls. A raw throughput number cannot distinguish a slow producer (master not offering data) from a congested consumer (slave withholding ready). The AXI Bus Meter separates these root causes by classifying every cycle into one of four mutually-exclusive buckets, so a post-run analysis can compute utilization and, more importantly, attribute lost cycles to the correct side of the interface.

The block is instantiated one per bus to be measured. On a read engine it drops onto the R channel (aggregate plus per-channel via `rid`); on a write engine it drops onto the W channel (aggregate plus per-channel via an engine-side sideband, since AXI4 W beats carry no id).

Use Cases: - Metering read R-bus and write W-bus utilization on the stream/rapids characterization engines - Root-causing throughput shortfalls (backpressure-bound vs starvation-bound datapaths) - Per-channel utilization breakdown in multi-channel DMA engines - On-silicon performance characterization runs driven from a host CSR interface

Key Benefit: Turns a single throughput figure into an attributable four-way cycle budget, so lost bandwidth can be pinned to producer starvation or consumer backpressure per channel.

18.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of per-channel bins (depth of the per-channel counter arrays)
CW	int	$(\text{NUM_CHANNELS} > 1) ? \lceil \log_2(\text{NUM_CHANNELS}) \rceil : 1$	Derived channel-id width; do not override

18.4 Port Groups

18.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

18.4.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	Synchronous one-cycle clear pulse; zeroes all counters and overflow stickies
i_freeze	input	1	When high, every counter and overflow sticky holds its value (window closed)

18.4.3 AXI Bus Snoop

Port	Direction	Width	Description
i_valid	input	1	Snooped channel valid (e.g. m_axi_rvalid / m_axi_wvalid)
i_ready	input	1	Snooped channel ready

Port	Direction	Width	Description
i_channel_id	input	CW	(e.g. m_axi_rready / m_axi_wready) Channel index selecting the per-channel bin to increment this cycle
i_channel_valid	input	1	Gates per-channel accumulation; high when a channel is on the hook this cycle

18.4.4 Aggregate Counters

Port	Direction	Width	Description
o_agg_productive	output	32	Cycles with valid && ready (data delivered)
o_agg_backpressure	output	32	Cycles with valid && !ready (master offering, slave stalling)
o_agg_starvation	output	32	Cycles with !valid && ready (slave ready, master not producing)
o_agg_idle	output	32	Cycles with !valid && !ready (both sides quiet)

18.4.5 Per-Channel Counters

Port	Direction	Width	Description
o_ch_productive	output	16 × NUM_CHANNELS	Per-channel productive-cycle counts
o_ch_backpressure	output	16 × NUM_CHANNELS	Per-channel backpressure-cycle counts
o_ch_starvation	output	16 × NUM_CHANNELS	Per-channel starvation-cycle counts

Port	Direction	Width	Description
o_ch_idle	output	16 × NUM_CHAN NELS	Per-channel idle-cycle counts
o_ch_overflow	output	NUM_CHAN NELS*4	Sticky overflow mask, packed {prod, bp, starv, idle} per channel

18.5 Functional Description

18.5.1 Bucket Classification

The four buckets are decoded combinationally from the handshake pair and are mutually exclusive — exactly one is asserted every cycle:

```

w_prod = i_valid && i_ready // productive  – data delivered
w_bp   = i_valid && !i_ready // backpressure – master wants to
send, slave stalls
w_starv = !i_valid && i_ready // starvation  – slave ready, master
not producing
w_idle  = !i_valid && !i_ready // idle       – both sides quiet

```

This is the reference methodology documented in `DMA_UTILIZATION_MEASUREMENT.md` §3.

18.5.2 Aggregate Counters

The four 32-bit aggregate counters always increment based on the raw bus state (subject only to `i_clear` and `i_freeze`). At 100 MHz a 32-bit counter lasts ~42.9 s before overflow — ample for a single characterization run. These provide the whole-bus utilization figure regardless of which channel was active.

18.5.3 Per-Channel Counters and Overflow Stickies

The per-channel arrays are 16 bits wide and `NUM_CHANNELS` deep. Only the bin selected by `i_channel_id` is incremented, and only when `i_channel_valid` is high. The caller asserts `i_channel_valid` whenever some channel is on the hook for the current cycle (for example mid-burst); for cycles where the engine has no work on any channel, `i_channel_valid` is left low and only the aggregate idle/starvation counts are attributed.

Because 16 bits wraps after 65 536 cycles (~655 μs at 100 MHz), each per-channel counter has a companion sticky overflow bit. When a counter is at 16'hFFFF and would increment again, its overflow bit latches high. The four bits per channel are packed {prod, bp, starv, idle} into `o_ch_overflow`. Software checks this mask after stopping the meter; any set bit means the burst

outran 16-bit per-channel resolution and the per-channel numbers for that channel should be discarded (the aggregate 32-bit figures remain valid).

18.5.4 Recommended Wiring

- **Read engine:** `i_valid = m_axi_rvalid`, `i_ready = m_axi_rready`,
`i_channel_id = m_axi_rid[CW-1:0]`, `i_channel_valid = m_axi_rvalid`
(rid is only meaningful when rvalid is high).
- **Write engine:** `i_valid = m_axi_wvalid`, `i_ready = m_axi_wready`,
`i_channel_id =` the engine's internal write-channel-id sideband,
`i_channel_valid =` a "W beats in flight on this channel" indicator (W beats carry no id in AXI4).

18.5.5 Clear and Freeze Semantics

`i_clear` is a synchronous one-cycle pulse: on the cycle it is high, all counters and all overflow stickies reset to zero. It is intended to be wired to the same control bit that clears the surrounding measurement state (debug SRAM, harness CRC) so a single CSR write atomically resets the entire measurement substrate.

`i_freeze` holds every counter and sticky frozen while high — no bucket increments, no overflow flips. It is driven from the characterization timer's done so the window closes the moment the workload finishes. Without it, the lifetime starvation count would drift upward at one bit per cycle during post-burst host polling, contaminating the in-window utilization math. Hold `i_freeze` low for unbounded free-running measurement.

18.6 Usage Example

```
// Meter the read engine's R channel: aggregate + per-channel by rid.
axi_bus_meter #(
    .NUM_CHANNELS    (8)
) u_r_meter (
    .aclk             (aclk),
    .aresetn          (aresetn),

    // Window control (share with the char timer + CRC clear)
    .i_clear          (perf_run_rising),    // one-cycle pulse on RUN
    rising edge
    .i_freeze         (~perf_run),          // freeze when the window
    is closed

    // R channel snoop
    .i_valid          (m_axi_rvalid),
    .i_ready          (m_axi_rready),
```

```

        .i_channel_id      (m_axi_rid[2:0]),
        .i_channel_valid   (m_axi_rvalid),      // rid valid only when
rvalid

        // Aggregate readback (to CSR / host)
        .o_agg_productive   (r_agg_prod),
        .o_agg_backpressure (r_agg_bp),
        .o_agg_starvation   (r_agg_starv),
        .o_agg_idle         (r_agg_idle),

        // Per-channel readback
        .o_ch_productive    (r_ch_prod),
        .o_ch_backpressure  (r_ch_bp),
        .o_ch_starvation    (r_ch_starv),
        .o_ch_idle          (r_ch_idle),
        .o_ch_overflow       (r_ch_overflow)
    );

    // Utilization = productive / (productive + backpressure + starvation
+ idle)
    // Backpressure-heavy => consumer bound; starvation-heavy => producer
bound.

```

18.7 Design Notes

18.7.1 Passive Observer

The meter drives nothing back onto the AXI bus — it only reads `valid/ready`. It can be added to or removed from a design without any protocol impact, and multiple meters can snoop different channels independently.

18.7.2 Counter Width Rationale

Aggregate counters are 32-bit because they must survive a full-length run (~42.9 s at 100 MHz). Per-channel counters are deliberately kept at 16 bits to bound area across `NUM_CHANNELS` bins; the sticky overflow mask makes the resulting ~655 μ s resolution limit self-diagnosing rather than silently wrong.

18.7.3 Verilator Output Drivers

The per-channel register arrays are copied to the unpacked output arrays through explicit `always_comb` loops, and the overflow stickies are packed into `o_ch_overflow` with a `c*4 +: 4` slice, because Verilator cannot infer unpacked-array output assignments directly from a register array.

18.7.4 Window Alignment

For correct in-window math, drive `i_clear` and `i_freeze` from the same window controller used by the companion `axi_perf_latency_hist` block so all meters open and close on the same cycle boundaries.

18.8 Related Modules

18.8.1 Used By

- `stream_char` datapath utilization metering (read R bus, write W bus)
- `rapids_char` AXI datapath meters (see `projects/NexysA7/.../rapids_char_harness.sv`)
- Host-driven on-silicon performance characterization harnesses

18.8.2 Uses

- `reset_defs.svh` - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

18.8.3 See Also

- `axis_bus_meter.sv` - AXIS analogue with byte/packet throughput counters
 - `axi_perf_latency_hist.sv` - Per-channel latency histogram, same window-control convention
-

18.9 References

18.9.1 Source Code

- RTL: `rtl/amba/shared/axi_bus_meter.sv`

18.9.2 Documentation

- Methodology: `docs/.../DMA_UTILIZATION_MEASUREMENT.md` (window semantics, four-bucket definition)
 - Architecture: `docs/markdown/RTLAmba/shared/README.md`
 - Design Guide: `rtl/amba/PRD.md`
-

Last Updated: 2026-07-15

18.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

19 AXI Burst Address Generator

Module: `axi_gen_addr.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

19.1 Overview

The AXI Burst Address Generator calculates the next address for AXI burst transactions, supporting FIXED, INCR, and WRAP burst types. It handles data width conversions and provides both next address and aligned address outputs for boundary-aware transaction processing.

19.1.1 Key Features

- Support for all AXI burst types (FIXED, INCR, WRAP)
- Data width conversion handling (DW != ODW)
- Next address calculation with proper increment
- Aligned address generation for boundary checking
- Pure combinational logic for zero-latency operation

19.2 Parameters

Parameter	Type	Default	Description
AW	int	32	Address bus width
DW	int	32	Input data width (internal)
ODW	int	32	Output data width (external bus)
LEN	int	8	Burst length field width

19.3 Ports

Port	Direction	Width	Description
curr_addr	input	AW	Current address
size	input	3	AXI size encoding (2 ^{size} bytes per beat)
burst	input	2	AXI burst type (00=FIXED, 01=INCR, 10=WRAP)
len	input	LEN	AXI burst length (beats - 1)
next_addr	output	AW	Next transaction address
next_addr_align	output	AW	Next address aligned to ODW

19.4 Functional Description

19.4.1 Increment Calculation

The address increment is determined by the size field:

```
increment_pre = (1 << size) // 2size bytes per beat
```

When data widths differ (DW != ODW), the increment is clamped:

```
if (increment_pre > ODWBYTES)
    increment = ODWBYTES;
```

19.4.2 Burst Type Handling

FIXED Burst (burst = 2'b00): - Address remains constant for all beats - next_addr = curr_addr

INCR Burst (burst = 2'b01): - Address increments by size each beat - $\text{next_addr} = \text{curr_addr} + \text{increment}$

WRAP Burst (burst = 2'b10): - Address wraps at burst boundary - Wrap mask = $(1 \ll (\text{size} + \log_2(\text{len}+1))) - 1$ - Aligned address = $(\text{curr_addr} + \text{increment}) \& \sim(\text{increment} - 1)$ - Wrap address = $(\text{curr_addr} \& \sim\text{wrap_mask}) | (\text{aligned_addr} \& \text{wrap_mask})$

19.4.3 Aligned Address Output

The aligned address output aligns the next address to the output data width:

```
alignment_mask = ODWBYTES - 1
next_addr_align = next_addr & ~alignment_mask
```

This is useful for boundary crossing detection in transaction splitters.

19.5 Usage Example

```
// Generate next address for AXI4 64-bit bus
axi_gen_addr #(
    .AW    (32),
    .DW    (64),    // Internal 64-bit
    .ODW   (64),    // External 64-bit
    .LEN   (8)
) u_addr_gen (
    .curr_addr    (32'h1000),
    .size         (3'b011),    // 8 bytes per beat
    .burst        (2'b01),    // INCR
    .len          (8'd3),    // 4 beats total
    .next_addr    (addr_next),
    .next_addr_align (addr_aligned)
);

// addr_next = 0x1000 + 8 = 0x1008
// addr_aligned = 0x1008 (already aligned to 8-byte boundary)
```

19.6 Design Notes

19.6.1 Data Width Conversion

When $\text{DW} \neq \text{ODW}$, the module handles width mismatches: - If $\text{increment} > \text{output bus bytes}$, clamp to output bus width - This prevents address increments larger than the physical bus

19.6.2 WRAP Burst Alignment

WRAP bursts require careful address calculation: - Wrap boundary determined by size and length
- Wrapped address stays within the wrap region - Example: 4-beat wrap at 0xFF8 wraps back to 0xFF8 (not 0x1000)

19.6.3 Pure Combinational

This module is purely combinational for zero latency: - No clock or reset - Instant address generation - Use in address calculation pipelines

19.7 Related Modules

19.7.1 Used By

- `axi_master_rd_splitter.sv` (boundary crossing detection)
- `axi_master_wr_splitter.sv` (boundary crossing detection)
- Address generation pipelines

19.7.2 Related Infrastructure

- `axi_split_combi.sv` (uses aligned addresses for split decisions)
-

19.8 References

19.8.1 Specifications

- ARM IHI 0022E: AMBA AXI4 Protocol Specification (Section A3.4 - Address Structure)
- Internal: `rtl/amba/PRD.md`

19.8.2 Source Code

- RTL: `rtl/amba/shared/axi_gen_addr.sv`
-

Last Updated: 2025-10-24

19.9 Navigation

- [Back to Shared Infrastructure Index](#)

- [Back to RTLamba Index](#)

20 AXI Master Read Splitter

Module: axi_master_rd_splitter.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

20.1 Overview

The AXI Master Read Splitter is a critical infrastructure module that splits single upstream AXI read transactions into multiple downstream transactions when boundary crossings occur. It enables address range interleaving, memory-mapped I/O access, and heterogeneous memory system integration while maintaining full AXI4 protocol compliance.

20.1.1 Key Features

- Automatic boundary crossing detection and transaction splitting
 - Configurable alignment boundary (4KB, 64KB, etc.)
 - AR channel forwarding with address/length calculation
 - R channel pass-through with transparent ID handling
 - Split transaction tracking via dedicated FIFO interface
 - Zero added latency for non-split transactions
 - Full AXI4 protocol compliance (burst, ID, user fields)
-

20.2 Module Purpose

In modern SoC designs, memory systems are often partitioned across multiple address ranges, memory controllers, or protection domains. The AXI Master Read Splitter solves the fundamental problem of handling transactions that cross these architectural boundaries.

Problem Statement: - Upstream master issues single AXI read (ADDR=0x0FC0, LEN=7, 8 beats total) - Transaction crosses 4KB boundary at 0x1000 - Downstream slaves require separate transactions per boundary region

Solution: - Module detects boundary crossing using combinational split logic - Automatically generates N split transactions (where $N \geq 1$) - First split: ADDR=0x0FC0, LEN=0 (1 beat to boundary) - Second split: ADDR=0x1000, LEN=6 (7 beats remaining) - All response data aggregated transparently

Key Invariants: - Upstream sees exactly ONE transaction request/acceptance - Downstream sees N split transactions (boundary-aligned) - Total response beats equals original transaction beat count - All AXI signaling (ID, USER, PROT, QOS) preserved across splits

20.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Width of AXI ID signals (ARID, RID)
AXI_ADDR_WIDTH	int	32	Width of AXI address bus (ARADDR)
AXI_DATA_WIDTH	int	32	Width of AXI data bus (RDATA), must be power of 2 (32-1024 bits)
AXI_USER_WIDTH	int	1	Width of AXI user extension fields (ARUSER, RUSER)
SPLIT_FIFO_DEPTH	int	4	Depth of split information tracking FIFO
IW	int	AXI_ID_WIDTH	Shorthand alias for ID width
AW	int	AXI_ADDR_WIDTH	Shorthand alias for address width
DW	int	AXI_DATA_WIDTH	Shorthand alias for data width
UW	int	AXI_USER_WIDTH	Shorthand alias for user width

20.4 Port Groups

20.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Global clock for all AXI interfaces
aresetn	input	1	Active-low

Port	Direction	Width	Description
			asynchronous reset

20.4.2 Configuration Interface

Port	Direction	Width	Description
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB boundaries)
block_read_y	input	1	Backpressure from error monitor (suppresses fub_arready when asserted)

20.4.3 Master AXI Read Interface (Downstream)

AR Channel (Address Read):

Port	Direction	Width	Description
m_axi_arid	output	IW	Transaction ID for split read transactions
m_axi_ara_ddr	output	AW	Calculated address for current split (aligned to boundaries)
m_axi_arlen	output	8	Calculated burst length for current split (AXI encoding: beats-1)
m_axi_arsize	output	3	Transfer size (preserved from original transaction)
m_axi_arburst	output	2	Burst type (preserved from original, typically INCR)
m_axi_arlock	output	1	Lock type (preserved from original)
m_axi_arcache	output	4	Cache attributes (preserved from original)
m_axi_arprot	output	3	Protection attributes

Port	Direction	Width	Description
rot			(preserved from original)
m_axi_arqos	output	4	Quality of Service (preserved from original)
m_axi_arregion	output	4	Region identifier (preserved from original)
m_axi_aruser	output	UW	User-defined extension (preserved from original)
m_axi_arvalid	output	1	Valid signal for current split transaction
m_axi_arready	input	1	Ready signal from downstream slave

R Channel (Read Data):

Port	Direction	Width	Description
m_axi_rid	input	IW	Response transaction ID (matches ARID)
m_axi_rdata	input	DW	Read data payload
m_axi_rresp	input	2	Response status (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_rlast	input	1	Last beat indicator for current split
m_axi_ruser	input	UW	User-defined extension for response
m_axi_rvalid	input	1	Valid signal for response data
m_axi_rready	output	1	Ready signal to downstream slave (pass-through from fub)

20.4.4 Slave AXI Read Interface (Upstream / FUB)

AR Channel (Address Read):

Port	Direction	Width	Description
fub_arid	input	IW	Original transaction ID from upstream master
fub_araddr	input	AW	Original starting address (may cross boundaries)
fub_arlen	input	8	Original burst length (may span multiple boundaries)
fub_arsize	input	3	Transfer size per beat
fub_arburst	input	2	Burst type (FIXED, INCR, WRAP)
fub_arlock	input	1	Lock type for atomic operations
fub_arcache	input	4	Cache attributes
fub_arprot	input	3	Protection attributes
fub_arqos	input	4	Quality of Service priority
fub_arregion	input	4	Region identifier
fub_aruser	input	UW	User-defined extension
fub_arvalid	input	1	Valid signal for original transaction
fub_arready	output	1	Ready signal to upstream master (suppressed until all splits complete)

R Channel (Read Data):

Port	Direction	Width	Description
fub_rid	output	IW	Response transaction ID (pass-through from m_axi)
fub_rdata	output	DW	Read data payload (pass-through from m_axi)
fub_rresp	output	2	Response status (pass-through from m_axi)
fub_rlast	output	1	Last beat indicator (pass-

Port	Direction	Width	Description
fub_ruser	output	UW	through from m_axi) User-defined extension (pass-through from m_axi)
fub_rvalid	output	1	Valid signal for response (pass-through from m_axi)
fub_rready	input	1	Ready signal from upstream master

20.4.5 Split Information Interface

Port	Direction	Width	Description
fub_split_a ddr	output	AW	Original transaction starting address
fub_split_i d	output	IW	Original transaction ID
fub_split_c nt	output	8	Number of split transactions generated (2+ if split occurred)
fub_split_v alid	output	1	Valid signal for split information
fub_split_r eady	input	1	Ready signal from split information consumer

20.5 Functional Description

20.5.1 Transaction Splitting Overview

The module operates as a transparent pass-through for non-split transactions and as an intelligent splitter for boundary-crossing transactions.

Non-Split Path (Fast): - Transaction fully contained within single boundary region - AR signals pass directly from fub to m_axi - fub_arready = m_axi_arready (immediate acceptance) - R channel completely transparent - Split count = 1

Split Path (Multi-Transaction): - Transaction crosses one or more boundary regions - AR signals buffered and regenerated for each split - fub_arready suppressed until final split accepted - R channel remains transparent (ID-based aggregation) - Split count = N (number of boundary crossings + 1)

20.5.2 State Machine Architecture

The module implements a simple two-state FSM:

IDLE State: - Awaits new transactions on fub_ar interface - Combinational split logic evaluates boundary crossing - If no split: immediate pass-through with fub_arready = m_axi_arready - If split: buffer transaction, transition to SPLITTING, suppress fub_arready

SPLITTING State: - Issue current split using buffered address/length - Update next_addr and remaining_len for subsequent split - If more splits needed: stay in SPLITTING, increment split_count - If final split: assert fub_arready when accepted, return to IDLE

20.5.3 Boundary Crossing Detection

Combinational logic module (axi_split_combi) provides real-time splitting decisions:

Inputs: - current_addr: Transaction start address - current_len: Total beats remaining (AXI encoding) - ax_size: Bytes per beat - alignment_mask: Boundary size (e.g., 0xFFF for 4KB)

Outputs: - split_required: Transaction crosses boundary - split_len: Beats to consume in current split (AXI encoding) - next_boundary_addr: Address at next boundary alignment - remaining_len_after_split: Beats remaining after current split

Key Calculation:

```
next_boundary_addr = (current_addr | alignment_mask) + 1
bytes_to_boundary = next_boundary_addr - current_addr
beats_to_boundary = bytes_to_boundary >> ax_size
split_len = beats_to_boundary - 1 // AXI encoding
```

20.5.4 AR Channel Forwarding Logic

Address/Length Calculation: - IDLE state: Use live fub_araddr and calculated split_len - SPLITTING state: Use buffered r_current_addr and split_len - Always output minimum of (split_len, current_len) for safety

Control Signals (ID, PROT, QOS, etc.): - IDLE state: Pass through from fub_ar* signals - SPLITTING state: Use buffered r_orig_ar* signals (captured at acceptance) - Ensures all split transactions carry identical attributes

Valid Signal Generation: - IDLE: m_axi_arvalid = fub_arvalid - SPLITTING: m_axi_arvalid = 1 (always asserting next split)

20.5.5 Ready Signal Management

Critical Protocol Constraint: - fub_arready must NOT assert until entire original transaction accepted - Upstream master expects single handshake for its request

Ready Logic: - IDLE + no split: fub_arready = m_axi_arready (immediate) - IDLE + split needed: fub_arready = 0 (suppress until done) - SPLITTING + intermediate: fub_arready = 0 - SPLITTING + final split: fub_arready = m_axi_arready (complete handshake)

20.5.6 R Channel Handling

The R channel is completely transparent:

Pass-Through Signals: - fub_rid = m_axi_rid - fub_rdata = m_axi_rdata - fub_rresp = m_axi_rresp - fub_rlast = m_axi_rlast - fub_ruser = m_axi_ruser - fub_rvalid = m_axi_rvalid - m_axi_rready = fub_rready

Why This Works: - Split transactions use same ID as original - Downstream slaves respond with matching ID - Upstream master sees aggregated response stream - Total beats = sum of all split lengths - Final RLAST indicates completion

20.5.7 Split Information Tracking

A dedicated FIFO captures metadata for each completed transaction:

FIFO Write Conditions: - Written when fub_arvalid && fub_arready (transaction accepted) - Contains: {original_addr, transaction_id, split_count}

Split Count Values: - No split: split_cnt = 1 - Split occurred: split_cnt = N (number of splits generated)

Use Cases: - Performance monitoring (detect excessive splitting) - Error correlation (map responses to original requests) - Debug visibility (trace transaction decomposition)

20.6 Usage Example

20.6.1 Basic Integration (4KB Boundary Alignment)

```
axi_master_rd_splitter #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .AXI_USER_WIDTH    (1),
    .SPLIT_FIFO_DEPTH  (16)
) u_rd_splitter (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // 4KB boundary alignment
    .alignment_mask     (12'hFFF),

    // Error monitor backpressure
```

```

.block_ready      (error_detected),

// Upstream AXI read interface (from master)
.fub_arid         (master_arid),
.fub_araddr       (master_araddr),
.fub_arlen        (master_arlen),
.fub_arsize       (master_arsize),
.fub_arburst      (master_arburst),
.fub_arlock       (master_arlock),
.fub_arcache      (master_arcache),
.fub_arprot       (master_arprot),
.fub_arqos        (master_arqos),
.fub_arregion     (master_arregion),
.fub_aruser       (master_aruser),
.fub_arvalid      (master_arvalid),
.fub_arready      (master_arready),

.fub_rid         (master_rid),
.fub_rdata        (master_rdata),
.fub_rresp        (master_rresp),
.fub_rlast        (master_rlast),
.fub_ruser        (master_ruser),
.fub_rvalid       (master_rvalid),
.fub_rready       (master_rready),

// Downstream AXI read interface (to memory system)
.m_axi_arid       (mem_arid),
.m_axi_araddr     (mem_araddr),
.m_axi_arlen      (mem_arlen),
.m_axi_arsize     (mem_arsize),
.m_axi_arburst    (mem_arburst),
.m_axi_arlock     (mem_arlock),
.m_axi_arcache    (mem_arcache),
.m_axi_arprot     (mem_arprot),
.m_axi_arqos      (mem_arqos),
.m_axi_arregion   (mem_arregion),
.m_axi_aruser     (mem_aruser),
.m_axi_arvalid    (mem_arvalid),
.m_axi_arready    (mem_arready),

.m_axi_rid       (mem_rid),
.m_axi_rdata     (mem_rdata),
.m_axi_rresp     (mem_rresp),
.m_axi_rlast     (mem_rlast),
.m_axi_ruser     (mem_ruser),
.m_axi_rvalid    (mem_rvalid),
.m_axi_rready    (mem_rready),

```



```

    // Split information for monitoring
    .fub_split_addr      (split_addr),
    .fub_split_id        (split_id),
    .fub_split_cnt       (split_count),
    .fub_split_valid     (split_valid),
    .fub_split_ready     (split_ready)
);

// Connect split information to monitoring infrastructure
gaxi_fifo_sync #(
    .DATA_WIDTH          (32 + 8 + 8), // addr + id + count
    .DEPTH               (64)
) u_split_fifo (
    .axi_aclk             (axi_clk),
    .axi_aresetn          (axi_rst_n),
    .wr_valid             (split_valid),
    .wr_data              ({split_addr, split_id, split_count}),
    .wr_ready             (split_ready),
    .rd_valid             (monitor_split_valid),
    .rd_data              (monitor_split_data),
    .rd_ready             (monitor_split_ready),
    .count                (split_fifo_level)
);

```

20.6.2 Transaction Splitting Example Scenario

Original Transaction: - Address: 0x0FC0 (4032 bytes into 4KB region) - Length: 7 (8 beats total in AXI encoding) - Size: 3'b011 (8 bytes per beat = 64 bits) - Total bytes: 8 beats × 8 bytes = 64 bytes - End address: 0x0FC0 + 64 - 1 = 0x0FFF

Boundary Analysis: - 4KB boundary at 0x1000 - Bytes to boundary: 0x1000 - 0x0FC0 = 64 bytes - Beats to boundary: 64 / 8 = 8 beats - Transaction crosses boundary: NO (ends exactly at boundary) - Split required: NO

Modified Scenario (Crosses Boundary): - Address: 0x0FC0 - Length: 8 (9 beats) - Total bytes: 9 × 8 = 72 bytes - End address: 0x0FC0 + 72 - 1 = 0x1007

Split Sequence: 1. First split: ADDR=0x0FC0, LEN=7 (8 beats to reach 0x1000) 2. Second split: ADDR=0x1000, LEN=0 (1 remaining beat) 3. Split count = 2

20.7 Design Notes

20.7.1 AXI Protocol Assumptions

The module enforces several critical assumptions for correct operation:

Assumption 1: Address Alignment - All AXI addresses aligned to data bus width - If DATA_WIDTH = 512 bits, ARADDR always 64-byte aligned - Verified by assertions in axi_split_combi

Assumption 2: Fixed Transfer Size - All transfers use maximum size equal to bus width - ARSIZE always matches $\log_2(\text{DATA_WIDTH}/8)$ - Example: 64-bit bus → ARSIZE = 3'b011 (8 bytes)

Assumption 3: Incrementing Bursts Only - Only INCR burst type supported (ARBURST = 2'b01) - No FIXED or WRAP burst handling - Simplifies address calculation logic

Assumption 4: No Address Wraparound - Transactions never wrap 0xFFFFFFFF → 0x00000000 - Guaranteed by system software/memory allocators - Enables simplified boundary crossing logic

20.7.2 Performance Characteristics

Latency: - No split: Zero added cycles (combinational pass-through) - Split required: 1 cycle per split transaction - Example: 3-way split adds 2 cycles total latency

Throughput: - Full AXI bandwidth for non-split traffic - Sustained split traffic: Limited by split calculation (1 cycle/split) - R channel: Full bandwidth (pass-through)

Area: - Minimal overhead: State machine + transaction buffer - Major component: Split information FIFO - Configurable via SPLIT_FIFO_DEPTH parameter

20.7.3 Error Handling

Transaction Table Exhaustion: - Split FIFO can fill if consumer stalls - Module continues operation (FIFO write stalls fub_arready) - Recommendation: Size FIFO $\geq$ max outstanding transactions

Protocol Violations: - Module assumes well-formed AXI transactions - Assertions validate assumptions (alignment, size, burst type) - Synthesis-time checks enforce parameter constraints

Backpressure Propagation: - block_ready input suppresses fub_arready - Enables error monitor integration - Prevents new transactions during error conditions

20.7.4 Integration Considerations

Alignment Mask Configuration: - 12-bit mask supports boundaries: 4KB to 8MB - Common values: 0xFFF (4KB), 0xFFFF (64KB) - Must match downstream memory region alignment

FIFO Sizing: - SPLIT_FIFO_DEPTH should accommodate burst traffic - Typical value: 16 entries for general-purpose use - High-performance: 32-64 entries

Clock Domain Constraints: - Single clock domain (aclk) for all interfaces - No CDC required within module - Ensure timing closure on split calculation path

20.8 Related Modules

20.8.1 Used By

- Memory controller wrappers requiring boundary-aligned transactions
- Multi-bank DRAM controllers with interleaving requirements
- Bridge modules interfacing heterogeneous memory regions
- DMA engines with address range constraints

20.8.2 Uses

- **axi_split_combi.sv** - Combinational splitting logic (boundary detection, length calculation)
- **gaxi_fifo_sync.sv** - Split information tracking FIFO
- **reset_defs.svh** - Standard reset macro definitions

20.8.3 See Also

- **axi_master_wr_splitter.sv** - Write channel splitting (AW + W channels)
 - **axi_split_combi.sv** - Combined read/write splitting wrapper
-

20.9 References

20.9.1 Source Code

- RTL: rtl/amba/shared/axi_master_rd_splitter.sv
- Tests: val/amba/test_axi_master_rd_splitter.py
- Testbench: bin/TBClasses/amba/axi_master_rd_splitter_tb.py

20.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - AXI Specification: AMBA AXI4 Protocol Specification (ARM IHI 0022E)
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2025-10-24

20.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

21 AXI Master Write Splitter

Module: `axi_master_wr_splitter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

21.1 Overview

The AXI Master Write Splitter handles the complex task of splitting boundary-crossing write transactions while maintaining data integrity and response consolidation. Unlike read splitting which simply passes through responses, write splitting must aggregate multiple split responses into a single consolidated response with proper error priority handling.

21.1.1 Key Features

- Automatic boundary crossing detection for write transactions
 - AW + W channel forwarding with synchronized address/data flow
 - B channel response consolidation (N responses → 1 consolidated response)
 - Error priority handling (DECERR > SLVERR > EXOKAY > OKAY)
 - WLAST regeneration for split boundaries
 - Zero added latency for non-split transactions
 - Full AXI4 protocol compliance with user extensions
-

21.2 Module Purpose

Write transaction splitting presents unique challenges compared to read splitting due to the tight coupling between address (AW), data (W), and response (B) channels. The AXI Master Write Splitter solves these challenges while maintaining protocol correctness.

Problem Statement: - Upstream master issues write (ADDR=0x0FC0, LEN=7, 8 data beats) - Transaction crosses 4KB boundary at 0x1000 - Downstream requires separate transactions per boundary - Each split generates independent response - Upstream expects single consolidated response

Solution: - Module detects boundary crossing using combinational split logic - AW channel: Generates N split address transactions - W channel: Regenerates WLAST at split boundaries - B channel: Consolidates N responses into 1 (error priority) - Example split: - Split 1: ADDR=0x0FC0,

LEN=0 (1 beat) → Response: OKAY - Split 2: ADDR=0x1000, LEN=6 (7 beats) → Response: OKAY - Consolidated: OKAY (worst error across all splits)

Key Invariants: - Upstream sees exactly ONE write transaction with ONE response - Downstream sees N split transactions with N responses - Consolidated response reflects worst error status - Write data flows through unchanged (WLAST regenerated) - All AXI signaling preserved across splits

21.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Width of AXI ID signals (AWID, BID)
AXI_ADDR_WIDTH	int	32	Width of AXI address bus (AWADDR)
AXI_DATA_WIDTH	int	32	Width of AXI data bus (WDATA), must be power of 2 (32-1024 bits)
AXI_USER_WIDTH	int	1	Width of AXI user extension fields (AWUSER, WUSER, BUSER)
SPLIT_FIFO_DEPTH	int	4	Depth of split information tracking FIFO
IW	int	AXI_ID_WIDTH	Shorthand alias for ID width
AW	int	AXI_ADDR_WIDTH	Shorthand alias for address width
DW	int	AXI_DATA_WIDTH	Shorthand alias for data width
UW	int	AXI_USER_WIDTH	Shorthand alias for user width

21.4 Port Groups

21.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Global clock for all AXI interfaces
aresetn	input	1	Active-low asynchronous reset

21.4.2 Configuration Interface

Port	Direction	Width	Description
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB boundaries)
block_read_y	input	1	Backpressure from error monitor (suppresses fub_awready when asserted)

21.4.3 Master AXI Write Interface (Downstream)

AW Channel (Address Write):

Port	Direction	Width	Description
m_axi_awid	output	IW	Transaction ID for split write transactions
m_axi_awaddr	output	AW	Calculated address for current split (boundary-aligned)
m_axi_awlenn	output	8	Calculated burst length for current split (AXI encoding: beats-1)
m_axi_awsiz	output	3	Transfer size (preserved from original transaction)
m_axi_awb	output	2	Burst type (preserved

Port	Direction	Width	Description
urst			from original, typically INCR)
m_axi_awlock	output	1	Lock type (preserved from original)
m_axi_awcache	output	4	Cache attributes (preserved from original)
m_axi_awprot	output	3	Protection attributes (preserved from original)
m_axi_awqos	output	4	Quality of Service (preserved from original)
m_axi_awregion	output	4	Region identifier (preserved from original)
m_axi_awuser	output	UW	User-defined extension (preserved from original)
m_axi_awvalid	output	1	Valid signal for current split transaction
m_axi_awready	input	1	Ready signal from downstream slave

W Channel (Write Data):

Port	Direction	Width	Description
m_axi_wdata	output	DW	Write data payload (pass-through from fub)
m_axi_wstrobe	output	DW/8	Write strobes (pass-through from fub)
m_axi_wlast	output	1	Last beat indicator (regenerated at split boundaries)
m_axi_wuser	output	UW	User-defined extension (pass-through from fub)
m_axi_wvalid	output	1	Valid signal for write data (pass-through from fub)
m_axi_wready	input	1	Ready signal from

Port	Direction	Width	Description
ady			downstream slave

B Channel (Write Response):

Port	Direction	Width	Description
m_axi_bid	input	IW	Response transaction ID (matches AWID)
m_axi_bresp	input	2	Response status (OKAY=00, EXOKAY=01, SLVERR=10, DECERR=11)
m_axi_buser	input	UW	User-defined extension for response
m_axi_bvalid	input	1	Valid signal for response
m_axi_bready	output	1	Ready signal to downstream slave (controlled during consolidation)

21.4.4 Slave AXI Write Interface (Upstream / FUB)

AW Channel (Address Write):

Port	Direction	Width	Description
fub_awid	input	IW	Original transaction ID from upstream master
fub_awaddr	input	AW	Original starting address (may cross boundaries)
fub_awlen	input	8	Original burst length (may span multiple boundaries)
fub_awsize	input	3	Transfer size per beat
fub_awburst	input	2	Burst type (FIXED, INCR, WRAP)
fub_awlock	input	1	Lock type for atomic operations
fub_awcac	input	4	Cache attributes

Port	Direction	Width	Description
he			
fub_awprot	input	3	Protection attributes
fub_awqos	input	4	Quality of Service priority
fub_awregion	input	4	Region identifier
fub_awuser	input	UW	User-defined extension
fub_awvalid	input	1	Valid signal for original transaction
fub_awready	output	1	Ready signal to upstream master (suppressed until all splits complete)

W Channel (Write Data):

Port	Direction	Width	Description
fub_wdata	input	DW	Write data payload from upstream master
fub_wstrb	input	DW/8	Write strobes from upstream master
fub_wlast	input	1	Last beat indicator from upstream master
fub_wuser	input	UW	User-defined extension
fub_wvalid	input	1	Valid signal for write data
fub_wready	output	1	Ready signal to upstream master (pass-through from m_axi)

B Channel (Write Response):

Port	Direction	Width	Description
fub_bid	output	IW	Consolidated response transaction ID (original AWID)

Port	Direction	Width	Description
fub_bresp	output	2	Consolidated response status (worst error across all splits)
fub_buser	output	UW	User-defined extension (from original transaction)
fub_bvalid	output	1	Valid signal for consolidated response (only after all splits received)
fub_bready	input	1	Ready signal from upstream master

21.4.5 Split Information Interface

Port	Direction	Width	Description
fub_split_addr	output	AW	Original transaction starting address
fub_split_id	output	IW	Original transaction ID
fub_split_count	output	8	Number of split transactions generated (2+ if split occurred)
fub_split_valid	output	1	Valid signal for split information
fub_split_ready	input	1	Ready signal from split information consumer

21.5 Functional Description

21.5.1 Write Transaction Splitting Overview

Write splitting requires coordinated management of three independent AXI channels with different timing characteristics.

Non-Split Path (Fast): - Transaction fully contained within single boundary - AW/W signals pass directly from fub to m_axi - fub_awready = m_axi_awready (immediate acceptance) - B channel passes through directly - Split count = 1

Split Path (Multi-Transaction + Response Consolidation): - Transaction crosses one or more boundaries - AW signals buffered and regenerated for each split - W channel WLAST regenerated at split boundaries - B channel: Collect N responses, consolidate into 1 - fub_awready suppressed until final split accepted - Split count = N (number of splits)

21.5.2 State Machine Architecture

Similar to read splitter but with additional W channel tracking:

IDLE State: - Awaits new transactions on fub_aw interface - Combinational split logic evaluates boundary crossing - If no split: pass-through mode (single response) - If split: buffer transaction, setup response consolidation, transition to SPLITTING

SPLITTING State: - Issue current split using buffered address/length - Track W channel beats to regenerate WLAST correctly - Collect B channel responses (consolidate errors) - Update next_addr and remaining_len for next split - Return to IDLE when final split accepted

21.5.3 Write Data Channel Management

WLAST Regeneration:

The critical challenge is regenerating WLAST at split boundaries:

Beat Counter Tracking: - r_expected_beats: Beats required for current split (from split_len + 1) - r_beat_counter: Current beat being transferred - w_split_boundary_reached: Counter matches expected beats

WLAST Logic:

```
If splitting:
    WLAST = (beat_counter + 1 >= expected_beats)
Else:
    WLAST = fub_wlast (pass-through)
```

Example (8 beats split into 1 + 7): - Original: 8 beats with WLAST on beat 8 - Split 1: Beat 1 → Generate WLAST (boundary reached) - Split 2: Beats 2-8 → Original WLAST on beat 8

21.5.4 Response Consolidation Logic

CRITICAL FEATURE: Multiple split responses must aggregate into single upstream response.

Consolidation State: - r_expected_response_count: Total splits generated - r_received_response_count: Responses received so far - r_waiting_for_responses: Consolidation active flag - r_consolidated_resp_status: Aggregated error status - r_original_txn_id: Transaction ID for consolidated response

Response Priority (Worst Error Wins):

Priority: DECERR (11) > SLVERR (10) > EXOKAY (01) > OKAY (00)

```
if (m_axi_bresp > r_consolidated_resp_status):
    r_consolidated_resp_status = m_axi_bresp
```

Consolidation Flow:

1. **Setup (at transaction start):**
 - Capture original AWID
 - Initialize consolidated_resp_status = OKAY
 - Set expected_response_count = estimated splits
 - Assert waiting_for_responses flag
2. **Collection (each B response):**
 - Accept response even if fub_bready not asserted
 - OR error status into consolidated_resp_status
 - Increment received_response_count
 - Keep fub_bvalid suppressed
3. **Completion (final response):**
 - Check: received_response_count >= expected_response_count
 - Assert fub_bvalid with consolidated status
 - Wait for fub_bready handshake
 - Clear waiting_for_responses flag

Pass-Through Mode (No Split): - waiting_for_responses = 0 - Direct connection: fub_b* = m_axi_b* - Single response, no aggregation needed

21.5.5 Boundary Crossing Detection

Reuses same combinational logic as read splitter (axi_split_combi):

Inputs: - current_addr, current_len, ax_size, alignment_mask - Same calculation as read splitter

Outputs: - split_required, split_len, next_boundary_addr, remaining_len_after_split - Used for both AW channel splitting and W channel beat tracking

21.5.6 AW Channel Forwarding Logic

Address/Length Calculation: - IDLE state: Use live fub_awaddr and calculated split_len - SPLITTING state: Use buffered r_current_addr and split_len

Control Signals: - IDLE state: Pass through from fub_aw* signals - SPLITTING state: Use buffered r_orig_aw* signals

Valid Signal: - IDLE: m_axi_awvalid = fub_awvalid - SPLITTING: m_axi_awvalid = 1 (asserting next split)

21.5.7 Ready Signal Management

fub_awready Logic: - IDLE + no split: fub_awready = m_axi_awready - IDLE + split needed: fub_awready = 0 (suppress) - SPLITTING + intermediate: fub_awready = 0 - SPLITTING + final split: fub_awready = m_axi_awready

W Channel Ready: - Always pass-through: fub_wready = m_axi_wready - Backpressure handled naturally by AXI protocol

B Channel Ready: - Consolidation mode: m_axi_bready = 1 (accept all responses) - Pass-through mode: m_axi_bready = fub_bready - Critical: Must accept all split responses even if upstream not ready

21.6 Usage Example

21.6.1 Basic Integration (4KB Boundary, Response Consolidation)

```
axi_master_wr_splitter #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .AXI_USER_WIDTH    (1),
    .SPLIT_FIFO_DEPTH  (16)
) u_wr_splitter (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // 4KB boundary alignment
    .alignment_mask     (12'hFFF),

    // Error monitor backpressure
    .block_ready        (error_detected),

    // Upstream AXI write interface (from master)
    .fub_awid           (master_awid),
    .fub_awaddr         (master_awaddr),
    .fub_awlen          (master_awlen),
    .fub_awsz           (master_awsz),
    .fub_awburst        (master_awburst),
    .fub_awlock         (master_awlock),
    .fub_awcache        (master_awcache),
    .fub_awprot         (master_awprot),
    .fub_awqos          (master_awqos),
    .fub_awregion       (master_awregion),
    .fub_awuser         (master_awuser),
    .fub_awvalid        (master_awvalid),
```

```

.fub_awready      (master_awready),

.fub_wdata        (master_wdata),
.fub_wstrb        (master_wstrb),
.fub_wlast        (master_wlast),
.fub_wuser        (master_wuser),
.fub_wvalid       (master_wvalid),
.fub_wready       (master_wready),

.fub_bid          (master_bid),
.fub_bresp        (master_bresp),
.fub_buser        (master_buser),
.fub_bvalid       (master_bvalid),
.fub_bready       (master_bready),

// Downstream AXI write interface (to memory)
.m_axi_awid       (mem_awid),
.m_axi_awaddr     (mem_awaddr),
.m_axi_awlen      (mem_awlen),
.m_axi_awsz       (mem_awsz),
.m_axi_awburst    (mem_awburst),
.m_axi_awlock     (mem_awlock),
.m_axi_awcache    (mem_awcache),
.m_axi_awprot     (mem_awprot),
.m_axi_awqos      (mem_awqos),
.m_axi_awregion   (mem_awregion),
.m_axi_awuser     (mem_awuser),
.m_axi_awvalid    (mem_awvalid),
.m_axi_awready    (mem_awready),

.m_axi_wdata      (mem_wdata),
.m_axi_wstrb      (mem_wstrb),
.m_axi_wlast      (mem_wlast),
.m_axi_wuser      (mem_wuser),
.m_axi_wvalid     (mem_wvalid),
.m_axi_wready     (mem_wready),

.m_axi_bid        (mem_bid),
.m_axi_bresp      (mem_bresp),
.m_axi_buser      (mem_buser),
.m_axi_bvalid     (mem_bvalid),
.m_axi_bready     (mem_bready),

// Split information tracking
.fub_split_addr   (split_addr),
.fub_split_id     (split_id),
.fub_split_cnt    (split_count),

```

```

        .fub_split_valid      (split_valid),
        .fub_split_ready     (split_ready)
    );

```

21.6.2 Response Consolidation Example

Scenario: 2-way split with error in second split

Original Transaction: - ADDR=0x0FC0, LEN=7 (8 beats), ID=0x42

Split Transactions: 1. Split 1: ADDR=0x0FC0, LEN=0 (1 beat) → Response: OKAY (00) 2. Split 2: ADDR=0x1000, LEN=6 (7 beats) → Response: SLVERR (10)

Consolidation Logic:

```

Initial: consolidated_resp = OKAY (00)
Split 1 response: OKAY (00)
    → 00 > 00? No → Keep OKAY
Split 2 response: SLVERR (10)
    → 10 > 00? Yes → Update to SLVERR

```

Final consolidated response: SLVERR (10)

Upstream View: - Master issues 1 write transaction (ADDR=0x0FC0, LEN=7, ID=0x42) - Receives 1 response (ID=0x42, BRESP=SLVERR) - Error correctly propagated despite partial success

21.7 Design Notes

21.7.1 AXI Protocol Assumptions

Same assumptions as read splitter:

Assumption 1: Address Alignment - All addresses aligned to data bus width

Assumption 2: Fixed Transfer Size - AWSIZE matches data width

Assumption 3: Incrementing Bursts Only - Only INCR burst type supported

Assumption 4: No Address Wraparound - Transactions never wrap address space

21.7.2 Response Consolidation Details

Why Consolidation is Critical: - AXI protocol: 1 address transaction → 1 response - Splitting creates N address transactions → N responses - Upstream expects exactly 1 response - Must aggregate N responses into 1 consolidated response

Error Priority Rationale:

DECERR (Decode Error):	Transaction to invalid address
SLVERR (Slave Error):	Valid address, processing error
EXOKAY (Exclusive Okay):	Exclusive access succeeded
OKAY:	Normal successful completion

Worst error wins → System sees most severe failure

Consolidation Timing: - Must wait for ALL split responses before asserting fub_bvalid - Prevents partial success reports - Ensures atomicity from upstream perspective

21.7.3 Performance Characteristics

Latency: - No split: Zero added cycles (pass-through) - Split required: 1 cycle per split + response consolidation - Response consolidation adds latency equal to slowest split

Throughput: - AW channel: Limited by split calculation (1 cycle/split) - W channel: Full bandwidth (pass-through with WLAST regeneration) - B channel: Responses accumulated (no upstream throughput impact)

Area: - Overhead: State machine + transaction buffer + response consolidation logic - Major components: Split FIFO + response tracking registers

21.7.4 Error Handling

Split Response Timeout: - Module waits indefinitely for all split responses - No timeout mechanism (relies on downstream protocol compliance) - Recommendation: Implement system-level timeout monitor

FIFO Overflow: - Split FIFO can fill if consumer stalls - Backpressure propagates to fub_awready - Size FIFO >= max outstanding writes

Data Channel Synchronization: - W channel must deliver exact beat count per split - WLAST regeneration critical for correctness - Assertions validate beat counter operation

21.7.5 Integration Considerations

Response Consolidation Requirements: - Downstream must return exactly 1 response per split - Response IDs must match split transaction IDs - All responses must eventually arrive

WLAST Timing: - Generated combinationally based on beat counter - Timing-critical path in high-speed designs - Consider registering if timing closure issues

Backpressure Handling: - B channel: Module accepts all split responses (m_axi_bready=1 during consolidation) - Upstream backpressure (fub_bready=0) only delays final consolidated response - Internal buffering for responses: Single register (r_consolidated_resp_status)

21.8 Related Modules

21.8.1 Used By

- Memory controller wrappers with boundary constraints
- Multi-bank DRAM controllers requiring aligned writes
- Bridge modules interfacing heterogeneous memory systems
- DMA engines with address range restrictions

21.8.2 Uses

- **axi_split_combi.sv** - Combinational splitting logic (shared with read splitter)
- **gaxi_fifo_sync.sv** - Split information tracking FIFO
- **reset_defs.svh** - Standard reset macro definitions

21.8.3 See Also

- **axi_master_rd_splitter.sv** - Read channel splitting (AR + R channels)
 - **axi_split_combi.sv** - Combined read/write splitting wrapper
-

21.9 References

21.9.1 Source Code

- RTL: rtl/amba/shared/axi_master_wr_splitter.sv
- Tests: val/amba/test_axi_master_wr_splitter.py
- Testbench: bin/TBClasses/amba/axi_master_wr_splitter_tb.py

21.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - AXI Specification: AMBA AXI4 Protocol Specification (ARM IHI 0022E)
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2025-10-24

21.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

22 AXI Performance Latency Histogram

Module: `axi_perf_latency_hist.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

22.1 Overview

The AXI Performance Latency Histogram captures per-transaction latency on an AXI master bus and bins it into a log2 histogram surfaced through a CSR-indexed readout (no MonBus packets). It is a self-contained snoop block — it does not gate or otherwise touch the AXI datapath — and is instantiated alongside the datapath monitors, deliberately leaving the shared `axi_monitor_base` untouched. For reads it measures both AR→first-R and AR→RLAST latency; for writes it measures AW→B latency.

22.1.1 Key Features

- Per-channel log2 latency histogram (default 16 bins: bin b counts latencies in $[2^b, 2^{(b+1)})$)
 - Read mode (`IS_READ=1`): two metrics — AR→first-R beat and AR→RLAST
 - Write mode (`IS_READ=0`): one metric — AW→B response
 - Per-channel command-timestamp FIFO matches completions to oldest outstanding command (same-ID in-order)
 - Four-stage histogram update pipeline for FPGA timing closure
 - Free-running (non-frozen) timestamp so latencies straddling the window boundary stay correct
 - CSR-indexed readout of any bin count plus each metric's transaction total
 - Window control (`i_clear` / `i_freeze`) mirroring `axi_bus_meter`
-

22.2 Module Purpose

Average latency hides the distribution that actually matters for QoS analysis — a bus can have a fine mean while a tail of slow transactions strangles a real workload. This block bins each transaction's measured latency into a log2 histogram, so the shape of the distribution (and its tail) is visible from a handful of CSR reads. It snoops the command and completion channels, timestamps each command as it is accepted, and on completion subtracts to get latency and

increments the matching log2 bin. It is purely observational and adds no load to the AXI path, so it can be dropped in for characterization without perturbing the design under measurement.

Use Cases: - Read/write latency distribution characterization on AXI master ports (perfmon Stage D) - Tail-latency analysis for QoS and arbitration tuning - Parity latency metrics alongside axi4_dma_observer per-port histograms - Host-driven on-silicon performance runs (CSR readout, no MonBus traffic)

Key Benefit: Exposes the full latency distribution (including the tail) per channel through cheap CSR reads, without touching the shared monitor base or adding any MonBus packet traffic.

22.3 Parameters

Parameter	Type	Default	Description
ID_WIDTH	int	8	AXI id width on the snooped bus
NUM_CHANNELS	int	8	Number of per-channel bins (channel = id[CW-1:0])
MAX_OUTSTANDING	int	8	Per-channel command-timestamp FIFO depth
NUM_BINS	int	16	Number of log2 histogram bins; bin b counts $[2^b, 2^{(b+1)})$
IS_READ	bit	1'b1	1 = read build (2 metrics), 0 = write build (1 metric)
CNT_W	int	32	Histogram bin and total counter width
CW	int	derived	$\$clog2(NUM_CHANNELS)$ — channel index width
PW	int	derived	$\$clog2(MAX_OUTSTANDING)$ — FIFO pointer width
PW1	int	derived	$PW + 1$ — occupancy width (extra bit for full)
BINW	int	derived	$\$clog2(NUM_BINS)$ — bin index width

22.4 Port Groups

22.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

22.4.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	One-cycle pulse (perf RUN rising edge): resets histogram + FIFOs
i_freeze	input	1	Hold high to freeze the histogram (counters stop; readback stable)

22.4.3 Command Channel (AR for reads, AW for writes)

Port	Direction	Width	Description
cmd_valid	input	1	Command-channel valid
cmd_ready	input	1	Command-channel ready (handshake timestamps the command)
cmd_id	input	ID_WIDTH	Command id; low CW bits select the channel

22.4.4 Read Data Channel (R) — used when IS_READ=1

Port	Direction	Width	Description
data_valid	input	1	R-channel valid
data_ready	input	1	R-channel ready
data_last	input	1	R-channel rlast (marks the RLAST metric / FIFO pop)

Port	Direction	Width	Description
data_id	input	ID_WIDTH	R-channel id; low CW bits select the channel

22.4.5 Write Response Channel (B) — used when IS_READ=0

Port	Direction	Width	Description
resp_valid	input	1	B-channel valid
resp_ready	input	1	B-channel ready
resp_id	input	ID_WIDTH	B-channel id; low CW bits select the channel

22.4.6 CSR-Indexed Readout

Port	Direction	Width	Description
i_hist_metric	input	1	Metric select: 0 = first-R/B, 1 = RLAST (reads only)
i_hist_bin	input	BINW	Bin index to read
o_hist_count	output	CNT_W	Selected bin's count for the selected metric
o_hist_total	output	CNT_W	Selected metric's total transaction count

22.5 Functional Description

22.5.1 Metrics

- **IS_READ=1 (read build), NUM_METRICS=2:** metric 0 = AR handshake → first R beat; metric 1 = AR handshake → RLAST beat.
- **IS_READ=0 (write build), NUM_METRICS=1:** metric 0 = AW handshake → B response.

The histogram storage is always sized for two metrics for uniform indexing; the write build uses only metric 0.

22.5.2 Transaction Matching

AXI requires same-ID responses to return in order, so completions can be matched to commands with a simple per-channel FIFO of command-phase timestamps rather than a CAM. On a command handshake (cmd_valid && cmd_ready) the current free-running time is pushed into

the channel's FIFO (bounded by `MAX_OUTSTANDING`). On completion the oldest timestamp for that channel (`r_ts[ch][head]`) is the start time, and the FIFO is popped when the completing beat is `last` (RLAST for reads; B is always “last” for writes). A single AXI data/response bus carries at most one beat per cycle, so at most one push and one pop occur per cycle — there is no multi-writer hazard on the shared histogram.

For reads, a per-channel `r_burst_active` flag distinguishes the first R beat (metric 0, AR→first-R) from subsequent beats of the same burst; it is set on the first beat and cleared when the burst pops on RLAST.

22.5.3 Log2 Binning

`latency_bin()` returns `floor(log2(lat))` clamped to `NUM_BINS - 1`; latencies of 0 or 1 map to bin 0. Bin `b` therefore counts latencies in $[2^b, 2^{(b+1)})$.

22.5.4 Free-Running Timestamp

`r_time` is a 32-bit counter that increments every cycle and is **not** frozen by `i_freeze`. Keeping it running means a latency measurement that straddles the window-close boundary still subtracts two consistent absolute timestamps and yields the correct value. Only the histogram accumulation is frozen.

22.5.5 Four-Stage Update Pipeline

The chain FIFO-read → latency subtract → log2 bin → indexed histogram increment is far too deep for a single 100 MHz cycle (it was the routed critical path), so it is split into four register stages:

- **Stage 0:** capture the event — {`start_ts`, `time`, metric flags `m0/m1`, `valid`}. Gated by `i_freeze` (a frozen window captures no new events).
- **Stage 1:** latency = completion time – start time.
- **Stage 2:** log2 bin from the latency.
- **Stage 3:** increment the selected histogram bin(s) and the per-metric total(s).

The increment is delayed a few cycles, but the host reads the window long after RUN drops to 0, by which time the pipeline has drained — `i_freeze` gates stage 0 while stages 1–3 keep advancing to flush. Because a single AXI bus completes at most one transaction per cycle and metrics `m0/m1` target different histograms, consecutive same-bin increments are hazard-free (the cross-cycle NBA read-after-write is correct).

22.5.6 Window Control

`i_clear` (a one-cycle pulse on the perf RUN rising edge) resets the histograms, totals, and all per-channel FIFOs and state. `i_freeze` (`~RUN`) holds the histogram so a closed window can be read back, while the free-running timestamp keeps counting.

22.5.7 Readout

Readout is a combinational mux: `o_hist_count` returns `r_hist[metric][bin]` for the CSR-selected metric and bin, and `o_hist_total` returns that metric's running transaction total (useful for normalizing bin counts into a distribution).

22.6 Usage Example

```
// Read-latency histogram on an AXI master read port (2 metrics:
first-R, RLAST).
axi_perf_latency_hist #(
    .ID_WIDTH           (8),
    .NUM_CHANNELS       (8),
    .MAX_OUTSTANDING    (8),
    .NUM_BINS           (16),
    .IS_READ            (1'b1)
) u_rd_lat_hist (
    .aclk                (aclk),
    .aresetn             (aresetn),

    // Window control (share with axi_bus_meter)
    .i_clear             (perf_run_rising),
    .i_freeze            (~perf_run),

    // AR command channel
    .cmd_valid           (m_axi_arvalid),
    .cmd_ready           (m_axi_arready),
    .cmd_id              (m_axi_arid),

    // R data channel
    .data_valid          (m_axi_rvalid),
    .data_ready          (m_axi_rready),
    .data_last           (m_axi_rlast),
    .data_id             (m_axi_rid),

    // B channel unused for reads
    .resp_valid          (1'b0),
    .resp_ready          (1'b0),
    .resp_id             ('0),

    // CSR readout
    .i_hist_metric       (csr_metric),    // 0 = AR->first-R, 1 = AR->RLAST
    .i_hist_bin          (csr_bin),
    .o_hist_count        (csr_bin_count),
    .o_hist_total        (csr_metric_total)
```

```
);
```

```
// Write build: set IS_READ=0, wire AW to cmd_* and B to resp_*, leave  
data_* tied off.
```

22.7 Design Notes

22.7.1 Snoop-Only, Base Untouched

The block is instantiated next to the datapath monitors but does not modify `axi_monitor_base`; it snoops the AXI channels read-only and emits no MonBus packets. Results leave exclusively through the CSR-indexed readout.

22.7.2 Timestamp Not Frozen

Only histogram accumulation freezes on `i_freeze`; `r_time` keeps counting so a transaction whose command was accepted before the window closed and whose completion arrives just after still yields a correct latency.

22.7.3 FIFO Depth vs Outstanding

`MAX_OUTSTANDING` bounds the per-channel timestamp FIFO. A command handshake only pushes when the channel FIFO is not full, and a completion only pops when the FIFO is non-empty, so an over-run on one channel cannot corrupt another. Size `MAX_OUTSTANDING` to the real per-channel outstanding depth of the bus.

22.7.4 Pipeline Drain Before Readback

Because increments land up to three cycles after the completing beat, always read the histogram after the window has been closed (`i_freeze` high) long enough for the pipeline to drain — the host reads long after `RUN` falls, so this is automatic in the intended usage.

22.8 Related Modules

22.8.1 Used By

- Perfmon Stage D characterization (see RFC below)
- Datapath-monitor instantiations needing latency distributions alongside `axi_bus_meter`

22.8.2 Uses

- **`reset_defs.svh`** - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

22.8.3 See Also

- **axi_bus_meter.sv** - Four-bucket utilization meter, shares the window-control convention
 - **axi4_dma_observer.sv** - DMA observability wrapper with per-port latency histograms
 - **axi_monitor_base.sv** - The shared monitor scaffold this block deliberately does not touch
-

22.9 References

22.9.1 Source Code

- RTL: rtl/amba/shared/axi_perf_latency_hist.sv

22.9.2 Documentation

- RFC: rtl/amba/PRD/RFCs/RFC-perfmon-window-buckets.md
 - Architecture: docs/markdown/RTLAmba/shared/README.md
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2026-07-15

22.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

23 AXI Split Combinational Logic

Module: axi_split_combi.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

23.1 Overview

The AXI Split Combinational Logic module provides pure combinational boundary crossing detection and split length calculation for AXI transactions. It serves as the core computational

engine for both read and write transaction splitters, enabling real-time decision-making without state machine overhead.

23.1.1 Key Features

- Pure combinational logic (zero latency)
 - Boundary crossing detection for arbitrary alignment boundaries
 - Automatic split length calculation (AXI encoding)
 - Optimized for synthesis (bit shifts instead of division)
 - Comprehensive assertion-based validation
 - Configurable address and data widths
 - No address wraparound assumption (simplified logic)
-

23.2 Module Purpose

AXI transaction splitting requires complex address arithmetic to determine: 1. Does this transaction cross a boundary? 2. How many beats fit before the boundary? 3. What address/length for the next split?

This module encapsulates all splitting calculations into reusable combinational logic, ensuring consistent behavior across read and write splitters.

Design Philosophy: - Separation of concerns: Calculation logic separate from state management - Reusability: Single module used by both read and write splitters - Performance: Combinational path enables single-cycle decisions - Correctness: Extensive assertions validate assumptions

23.3 Parameters

Parameter	Type	Default	Description
AW	int	32	Address width in bits (typically 32 or 64)
DW	int	32	Data width in bits, must be power of 2 (32, 64, 128, 256, 512, 1024)

Derived Constants (localparam):

Constant	Calculation	Description
BYTES_PER_BEAT	DW / 8	Bytes transferred per beat

Constant	Calculation	Description
LOG2_BYTES_PER_BEAT	$\$clog2(\text{BYTES_PER_BEAT})$	(e.g., 8 for 64-bit bus) Bit shift amount for beat calculations
EXPECTED_AX_SIZE	LOG2_BYTES_PER_BEAT	Expected value of ax_size input (for assertions)
ADDR_ALIGN_MASK	BYTES_PER_BEAT - 1	Mask for address alignment checking

23.4 Port Groups

23.4.1 Clock and Reset (For Assertions Only)

Port	Direction	Width	Description
aclk	input	1	Clock for assertion evaluation (not used in functional logic)
aresetn	input	1	Active-low reset for assertion gating (not used in functional logic)

23.4.2 Input Signals

Port	Direction	Width	Description
current_address	input	AW	Current transaction starting address (may be mid-split)
current_length	input	8	Current transaction length in AXI encoding (beats - 1)
ax_size	input	3	AXI size field (log2 of bytes per beat)
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB)
is_idle_state	input	1	Indicates whether state machine is in IDLE (first transaction)

Port	Direction	Width	Description
transaction_valid	input	1	Valid transaction present on inputs

23.4.3 Output Signals

Port	Direction	Width	Description
split_required	output	1	Transaction crosses boundary and requires splitting
split_len	output	8	Length of current split in AXI encoding (beats - 1)
next_boundary_addr	output	AW	Address at next boundary alignment
remaining_len_after_split	output	8	Remaining length after current split (AXI encoding)
new_split_needed	output	1	Helper signal: split_required AND is_idle_state AND transaction_valid

23.5 Functional Description

23.5.1 Boundary Crossing Detection Algorithm

The module implements a simplified boundary detection algorithm based on key AXI assumptions:

Step 1: Calculate Transaction Parameters

```
total_bytes = (current_len + 1) << ax_size
transaction_end_addr = current_addr + total_bytes - 1
```

Step 2: Calculate Next Boundary

```
next_boundary_addr = (current_addr | alignment_mask) + 1
```

Example (4KB boundary, ADDR=0x0FC0, mask=0xFFF):

```
current_addr      = 0x0FC0
alignment_mask    = 0xFFFF
```

```
OR result          = 0x0FFF
next_boundary      = 0x0FFF + 1 = 0x1000
```

Step 3: Calculate Space to Boundary

```
bytes_to_boundary = next_boundary_addr - current_addr
beats_to_boundary = bytes_to_boundary >> ax_size
```

Example (continuing above, ax_size=3 for 8-byte beats):

```
bytes_to_boundary = 0x1000 - 0x0FC0 = 64 bytes
beats_to_boundary = 64 >> 3 = 8 beats
```

Step 4: Determine Split Requirement

Transaction requires splitting if: 1. It ends at or past the next boundary (crosses_boundary) 2. There is space for at least one beat before boundary (has_beats_before_boundary) 3. The beats to boundary fit within total transaction (beats_fit_before_boundary)

```
crosses_boundary = (transaction_end_addr >= next_boundary_addr)
has_beats_before_boundary = (beats_to_boundary > 0)
beats_fit_before_boundary = (beats_to_boundary <= (current_len + 1))
```

```
split_required = crosses_boundary AND has_beats_before_boundary
                AND beats_fit_before_boundary
```

23.5.2 Split Length Calculation

If splitting is required, the split length represents how many beats to consume before the boundary:

```
split_len = split_required ? (beats_to_boundary - 1) : current_len
```

AXI Encoding: Length is always encoded as (beats - 1): - 1 beat → LEN=0 - 2 beats → LEN=1 - 8 beats → LEN=7

23.5.3 Remaining Length Calculation

After consuming the current split, calculate remaining beats:

```
original_beats_total = current_len + 1
split_beats_actual = split_required ? beats_to_boundary :
original_beats_total
remaining_beats_actual = split_required ? (original_beats_total -
split_beats_actual) : 0
```

```
// Convert back to AXI encoding
remaining_len_after_split = split_required ?
                          ((remaining_beats_actual > 0) ?
```

```
(remaining_beats_actual - 1) : 0)
                          : 0
```

23.5.4 State Machine Helper

The `new_split_needed` output simplifies state machine control:

```
new_split_needed = split_required AND is_idle_state AND
transaction_valid
```

This signal indicates a new split sequence is starting (as opposed to continuing an existing split).

23.6 Design Assumptions

The module enforces several critical assumptions to enable simplified, high-performance logic:

23.6.1 Assumption 1: Address Alignment

Assumption: All AXI addresses are aligned to the data bus width.

Implication: - If `DATA_WIDTH` = 512 bits (64 bytes), address is always 64-byte aligned - Low-order address bits are always zero

Verification:

```
assert ((current_addr & ADDR_ALIGN_MASK) == '0')
```

23.6.2 Assumption 2: Fixed Transfer Size

Assumption: All AXI transfers use maximum transfer size equal to bus width.

Implication: - `ax_size` always matches $\log_2(\text{DATA_WIDTH}/8)$ - Example: 64-bit bus $\rightarrow$ `ax_size` = 3'b011 (8 bytes)

Verification:

```
assert (ax_size == EXPECTED_AX_SIZE)
```

23.6.3 Assumption 3: Incrementing Bursts Only

Assumption: All bursts use incrementing address mode (`AxBURST` = 2'b01).

Implication: - No FIXED or WRAP burst handling - Address calculation simplified

Rationale: - Covers 95%+ of real-world use cases - FIXED bursts rarely cross boundaries - WRAP bursts require complex modulo arithmetic

23.6.4 Assumption 4: No Address Wraparound

Assumption: Transactions never wrap around top of address space (0xFFFFFFFF → 0x00000000).

Implication: - No wraparound detection or handling - Simplified comparison logic

Verification:

```
assert (transaction_end_addr >= current_addr) // No overflow
assert (next_boundary_addr > current_addr)    // Boundary doesn't wrap
```

Rationale: - Top of address space typically reserved for device registers - Memory allocators avoid near-boundary allocations - Real systems never allow this condition

23.7 Usage Example

23.7.1 Integration with Read Splitter

```
// Instantiate split logic within read splitter
axi_split_combi #(
    .AW          (32),
    .DW          (64)
) u_split_logic (
    // Clock/reset for assertions only
    .aclk        (aclk),
    .aresetn     (aresetn),

    // Transaction parameters
    .current_addr (w_current_addr),      // From state
machine
    .current_len  (w_current_len),      // From state
machine
    .ax_size      (w_current_size),     // ARSIZE
    .alignment_mask (12'hFFF),         // 4KB
boundary
    .is_idle_state (r_state == IDLE),
    .transaction_valid (fub_arvalid),

    // Splitting decision outputs
    .split_required (w_split_required),
    .split_len      (w_split_len),
    .next_boundary_addr (w_next_boundary_addr),
    .remaining_len_after_split (w_remaining_len_after_split),
    .new_split_needed (w_new_split_needed)
);
```

```

// Use outputs in state machine
always_comb begin
    case (r_state)
        IDLE: begin
            if (fub_arvalid && m_axi_arready) begin
                if (w_new_split_needed) begin
                    // Split required - start sequence
                    next_state = SPLITTING;
                    // Buffer next split parameters
                    buffer_addr = w_next_boundary_addr;
                    buffer_len = w_remaining_len_after_split;
                end else begin
                    // No split - pass through
                    next_state = IDLE;
                end
            end
        end

        SPLITTING: begin
            if (m_axi_arvalid && m_axi_arready) begin
                if (w_split_required) begin
                    // More splits needed
                    buffer_addr = w_next_boundary_addr;
                    buffer_len = w_remaining_len_after_split;
                end else begin
                    // Final split complete
                    next_state = IDLE;
                end
            end
        end
    endcase
end

// Use split_len for address channel
assign m_axi_araddr = w_current_addr;
assign m_axi_arlen = w_split_required ? w_split_len : w_current_len;

```

23.7.2 Example Calculation Walkthrough

Scenario: 4KB boundary, 64-bit data bus

Inputs: - current_addr = 0x0FC0 - current_len = 7 (8 beats in AXI encoding) - ax_size = 3 (8 bytes per beat) - alignment_mask = 0xFFFF

Calculations:

Step 1: Transaction parameters


```
total_bytes = (7 + 1) << 3 = 8 << 3 = 64 bytes
transaction_end_addr = 0x0FC0 + 64 - 1 = 0x0FFF
```

Step 2: Next boundary

```
next_boundary_addr = (0x0FC0 | 0x0FFF) + 1
                   = 0x0FFF + 1
                   = 0x1000
```

Step 3: Space to boundary

```
bytes_to_boundary = 0x1000 - 0x0FC0 = 64 bytes
beats_to_boundary = 64 >> 3 = 8 beats
```

Step 4: Split decision

```
crosses_boundary = (0x0FFF >= 0x1000) = False
split_required = False
```

Outputs: - split_required = 0 (transaction ends exactly at boundary, no split) - split_len = 7 (pass through original length) - remaining_len_after_split = 0

Modified Scenario: Transaction crosses boundary

Inputs: - current_addr = 0x0FC0 - current_len = 8 (9 beats) - ax_size = 3 - alignment_mask = 0xFFF

Calculations:

Step 1: Transaction parameters

```
total_bytes = (8 + 1) << 3 = 72 bytes
transaction_end_addr = 0x0FC0 + 72 - 1 = 0x1007
```

Step 4: Split decision

```
crosses_boundary = (0x1007 >= 0x1000) = True
has_beats_before_boundary = (8 > 0) = True
beats_fit_before_boundary = (8 <= 9) = True
split_required = True
```

Outputs: - split_required = 1 - split_len = 7 (8 beats - 1 in AXI encoding) - next_boundary_addr = 0x1000 - remaining_len_after_split = 0 (1 beat - 1 in AXI encoding)

23.8 Design Notes

23.8.1 Synthesis Optimization

Bit Shifts Instead of Division:

```
// Original (expensive):  
beats_to_boundary = bytes_to_boundary / (1 << ax_size)
```

```
// Optimized (single barrel shifter):  
beats_to_boundary = bytes_to_boundary >> ax_size
```

Fixed Alignment Assumptions: - Removes modulo arithmetic - Enables simple OR-based boundary calculation - Critical for timing closure in high-speed designs

23.8.2 Assertion Coverage

The module includes comprehensive assertions for validation:

Input Validation (Triggered on `transaction_valid` && `is_idle_state`): - Address alignment to data width - `ax_size` matches expected value for data width - No address wraparound in transaction - No boundary calculation wraparound

Output Validation (Triggered on `transaction_valid`): - Beats to boundary within reasonable range - Split length conservation (`split_beats` + `remaining_beats` = `original_beats`) - Boundary alignment verification - Comparison logic consistency

23.8.3 Timing Considerations

Critical Path: 1. Address OR with `alignment_mask` (`next_boundary_addr`) 2. Subtraction (`bytes_to_boundary`) 3. Variable barrel shift by `ax_size` (`beats_to_boundary`) 4. Comparison and output muxing (`split_required`, `split_len`)

Optimization Strategies: - Pipeline if timing closure issues (adds 1 cycle latency) - Use registered `alignment_mask` if it's static - Consider `ax_size` as constant parameter if fixed data width

23.8.4 Debug Support

DEBUG_AXI_SPLIT Macro:

Enable detailed transaction logging:

```
`define DEBUG_AXI_SPLIT  
  
// Generates console output:  
=== AXI_SPLIT_DEBUG (NO_WRAPAROUND) ===  
current_addr = 0x00000FC0  
current_len = 7 (total beats = 8)  
ax_size = 3 (bytes_per_beat = 8)  
alignment_mask = 0xFFF  
transaction_end_addr = 0x00000FFF  
next_boundary_addr = 0x00001000  
bytes_to_boundary = 64  
beats_to_boundary = 8  
crosses_boundary = 0
```

```
split_required = 0
split_len = 7 (split beats = 8)
remaining_len_after_split = 0 (remaining beats = 0)
=====
```

23.9 Related Modules

23.9.1 Used By

- **axi_master_rd_splitter.sv** - Read transaction splitting (AR channel)
- **axi_master_wr_splitter.sv** - Write transaction splitting (AW channel)

23.9.2 Dependencies

- None (pure combinational logic)

23.9.3 See Also

- **axi_master_rd_splitter.sv** - Full read splitter with state machine
 - **axi_master_wr_splitter.sv** - Full write splitter with response consolidation
-

23.10 References

23.10.1 Source Code

- RTL: rtl/amba/shared/axi_split_combi.sv
- Tests: Indirectly tested via splitter tests (test_axi_master_rd_splitter.py, test_axi_master_wr_splitter.py)

23.10.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - AXI Specification: AMBA AXI4 Protocol Specification (ARM IHI 0022E)
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2025-10-24

23.11 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

24 AXI-Stream Master Pattern Generator

Module: `axis4_master_pattern_gen.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

24.1 Overview

`axis4_master_pattern_gen` is an AXI-Stream master that emits a deterministic, per-channel LFSR data pattern for characterization and verification. It is the AXIS *sink* stimulus for the RAPIDS characterization harness, and it is pattern/CRC-consistent with the AXI4 pattern blocks: the per-channel 32-bit CRC-32 it computes is bit-identical to what `axi4_slave_wr_crc_check` produces for the same emitted stream (self-check path: `axis_gen` → `m_axi_wr` → `wr_crc_check`).

24.1.1 Key Features

- AXI-Stream master (`m_axis_*`) with no address channel — simpler than the AXI4 pattern generators
 - Per-channel independent LFSR (seed ^ ch) and CRC-32, copied verbatim from `axi4_slave_rd_pattern_gen`
 - Sequential per-channel scheduling — finish one channel's beats, then the next, so each channel's LFSR sequence is contiguous
 - Channel mask (`cfg_channel_mask`) selects active channels (0 → all)
 - Programmable beats-per-channel and tlast cadence (`cfg_beats_per_pkt`)
 - Per-channel expected-CRC / beat-count telemetry plus an aggregate total
 - LFSR/CRC advance only on accepted beats, so beat N of channel C is a deterministic function of (seed ^ C, N) independent of tready stalls
-

24.2 Module Purpose

Characterizing a stream-consuming engine (a “sink”) needs a deterministic source that a downstream checker can predict. This block streams a known LFSR pattern per channel and computes the CRC-32 that the same data will produce when it lands in a CRC-checking sink. Because the LFSR advances only on accepted beats, backpressure never perturbs the sequence —

the emitted data is a pure function of channel and beat index, and the exported per-channel CRC is the golden value for end-to-end integrity.

Use Cases: - Driving a RAPIDS sink engine's AXIS input during characterization - Sink self-check: stream into a DMA write path terminated by `axi4_slave_wr_crc_check` and compare CRCs - Per-channel throughput / backpressure sweeps with deterministic, reproducible data - On-chip (FPGA) stream stimulus in the RAPIDS characterization harness

Key Benefit: A memory-free, backpressure-insensitive stream source whose per-channel CRC-32 matches the AXI4 CRC blocks bit-for-bit, so a stream integrity test reduces to a single register compare.

24.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts; channel index drives <code>tid</code>
AXIS_DATA_WIDTH	int	512	Stream data width (must be a multiple of <code>LFSR_WIDTH</code>)
AXIS_ID_WIDTH	int	8	<code>tid</code> width
AXIS_DEST_WIDTH	int	4	<code>tdest</code> width
AXIS_USER_WIDTH	int	1	<code>tuser</code> width
LFSR_WIDTH	int	32	LFSR width (fixed; must match the AXI4 blocks)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel <code>N</code> uses seed $\wedge N$
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value

Parameter	Type	Default	Description
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input
CRC_REFOUT	int	1	Reflect output
BEAT_COUNT_WIDTH	int	32	Width of beat / packet counters
STRB_WIDTH	int	AXIS_DATA_WIDTH/8	Derived: tstrb width
REP	int	AXIS_DATA_WIDTH/ LFSR_WIDTH	Derived: 32-bit LFSR copies per beat
CIW	int	(NUM_CHANNELS > 1) ? \$clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: LFSR + CRC parameters must match axi4_slave_rd_pattern_gen / axi4_slave_wr_crc_check for cross-block CRC consistency.

24.4 Port Groups

24.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Stream clock
rst_n	input	1	Active-low asynchronous reset

24.4.2 Configuration / Control

Port	Direction	Width	Description
cfg_start	input	1	Pulse to begin a run (seeds all channels)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use LFSR_SEED param)

Port	Direction	Width	Description
cfg_channel_mask	input	NUM_CHANNELS	Active channels (0 → all channels active)
cfg_num_beats	input	BEAT_COUNT_WIDTH	Beats to send per channel
cfg_beats_per_pkt	input	BEAT_COUNT_WIDTH	last cadence (0 → one packet per channel)
cfg_tdest	input	AXIS_DEST_WIDTH	Value driven on tdest
cfg_busy	output	1	High while running (state != IDLE)
cfg_done	output	1	One-cycle pulse at end of run

24.4.3 Per-Channel Telemetry

Port	Direction	Width	Description
o_expected_crc	output	NUM_CHANNELS × 32	Per-channel expected CRC-32
o_expected_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid
o_beat_count	output	NUM_CHANNELS × 32	Per-channel beats emitted
o_beat_count_total	output	32	Sum across channels (harness stop trigger)

24.4.4 AXI-Stream Master (m_axis)

Port	Direction	Width	Description
m_axis_tvalid	output	1	Beat valid (asserted in the RUN state)
m_axis_tready	input	1	Downstream ready
m_axis_tdata	output	AXIS_DATA_WIDTH	{REP{lfsr_out[ch]}} — 32-bit LFSR replicated to fill the bus
m_axis_tstrb	output	STRB_WIDTH	All-ones (full beat)

Port	Direction	Width	Description
b			
m_axis_tlast	output	1	Packet last (per cfg_beats_per_pkt, and each channel's final beat)
m_axis_tidx	output	AXIS_ID_WIDTH	Channel index of the current beat
m_axis_tdest	output	AXIS_DEST_WIDTH	Driven from cfg_tdest
m_axis_tuser	output	AXIS_USER_WIDTH	Tied to 0

24.5 Functional Description

24.5.1 Scheduling FSM

A three-state FSM (IDLE, RUN, DONE) sequences a run. On `cfg_start` the effective channel mask, beats-per-channel, and `tlast` cadence are latched; if there are no beats or no active channel it drops straight to DONE, otherwise it selects the first active channel and enters RUN. DONE pulses `cfg_done` for one cycle and returns to IDLE. `m_axis_tvalid` is asserted exactly in RUN.

24.5.2 Sequential Per-Channel Scheduling

Channels are streamed one at a time in ascending index order among the masked set. A `f_next_active_after` function priority-scans the mask for the lowest active channel index strictly greater than the current one (or the first active channel at start). When the current channel's beats are exhausted (`w_ch_last_beat`), the FSM advances to the next active channel and reloads its beat count; when none remain it goes to DONE. Because channels run to completion in turn, each channel's LFSR sequence is contiguous and uninterrupted.

24.5.3 Per-Channel LFSR + CRC (Verbatim from the AXI4 blocks)

Each channel owns a `shifter_lfsr_fibonacci` (32-bit, taps {32,22,2,1}, seed `w_seed ^ ch`) and a `dataint_crc` (CRC-32/Ethernet, `cascade_sel = 4'b1000`). The active channel advances on its accepted beat (`ch_beat = w_beat && r_ch == ch`); all channels reload their seed / clear their CRC on the global load pulse (`cfg_start` in IDLE). The seed, taps, replication, CRC instantiation, and gating are copied verbatim from `axi4_slave_rd_pattern_gen` so the per-channel 32-bit CRC is bit-identical. Per-channel valid flags and beat counters track alongside, and `o_beat_count_total` is a combinational sum for the harness stop trigger.

24.5.4 Stream Outputs and tlast Cadence

`m_axis_tdata` is the active channel's LFSR output replicated REP times; `tid` carries the channel index; `tstrb` is all-ones (full beats); `tdest` is driven from `cfg_tdest`; `tuser` is 0. `tlast` asserts on each channel's final beat, and additionally every `cfg_beats_per_pkt` beats when that cadence is non-zero (0 → one packet spanning the channel's whole run). A per-channel packet counter (`r_pkt_cnt`) resets at each `tlast` and at channel boundaries.

24.5.5 Backpressure Insensitivity

Because the LFSR and CRC advance only on accepted beats (`w_beat = tvalid && tready`), a downstream `tready` stall simply holds the current beat — the emitted value for beat *N* of channel *C* is always `f(seed ^ C, N)` regardless of stall pattern. This is what lets the exported per-channel CRC serve as a stable golden value.

24.6 Usage Example

```
// AXIS stimulus for a 4-channel RAPIDS sink characterization run.
axis4_master_pattern_gen #(
    .NUM_CHANNELS      (4),
    .AXIS_DATA_WIDTH   (512),
    .AXIS_ID_WIDTH     (8),
    .LFSR_SEED         (32'hDEADBEEF)
) u_axis_gen (
    .clk (clk),
    .rst_n (rst_n),

    // Control
    .cfg_start          (csr_start_pulse),
    .cfg_lfsr_seed       (32'd0),           // use param default
    .cfg_channel_mask    (4'b1111),        // all channels
    .cfg_num_beats       (32'd1024),        // per channel
    .cfg_beats_per_pkt   (32'd16),          // tlast every 16 beats
    .cfg_tdest           (4'd0),
    .cfg_busy            (gen_busy),
    .cfg_done            (gen_done),

    // Golden CRCs for the downstream checker
    .o_expected_crc      (gen_crc),
    .o_expected_crc_valid (gen_crc_valid),
    .o_beat_count        (gen_beats),
    .o_beat_count_total   (gen_beats_total),

    // Stream to the DUT sink
    .m_axis_tvalid (s_tvalid), .m_axis_tready (s_tready),
```

```
.m_axis_tdata (s_tdata), .m_axis_tstrb (s_tstrb),  
.m_axis_tlast (s_tlast), .m_axis_tid (s_tid),  
.m_axis_tdest (s_tdest), .m_axis_tuser (s_tuser)  
);
```

24.7 Design Notes

- **CRC-consistency by construction:** the LFSR and CRC blocks are copied verbatim from `axi4_slave_rd_pattern_gen`, so a stream emitted here and re-CRC'd by `axi4_slave_wr_crc_check` (or checked by `axis4_slave_pattern_check`) yields identical per-channel CRCs.
 - **Sequential scheduling keeps sequences contiguous:** finishing one channel before starting the next means each channel's LFSR runs as an unbroken sequence, matching how the checker regenerates it.
 - **No address channel:** as a pure stream the generator is simpler than the AXI4 pattern gens — no `dma_address_gen`, no burst FSM, just per-channel beat counting.
 - **Backpressure-safe:** advancing only on accepted beats decouples the data sequence from `tready` timing.
 - **Data width constraint:** `AXIS_DATA_WIDTH` must be a multiple of `LFSR_WIDTH` since `tdata = REP × lfsr_out`.
-

24.8 Related Modules

24.8.1 Used By

- `projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip AXIS sink stimulus
- RAPIDS sink-path characterization flows

24.8.2 Uses

- **`shifter_lfsr_fibonacci.sv`** — per-channel maximal-length Fibonacci LFSR
- **`dataint_crc.sv`** — per-channel CRC-32 accumulator

24.8.3 See Also

- **`axis4_slave_pattern_check.sv`** — the matching AXIS checker (same LFSR/CRC config)

- **axi4_slave_rd_pattern_gen.sv** — the AXI4 read pattern source this is copied from
 - **axi4_slave_wr_crc_check.sv** — CRC sink whose values match this generator
-

24.9 References

24.9.1 Source Code

- RTL: rtl/amba/shared/axis4_master_pattern_gen.sv

24.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Harness:
projects/components/dmas/rapids/CONTROL_ENGINE_INTEGRATION.md
-

Last Updated: 2026-07-15

24.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

25 AXI-Stream Slave Pattern Checker

Module: axis4_slave_pattern_check.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

25.1 Overview

axis4_slave_pattern_check is an AXI-Stream slave that consumes a stream and checks it against a deterministic, per-channel LFSR pattern, computing a per-channel CRC-32 as it goes. It is the AXIS *source* checker for the RAPIDS characterization harness and is pattern/CRC-consistent with axi4_slave_rd_pattern_gen: the per-channel CRC-32 it computes is bit-identical to

`axi4_slave_rd_pattern_gen.read_crc_value` for the same stream (self-check path: `rd_gen` → `m_axis` → `axis_check`).

25.1.1 Key Features

- AXI-Stream slave (`s_axis_*`), pure sink with `tready` driven by a `ready_en` input to model backpressure
 - Per-channel independent LFSR (seed ^ `ch`) and CRC-32, copied verbatim from `axi4_slave_rd_pattern_gen`
 - Incoming beats demuxed by `s_axis_tid[CIW-1:0]` into the matching channel context
 - Per-beat compare against the locally regenerated pattern; sticky `o_data_error` on any mismatch
 - Per-channel actual-CRC / beat-count telemetry plus aggregate beat and packet (`tlast`) counters
 - LFSR advances only on accepted beats, so the check is independent of upstream stalls and cross-channel interleave
-

25.2 Module Purpose

Characterizing a stream-*producing* engine (a “source”) needs a checker that knows what the engine should emit. This block seeds every channel identically to the generator and, on each accepted beat, regenerates the expected pattern for that channel and compares it against the received `tdata`. It also folds the regenerated data into a per-channel CRC-32 for a whole-run summary. Because the LFSR advances only on accepted beats and is demuxed by `tid`, the check is robust to backpressure and to arbitrary interleave of channels on the shared stream.

Use Cases: - Terminating and verifying a RAPIDS source engine’s AXIS output during characterization - Source self-check: check a stream produced from `axi4_slave_rd_pattern_gen` and compare CRCs - Per-channel integrity checking under backpressure (via `ready_en`) - On-chip (FPGA) stream checker in the RAPIDS characterization harness

Key Benefit: A memory-free stream checker that both pinpoints (sticky `o_data_error` per beat) and summarizes (per-channel CRC-32 identical to the generator) integrity, robust to stalls and channel interleave.

25.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts; channel = tid low bits
AXIS_DATA_WIDTH	int	512	Stream data width (must be a multiple of LFSR_WIDTH)
AXIS_ID_WIDTH	int	8	tid width
AXIS_DEST_WIDTH	int	4	tdest width
AXIS_USER_WIDTH	int	1	tuser width
LFSR_WIDTH	int	32	LFSR width (fixed; must match the generator)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses seed ^ N
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input
CRC_REFOUT	int	1	Reflect output
BEAT_COUNT_WIDTH	int	32	Width of beat / packet counters
STRB_WIDTH	int	AXIS_DATA_WIDTH/8	Derived: tstrb width
REP	int	AXIS_DATA_WIDTH/TH/	Derived: 32-bit LFSR copies per beat

Parameter	Type	Default	Description
CIW	int	LFSR_WIDTH (NUM_CHANNELS > 1) ? \$clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: LFSR + CRC parameters must match axi4_slave_rd_pattern_gen / axis4_master_pattern_gen for cross-block CRC consistency.

25.4 Port Groups

25.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Stream clock
rst_n	input	1	Active-low asynchronous reset

25.4.2 Configuration / Control

Port	Direction	Width	Description
cfg_start	input	1	Pulse to arm and seed all channels (also clears errors/counters)
cfg_lfsr_seeded	input	LFSR_WIDTH	Seed override (0 → use LFSR_SEED param)
ready_en	input	1	Drives s_axis_tready (tie high for a pure sink; deassert to model backpressure)

25.4.3 Telemetry

Port	Direction	Width	Description
o_actual_crc	output	NUM_CHANNELS × 32	Per-channel actual CRC-32

Port	Direction	Width	Description
o_actual_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid
o_data_error	output	1	Sticky: any beat mismatched the expected pattern
o_beat_count	output	NUM_CHANNELS × 32	Per-channel beats received
o_beat_count_total	output	32	Sum across channels (harness stop trigger)
o_pkt_count	output	BEAT_COUNT_WIDTH	Aggregate total beats received

25.4.4 AXI-Stream Slave (s_axis)

Port	Direction	Width	Description
s_axis_tvalid	input	1	Beat valid
s_axis_tready	output	1	Ready (= ready_en)
s_axis_tdata	input	AXIS_DATA_WIDTH	Received data (compared to {REP{lfsr_out[ch]}})
s_axis_tstrob	input	STRB_WIDTH	Byte strobes (not used by the check)
s_axis_tlast	input	1	Packet last (counted into o_pkt_count)
s_axis_tid	input	AXIS_ID_WIDTH	Channel selector (tid[CIW-1:0])
s_axis_tdest	input	AXIS_DEST_WIDTH	Destination (unused)
s_axis_tuser	input	AXIS_USER_WIDTH	User sideband (unused)

25.5 Functional Description

25.5.1 Readiness and Beat Handshake

`s_axis_tready` is driven directly from `ready_en` — tie it high for a pure, always-ready sink, or deassert it to model downstream backpressure. An accepted beat is `w_beat = tvalid && tready`. Because the same handshake gates both the compare and the LFSR/CRC advance, the checker and the upstream generator advance in lockstep. The active channel for a beat is `s_axis_tid[CIW-1:0]`.

25.5.2 Per-Channel LFSR + CRC Regeneration (Verbatim from the Generator)

Each channel owns a `shifter_lfsr_fibonacci` (32-bit, taps {32,22,2,1}, seed `w_seed ^ ch`) and a `dataint_crc` (CRC-32/Ethernet, `cascade_sel = 4'b1000`). On `cfg_start` (the arm/load pulse) every channel reloads its seed and clears its CRC / counters. A channel advances only on an accepted beat carrying its `tid` (`ch_beat = w_beat && w_ch == ch`). The seed, taps, replication, CRC instantiation, and gating are copied verbatim from `axi4_slave_rd_pattern_gen`, so the per-channel CRC is bit-identical. Each channel exposes `o_actual_crc`, `o_actual_crc_valid`, and `o_beat_count`; `o_beat_count_total` is a combinational sum.

25.5.3 Per-Beat Compare and Sticky Error

The expected data for the active channel is its current LFSR value replicated to the bus width (`expected_data_per_ch[w_ch] = {REP{lfsr_out[ch]}}`), sampled *before* the LFSR advances on the same edge. A mismatch (`w_beat && tdata != expected`) latches `o_data_error` sticky until the next `cfg_start`. Because the compare uses the pre-advance value and the LFSR steps only on accepted beats, the check is independent of upstream stalls and of how channels interleave on the shared stream.

25.5.4 Packet Counting

`o_pkt_count` increments on every accepted beat that carries `tlast`, giving an aggregate packet count across channels. Both `o_data_error` and `o_pkt_count` are cleared by `cfg_start`.

25.6 Usage Example

```
// AXIS checker terminating a 4-channel RAPIDS source engine.
axis4_slave_pattern_check #(
    .NUM_CHANNELS      (4),
    .AXIS_DATA_WIDTH   (512),
    .AXIS_ID_WIDTH     (8),
    .LFSR_SEED         (32'hDEADBEEF)
) u_axis_chk (
    .clk (clk),
    .rst_n (rst_n),
```



```

// Control
.cfg_start      (csr_arm_pulse),
.cfg_lfsr_seed  (32'd0),          // use param default
.ready_en       (sink_ready),    // 1 = always ready; toggle to
backpressure

// Telemetry to the harness CSR block
.o_actual_crc    (chk_crc),
.o_actual_crc_valid (chk_crc_valid),
.o_data_error    (chk_data_error),
.o_beat_count    (chk_beats),
.o_beat_count_total (chk_beats_total),
.o_pkt_count     (chk_pkts),

// Stream from the DUT source
.s_axis_tvalid   (m_tvalid), .s_axis_tready (m_tready),
.s_axis_tdata    (m_tdata),  .s_axis_tstrb  (m_tstrb),
.s_axis_tlast    (m_tlast),  .s_axis_tid    (m_tid),
.s_axis_tdest    (m_tdest),  .s_axis_tuser  (m_tuser)
);

// Clean check: no mismatch and per-channel CRC matches the
generator's.
assign stream_ok = !chk_data_error && (chk_crc[ch] ==
gen_expected_crc[ch]);

```

25.7 Design Notes

- **CRC-consistency by construction:** the LFSR and CRC blocks are copied verbatim from `axi4_slave_rd_pattern_gen`, so a stream produced by that generator (or by `axis4_master_pattern_gen`) yields identical per-channel CRCs here.
- **Compare uses the pre-advance LFSR value:** the expected data is sampled before the channel's LFSR steps on the same clock edge, so the beat is compared against the value the generator emitted, not the next one.
- **Two integrity signals:** sticky `o_data_error` pinpoints that *some* beat disagreed, while the per-channel CRC lets the harness confirm the exact stream matched end-to-end.

- **Backpressure via ready_en:** exposing readiness as a config input lets sweeps drive the checker as an always-ready sink or as a throttling one without extra logic.
 - **Data width constraint:** `AXIS_DATA_WIDTH` must be a multiple of `LFSR_WIDTH` since the expected data is $\text{REP} \times \text{lfsr_out}$.
-

25.8 Related Modules

25.8.1 Used By

- `projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip AXIS source checker
- RAPIDS source-path characterization flows

25.8.2 Uses

- `shifter_lfsr_fibonacci.sv` — per-channel maximal-length Fibonacci LFSR
- `dataint_crc.sv` — per-channel CRC-32 accumulator

25.8.3 See Also

- `axis4_master_pattern_gen.sv` — the matching AXIS generator (same LFSR/CRC config)
 - `axi4_slave_rd_pattern_gen.sv` — the AXI4 read pattern source this is copied from
 - `axi4_master_rd_crc_check.sv` — the AXI4 read-side per-beat compare + CRC counterpart
-

25.9 References

25.9.1 Source Code

- RTL: `rtl/amba/shared/axis4_slave_pattern_check.sv`

25.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`
 - Index: `docs/markdown/RTLAmba/index.md`
 - Harness:
`projects/components/dmas/rapids/CONTROL_ENGINE_INTEGRATION.md`
-

25.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

26 AXIS Bus Meter

Module: `axis_bus_meter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

26.1 Overview

The AXIS Bus Meter is the AXI-Stream analogue of `axi_bus_meter`. It performs the same four-bucket per-cycle valid/ready classification (productive / backpressure / starvation / idle), and adds AXIS-native throughput counters that are **window-independent**: a payload-byte count derived from `tstrb` popcount and a packet count derived from `tlast`. Because bytes and packets are counted only on productive beats, they measure exactly how much data moved regardless of how long the measurement window stays open — so throughput computed as bytes / busy-time is robust to backpressure and idle padding that would distort a pure cycle-utilization figure. Like its AXI cousin, it is a pure observer and drives nothing back onto the bus.

26.1.1 Key Features

- Four-bucket per-cycle classification identical to `axi_bus_meter`
 - Window-independent AXIS throughput counters: 64-bit bytes (via `tstrb` popcount), 32-bit beats, 32-bit packets (via `tlast`)
 - Aggregate 32-bit cycle-bucket counters (~42.9 s at 100 MHz before wrap)
 - Per-channel 16-bit cycle buckets binned by `tid`, with per-channel 4-bit sticky overflow
 - Synchronous one-cycle `i_clear` and `i_freeze` window control
 - Passive snoop of `tvalid/tready/tlast/tstrb/tid` — no bus interaction
-

26.2 Module Purpose

Cycle-utilization alone is a fragile throughput proxy on a stream bus: if a window is held open for host polling after the last beat, idle cycles inflate and the utilization ratio drops even though the same number of bytes moved. The AXIS Bus Meter fixes this by counting the actual payload

transferred. Every productive beat contributes `popcount(tstrb)` bytes to a 64-bit accumulator and, when `tlast` is set, one packet to a 32-bit accumulator. These are byte-exact and independent of window length, so the honest throughput figure is `bytes / busy_time`. The four cycle buckets are retained alongside for root-causing where non-productive time went.

The block is instantiated one per AXIS bus to be measured, snooping the stream signals as a passive observer.

Use Cases: - Metering AXIS datapath throughput in the `rapids_char` characterization harness - Byte-exact throughput measurement immune to window-hold artifacts - Per-stream (per-tid) cycle-utilization breakdown on multi-stream buses - Packet-rate measurement via `tlast` on framed streams

Key Benefit: Window-independent byte and packet counts give a throughput number that cannot be distorted by how long the host holds the measurement window open.

26.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	512	Stream data width in bits (sets <code>tstrb</code> /byte-lane width)
NUM_CHANNELS	int	8	Number of per-channel (tid) bins
TID_WIDTH	int	$(\text{NUM_CHANNELS} > 1) ? \lceil \log_2(\text{NUM_CHANNELS}) \rceil : 1$	tid bus width; derived, do not override
SW	int	<code>DATA_WIDTH / 8</code>	<code>tstrb</code> width (byte lanes); derived
CW	int	$(\text{NUM_CHANNELS} > 1) ? \lceil \log_2(\text{NUM_CHANNELS}) \rceil : 1$	Channel-bin index width; derived

26.4 Port Groups

26.4.1 Clock and Reset

Port	Direction	Width	Description
<code>aclk</code>	input	1	Clock

Port	Direction	Width	Description
aresetn	input	1	Active-low asynchronous reset

26.4.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	Synchronous one-cycle clear pulse; zeroes every counter and sticky
i_freeze	input	1	Hold high to close the measurement window (no counter moves)

26.4.3 AXIS Bus Snoop

Port	Direction	Width	Description
i_tvalid	input	1	Stream tvalid
i_tready	input	1	Stream tready
i_tlast	input	1	Stream tlast (increments packet count on a productive beat)
i_tstrb	input	SW	Byte-lane strobes; popcount gives this beat's payload byte count
i_tid	input	TID_WIDTH	Stream id; its low CW bits select the per-channel bin

26.4.4 Aggregate Cycle Buckets

Port	Direction	Width	Description
o_agg_prod uctive	output	32	Cycles with tvalid && tready (beat transferred)
o_agg_back pressure	output	32	Cycles with tvalid && ! tready
o_agg_star	output	32	Cycles with !tvalid &&

Port	Direction	Width	Description
vation			tready
o_agg_idle	output	32	Cycles with !tvalid && !tready

26.4.5 Aggregate AXIS-Native Throughput (productive beats only)

Port	Direction	Width	Description
o_agg_bytes	output	64	Sum of popcount(tstrb) over productive beats — exact payload bytes moved
o_agg_beats	output	32	Productive beat count (equals o_agg_productive; provided for convenience)
o_agg_packets	output	32	Productive beats with tlast asserted (stream packets)

26.4.6 Per-Channel Cycle Buckets (binned by tid)

Port	Direction	Width	Description
o_ch_productive	output	16 × NUM_CHANNELS	Per-channel productive-cycle counts
o_ch_backpressure	output	16 × NUM_CHANNELS	Per-channel backpressure-cycle counts
o_ch_starvation	output	16 × NUM_CHANNELS	Per-channel starvation-cycle counts
o_ch_idle	output	16 × NUM_CHANNELS	Per-channel idle-cycle counts (always zero — see Design Notes)
o_ch_overflow	output	NUM_CHANNELS*4	Sticky overflow mask, packed {prod, bp, starv, idle} per channel

26.5 Functional Description

26.5.1 Bucket Classification

Identical to `axi_bus_meter`: the handshake pair decodes combinationally into four mutually-exclusive buckets.

```
w_prod = i_tvalid && i_tready // productive – beat transferred
w_bp   = i_tvalid && !i_tready // backpressure – master wants to
send, slave stalls
w_starv = !i_tvalid && i_tready // starvation – slave ready,
master not producing
w_idle  = !i_tvalid && !i_tready // idle – both sides quiet
```

See `DMA_UTILIZATION_MEASUREMENT.md` §3 for the reference methodology.

26.5.2 Payload-Byte Popcount

Each beat's byte count is the number of asserted `tstrb` lanes, computed combinationally by summing the strobe bits into `w_beat_bytes`. If the stream has no meaningful `tstrb`, tie it all-ones so bytes accumulate as `beats × (DATA_WIDTH/8)`.

26.5.3 AXIS-Native Throughput Counters

On every productive beat (`tvalid && tready`, window open):

- `o_agg_bytes += this beat's popcount(tstrb)` — a 64-bit accumulator that will not wrap in any realistic run.
- `o_agg_beats += 1` — a convenience mirror of `o_agg_productive`.
- `o_agg_packets += 1` when `i_tlast` is set — counts framed stream packets.

Because these advance only on productive beats, they are window-length independent: holding `i_freeze` low longer only accumulates more idle cycles in the aggregate buckets, never more bytes or packets.

26.5.4 Per-Channel Cycle Buckets

Per-channel buckets are 16-bit, `NUM_CHANNELS` deep, binned by the low CW bits of `tid` (`w_ch`). `AXIS` carries `tid` on every valid cycle, so the productive, backpressure, and starvation buckets are attributed to `w_ch` whenever the bus is active. Each per-channel counter has a companion sticky overflow bit (`{prod, bp, starv, idle}` packed into `o_ch_overflow`) that latches when the counter would advance past `16'hFFFF`, exactly as in `axi_bus_meter`. Software discards the per-channel numbers for any channel whose overflow mask is nonzero.

26.5.5 Clear and Freeze Semantics

`i_clear` is a synchronous one-cycle pulse that zeroes every counter and sticky. `i_freeze` holds all counters frozen while high and is driven from the characterization window controller so the window closes the moment the workload finishes.

26.6 Usage Example

```
// Meter an AXIS bus: cycle buckets + byte/packet throughput, per-tid
bins.
axis_bus_meter #(
    .DATA_WIDTH      (512),
    .NUM_CHANNELS    (8)
) u_axis_meter (
    .aclk              (aclk),
    .aresetn           (aresetn),

    .i_clear           (perf_run_rising),
    .i_freeze          (~perf_run),

    // Stream snoop (all inputs – passive)
    .i_tvalid          (s_axis_tvalid),
    .i_tready          (s_axis_tready),
    .i_tlast           (s_axis_tlast),
    .i_tstrb           (s_axis_tstrb),           // tie all-ones if the
stream has no tstrb
    .i_tid             (s_axis_tid),

    // Aggregate cycle buckets
    .o_agg_productive  (agg_prod),
    .o_agg_backpressure (agg_bp),
    .o_agg_starvation  (agg_starv),
    .o_agg_idle        (agg_idle),

    // Window-independent throughput
    .o_agg_bytes       (agg_bytes),
    .o_agg_beats        (agg_beats),
    .o_agg_packets      (agg_packets),

    // Per-channel (by tid)
    .o_ch_productive   (ch_prod),
    .o_ch_backpressure (ch_bp),
    .o_ch_starvation   (ch_starv),
    .o_ch_idle         (ch_idle),
    .o_ch_overflow      (ch_overflow)
```



```
);
```

```
// Honest throughput = o_agg_bytes / busy_time (immune to window-hold padding)
```

26.7 Design Notes

26.7.1 Why Byte/Packet Counters Are Window-Independent

A pure cycle-utilization ratio drops if the window is held open past the last beat (host polling adds idle cycles). Byte and packet counters advance only on productive beats, so they capture the true transferred payload no matter how long the window is held. Reporting `bytes / busy_time` sidesteps the padding-sensitivity of `productive / total_cycles`.

26.7.2 Per-Channel Idle Is Not Attributed

Idle cycles (`!tvalid && !tready`) cannot be assigned to any stream — no `tid` is meaningful when nothing is on the bus — so `r_ch_idle` is never incremented and `o_ch_idle` stays zero. Idle cycles land only in the aggregate `o_agg_idle`. The per-channel idle output and its overflow bit are kept for structural symmetry with `axi_bus_meter`.

26.7.3 tstrb Handling

If the source does not drive a meaningful `tstrb`, tie it all-ones so `o_agg_bytes` equals `beats × (DATA_WIDTH/8)`. Otherwise `o_agg_bytes` reflects sparse/partial beats exactly.

26.7.4 Counter Widths

Cycle buckets are 32-bit (aggregate) / 16-bit (per-channel) matching `axi_bus_meter`. Bytes are 64-bit to guarantee no wrap; beats and packets are 32-bit.

26.8 Related Modules

26.8.1 Used By

- `rapids_char` AXIS datapath meters (characterization harness)
- Host-driven AXIS throughput characterization runs

26.8.2 Uses

- `reset_defs.svh` - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

26.8.3 See Also

- **axi_bus_meter.sv** - AXI channel four-bucket meter (this module's origin)
 - **axi_perf_latency_hist.sv** - Latency histogram sharing the same window-control convention
-

26.9 References

26.9.1 Source Code

- RTL: rtl/amba/shared/axis_bus_meter.sv

26.9.2 Documentation

- Methodology: docs/.../DMA_UTILIZATION_MEASUREMENT.md (four-bucket definition, window semantics)
 - Architecture: docs/markdown/RTLAmba/shared/README.md
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2026-07-15

26.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

27 Clock-Gated Variants Guide

Category: Infrastructure Documentation **Location:**

rtl/amba/shared/amba_clock_gate_ctrl.sv, rtl/common/clock_gate_ctrl.sv **Applies**

To: All AMBA modules with a _cg suffix **Status:** Production Ready (transport modules) — see

[Known Gaps](#) for monitor variants

27.1 Overview

Most AMBA protocol modules have a clock-gated (_cg) variant that wraps the base module, drives it from a gated clock, and gates that clock after a programmable idle period. The gating decision is

made entirely at runtime through configuration inputs — there is no compile-time enable parameter on the wrapper.

Naming Convention: {base_module}_cg.sv

Examples: - axi4_master_rd.sv → axi4_master_rd_cg.sv - apb_slave.sv → apb_slave_cg.sv - axis_master.sv → axis_master_cg.sv

27.2 Key Principle

Clock-gated variants preserve the functional behavior of the base module and add runtime power management.

Base Module + amba_clock_gate_ctrl + activity detection = _cg Variant

All base-module parameters and ports are preserved. The wrapper adds one parameter, two configuration inputs, and one or two status outputs.

Note that a _cg variant is not cycle-identical to its base module. Most _cg wrappers force the relevant *ready signals low while the clock is gated, so the first transfer that arrives out of a gated period is backpressured for the wake-up cycle. Transfers are delayed, never dropped. See [Wake-Up Behavior](#).

27.3 Gating Infrastructure

The gating logic lives in two shared modules, not in the wrappers themselves.

27.3.1 amba_clock_gate_ctrl (rtl/amba/shared/)

The AMBA-facing adapter. It ORs the two activity inputs, registers the result into r_wakeup, exposes idle = ~r_wakeup, and passes r_wakeup to the generic controller.

Port	Direction	Description
clk_in	Input	Ungated clock
aresetn	Input	Asynchronous active-low reset
cfg_cg_enable	Input	Global gating enable (0 = clock always runs)
cfg_cg_idle_count	Input	Idle countdown value, CG_IDLE_COUNT_WIDTH bits
user_valid	Input	Any user-side valid/activity

Port	Direction	Description
		signal
axi_valid	Input	Any bus-side valid/activity signal
clk_out	Output	Gated clock
gating	Output	Clock currently gated
idle	Output	No activity seen in the previous cycle

27.3.2 clock_gate_ctrl (rtl/common/)

The generic countdown controller. It loads `cfg_cg_idle_count` on reset, on wakeup, and whenever `cfg_cg_enable` is low; otherwise it decrements to zero and holds. The gate condition is:

```
w_gate_enable = cfg_cg_enable && !wakeup && (r_idle_counter == 0);
```

`w_gate_enable` drives the `icg` cell (inverted, since the cell enable is active-high) and is also driven out as `gating`.

27.4 Additional Parameter (All _cg Modules)

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of the idle counter. Maximum idle count is $2^{CG_IDLE_COUNT_WIDTH} - 1$ (15 cycles at the default width). Sets the width of <code>cfg_cg_idle_count</code> .

`CG_IDLE_COUNT_WIDTH` is a counter **width**, not a cycle count. To change the idle threshold at runtime, drive `cfg_cg_idle_count`; widen `CG_IDLE_COUNT_WIDTH` only when the threshold you need exceeds the current counter range.

All other parameters are identical to the base module.

27.5 Additional Ports (All _cg Modules)

27.5.1 Configuration Inputs

Port	Direction	Width	Description
cfg_cg_enable	Input	1	Enable clock gating. 0 = clock always runs (runtime bypass).
cfg_cg_idle_count	Input	CG_IDLE_COUNT_WIDTH	Idle countdown value. Gating engages $\text{cfg_cg_idle_count} + 1$ clocks after the internal wakeup deasserts, which is $\text{cfg_cg_idle_count} + 2$ clocks after the last bus activity on single-stage families and $+ 3$ on two-stage families. See Gating Latency .

27.5.2 Status Outputs

Status port naming differs by protocol family. Check the module port list before wiring.

Family	Status Ports
AXI4, AXI5, AXI4-Lite, AXI-Stream (axis4)	cg_gating, cg_idle
AXI5 monitor (*_mon_cg)	cg_gating, cg_idle
AXI5-Stream (axis5)	axis_clock_gating
APB, APB5 (non-CDC)	apb_clock_gating (the controller idle output is left unconnected)
APB5 CDC (apb5_slave_cdc_cg)	apb_clock_gating
APB CDC (apb_slave_cdc_cg)	pclk_cg_gating, pclk_cg_idle, aclk_cg_gating, aclk_cg_idle (two gating domains, two controller instances)

All other ports are identical to the base module.

27.5.3 Ports That Do Not Exist

These are common misconceptions; none of them are present in the RTL:

- **No `cg_clk_count` / gated-cycle counter.** The wrappers expose instantaneous state only. Accumulate `cg_gating` externally if a power metric is needed — see [Measuring Gated Cycles](#).
 - **No scan or test-bypass port.** For DFT, hold `cfg_cg_enable` low (`.cfg_cg_enable(~scan_mode)`).
 - **No `ENABLE_CLOCK_GATING`, `CG_IDLE_CYCLES`, or `CG_GATE_*` parameters** on any transport-level `_cg` module. Those names appear only in the legacy AXI4 and AXI4-Lite `*_mon_cg` wrappers described under [Known Gaps](#).
-

27.6 Clock Gating Behavior

27.6.1 Activity Detection

Each wrapper builds `user_valid` and `axi_valid` from the valid signals (and, on the AXI transport wrappers, the base module's busy output) of the channels it owns. For example, `axi4_master_rd_cg`:

```
assign user_valid = fub_axi_arvalid || fub_axi_rready || int_busy;  
assign axi_valid  = m_axi_arvalid  || m_axi_rvalid;
```

27.6.2 Gating Conditions (All Must Be True)

1. `cfg_cg_enable = 1`
2. The internal `r_wakeup` is deasserted (no activity in the previous cycle)
3. The idle counter has decremented to zero

27.6.3 Counting Convention

Two numbers are quoted throughout this book, and they must not be confused:

- **Register stages** — how many flops an activity edge passes through before it reaches the ICG enable.
- **First usable gated-clock edge** — how many clocks after activity asserts before the gated block sees a rising edge it can work on. This is always one more than the register stage count, because the ICG enable itself is combinational and the released clock is only usable on the following edge.

Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. The first gated-clock rising edge available

to the block therefore arrives **2 clocks** (single-stage families) or **3 clocks** (two-stage families) after activity asserts.

clock_gate_ctrl itself adds **no** flop on the wake-up path: w_gate_enable is a combinational function of wakeup, feeding the icg enable directly. Its header comment “Wakeup: 1 clock from wakeup assertion to clock restoration” describes the resulting clock edge, not a register stage.

27.6.4 Register Stages by Family

Family	Wrapper flop	amba_clock_gate_ctrl flop	Stages	First usable edge
AXI4, AXI5, AXI4- Lite _cg	No (combinatio nal)	Yes	1	2 clocks
axis_ maste r_cg, axis_ slave _cg (AXI4- Strea m)	No (combinatio nal)	Yes	1	2 clocks
All *_mon _cg monit or wrap pers	No (combinatio nal)	Yes	1	2 clocks
apb_s lave_ cdc_ cg	No (combinatio nal)	Yes	1	2 clocks
apb_m aster _cg, apb_s lave_ cg	Yes	Yes	2	3 clocks
apb5_ maste	Yes	Yes	2	3 clocks

Family	Wrapper flop	amba_clock_gate_ctrl flop	Stages	First usable edge
r_cg, apb5_ slave _cg, apb5_ slave _cdc_ cg				
axis5 _mast er_cg, axis5 _slav e_cg (AXI5- Strea m)	Yes	Yes	2	3 clocks

Note: the AXI5-Stream _cg wrappers register activity locally and so behave like the APB family, not like the AXI4-Stream wrappers. Conversely apb_slave_cdc_cg drives the activity terms combinationally and so behaves like the AXI family. On both CDC wrappers the cross-domain activity term additionally pays the usual two-flop synchronizer delay in the receiving domain, on top of the stages counted above.

27.6.5 Gating Latency

gating asserts `cfg_cg_idle_count + 1` clocks after the last cycle in which the internal wakeup was high. Measured from the last bus activity, that is:

- `cfg_cg_idle_count + 2` clocks on the single-stage families (AXI4, AXI5, AXI4-Lite, AXI4-Stream, the monitor _cg wrappers, apb_slave_cdc_cg)
- `cfg_cg_idle_count + 3` clocks on the two-stage families (APB, APB5, AXI5-Stream)

The extra clock in each case is the time activity takes to appear on `r_wakeup`.

With `cfg_cg_idle_count = 0`, gating engages on the next clock after wakeup deasserts.

27.6.6 Wake-Up Behavior

Wake-up latency is not zero. Activity is registered before it can release the gate.

AXI4, AXI5, AXI4-Lite, AXI4-Stream: the wrapper combines the valid signals combinationally, so the only flop in the path is `r_wakeup` inside `amba_clock_gate_ctrl` — **1 register stage, first usable clock edge 2 clocks** after activity asserts:

Cycle N: Clock gated (`cg_gating = 1`). `ARVALID` asserts.
 `user_valid = 1` combinationally; `ARREADY` forced low, so no handshake.
 Cycle N+1: `r_wakeup = 1` -> gate released combinationally, `cg_gating = 0`.
 Cycle N+2: First usable gated-clock rising edge. `ARREADY` reflects the base
 module; the transfer can complete.

APB, APB5, AXI5-Stream: these wrappers register activity into their own `r_wakeup` flop before handing it to `amba_clock_gate_ctrl`, which registers it again — **2 register stages, first usable clock edge 3 clocks** after activity asserts.

27.6.7 Ready Forcing While Gated

The following wrappers drive the relevant `*ready` outputs to zero while gated, so the first beat out of a gated period is backpressured rather than lost:

- All AXI4, AXI5, and AXI4-Lite transport `_cg` modules
- `axis_master_cg`, `axis_slave_cg`
- All four AXI5 `*_mon_cg` modules
- `apb_slave_cdc_cg` (both clock domains)

`apb_master_cg`, `apb_slave_cg`, the APB5 variants, and the AXI5-Stream variants do not force `ready`.

```
stateDiagram-v2
    [*] --> RUNNING

    RUNNING --> COUNTING : wakeup deasserted
    COUNTING --> RUNNING : activity detected<br/>(counter reloads)
    COUNTING --> GATED : counter reaches 0<br/>&& cfg_cg_enable
    GATED --> RUNNING : activity detected<br/>(1 stage / 2 clocks to
first edge;<br/>2 stages / 3 clocks on APB, APB5, AXI5-Stream)

    state RUNNING {
        note right of RUNNING : Clock running, counter reloaded
    }

    state COUNTING {
        note right of COUNTING : Counter decrementing to 0
    }
```

```

state GATED {
    note right of GATED : Clock stopped, ready forced low
}

```

27.7 Usage Patterns

27.7.1 Pattern 1: Aggressive Gating

Use Case: Bursty traffic, low duty cycle, power-constrained systems.

```

axi4_master_rd_cg #(
    // Base module parameters (same as the non-CG variant)
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),

    // Clock gating
    .CG_IDLE_COUNT_WIDTH (4)           // counter range 0-15
) u_cg_aggressive (
    .aclk                (clk),
    .aresetn              (rst_n),

    .cfg_cg_enable        (1'b1),
    .cfg_cg_idle_count    (4'd1),      // gate 2 clocks after wakeup
deasserts

    // ... all other ports same as the base module ...

    .cg_gating            (rd_gated),
    .cg_idle               (rd_idle)
);

```

27.7.2 Pattern 2: Runtime-Adjustable Threshold

Use Case: A power-management controller trades wake-up cost against gated time.

```

logic [3:0] idle_threshold;

always_comb begin
    case (power_mode)
        POWER_HIGH_PERF: idle_threshold = 4'd15; // conservative
        POWER_BALANCED:  idle_threshold = 4'd5;  // moderate
        POWER_LOW:       idle_threshold = 4'd1;  // aggressive
        default:         idle_threshold = 4'd5;
    endcase

```

end

```
axi4_master_wr_cg #(
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg_dynamic (
    .cfg_cg_enable      (power_mgmt_enable),
    .cfg_cg_idle_count  (idle_threshold),
    .cg_gating          (wr_gated),
    .cg_idle            (wr_idle)
    // ... base module ports ...
);
```

27.7.3 Pattern 3: Gating Bypassed

Use Case: Functional verification, DFT scan, latency-critical operation.

```
axi4_master_rd_cg #(
    .CG_IDLE_COUNT_WIDTH (4)
) u_cg_bypassed (
    .cfg_cg_enable      (1'b0),           // clock always runs
    .cfg_cg_idle_count  (4'd0),           // ignored while cfg_cg_enable =
0
    // ... base module ports ...
);
```

With `cfg_cg_enable = 0` the counter is continuously reloaded, gating stays low, the ICG enable stays high, and the ready-forcing terms resolve to the base module's own ready signals. The wrapper is then behaviorally equivalent to the base module.

This is the only bypass mechanism. There is no separate scan or test-mode port.

27.8 Measuring Gated Cycles

The wrappers do not count gated cycles. Accumulate the status output where the metric is needed:

```
logic [31:0] total_cycles, gated_cycles;

always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        total_cycles <= total_cycles + 1'b1;
        if (cg_gating) gated_cycles <= gated_cycles + 1'b1;
    end
end
```

```
// Gated fraction = (gated_cycles / total_cycles) x 100%
```

Note that this counter must itself be clocked from an ungated clock.

27.9 Synthesis and Portability

27.9.1 The ICG Cell

`clock_gate_ctrl` instantiates a bare `icg` primitive by name:

```
icg u_icg (
    .clk (clk_in),
    .en  (~w_gate_enable),
    .gclk(clk_out)
);
```

`rtl/common/icg.sv` provides a behavioral model (a low-phase `always_latch` on the enable ANDed with the clock) that is correct for simulation and for a standard-cell flow where the name maps to a library integrated clock gate.

ASIC: map `icg` to the foundry integrated clock-gating cell. Verify the enable setup and hold requirements of the chosen cell, and add clock-gating checks to the timing constraints.

FPGA portability note: the `icg` behavioral model is not an FPGA-friendly construct. An inferred latch feeding an AND gate on a clock net will either be rejected or produce a glitch-prone, unconstrained clock. For FPGA targets, either

- replace `icg` with a vendor clock-enable primitive (Xilinx `BUFGCE`, Intel `ALTCLKCTRL`, and equivalents), or
- hold `cfg_cg_enable` low and use the base (non-`_cg`) module, converting the gating intent into ordinary clock enables that the synthesis tool can infer.

This is a real portability constraint, not a tuning preference. Do not push a `_cg` variant through an FPGA flow without addressing it.

27.10 Verification Considerations

27.10.1 Functional Verification

Hold `cfg_cg_enable` low for functional regression runs. This yields simpler waveforms, faster simulation, and removes the wake-up backpressure cycle from the transfer timing.

27.10.2 Gating Verification

For gating-specific tests, drive `cfg_cg_enable` high and:

1. Sweep `cfg_cg_idle_count` and confirm `cg_gating` asserts `cfg_cg_idle_count + 1` clocks after the last wakeup.
 2. Confirm the wake-up latency — 1 register stage and a first usable gated-clock edge 2 clocks after activity asserts, or 2 stages and 3 clocks on APB, APB5, and AXI5-Stream — and that no transfer is lost across a gate-to-ungate transition.
 3. Confirm that `*ready` is low for the whole gated interval on the wrappers listed under [Ready Forcing While Gated](#).
 4. Confirm `cfg_cg_enable = 0` reproduces base-module timing exactly.
-

27.11 Available Clock-Gated Variants

27.11.1 Transport Modules (22)

Every module below instantiates `amba_clock_gate_ctrl` and takes `CG_IDLE_COUNT_WIDTH`, `cfg_cg_enable`, and `cfg_cg_idle_count`.

Protocol	Location	CG Variants
AXI4	rtl/amba/axi4/	axi4_master_rd_cg, axi4_master_wr_cg, axi4_slave_rd_cg, axi4_slave_wr_cg
AXI5	rtl/amba/axi5/	axi5_master_rd_cg, axi5_master_wr_cg, axi5_slave_rd_cg, axi5_slave_wr_cg
AXI4-Lite	rtl/amba/axil4/	axil4_master_rd_cg, axil4_master_wr_cg, axil4_slave_rd_cg, axil4_slave_wr_cg
APB	rtl/amba/apb/	apb_master_cg, apb_slave_cg, apb_slave_cdc_cg (two gating domains)

Protocol	Location	CG Variants
APB5	rtl/amba/apb5/	apb5_master_cg, apb5_slave_cg, apb5_slave_cdc_cg
AXI-Stream	rtl/amba/axis4/	axis_master_cg, axis_slave_cg
AXI5-Stream	rtl/amba/axis5/	axis5_master_cg, axis5_slave_cg

27.11.2 Monitor Modules (12)

Protocol	Location	CG Variants	Gating Implemented
AXI5	rtl/amba/ monitor/	axi5_master_rd_ mon_cg, axi5_master_wr_ mon_cg, axi5_slave_rd_m on_cg, axi5_slave_wr_m on_cg	Yes
AXI4	rtl/amba/ monitor/	axi4_master_rd_ mon_cg, axi4_master_wr_ mon_cg, axi4_slave_rd_m on_cg, axi4_slave_wr_m on_cg	No — see Known Gaps
AXI4-Lite	rtl/amba/ monitor/	axil4_master_rd_ _mon_cg, axil4_master_wr_ _mon_cg, axil4_slave_rd_ mon_cg, axil4_slave_wr_ mon_cg	No — see Known Gaps

Total: 34 _cg modules, of which 26 implement clock gating.

27.12 Known Gaps

The eight AXI4 and AXI4-Lite *_mon_cg wrappers predate amba_clock_gate_ctrl and use a different, incomplete scheme. None of them instantiates amba_clock_gate_ctrl, and none of them takes CG_IDLE_COUNT_WIDTH.

AXI4 *_mon_cg (axi4_master_rd_mon_cg, axi4_master_wr_mon_cg, axi4_slave_rd_mon_cg, axi4_slave_wr_mon_cg):

- Carry legacy parameters ENABLE_CLOCK_GATING, CG_IDLE_CYCLES, CG_GATE_MONITOR, CG_GATE_REPORTER, CG_GATE_TIMERS, and legacy inputs cfg_cg_idle_threshold, cfg_cg_force_on, cfg_cg_gate_monitor, cfg_cg_gate_reporter, cfg_cg_gate_timers.
- Build per-domain gated clocks (aclk_monitor, aclk_reporter, aclk_timers) with combinational always_comb blocks rather than ICG cells.
- **Do not connect those gated clocks to anything.** The underlying monitor is instantiated with .aclk(aclk) and an in-source TODO: Use aclk_monitor once ICG cells replace combinational gating. The gating logic is therefore dead, and the cg_monitor_gated/cg_reporter_gated/cg_timers_gated/cg_cycles_saved outputs report a model that has no effect on the clock.

AXI4-Lite *_mon_cg (axil4_master_rd_mon_cg, axil4_master_wr_mon_cg, axil4_slave_rd_mon_cg, axil4_slave_wr_mon_cg):

- Carry legacy parameters ENABLE_CLOCK_GATING and CG_IDLE_CYCLES plus inputs cfg_cg_enable and cfg_cg_idle_threshold.
- Contain no gating structure at all. cfg_cg_enable is folded into cg_monitor_enable, and cg_cycles_saved is a plain idle-cycle counter.

Implication: treat the AXI4 and AXI4-Lite *_mon_cg modules as functionally equivalent to their non-_cg base monitors for power purposes. They save no dynamic power today. The AXI5 *_mon_cg modules are the reference for how a monitor CG wrapper should look.

27.13 Related Documentation

- **Gating Controller:** [amba_clock_gate_ctrl.md](#)
- **Per-Protocol Guides:**
 - [AXI4 Clock Gating Guide](#)
 - [AXI4-Lite Clock Gating Guide](#)

- [AXI-Stream Clock Gating Guide](#)
 - **Base Module Documentation:** see the per-protocol directories (axi4/, axi5/, axil4/, apb/, apb5/, axis4/, axis5/)
 - **AMBA Overview:** [overview.md](#)
-

27.14 Navigation

- [← Back to Shared Infrastructure Index](#)
- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

28 SDPRAM Core

Module: `sdpram_core.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

28.1 Overview

`sdpram_core` is the protocol-agnostic Simple Dual-Port RAM backend shared by the four `sdpram_slave_*` protocol wrappers. It owns the inferred dual-port BRAM array, a burst-aware write tracker, a burst-aware read tracker, and a bulk-clear FSM. It exposes a single FUB-shaped slave interface (an AXI superset with `id` / `addr` / `len` / `size` / `burst` on the address channels and `id` / `resp` / `last` on the response channels) so that a protocol-specific wrapper can drop straight on top without any string-switch generate plumbing.

28.1.1 Key Features

- Inferred simple dual-port BRAM: one write port (port A) and one read port (port B)
- No reset on the RAM array (BRAM contents are undefined at power-up, cleared only via the bulk-clear FSM)
- FPGA `ram_style = "auto"` attribute for tool-driven BRAM inference
- Burst-aware write and read trackers driven by `axi_gen_addr` (INCR / FIXED any length up to AXI4's 256-beat max)
- Byte-enabled writes via `fub_wstrb`
- Single-cycle read latency on port B
- Bulk-clear FSM that walks the whole array to zero, gated on both sides idle

- FUB-shaped interface that degenerates cleanly to single-beat for AXIL wrappers
 - Observation outputs (valid/ready snapshot, BRAM write/read fire pulses)
-

28.2 Module Purpose

The `sdpram_slave_*` family needs one memory kernel that behaves identically regardless of which AMBA protocol drives its write and read sides. Rather than duplicate the BRAM, burst logic, and clear FSM in every protocol permutation, that logic lives once here, and the wrappers translate their protocol's FUB into this core's AXI-shaped FUB.

The core speaks exactly one wire format. AXI4 wrappers pass the real `awlen` / `awsize` / `awburst` / `awid` fields straight through; AXIL wrappers, which have no burst or ID fields, feed single-beat defaults (`awlen=0`, `awsize=$clog2(STRB_W)`, `awburst=INCR`, `awid=0`) so the burst tracker collapses to a single-beat path.

Use Cases: - Shared backend for all four `sdpram_slave_{axi4,axil}_{axi4,axil}` wrappers - Memory model / descriptor-RAM / semaphore-RAM inside characterization harnesses - Memory ring backend behind a monitor-bus capture master-write port - Any test bench needing a synthesizable dual-port scratch memory with a fast clear

Key Benefit: One compute kernel with all the burst and clear complexity in a single place, so the protocol wrappers stay thin and no “fake AXI4” fields leak into any external port list.

28.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	FUB transaction-ID width. Passthrough on AXI4 wrappers; tied to a 1-bit zero by AXIL wrappers
ADDR_WIDTH	int	32	Byte-address width on both FUB address channels
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
MEM_DEPTH	int	2048	Number of BRAM words (array depth)

Derived localparams: - STRB_W = DATA_WIDTH / 8 — write-strobe / byte-enable width - ADDR_LSB = $\$clog2(\text{STRB_W})$ — byte-offset bits below the word address - MEM_AW = $\$clog2(\text{MEM_DEPTH})$ — BRAM word-address width - WORD_AW = ADDR_WIDTH - ADDR_LSB — full word-address width

28.4 Port Groups

28.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset (control logic only; never resets the BRAM array)

28.4.2 FUB Write Side (AW + W + B)

Port	Direction	Width	Description
fub_awid	input	AXI_ID_WIDTH	Write transaction ID (echoed back on fub_bid)
fub_awaddr	input	ADDR_WIDTH	Write burst base byte-address
fub_awlen	input	8	Burst length minus 1 (0 = single beat)
fub_awsiz	input	3	Beat size (log2 bytes) fed to axi_gen_addr
fub_awburst	input	2	Burst type: INCR (2'b01) / FIXED (2'b00) / WRAP (2'b10)
fub_awvalid	input	1	Write-address valid
fub_awready	output	1	Write-address ready (accepts when no write active, no B pending, not clearing)
fub_wdata	input	DATA_WIDTH	Write data

Port	Direction	Width	Description
		H	
fub_wstrb	input	DATA_WIDT H/8	Per-byte write strobe
fub_wvalid	input	1	Write-data valid
fub_wready	output	1	Write-data ready (asserted while a write burst is active and not clearing)
fub_bid	output	AXI_ID_WID TH	Write-response ID (captured from fub_awid)
fub_bresp	output	2	Write response (OKAY 2'b00; SLVERR 2'b10 reserved for out-of-range)
fub_bvalid	output	1	Write-response valid (asserted while B pending)
fub_bready	input	1	Write-response ready

28.4.3 FUB Read Side (AR + R)

Port	Direction	Width	Description
fub_arid	input	AXI_ID_WID TH	Read transaction ID (echoed on fub_rid)
fub_araddr	input	ADDR_WIDT H	Read burst base byte-address
fub_arlen	input	8	Burst length minus 1 (0 = single beat)
fub_arsize	input	3	Beat size (log2 bytes) fed to axi_gen_addr
fub_arburst	input	2	Burst type: INCR / FIXED / WRAP
fub_arvalid	input	1	Read-address valid
fub_arready	output	1	Read-address ready (accepts when no read

Port	Direction	Width	Description
			active, not clearing)
fub_rid	output	AXI_ID_WIDTH	Read-data ID (from the inflight tracker)
fub_rdata	output	DATA_WIDTH	Read data (BRAM port-B output)
fub_rresp	output	2	Read response (OKAY / SLVERR reserved)
fub_rlast	output	1	Last-beat marker of the read burst
fub_rvalid	output	1	Read-data valid (reflects the inflight tracker)
fub_rready	input	1	Read-data ready

28.4.4 Bulk-Clear Control

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request a full-array clear (accepted only when both sides idle)
o_cfg_done_clear	output	1	Asserted when the clear walk has finished

28.4.5 Observation

Port	Direction	Width	Description
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot: {rready, rvalid, arready, arvalid, bready, bvalid, wready, wvalid, awready, awvalid}
o_dbg_bram_wr	output	1	One-cycle pulse when a BRAM port-A write fires
o_dbg_bram_rd	output	1	One-cycle pulse when a BRAM port-B read issues

28.5 Functional Description

28.5.1 FUB Interface Contract

The core speaks a single wire format — a FUB-shaped AXI superset — and nothing else. On the address channels it carries `id / addr / len / size / burst`; on the response channels it carries `id / resp / last`. Wrappers translate their protocol into this shape:

- **AXI4 wrappers** pass the real `awlen / awsize / awburst / awid` (and the AR equivalents) straight through.
- **AXIL wrappers** feed defaults: `awlen = 0, awsize = $clog2(STRB_W), awburst = 2'b01 (INCR), awid = 0`. With `len = 0`, the burst tracker degenerates to a single-beat path.

28.5.2 Write Path — Burst Tracker

On `fub_awvalid && fub_awready`, the tracker latches the burst's ID, base address, beat count (`awlen`), size, and burst type, and asserts `r_wr_active`. Each accepted W beat (`fub_wvalid && fub_wready`) writes byte-enabled data into BRAM port A at the current word address, then advances the address through `axi_gen_addr` and decrements the beat counter. When the last beat is accepted (`r_wr_beats_left == 0`), the tracker drops `r_wr_active`, latches a pending B response (`r_b_pending`), and captures the ID and response code. `fub_awready` deasserts while a write is active or a B is pending, so only one write burst is in flight at a time.

28.5.3 Read Path — Burst Tracker

On `fub_arvalid && fub_arready`, the read tracker latches the burst parameters and asserts `r_rd_active`. A read beat issues (`read_issue`) when the burst is active, the core is not clearing, and either the inflight register is empty or the current beat is being consumed (`!r_inflight || fub_rready`). Each issue reads BRAM port B at the current word address (one-cycle latency), advances the address through `axi_gen_addr`, decrements the beat count, and — on the last beat — drops `r_rd_active`. An inflight tracker holds the (`id, rresp, rlast`) for the beat currently presented on `fub_r`, driving `fub_rvalid`; it clears on handshake unless a new issue refills it the same cycle.

28.5.4 Address Generation

Both paths use an `axi_gen_addr` instance (parameterized `AW=ADDR_WIDTH, DW=ODW=DATA_WIDTH, LEN=8`) to compute the next beat address from the current address, size, and burst type. INCR and FIXED are handled directly by the glue. WRAP is computed correctly by `axi_gen_addr` but the BRAM address glue advances linearly, so the AXI4 wrappers carry a sim-only assertion flagging WRAP at the sim boundary until it has been exercised.

28.5.5 BRAM Array

The memory is an inferred dual-port array ((* ram_style = "auto" *) logic [DATA_WIDTH-1:0] r_mem [MEM_DEPTH]). It is **never reset** — BRAM contents come up undefined and are only zeroed by the bulk-clear FSM. Port A is shared between the clear FSM (when clearing) and byte-enabled writes; port B is a registered read (single-cycle latency). A benign WIDTHTRUNC lint-off covers Verilator computing an extra index bit for the never-taken clear wrap path; every index that reaches the array is provably in [0, MEM_DEPTH-1] by tracker construction.

28.5.6 Bulk-Clear FSM

The clear FSM has two states (CLR_IDLE, CLR_BUSY). It accepts i_cfg_start_clear only when both sides report idle (!r_wr_active && !r_b_pending && !r_rd_active && !r_inflight), which keeps it from glitching fub_*_ready mid-transaction. Once busy, it walks r_clear_addr from 0 to MEM_DEPTH-1, writing zero to BRAM port A each cycle, then returns to idle and asserts o_cfg_done_clear. While w_clearing is asserted, fub_awready, fub_wready, and fub_arready are all deasserted so the FSM owns port A cleanly.

28.5.7 Observation Outputs

o_dbg_fub_vr packs the ten FUB valid/ready bits for waveform-free probing. o_dbg_bram_wr pulses on a real byte-enabled write fire, and o_dbg_bram_rd pulses on each read issue.

28.6 Usage Example

sdpram_core is not usually instantiated directly — the sdpram_slave_* wrappers do that. The pattern (as used inside every wrapper) is:

```
sdpram_core #(
    .AXI_ID_WIDTH (AXI_ID_WIDTH),
    .ADDR_WIDTH   (ADDR_WIDTH),
    .DATA_WIDTH   (DATA_WIDTH),
    .MEM_DEPTH    (MEM_DEPTH)
) u_core (
    .clk      (clk),
    .aresetn  (aresetn),

    // Write FUB (from a slave-write leaf, or single-beat defaults for
    // AXIL)
    .fub_awid      (fub_awid),
    .fub_awaddr    (fub_awaddr),
    .fub_awlen     (fub_awlen),           // 8'h00 for AXIL
    .fub_awsz      (fub_awsz),           // $clog2(STRB_W) for AXIL
    .fub_awburst   (fub_awburst),        // 2'b01 (INCR) for AXIL
```

```

.fub_awvalid (fub_awvalid),
.fub_awready (fub_awready),
.fub_wdata   (fub_wdata),
.fub_wstrb   (fub_wstrb),
.fub_wvalid  (fub_wvalid),
.fub_wready  (fub_wready),
.fub_bid     (fub_bid),
.fub_bresp   (fub_bresp),
.fub_bvalid  (fub_bvalid),
.fub_bready  (fub_bready),

// Read FUB (from a slave-read leaf)
.fub_arid    (fub_arid),
.fub_araddr  (fub_araddr),
.fub_arlen   (fub_arlen),
.fub_arsize  (fub_arsize),
.fub_arburst (fub_arburst),
.fub_arvalid (fub_arvalid),
.fub_arready (fub_arready),
.fub_rid     (fub_rid),
.fub_rdata   (fub_rdata),
.fub_rresp   (fub_rresp),
.fub_rlast   (fub_rlast),
.fub_rvalid  (fub_rvalid),
.fub_rready  (fub_rready),

// Bulk clear + debug
.i_cfg_start_clear (i_cfg_start_clear),
.o_cfg_done_clear  (o_cfg_done_clear),
.o_dbg_fub_vr      (o_dbg_fub_vr),
.o_dbg_bram_wr     (o_dbg_bram_wr),
.o_dbg_bram_rd     (o_dbg_bram_rd)
);

```

28.7 Design Notes

28.7.1 No Reset on the RAM Array

The BRAM array is deliberately not reset. Resetting a large inferred RAM prevents BRAM inference on most FPGA flows and costs an enormous number of flip-flops on ASIC. Contents are undefined at power-up; software (or a harness) issues `i_cfg_start_clear` to zero the array when a defined starting state is required.

28.7.2 One Wire Format, Four Wrappers

The core exists so that the burst logic, byte-enable write, single-cycle read, and clear FSM are written exactly once. The four protocol permutations are thin wrappers that only translate their protocol's FUB into this AXI-shaped FUB. AXIL sides supply the burst/ID defaults in exactly one place per wrapper, so no external port list ever exposes a “fake AXI4” field.

28.7.3 WRAP Bursts

`axi_gen_addr` computes WRAP addresses correctly, but the BRAM glue here advances linearly. Until the WRAP path is exercised end-to-end, the AXI4 wrappers assert (sim-only) that `awburst/arbust` is not WRAP at the interface boundary. INCR and FIXED are fully supported.

28.7.4 Single Outstanding Burst Per Side

Each side accepts one burst at a time — `fub_awready` deasserts while a write is active or a B is pending, and `fub_arready` deasserts while a read is active. This keeps the tracker state minimal and matches the descriptor-RAM / scratch-memory use cases the family targets.

28.8 Related Modules

28.8.1 Used By

- `sdpram_slave_axi4_axi4.sv` — AXI4 write + AXI4 read wrapper
- `sdpram_slave_axi4_axil.sv` — AXI4 write + AXIL read wrapper
- `sdpram_slave_axil_axi4.sv` — AXIL write + AXI4 read wrapper
- `sdpram_slave_axil_axil.sv` — AXIL write + AXIL read wrapper
- Characterization harnesses (e.g. `rapids_char_harness` descriptor RAM, `stream_char_harness` memory ring)

28.8.2 Uses

- `axi_gen_addr.sv` — Next-beat address generation for both trackers
- `reset_defs.svh` — `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

28.8.3 See Also

- `sdpram_slave_axi4_axi4.sv` — the AXI4/AXI4 protocol wrapper
 - `sdpram_slave_axil_axil.sv` — the AXIL/AXIL protocol wrapper
 - `monbus_group_core.sv` — a comparable “one core + protocol wrappers” split for the monitor-bus delivery layer
-

28.9 References

28.9.1 Source Code

- RTL: rtl/amba/shared/sdpram_core.sv
- Address gen: rtl/amba/shared/axi_gen_addr.sv
- Wrappers: rtl/amba/shared/sdpram_slave_*.sv

28.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Subsystem guide: rtl/amba/CLAUDE.md
-

Last Updated: 2026-07-15

28.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

29 Simple Dual-Port BRAM Slave Family

Modules: - sdpram_slave.sv — common backend (BRAM glue + clear FSM + protocol skids) -
sdpram_slave_axi4_axi4.sv — wrapper: AXI4 write, AXI4 read -
sdpram_slave_axi4_axil.sv — wrapper: AXI4 write, AXIL read -
sdpram_slave_axil_axi4.sv — wrapper: AXIL write, AXI4 read -
sdpram_slave_axil_axil.sv — wrapper: AXIL write, AXIL read

Location: rtl/amba/shared/ **Category:** Memory / BRAM Slave **Status:** Production Ready

29.1 Overview

The sdpram_slave family provides a BRAM-backed simple-dual-port slave with **independent protocol choice on each side** (write, read). It unifies what used to be two separate modules (axi4_sdpram_slave + axil_sdpram_slave) into a single common backend plus four protocol-specific wrappers that expose the exact port shape for their configuration.

The most common deployment in this repo is the **monitor bus memory dump ring** — monbus_axil_axil_group's m_axil_* master writes the compressed (or raw) trace into a sdpram_slave_axil_axil, and the host CPU reads it back through the same slave's read port.

Wide AXI4 descriptor RAMs (e.g. `stream_char_harness::u_desc_ram`, 256-bit data, 8-bit AXI ID) instantiate `sdpram_slave_axi4_axi4`.

29.2 Why four wrappers + a backend?

SystemVerilog cannot conditionally include or exclude ports from a single module's port list. The port list is a static syntactic construct, fixed at module declaration. So to give each protocol combination an exact port shape — no spurious AXI4-only fields for the caller to tie off — the family is split:

- **One common backend** (`sdpram_slave.sv`) with the BRAM port-A/B glue, bulk-clear FSM, burst tracker, and the two protocol-skid generate blocks (gated by `WR_PROTOCOL` / `RD_PROTOCOL` string parameters). The backend's port list is the AXI4 superset; in AXIL mode the unused fields are ignored on inputs / driven to 0 on outputs.
- **Four thin wrappers**, each with the right protocol-shaped port list, that instantiate the backend with the matching parameter setting and tie off any AXI4-only fields for an AXIL side.

Wrapper	Wr ports	Rd ports
<code>sdpram_slave_axi4_axi4</code>	<code>s_axi_aw*/w*/b*</code> (full AXI4)	<code>s_axi_ar*/r*</code> (full AXI4)
<code>sdpram_slave_axi4_axil</code>	<code>s_axi_aw*/w*/b*</code> (full AXI4)	<code>s_axil_ar*/r*</code> (AXIL only)
<code>sdpram_slave_axil_axi4</code>	<code>s_axil_aw*/w*/b*</code> (AXIL only)	<code>s_axi_ar*/r*</code> (full AXI4)
<code>sdpram_slave_axil_axil</code>	<code>s_axil_aw*/w*/b*</code> (AXIL only)	<code>s_axil_ar*/r*</code> (AXIL only)

Callers pick the wrapper module name matching their fabric. The bare `sdpram_slave` remains directly callable for unusual configs (or for the existing `test_sdpram_slave.py` regression which exercises both sides independently via the parameter knobs).

29.3 Common Parameters

All wrappers expose the same scaling knobs as the backend:

Parameter	Default	Notes
ADDR_WIDTH	32	Address width.
DATA_WIDTH	256 (axi4_*) / 64 (axil_axil)	Data-bus width.
MEM_DEPTH	2048 (axi4_*) / 1024 (axil_axil)	BRAM word count.
AXI_ID_WIDTH	8	Present only when at least one side is AXI4.
USER_WIDTH	1	Present only when at least one side is AXI4.
SKID_DEPTH_AW/W/B/AR/R	2/2/2/2/4	Skid buffer depths for each channel.

29.4 Burst Support

- **AXI4 mode** supports INCR (awburst/arbust = 2'b01) and FIXED (2'b00) of any length up to AXI4's 256-beat maximum. WRAP (2'b10) is rejected by a SIMULATION-only \$error and treated as INCR in synth (the BRAM glue advances linearly).
- **AXIL mode** is single-beat by construction — the AXIL skid ties the fub-side awlen/arlen to 0, so the burst-aware backend produces exactly one beat per AW/AR. Multi-beat transactions are not expressible in AXIL anyway.

29.5 Bulk Clear

All wrappers expose:

- `i_cfg_start_clear` (input) — pulse high to start a memory-wide clear.
- `o_cfg_done_clear` (output) — pulses high when the clear FSM finishes.

The clear FSM owns BRAM port A while it walks the whole memory writing zeros. It waits for both sides idle before claiming the port, so an in-flight write or read completes cleanly before the clear begins.

29.6 Debug / Observation Outputs

Common to all wrappers (and the backend):

Output	Width	Meaning
o_dbg_vr	10	External {aw,w,b,ar,r}_{valid,ready} (AW = [9:8], W = [7:6], B = [5:4], AR = [3:2], R = [1:0])
o_dbg_fub_vr	10	Fub-side (post-skid) valid/ready for the same five channels
o_dbg_bram_wr	1	One-cycle pulse on BRAM port-A write fire
o_dbg_bram_rd	1	One-cycle pulse on BRAM port-B read fire
o_dbg_busy_wr	1	Write-side skid busy
o_dbg_busy_rd	1	Read-side skid busy

29.7 Use in the Monitor System

The `sdpram_slave_axil_axil` wrapper is the canonical SRAM-ring backend for the `monbus_axil_axil_group` master writer — both sides AXIL, so no AXI4-only fields anywhere on the harness wiring. For details on the slot stream landing in the ring, see `monbus_compressor.md` (the compressed case) and the raw-record beat layout described in `monbus_group.md`.

29.8 Test

`val/amba/test_sdpram_slave.py` exercises the bare `sdpram_slave` backend across all four (WR_PROTOCOL, RD_PROTOCOL) combinations via `@pytest.mark.parametrize`. The wrappers are thin pass-throughs so the backend regression is the load-bearing one. Sub-tests cover:

1. Single-beat AW/W/B + AR/R round-trip (all 4 combos)
2. AXI4 INCR burst write + read (AXI4-only)
3. AXI4 FIXED burst write (last beat wins) + read (AXI4-only)

4. Random fill + readback (all 4 combos)
pytest val/amba/test_sdpram_slave.py -v

29.9 Migration

If you have an old caller that instantiates the bare sdpram_slave:

```
// Old: full AXI4-shape ports, tie off AXI4-only for AXIL side
sdpram_slave #(
    .WR_PROTOCOL ("AXIL"),
    .RD_PROTOCOL ("AXIL"),
    .AXI_ID_WIDTH (1), // unused
    ...
) u_dump (
    .s_axi_awid      (1'b0),           // ← unsightly tie-off
    .s_axi_awaddr    (axil_awaddr),
    .s_axi_awlen     (8'h00),         // ← unsightly tie-off
    .s_axi_awsz      (3'h0),         // ← unsightly tie-off
    ...
);
```

After migrating to the matching wrapper:

```
// New: AXIL-only ports, no tie-offs needed
sdpram_slave_axil_axil #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (64),
    .MEM_DEPTH  (1024)
) u_dump (
    .s_axil_awaddr  (axil_awaddr),
    .s_axil_awprot  (axil_awprot),
    .s_axil_awvalid (axil_awvalid),
    .s_axil_awready (axil_awready),
    ...
);
```

The backend is unchanged so functional behavior is identical — only the port list shape and signal names differ.

29.10 Related Modules

Module	Role
monbus_axil_axil_group	Most common consumer (memory-

Module	Role
	dump ring)
monbus_compressor	Source of the compressed slot stream landing in the ring
axi4_slave_wr / axi4_slave_rd	AXI4-side skids instantiated by the backend
axil4_slave_wr / axil4_slave_rd	AXIL-side skids instantiated by the backend

30 SDPRAM Slave — AXI4 Write / AXI4 Read

Module: sdpram_slave_axi4_axi4.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

30.1 Overview

sdpram_slave_axi4_axi4 is one of four protocol-pair wrappers that place a Simple Dual-Port RAM behind two independent AMBA slave interfaces — a write-side slave and a read-side slave. This variant exposes a **full AXI4 slave** on both the write side (AW + W + B) and the read side (AR + R). It directly instantiates the native axi4_slave_wr and axi4_slave_rd skid leaf modules and pipes their FUB-side outputs into the shared sdpram_core backend.

30.1.1 Key Features

- Full AXI4 slave on both write and read sides (burst, ID, size)
- Native axi4_slave_wr + axi4_slave_rd skid leaves (no protocol emulation)
- Shared sdpram_core backend — one BRAM, one clear FSM, one burst tracker set
- Byte-enabled writes, single-cycle-latency reads
- Bulk-clear control input and done flag
- Full complement of debug taps (external and FUB valid/ready, BRAM fire pulses, leaf busy)
- Sim-only assertion flagging unvalidated WRAP bursts

30.2 Module Purpose

Characterization harnesses need a synthesizable memory that a device-under-test can read and write over AXI4 while a separate host or checker uses the other port. Splitting the write side and

read side into two independent AXI4 slaves lets one agent stream data in while another drains it out, with the BRAM as the shared medium.

This is the pure-AXI4 permutation of the family. When both the producer and consumer speak full AXI4 (bursts, IDs), this wrapper avoids any width- or burst-adaptation on either side.

Use Cases: - Memory model behind an AXI4 DUT in a characterization harness - Descriptor RAM with an AXI4 host-write port and an AXI4 engine-read port - Semaphore / scratch RAM shared between two AXI4 masters - General synthesizable dual-port memory with a fast bulk clear

Key Benefit: Full AXI4 on both sides with zero fake fields — the burst/ID information flows straight through the skid leaves into the shared core.

30.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI transaction-ID width on both sides
ADDR_WIDTH	int	32	Byte-address width
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
USER_WIDTH	int	1	AXI USER-signal width (carried by leaves, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

30.4 Port Groups

30.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

30.4.2 AXI4 Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write-address ID
s_axi_awaddr	input	ADDR_WIDTH	Write burst base address
s_axi_awlen	input	8	Burst length minus 1
s_axi_awsize	input	3	Beat size (log2 bytes)
s_axi_awburst	input	2	Burst type (INCR / FIXED / WRAP)
s_axi_awlock	input	1	Lock (carried by leaf, ignored by core)
s_axi_awcache	input	4	Cache attributes (ignored by core)
s_axi_awprot	input	3	Protection attributes (ignored by core)
s_axi_awqos	input	4	QoS (ignored by core)
s_axi_awregion	input	4	Region (ignored by core)
s_axi_awuser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_awvalid	input	1	Write-address valid

Port	Direction	Width	Description
s_axi_awready	output	1	Write-address ready
s_axi_wdata	input	DATA_WIDTH	Write data
s_axi_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axi_wlast	input	1	Write-data last beat
s_axi_wuser	input	USER_WIDTH	Write-data user sideband
s_axi_wvalid	input	1	Write-data valid
s_axi_wready	output	1	Write-data ready
s_axi_bid	output	AXI_ID_WIDTH	Write-response ID
s_axi_bresp	output	2	Write response
s_axi_buser	output	USER_WIDTH	Write-response user (hardwired to 0)
s_axi_bvalid	output	1	Write-response valid
s_axi_bready	input	1	Write-response ready

30.4.3 AXI4 Slave Read (AR + R)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read-address ID
s_axi_araddr	input	ADDR_WIDTH	Read burst base address
s_axi_arlen	input	8	Burst length minus 1
s_axi_arsize	input	3	Beat size (log2

Port	Direction	Width	Description
			bytes)
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock (ignored by core)
s_axi_arcache	input	4	Cache attributes (ignored by core)
s_axi_arprot	input	3	Protection attributes (ignored by core)
s_axi_arqos	input	4	QoS (ignored by core)
s_axi_arregion	input	4	Region (ignored by core)
s_axi_aruser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_arvalid	input	1	Read-address valid
s_axi_arready	output	1	Read-address ready
s_axi_rid	output	AXI_ID_WIDTH	Read-data ID
s_axi_rdata	output	DATA_WIDTH	Read data
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read-data last beat
s_axi_ruser	output	USER_WIDTH	Read-data user (hardwired to 0)
s_axi_rvalid	output	1	Read-data

Port	Direction	Width	Description
s_axi_rready	input	1	valid Read-data ready

30.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot {rready, rvalid, arready, arvalid, bready, bvalid, wready, wvalid, awready, awvalid}
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_busy_wr	output	1	Write-side skid-leaf busy
o_dbg_busy_rd	output	1	Read-side skid-leaf busy

30.5 Functional Description

30.5.1 Structure

The wrapper is three instances and a little glue:

1. `axi4_slave_wr` accepts the external AXI4 write channels and presents a FUB-shaped write interface.
2. `axi4_slave_rd` accepts the external AXI4 read channels and presents a FUB-shaped read interface.
3. `sdpram_core` consumes both FUBs, owns the BRAM, the burst trackers, and the clear FSM.

30.5.2 Field Pass-Through and Tie-Offs

Because both sides are full AXI4, the real `awid` / `awlen` / `awsize` / `awburst` (and the AR equivalents) flow straight through the leaves into the core. The AXI4-only qualifier fields the core does not consume — `lock`, `cache`, `prot`, `qos`, `region`, `user`, and `wlast` — are carried through the leaf FUB and terminated on `*_unused` nets. The core does not preserve `user` across the BRAM, so `s_axi_buser` and `s_axi_ruser` are hardwired to zero (matching legacy behavior).

30.5.3 Memory Behavior

All memory semantics live in `sdpram_core`: byte-enabled writes on BRAM port A, single-cycle-latency reads on port B, one outstanding burst per side, and the bulk-clear FSM gated on both sides idle. See `sdpram_core` for the full backend description.

30.5.4 Debug Taps

`o_dbg_vr` mirrors the five external channel valid/ready pairs; `o_dbg_fub_vr` mirrors the same for the internal FUB. `o_dbg_bram_wr` / `o_dbg_bram_rd` pulse on real BRAM accesses, and `o_dbg_busy_wr` / `o_dbg_busy_rd` expose each skid leaf's busy flag.

30.5.5 WRAP Assertion

A `translate_off` block asserts on each accepted AW and AR that `awburst`/`arburst` is not WRAP (2'b10). WRAP is computed by `axi_gen_addr` but not yet validated through the BRAM glue, so the assertion warns at the sim boundary.

30.6 Usage Example

```
sdpram_slave_axi4_axi4 #(
    .AXI_ID_WIDTH  (8),
    .ADDR_WIDTH    (32),
    .DATA_WIDTH    (256),
    .MEM_DEPTH     (2048)
) u_mem (
    .aclk          (aclk),
    .aresetn       (aresetn),
```

```

// Write-side AXI4 slave (from the producer master)
.s_axi_awid      (dut_awid),
.s_axi_awaddr    (dut_awaddr),
.s_axi_awlen     (dut_awlen),
.s_axi_awsz      (dut_awsz),
.s_axi_awburst   (dut_awburst),
.s_axi_awlock    (dut_awlock),
.s_axi_awcache   (dut_awcache),
.s_axi_awprot    (dut_awprot),
.s_axi_awqos     (dut_awqos),
.s_axi_awregion  (dut_awregion),
.s_axi_awuser    (dut_awuser),
.s_axi_awvalid   (dut_awvalid),
.s_axi_awready   (dut_awready),
// ... W, B, and the AR/R read channels ...

.i_cfg_start_clear (clear_pulse),
.o_cfg_done_clear  (clear_done),
.o_dbg_vr          (dbg_vr),
.o_dbg_fub_vr      (dbg_fub_vr),
.o_dbg_bram_wr     (dbg_bram_wr),
.o_dbg_bram_rd     (dbg_bram_rd),
.o_dbg_busy_wr     (dbg_busy_wr),
.o_dbg_busy_rd     (dbg_busy_rd)
);

```

30.7 Design Notes

30.7.1 Native AXI4, No Emulation

Unlike the AXIL-side wrappers, this variant needs no default-field synthesis — both sides are already AXI4, so the burst/ID fields pass through unchanged. The only tie-offs are the qualifier fields the core intentionally ignores, plus the user outputs the BRAM does not preserve.

30.7.2 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. SystemVerilog cannot conditionally include ports in a single module's port list, so each protocol combination gets its own thin wrapper with the exact port shape; the memory kernel lives once in the core.

30.7.3 WRAP Support Is Sim-Gated

INCR and FIXED bursts of any length are supported. WRAP is flagged by assertion until validated. Callers relying on WRAP semantics should exercise and confirm the path first.

30.8 Related Modules

30.8.1 Used By

- Characterization harnesses (memory model, descriptor RAM, semaphore RAM)
- Any AXI4-to-AXI4 shared-memory test scaffold

30.8.2 Uses

- **axi4_slave_wr.sv** — Write-side AXI4 skid leaf
- **axi4_slave_rd.sv** — Read-side AXI4 skid leaf
- **sdpram_core.sv** — Shared SDPRAM backend

30.8.3 See Also

- **sdpram_core.sv** — Protocol-agnostic backend ([doc](#))
 - **sdpram_slave_axi4_axil.sv** — AXI4 write + AXIL read variant
 - **sdpram_slave_axil_axi4.sv** — AXIL write + AXI4 read variant
 - **sdpram_slave_axil_axil.sv** — AXIL write + AXIL read variant
-

30.9 References

30.9.1 Source Code

- RTL: rtl/amba/shared/sdpram_slave_axi4_axi4.sv
- Backend: rtl/amba/shared/sdpram_core.sv

30.9.2 Documentation

- Backend spec: docs/markdown/RTLamba/shared/sdpram_core.md
 - Architecture: docs/markdown/RTLamba/shared/README.md
-

Last Updated: 2026-07-15

30.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

31 SDPRAM Slave — AXI4 Write / AXIL Read

Module: `sdpram_slave_axi4_axil.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

31.1 Overview

`sdpram_slave_axi4_axil` is the family permutation that exposes a **full AXI4 slave on the write side** (AW + W + B) and a **single-beat AXI4-Lite slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. The write side instantiates `axi4_slave_wr` (full AXI4 with `id / len / size / burst`); the read side instantiates `axi4_slave_rd`, and the wrapper bridges the AXIL read FUB into the core's AXI-shaped read FUB using single-beat defaults.

31.1.1 Key Features

- Full AXI4 write slave (burst, ID, size)
 - Single-beat AXIL read slave (no burst / ID)
 - AXIL read FUB bridged into the core with `arlen=0`, `arsize=$clog2(STRB_W)`, `arburst=INCR`, `arid=0`
 - Shared `sdpram_core` backend
 - Byte-enabled writes, single-cycle-latency reads
 - Bulk-clear control and debug taps
 - Sim-only WRAP-burst assertion on the AXI4 write side
-

31.2 Module Purpose

Some harness topologies write bulk data over a high-throughput AXI4 burst port but only need simple, single-word AXIL reads for status polling or spot checks. This wrapper matches that shape: fast AXI4 in, lightweight AXIL out, over one shared BRAM.

The AXIL read side has no burst or ID fields, so the wrapper supplies the missing AXI-shaped fields at the single point where the AXIL read FUB meets the core, letting the core's burst tracker collapse to a single-beat read path.

Use Cases: - AXI4 bulk-write memory model with an AXIL status/read-back port - Descriptor RAM written by an AXI4 engine and polled over AXIL - Any shared memory where the read side is a simple register-style AXIL access

Key Benefit: Full AXI4 write throughput paired with a minimal AXIL read port, with the single-beat adaptation isolated to one place in the wrapper.

31.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI4 write-side transaction-ID width
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
USER_WIDTH	int	1	AXI4 write-side USER width (carried by leaf, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

Derived localparams: STRB_W = DATA_WIDTH/8, FUB_ARSIZE_DEFAULT = $\lceil \log_2(\text{STRB_W}) \rceil$ (the arsize fed to the core for the single-beat read path).

31.4 Port Groups

31.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

31.4.2 AXI4 Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write-address ID
s_axi_awa ddr	input	ADDR_WIDTH	Write burst base address
s_axi_awle n	input	8	Burst length minus 1
s_axi_awsi ze	input	3	Beat size (log2 bytes)
s_axi_awb urst	input	2	Burst type (INCR / FIXED / WRAP)
s_axi_awlo ck	input	1	Lock (ignored by core)
s_axi_awca che	input	4	Cache attributes (ignored by core)
s_axi_awpr ot	input	3	Protection attributes (ignored by core)
s_axi_awq os	input	4	QoS (ignored by core)
s_axi_awre gion	input	4	Region (ignored by core)
s_axi_awus er	input	USER_WIDTH	User sideband (ignored by core)
s_axi_awva lid	input	1	Write-address valid
s_axi_awre ady	output	1	Write-address ready
s_axi_wdat a	input	DATA_WIDTH	Write data
s_axi_wstr b	input	DATA_WIDTH/8	Per-byte write strobe
s_axi_wlast	input	1	Write-data last beat
s_axi_wuse	input	USER_WIDTH	Write-data user sideband

Port	Direction	Width	Description
r		H	
s_axi_wvalid	input	1	Write-data valid
s_axi_wready	output	1	Write-data ready
s_axi_bid	output	AXI_ID_WIDTH	Write-response ID
s_axi_bresp	output	2	Write response
s_axi_buser	output	USER_WIDTH	Write-response user (hardwired to 0)
s_axi_bvalid	output	1	Write-response valid
s_axi_bready	input	1	Write-response ready

31.4.3 AXIL Slave Read (AR + R)

Port	Direction	Width	Description
s_axil_araddr	input	ADDR_WIDTH	Read address (single beat)
s_axil_arprot	input	3	Read protection attributes
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address ready
s_axil_rdata	output	DATA_WIDTH	Read data
s_axil_rresp	output	2	Read response
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready

31.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot {rready, rvalid, arready, arvalid, bready, bvalid, wready, wvalid, awready, awvalid} (AXIL R/AR on top, AXI4 B/W/AW below)
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_bus_y_wr	output	1	Write-side (AXI4) skid-leaf busy
o_dbg_bus_y_rd	output	1	Read-side (AXIL) skid-leaf busy

31.5 Functional Description

31.5.1 Structure

Three instances: axi4_slave_wr (write), axil4_slave_rd (read), and sdpram_core (backend). The write leaf's FUB carries full AXI4 write fields into the core. The read leaf's AXIL FUB carries only address and data; the wrapper supplies the burst/ID fields the core expects.

31.5.2 AXIL Read Bridge

The AXIL read FUB (fub_axil_araddr / arvalid / arready and rdata / rresp / rvalid / rready) is wired to the core's read FUB, and the AXI-shaped fields the core needs are tied off at that boundary:

- fub_arid = '0
- fub_arlen = 8'h00 (single beat)
- fub_arsize = 3' (FUB_ARSIZE_DEFAULT) = $\$clog2(\text{STRB_W})$ (one full data word per beat)
- fub_arburst = 2'b01 (INCR)

The core's fub_rid and fub_rlast outputs go to *_unused nets — AXIL has no ID or last field. With arlen = 0, each AXIL read is a single-beat access into BRAM.

31.5.3 Write Path and Tie-Offs

The write side passes full AXI4 fields straight through. The AXI4 qualifier fields the core ignores (lock, cache, prot, qos, region, user, wlast) terminate on *_unused nets. s_axi_buser is hardwired to zero since the core does not preserve user across the BRAM.

31.5.4 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding burst per side, and the bulk-clear FSM all live in sdpram_core — see sdpram_core.

31.5.5 WRAP Assertion

A translate_off block asserts on each accepted AW that awburst is not WRAP. Only the write side carries this assertion; the AXIL read side is inherently single-beat INCR.

31.6 Usage Example

```
sdpram_slave_axi4_axil #(
    .AXI_ID_WIDTH  (8),
    .ADDR_WIDTH    (32),
    .DATA_WIDTH     (256),
    .MEM_DEPTH      (2048)
) u_mem (
    .aclk           (aclk),
    .aresetn        (aresetn),

    // AXI4 burst write in
    .s_axi_awid      (eng_awid),
    .s_axi_awaddr    (eng_awaddr),
```

```

.s_axi_awlen    (eng_awlen),
.s_axi_awsz     (eng_awsz),
.s_axi_awburst  (eng_awburst),
/* ...remaining AW/W/B... */
.s_axi_bvalid   (eng_bvalid),
.s_axi_bready   (eng_bready),

// AXIL single-beat read-back
.s_axil_araddr  (cpu_araddr),
.s_axil_arprot  (cpu_arprot),
.s_axil_arvalid (cpu_arvalid),
.s_axil_arready (cpu_arready),
.s_axil_rdata   (cpu_rdata),
.s_axil_rresp   (cpu_rresp),
.s_axil_rvalid  (cpu_rvalid),
.s_axil_rready  (cpu_rready),

.i_cfg_start_clear (clear_pulse),
.o_cfg_done_clear  (clear_done),
.o_dbg_vr          (dbg_vr),
.o_dbg_fub_vr      (dbg_fub_vr),
.o_dbg_bram_wr     (dbg_bram_wr),
.o_dbg_bram_rd     (dbg_bram_rd),
.o_dbg_busy_wr     (dbg_busy_wr),
.o_dbg_busy_rd     (dbg_busy_rd)
);

```

31.7 Design Notes

31.7.1 Single-Beat Adaptation Lives in One Place

The only “fake AXI4” fields anywhere in this wrapper are the four read-side defaults (`arid / arlen / arsize / arbust`) fed to the core at the AXIL read FUB boundary. Everything else on the write side is genuine AXI4.

31.7.2 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

31.7.3 Read Beat Size

`FUB_ARSIZE_DEFAULT = $clog2(STRB_W)` selects a full-data-word beat for each AXIL read, matching the BRAM word width. Sub-word AXIL reads are not modeled — a read returns the whole BRAM word at the addressed word index.

31.8 Related Modules

31.8.1 Used By

- Characterization harnesses with AXI4 bulk-write and AXIL read-back

31.8.2 Uses

- **axi4_slave_wr.sv** — Write-side AXI4 skid leaf
- **axil4_slave_rd.sv** — Read-side AXIL skid leaf
- **sdpram_core.sv** — Shared SDPRAM backend

31.8.3 See Also

- **sdpram_core.sv** — Protocol-agnostic backend ([doc](#))
 - **sdpram_slave_axi4_axi4.sv** — AXI4 write + AXI4 read variant
 - **sdpram_slave_axil_axi4.sv** — AXIL write + AXI4 read variant
 - **sdpram_slave_axil_axil.sv** — AXIL write + AXIL read variant
-

31.9 References

31.9.1 Source Code

- RTL: rtl/amba/shared/sdpram_slave_axi4_axil.sv
- Backend: rtl/amba/shared/sdpram_core.sv

31.9.2 Documentation

- Backend spec: docs/markdown/RTLAmba/shared/sdpram_core.md
 - Architecture: docs/markdown/RTLAmba/shared/README.md
-

Last Updated: 2026-07-15

31.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

32 SDPRAM Slave — AXIL Write / AXI4 Read

Module: `sdpram_slave_axil_axi4.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

32.1 Overview

`sdpram_slave_axil_axi4` is the family permutation with a **single-beat AXI4-Lite slave on the write side** (AW + W + B) and a **full AXI4 slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. The write side instantiates `axil4_slave_wr`; the read side instantiates `axi4_slave_rd` (full AXI4 with id / len / size / burst). The wrapper bridges the AXIL write FUB into the core's AXI-shaped write FUB using single-beat defaults.

32.1.1 Key Features

- Single-beat AXIL write slave (no burst / ID)
 - Full AXI4 read slave (burst, ID, size)
 - AXIL write FUB bridged into the core with `awlen=0`, `awsize=$clog2(STRB_W)`, `awburst=INCR`, `awid=0`
 - Shared `sdpram_core` backend
 - Byte-enabled writes, single-cycle-latency reads
 - Bulk-clear control and debug taps
 - Sim-only WRAP-burst assertion on the AXI4 read side
-

32.2 Module Purpose

This shape fits a common characterization pattern: a host CPU populates a memory a word at a time over a simple AXIL write port (for example, a descriptor RAM's host-write port), while a high-throughput engine reads bursts back over full AXI4.

The AXIL write side has no burst or ID fields, so the wrapper supplies the missing AXI-shaped fields at the single point where the AXIL write FUB meets the core, collapsing the core's write burst tracker to a single-beat path.

Use Cases: - Descriptor RAM: host writes descriptors over AXIL, an engine reads them over AXI4 bursts - Semaphore / mailbox RAM: single-word AXIL writes, bursty AXI4 reads - Memory model populated by a control-plane CPU and consumed by a data-plane master

Key Benefit: A simple word-at-a-time AXIL write port with full AXI4 burst read throughput, with the single-beat adaptation isolated to one place in the wrapper.

32.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI4 read-side transaction-ID width
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
USER_WIDTH	int	1	AXI4 read-side USER width (carried by leaf, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

Derived localparams: STRB_W = DATA_WIDTH/8, FUB_AWSIZE_DEFAULT = \$clog2(STRB_W)
(the awsize fed to the core for the single-beat write path).

32.4 Port Groups

32.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

32.4.2 AXIL Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axil_awaddr	input	ADDR_WIDTH	Write address (single beat)
s_axil_awprot	input	3	Write protection attributes
s_axil_awvalid	input	1	Write-address valid
s_axil_awready	output	1	Write-address ready
s_axil_wdata	input	DATA_WIDTH	Write data
s_axil_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axil_wvalid	input	1	Write-data valid
s_axil_wready	output	1	Write-data ready
s_axil_bresp	output	2	Write response
s_axil_bvalid	output	1	Write-response valid
s_axil_bready	input	1	Write-response ready

32.4.3 AXI4 Slave Read (AR + R)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read-address ID
s_axi_araddr	input	ADDR_WIDTH	Read burst base address
s_axi_arlen	input	8	Burst length minus 1
s_axi_arsize	input	3	Beat size (log2 bytes)

Port	Direction	Width	Description
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock (ignored by core)
s_axi_arcache	input	4	Cache attributes (ignored by core)
s_axi_arprot	input	3	Protection attributes (ignored by core)
s_axi_arqos	input	4	QoS (ignored by core)
s_axi_arregion	input	4	Region (ignored by core)
s_axi_aruser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_arvalid	input	1	Read-address valid
s_axi_arready	output	1	Read-address ready
s_axi_rid	output	AXI_ID_WIDTH	Read-data ID
s_axi_rdata	output	DATA_WIDTH	Read data
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read-data last beat
s_axi_ruser	output	USER_WIDTH	Read-data user (hardwired to 0)
s_axi_rvalid	output	1	Read-data valid

Port	Direction	Width	Description
s_axi_rready	input	1	Read-data ready

32.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot {rready,rvalid,arready,arvalid,bready,bvalid,wready,wvalid,awready,awvalid} (AXI4 R/AR on top, AXIL B/W/AW below)
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_bus_y_wr	output	1	Write-side (AXIL) skid-leaf busy
o_dbg_bus_y_rd	output	1	Read-side (AXI4) skid-leaf busy

32.5 Functional Description

32.5.1 Structure

Three instances: `axil4_slave_wr` (write), `axil4_slave_rd` (read), and `sdpram_core` (backend). The read leaf's FUB carries full AXI4 read fields into the core. The write leaf's AXIL FUB carries only address and data; the wrapper supplies the burst/ID fields the core expects.

32.5.2 AXIL Write Bridge

The AXIL write FUB (`fub_axil_awaddr` / `awvalid` / `awready`, `wdata` / `wstrb` / `wvalid` / `wready`, `bresp` / `bvalid` / `bready`) is wired to the core's write FUB, and the AXI-shaped fields the core needs are tied off at that boundary:

- `fub_awid` = '0
- `fub_awlen` = 8'h00 (single beat)
- `fub_awsz` = 3'(FUB_AWSIZE_DEFAULT) = $\$clog2(\text{STRB_W})$ (one full data word per beat)
- `fub_awburst` = 2'b01 (INCR)

The core's `fub_bid` output goes to a `*_unused` net — AXIL has no B-channel ID. With `awlen` = 0, each AXIL write is a single-beat access into BRAM.

32.5.3 Read Path and Tie-Offs

The read side passes full AXI4 fields straight through. The AXI4 qualifier fields the core ignores (lock, cache, prot, qos, region, user) terminate on `*_unused` nets. `s_axi_ruser` is hardwired to zero since the core does not preserve user across the BRAM.

32.5.4 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding burst per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

32.5.5 WRAP Assertion

A `translate_off` block asserts on each accepted AR that `arburst` is not WRAP. Only the read side carries this assertion; the AXIL write side is inherently single-beat INCR.

32.6 Usage Example

```
sdpram_slave_axil_axi4 #(
    .AXI_ID_WIDTH  (8),
    .ADDR_WIDTH    (32),
    .DATA_WIDTH    (256),
```

```

        .MEM_DEPTH      (2048)
    ) u_desc_ram (
        .aclk            (aclk),
        .aresetn         (aresetn),

        // AXIL host-write port (descriptor population)
        .s_axil_awaddr    (host_awaddr),
        .s_axil_awprot    (host_awprot),
        .s_axil_awvalid   (host_awvalid),
        .s_axil_awready   (host_awready),
        .s_axil_wdata     (host_wdata),
        .s_axil_wstrb     (host_wstrb),
        .s_axil_wvalid    (host_wvalid),
        .s_axil_wready    (host_wready),
        .s_axil_bresp     (host_bresp),
        .s_axil_bvalid    (host_bvalid),
        .s_axil_bready    (host_bready),

        // AXI4 burst read port (engine fetch)
        .s_axi_arid       (eng_arid),
        .s_axi_araddr     (eng_araddr),
        .s_axi_arlen      (eng_arlen),
        /* ...remaining AR/R... */
        .s_axi_rvalid     (eng_rvalid),
        .s_axi_rready     (eng_rready),

        .i_cfg_start_clear (clear_pulse),
        .o_cfg_done_clear  (clear_done),
        .o_dbg_vr          (dbg_vr),
        .o_dbg_fub_vr      (dbg_fub_vr),
        .o_dbg_bram_wr     (dbg_bram_wr),
        .o_dbg_bram_rd     (dbg_bram_rd),
        .o_dbg_busy_wr     (dbg_busy_wr),
        .o_dbg_busy_rd     (dbg_busy_rd)
    );

```

32.7 Design Notes

32.7.1 Descriptor-RAM Fit

This is the natural shape for a descriptor / semaphore RAM: a control-plane CPU pokes single words over AXIL, and a data-plane engine burst-reads them over AXI4. The `rapids_char_harness` descriptor RAM host-write port is exactly this pattern.

32.7.2 Single-Beat Adaptation Lives in One Place

The only “fake AXI4” fields anywhere in this wrapper are the four write-side defaults (`awid` / `awlen` / `awsize` / `awburst`) fed to the core at the AXIL write FUB boundary. Everything else on the read side is genuine AXI4.

32.7.3 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

32.8 Related Modules

32.8.1 Used By

- `rapids_char_harness` descriptor RAM (host-write port)
- Characterization harnesses with AXIL host-write and AXI4 engine-read

32.8.2 Uses

- `axil4_slave_wr.sv` — Write-side AXIL skid leaf
- `axi4_slave_rd.sv` — Read-side AXI4 skid leaf
- `sdpram_core.sv` — Shared SDPRAM backend

32.8.3 See Also

- `sdpram_core.sv` — Protocol-agnostic backend ([doc](#))
 - `sdpram_slave_axi4_axi4.sv` — AXI4 write + AXI4 read variant
 - `sdpram_slave_axi4_axil.sv` — AXI4 write + AXIL read variant
 - `sdpram_slave_axil_axil.sv` — AXIL write + AXIL read variant
-

32.9 References

32.9.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axil_axi4.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

32.9.2 Documentation

- Backend spec: `docs/markdown/RTLAmba/shared/sdpram_core.md`
- Architecture: `docs/markdown/RTLAmba/shared/README.md`

32.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

33 SDPRAM Slave — AXIL Write / AXIL Read

Module: `sdpram_slave_axil_axil.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

33.1 Overview

`sdpram_slave_axil_axil` is the all-AXI4-Lite permutation of the family: a **single-beat AXIL slave on the write side** (AW + W + B) and a **single-beat AXIL slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. It directly instantiates the native `axil4_slave_wr` and `axil4_slave_rd` skid leaves and bridges their AXIL FUB outputs into the core's AXI-shaped FUB by supplying single-beat defaults on both sides. No AXI4-only fields appear anywhere on the wrapper's external boundary.

33.1.1 Key Features

- Single-beat AXIL slave on both write and read sides
 - Native `axil4_slave_wr` + `axil4_slave_rd` skid leaves
 - Both FUBs bridged into the core with `len=0`, `size=$clog2(STRB_W)`, `burst=INCR`, `id=0`
 - A 1-bit zero ID carried through the core for typing only (`CORE_ID_WIDTH = 1`)
 - Shared `sdpram_core` backend
 - Byte-enabled writes, single-cycle-latency reads
 - Bulk-clear control and debug taps
 - No WRAP assertion needed (both sides are inherently single-beat INCR)
-

33.2 Module Purpose

The simplest members of the family need a lightweight, register-style memory shared between two AXIL agents — one writing, one reading. This wrapper provides exactly that with no burst machinery exposed: both sides are single-word AXIL, and the shared backend supplies the burst/ID scaffolding internally.

Because AXIL carries no transaction ID, the wrapper carries a 1-bit zero ID through `sdpram_core` purely for type-width bookkeeping. The single-beat defaults are the only “fake” AXI4 fields in the design, and they live in exactly one place per side.

Use Cases: - Lightweight scratch / mailbox RAM shared between two AXIL agents - Small semaphore RAM in a control-plane fabric - Memory-ring backend for the AXIL/AXIL monitor-bus capture master-write port (pairs with `monbus_axil_axil_group`) - Any AXIL-only shared-memory test scaffold

Key Benefit: A minimal AXIL-in / AXIL-out shared memory with all burst and clear complexity hidden in the shared core, and no spurious AXI4 fields on the external ports.

33.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	64	Data-bus / BRAM word width (bits)
MEM_DEPTH	int	1024	BRAM depth in words
SKID_DEPTH_A W	int	2	Write-address skid depth
SKID_DEPTH_ W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_A R	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid

Parameter	Type	Default	Description
			depth

Derived localparams: STRB_W = DATA_WIDTH/8, FUB_AWSIZE_DEFAULT = \$clog2(STRB_W) (the awsize/arsize fed to the core), CORE_ID_WIDTH = 1 (a 1-bit zero ID carried through the core for typing only — AXIL has no transaction ID).

Note: unlike the AXI4-bearing wrappers, this module has **no** AXI_ID_WIDTH or USER_WIDTH parameter, and its DATA_WIDTH / MEM_DEPTH defaults are smaller (64 / 1024), reflecting its lightweight role.

33.4 Port Groups

33.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

33.4.2 AXIL Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axil_awaddr	input	ADDR_WIDTH	Write address (single beat)
s_axil_awprot	input	3	Write protection attributes
s_axil_awvalid	input	1	Write-address valid
s_axil_awready	output	1	Write-address ready
s_axil_wdata	input	DATA_WIDTH	Write data
s_axil_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axil_wvalid	input	1	Write-data valid

Port	Direction	Width	Description
s_axil_wready	output	1	Write-data ready
s_axil_bresp	output	2	Write response
s_axil_bvalid	output	1	Write-response valid
s_axil_bready	input	1	Write-response ready

33.4.3 AXIL Slave Read (AR + R)

Port	Direction	Width	Description
s_axil_araddr	input	ADDR_WIDTH	Read address (single beat)
s_axil_arprot	input	3	Read protection attributes
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address ready
s_axil_rdata	output	DATA_WIDTH	Read data
s_axil_rresp	output	2	Read response
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready

33.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready

Port	Direction	Width	Description
			snapshot {rready,rvalid, arready,arvalid, bready,bvalid, wready,wvalid, awready,awvalid}
o_dbg_fub_ vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bra m_wr	output	1	BRAM write-fire pulse
o_dbg_bra m_rd	output	1	BRAM read-issue pulse
o_dbg_bus y_wr	output	1	Write-side (AXIL) skid-leaf busy
o_dbg_bus y_rd	output	1	Read-side (AXIL) skid-leaf busy

33.5 Functional Description

33.5.1 Structure

Three instances: axil4_slave_wr (write), axil4_slave_rd (read), and sdpram_core (backend, parameterized with AXI_ID_WIDTH = CORE_ID_WIDTH = 1). Both AXIL FUBs carry only address and data; the wrapper supplies the burst/ID fields the core expects.

33.5.2 Single-Beat Defaults on Both Sides

At each FUB-to-core boundary the wrapper ties the AXI-shaped fields the core needs:

Write side: - fub_awid = 1'b0 - fub_awlen = 8'h00 - fub_awsz = 3' (FUB_AWSIZE_DEFAULT) = \$clog2(STRB_W) - fub_awburst = 2'b01 (INCR)

Read side: - fub_arid = 1'b0 - fub_arlen = 8'h00 - fub_arsz = 3' (FUB_AWSIZE_DEFAULT) - fub_arburst = 2'b01 (INCR)

The core's fub_bid, fub_rid, and fub_rlast outputs terminate on *_unused nets. With len = 0 on both sides, every access is a single-beat BRAM access.

33.5.3 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding access per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

33.5.4 No WRAP Assertion

Neither side can express a burst, so there is no WRAP-burst assertion in this wrapper (the AXI4-bearing wrappers carry one because their burst-capable side could request WRAP).

33.6 Usage Example

```
sdpram_slave_axil_axil #(
    .ADDR_WIDTH (32),
    .DATA_WIDTH (64),
    .MEM_DEPTH  (1024)
) u_scratch (
    .aclk      (aclk),
    .aresetn   (aresetn),

    // AXIL write port
    .s_axil_awaddr  (wr_awaddr),
    .s_axil_awprot  (wr_awprot),
    .s_axil_awvalid (wr_awvalid),
    .s_axil_awready (wr_awready),
    .s_axil_wdata   (wr_wdata),
    .s_axil_wstrb   (wr_wstrb),
    .s_axil_wvalid  (wr_wvalid),
    .s_axil_wready  (wr_wready),
    .s_axil_bresp   (wr_bresp),
    .s_axil_bvalid  (wr_bvalid),
    .s_axil_bready  (wr_bready),

    // AXIL read port
    .s_axil_araddr  (rd_araddr),
    .s_axil_arprot  (rd_arprot),
    .s_axil_arvalid (rd_arvalid),
    .s_axil_arready (rd_arready),
    .s_axil_rdata   (rd_rdata),
    .s_axil_rresp   (rd_rresp),
    .s_axil_rvalid  (rd_rvalid),
    .s_axil_rready  (rd_rready),

    .i_cfg_start_clear (clear_pulse),
    .o_cfg_done_clear  (clear_done),
    .o_dbg_vr          (dbg_vr),
```

```
.o_dbg_fub_vr      (dbg_fub_vr),  
.o_dbg_bram_wr     (dbg_bram_wr),  
.o_dbg_bram_rd     (dbg_bram_rd),  
.o_dbg_busy_wr     (dbg_busy_wr),  
.o_dbg_busy_rd     (dbg_busy_rd)  
);
```

33.7 Design Notes

33.7.1 The Only “Fake” Fields, In One Place Per Side

AXIL has no id / len / burst, so the wrapper feeds sdpram_core single-beat INCR defaults. These are the only synthetic AXI4 fields anywhere in the design, and they live in exactly one place per side — the smell of the legacy generate-if base is gone.

33.7.2 Monitor-Bus Pairing

This wrapper is the canonical memory-ring backend for the AXIL/AXIL monitor-bus capture path: monbus_axil_axil_group’s master-write port streams records into an sdpram_slave_axil_axil acting as the dump ring.

33.7.3 One Backend, Four Wrappers

This wrapper shares sdpram_core with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

33.8 Related Modules

33.8.1 Used By

- monbus_axil_axil_group master-write dump ring (memory-ring backend)
- Lightweight AXIL-only shared-memory test scaffolds

33.8.2 Uses

- **axil4_slave_wr.sv** — Write-side AXIL skid leaf
- **axil4_slave_rd.sv** — Read-side AXIL skid leaf
- **sdpram_core.sv** — Shared SDPRAM backend

33.8.3 See Also

- **sdpram_core.sv** — Protocol-agnostic backend ([doc](#))
- **sdpram_slave_axi4_axi4.sv** — AXI4 write + AXI4 read variant

- **sdpram_slave_axi4_axil.sv** — AXI4 write + AXIL read variant
 - **sdpram_slave_axil_axi4.sv** — AXIL write + AXI4 read variant
 - **monbus_axil_axil_group.sv** — Monitor-bus delivery wrapper this RAM backs
-

33.9 References

33.9.1 Source Code

- RTL: rtl/amba/shared/sdpram_slave_axil_axil.sv
- Backend: rtl/amba/shared/sdpram_core.sv

33.9.2 Documentation

- Backend spec: docs/markdown/RTLamba/shared/sdpram_core.md
 - Architecture: docs/markdown/RTLamba/shared/README.md
-

Last Updated: 2026-07-15

33.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

34 AMBA Protocol Shims

Location: rtl/amba/shims/ **Test Location:** val/amba/, val/integ_amba/ **Status:** ✓ Production Ready

34.1 Overview

The shims subsystem provides protocol conversion modules that bridge between different bus protocols and interface standards. These adapters enable seamless integration of heterogeneous components within the AMBA ecosystem.

34.1.1 Key Features

- ✓ **Protocol Bridging:** Convert between AXI4, APB, and custom interfaces

- ✓ **Clock Domain Crossing:** Dual-clock support with proper CDC
 - ✓ **Width Adaptation:** Automatic data width conversion
 - ✓ **Burst Decomposition:** Complex transactions → Simple protocol transfers
 - ✓ **Standards Compliance:** PeakRDL integration, APB/AXI4 conformance
 - ✓ **Production Ready:** Comprehensive testing and verification
-

34.2 Available Shims

Shim	Purpose	Documentation	Status
axi4_to_apb_shim	AXI4 to APB bridge with dual-clock CDC	axi4_to_apb_shim.md	✓ Documented
axi4_to_apb_converter	Core AXI4→APB conversion logic	axi4_to_apb_convert.md	✓ Documented
peakrdl_to_cmdrsp	PeakRDL passthrough → cmd/rsp adapter	peakrdl_to_cmdrsp.md	✓ Documented

34.3 Quick Start

34.3.1 AXI4 to APB Bridge

```
// Bridge AXI4 master to APB peripheral bus
axi4_to_apb_shim #(
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(32),
    .APB_ADDR_WIDTH(32),
    .APB_DATA_WIDTH(32)
) u_bridge (
    .aclk(axi_clk),
    .aresetn(axi_resetn),
    .pclk(apb_clk),
    .presetn(apb_resetn),

    // AXI4 slave interface
    .s_axi_awid(axi_awid),
    .s_axi_awaddr(axi_awaddr),
    // ... AXI signals ...

```

```

    // APB master interface
    .m_apb_PSEL(apb_psel),
    .m_apb_PADDR(apb_paddr),
    // ... APB signals ...
);

34.3.2 PeakRDL Register Integration

// Integrate PeakRDL register block
peakrdl_to_cmdrsp #(
    .ADDR_WIDTH(12),
    .DATA_WIDTH(32)
) u_adapter (
    .aclk(clk),
    .aresetn(rst_n),

    // From APB master stub (cmd/rsp)
    .cmd_valid(cmd_valid),
    .cmd_ready(cmd_ready),
    .cmd_pwrite(cmd_pwrite),
    .cmd_paddr(cmd_paddr),
    .cmd_pwdata(cmd_pwdata),
    .cmd_pstrb(cmd_pstrb),
    .rsp_valid(rsp_valid),
    .rsp_ready(rsp_ready),
    .rsp_prdata(rsp_prdata),
    .rsp_pslverr(rsp_pslverr),

    // To PeakRDL register block
    .regblk_req(regblk_req),
    .regblk_req_is_wr(regblk_req_is_wr),
    // ... PeakRDL cpuif ...
);

```

34.4 Design Patterns

34.4.1 Pattern 1: CPU to Peripherals Bridge

Use AXI4-to-APB shim for CPU master accessing peripheral registers:

```

// ARM Cortex-M processor → APB peripherals
axi4_to_apb_shim #(
    .AXI_ID_WIDTH(12),           // CPU master ID width
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(32),

```



```

        .APB_ADDR_WIDTH(32),
        .APB_DATA_WIDTH(32),
        .DEPTH_AW(2),           // Shallow buffers for low latency
        .DEPTH_AR(2)
    ) u_cpu_periph_bridge (
        .aclk(cpu_clk),
        .aresetn(cpu_resetn),
        .pclk(periph_clk),
        .presetn(periph_resetn),
        // ... connections ...
    );

```

Characteristics: - Low latency (shallow buffers) - Single-beat transactions typical - Dual-clock support

34.4.2 Pattern 2: High-Speed to Low-Speed Bridge

Use width conversion for data rate adaptation:

```

// 64-bit AXI @ 200 MHz → 32-bit APB @ 100 MHz
axi4_to_apb_shim #(
    .AXI_DATA_WIDTH(64),       // High-speed side
    .APB_DATA_WIDTH(32),       // Low-speed side (2:1)
    .DEPTH_AW(4),              // Deeper for CDC + bursts
    .DEPTH_W(8),               // Extra depth for width conversion
    .DEPTH_R(8),
    .SIDE_DEPTH(8)
) u_width_bridge (
    .aclk(axi_clk_200mhz),
    .pclk(apb_clk_100mhz),
    // ... connections ...
);

```

Characteristics: - Width adaptation (automatic slicing) - Deeper buffers for burst traffic - Optimized for throughput

34.4.3 Pattern 3: Register Block Integration

Use PeakRDL adapter for register automation:

APB Bus → apb_slave_stub → peakrdl_to_cmdrsp → PeakRDL Register Block
 (APB→cmd/rsp) (cmd/rsp→cpuif) (generated from .rdl)

Benefits: - Register automation via PeakRDL - Standard cmd/rsp intermediate interface - Error propagation - Stall handling

34.5 Performance Characteristics

34.5.1 AXI4 to APB Bridge

Latency:

Configuration	Typical Latency	Notes
Same clock, no width convert	5-7 cycles	Minimal overhead
Same clock, 2:1 width convert	8-12 cycles	+1 cycle per APB beat
Dual-clock, no width convert	12-16 cycles	+3-5 cycles per CDC
Dual-clock, 2:1 width convert	15-20 cycles	Combined overhead

Throughput:

Width Ratio	Max Throughput	Notes
1:1	0.5 transfers/cycle	APB overhead
2:1	0.25 transfers/cycle	Width conversion
4:1	0.125 transfers/cycle	More APB beats

34.5.2 PeakRDL Adapter

Latency: - Read: 2 cycles (no stall) - Write: 2 cycles (no stall) - +N cycles if PeakRDL stalls request

Throughput: - Maximum: 1 transaction per 2 cycles - Typical: 1 transaction per 3-5 cycles (with APB)

34.6 Testing

Test AXI to APB bridge

```
pytest val/integ_amba/test_axi2apb_shim.py -v
```

Test PeakRDL adapter

```
pytest val/amba/test_peakrdl_to_cmdrsp.py -v
```

Run all shims tests

```
pytest val/amba/test_*shim*.py val/integ_amba/test_*shim*.py -v
```

```
# Generate waveforms
```

```
pytest val/integ_amba/test_axi2apb_shim.py --vcd=bridge.vcd -v  
gtkwave bridge.vcd
```

34.7 Common Use Cases

34.7.1 Use Case 1: SoC Integration

Scenario: ARM Cortex-M processor with AXI4 master accessing APB peripherals

Solution:

```
axi4_to_apb_shim u_cpu_bridge (  
    .aclk(cpu_axi_clk),  
    .pclk(periph_apb_clk),  
    // Connect CPU master → APB peripheral bus  
);
```

Benefits: - Enables CPU access to APB peripherals - Handles clock domain crossing - Minimal latency for register access

34.7.2 Use Case 2: DMA to Peripherals

Scenario: DMA engine (AXI4 master) writing to APB peripheral registers

Solution:

```
axi4_to_apb_shim #(  
    .DEPTH_AW(8),      // Deep buffers for burst DMA  
    .DEPTH_W(16)  
) u_dma_bridge (  
    .aclk(dma_clk),  
    .pclk(periph_clk),  
    // Connect DMA master → APB registers  
);
```

Benefits: - Burst decomposition (DMA bursts → APB single-beats) - Deep buffering for DMA traffic
- Error reporting back to DMA

34.7.3 Use Case 3: Configuration Registers

Scenario: Control registers for IP block, auto-generated from SystemRDL

Solution:

APB → apb_slave_stub → peakrdl_to_cmdrsp → PeakRDL config_regs

Benefits: - Register automation (field reset, RW access, interrupts) - Documentation generation - Software header generation - Type safety

34.8 Parameter Selection Guidelines

34.8.1 AXI to APB Bridge

Skid Buffer Depths:

Application	DEPTH_AW/ AR	DEPTH_W	DEPTH_R	Notes
Low-latency CPU	2	2-4	2-4	Minimize latency
Moderate DMA	4	8	8	Balance latency/throughput
Burst DMA	8	16	16	Maximize throughput

Width Conversion:

AXI Width	APB Width	Ratio	Typical Use
32	32	1:1	Simple bridging
64	32	2:1	Modern CPU → legacy periph
128	32	4:1	Wide DMA → narrow periph
64	64	1:1	High-bandwidth peripherals

Side FIFO Depth: - $SIDE_DEPTH \geq \max(\text{AXI burst length} \times \text{width ratio})$ - Typical: 4-8 for CPU, 8-16 for DMA

34.8.2 PeakRDL Adapter

Address Width: - Match PeakRDL generation: `--cpuif-addr-width N` - Typical: 12 bits (4 KB register space)

Data Width: - Must match PeakRDL generation (usually 32) - No width conversion supported

34.9 Design Tradeoffs

34.9.1 AXI to APB Bridge

✓ **Advantages:** - Standard protocols (AXI4 ↔ APB) - Dual-clock CDC support - Automatic width conversion - Burst decomposition

✗ **Limitations:** - APB serialization (no concurrent transactions) - Performance overhead (APB slower than AXI) - No AXI exclusive access support - Resource cost (~800-1500 LUTs)

Alternatives: - Native APB masters (if no AXI required) - AXI crossbar (if staying in AXI domain)

34.9.2 PeakRDL Adapter

✓ **Advantages:** - Register automation - Standards-based (SystemRDL) - Documentation generation - Low overhead (~50 LUTs)

✗ **Limitations:** - PeakRDL dependency - Learning curve (SystemRDL) - 32-bit only (typical)

Alternatives: - Custom register implementation - PeakRDL native APB cpuif - Commercial register tools

34.10 Synthesis Notes

34.10.1 Resource Usage

Module	LUTs	FFs	BRAM	Notes
axi4_to_apb_shim (32/32)	~800	~600	0	Minimal config
axi4_to_apb_shim (64/32)	~1200	~900	0	Width conversion
axi4_to_apb_convert	~400	~300	0	Core logic only
peakrdl_to_cmdrsp	~50	~100	0	Lightweight

34.10.2 Timing Closure

Critical Paths: - AXI packet unpacking (combinatorial) - Width conversion muxing - CDC synchronizers

Recommendations:

```
# Constrain CDC paths in axi4_to_apb_shim
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
set_max_delay -from */cdc_handshake/src_* -to */cdc_handshake/dst_*
10.0
```

```
# Register insertion for wide data paths
set_max_fanout 16 [get_pins */data_shift_reg*]
```

34.11 Related Documentation

34.11.1 Protocol Specifications

- ARM AMBA 4 AXI4 Specification
- ARM AMBA 3 APB Protocol Specification v2.0
- SystemRDL 2.0 Specification (PeakRDL)

34.11.2 RTL Design Sherpa Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem
- [AXI4 Modules](#) - AXI4 infrastructure
- [APB Modules](#) - APB infrastructure
- [Shared Utilities](#) - Common modules (CDC, FIFOs)

34.11.3 External Resources

- **PeakRDL:** <https://github.com/SystemRDL/PeakRDL>
- **PeakRDL regblock:** <https://github.com/SystemRDL/PeakRDL-regblock>
- **PeakRDL Workflow Guide:** [rtl/amba/shims/peakrdl_adapter_README.md](#)

34.11.4 Source Code

- RTL: [rtl/amba/shims/](#)
 - Tests: [val/amba/test_peakrdl*.py](#), [val/integ_amba/test_axi2apb*.py](#)
 - Framework: [bin/TBClasses/components/](#)
-

34.12 Design Notes

34.12.1 When to Use Shims

✓ **Use AXI4-to-APB Shim When:** - Bridging AXI4 master to APB slave - Different clock domains required - Width conversion needed - Burst decomposition required

✓ **Use PeakRDL Adapter When:** - Automating register block generation - Using SystemRDL descriptions - Need documentation/header generation - Integrating with cmd/rsp infrastructure

✗ **Consider Alternatives When:** - Same protocol both sides (use crossbar) - Need high performance (APB is slow) - Simple registers (custom implementation may be lighter)

34.12.2 Clock Domain Crossing

AXI to APB CDC: - Uses cdc_handshake module - Gray-code pointer-based synchronization - Safe for arbitrary clock ratios - Proper reset synchronization required

Constraints Required:

```
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
set_false_path -from [get_clocks pclk] -to [get_clocks aclk]
```

34.13 Migration Guide

34.13.1 Upgrading from Legacy AXI-APB Bridges

Before (legacy):

```
axi_apb_bridge u_bridge (
    .axi_*, // Legacy AXI naming
    .apb_*  // Legacy APB naming
);
```

After (RTL Design Sherpa):

```
axi4_to_apb_shim u_bridge (
    .s_axi_*, // AXI slave interface
    .m_apb_*  // APB master interface
);
```

Key Changes: - Explicit slave/master prefixes - Standardized signal naming - Added CDC support - Parameterized buffering

34.14 Troubleshooting

34.14.1 Issue 1: High Latency

Symptoms: Slow register access, performance degradation

Debug: 1. Check buffer depths (DEPTH_*) - deeper = higher latency 2. Verify clock domain crossing overhead 3. Check for width conversion ratio 4. Monitor APB protocol overhead

Solutions: - Reduce buffer depths for low latency - Same-clock operation if possible - Minimize width ratio (prefer 1:1)

34.14.2 Issue 2: PeakRDL Stalls

Symptoms: Long delays, timeout errors

Debug: 1. Check regblk_req_stall_* signals 2. Verify PeakRDL internal logic 3. Check for field access conflicts

Solutions: - Review PeakRDL register arbitration - Check for hardware write conflicts - Verify interrupt handling

34.14.3 Issue 3: Data Corruption

Symptoms: Incorrect read data, write failures

Debug: 1. Verify width conversion logic 2. Check byte strobes (WSTRB → bit enables) 3. Monitor error signals (PSLVERR, BRESP, RRESP)

Solutions: - Verify parameter configuration - CheckWSTRB alignment - Enable error checking in testbench

Last Updated: 2025-10-20

34.15 Navigation

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

35 axi4_to_apb_convert

Module: axi4_to_apb_convert.sv **Location:** rtl/amba/shims/ **Status:** ✓ Production Ready

35.1 Overview

Core conversion logic that transforms AXI4 memory-mapped transactions into APB peripheral bus protocol. This module handles burst decomposition, width adaptation, and protocol conversion within a single clock domain. Used internally by [axi4_to_apb_shim](#) but can also be instantiated standalone for same-clock bridging.

35.1.1 Key Features

- ✓ **Burst Decomposition:** AXI bursts (AWLEN/ARLEN) → APB single-beat transfers
- ✓ **Width Adaptation:** Automatic conversion for AXI_DATA_WIDTH > APB_DATA_WIDTH
- ✓ **Address Generation:** Proper address incrementing per AXI burst type (INCR, WRAP)
- ✓ **Transaction Tracking:** Side FIFO maintains ID, LAST, USER across conversion
- ✓ **Error Handling:** Accumulates APB PSLVERR → AXI BRESP/RRESP
- ✓ **Zero Data Loss:** Proper flow control and buffering

35.2 Module Declaration

```
module axi4_to_apb_convert #(
    parameter int SIDE_DEPTH           = 6,    // Side info FIFO depth
    parameter int AXI_ID_WIDTH         = 8,
    parameter int AXI_ADDR_WIDTH       = 32,
    parameter int AXI_DATA_WIDTH       = 32,
    parameter int AXI_USER_WIDTH       = 1,
    parameter int APB_ADDR_WIDTH       = 32,
    parameter int APB_DATA_WIDTH       = 32,
    parameter int AXI_WSTRB_WIDTH      = AXI_DATA_WIDTH / 8,
    parameter int APB_WSTRB_WIDTH      = APB_DATA_WIDTH / 8
) (
    // Clock and Reset
    input  logic          aclk,
    input  logic          aresetn,

    // AXI Slave Interface (packet format from axi4_slave_stub)
    // AW channel
    input  logic [AWSize-1:0] r_s_axi_aw_pkt,
    input  logic [3:0]        r_s_axi_aw_count,
    input  logic              r_s_axi_awvalid,
    output logic              w_s_axi_awready,
```

```

// W channel
input logic [WSize-1:0]      r_s_axi_w_pkt,
input logic                  r_s_axi_wvalid,
output logic                 w_s_axi_wready,

// B channel
output logic [BSize-1:0]     r_s_axi_b_pkt,
output logic                 w_s_axi_bvalid,
input logic                  r_s_axi_bready,

// AR channel
input logic [ARSize-1:0]     r_s_axi_ar_pkt,
input logic [3:0]            r_s_axi_ar_count,
input logic                  r_s_axi_arvalid,
output logic                 w_s_axi_arready,

// R channel
output logic [RSize-1:0]     r_s_axi_r_pkt,
output logic                 w_s_axi_rvalid,
input logic                  r_s_axi_rready,

// APB Master Interface (command/response packets)
output logic                 w_cmd_valid,
input logic                  r_cmd_ready,
output logic [APBCmdWidth-1:0] r_cmd_data,

input logic                  r_rsp_valid,
output logic                 w_rsp_ready,
input logic [APBRspWidth-1:0] r_rsp_data
);

```

35.3 Parameters

Parameter	Default	Description
SIDE_DEPTH	6	Side information FIFO depth (tracks ID, last, user)
AXI_ID_WIDTH	8	AXI transaction ID width
AXI_ADDR_WIDTH	32	AXI address width
AXI_DATA_WIDTH	32	AXI data width (32, 64, 128, 256, 512)

Parameter	Default	Description
AXI_USER_WIDTH	1	AXI user sideband width
APB_ADDR_WIDTH	32	APB address width (must match AXI)
APB_DATA_WIDTH	32	APB data width (8, 16, 32, 64)

Derived Parameters: - $AXI2APBRATIO = AXI_DATA_WIDTH / APB_DATA_WIDTH$ (automatically calculated) - $APBCmdWidth = APB$ command packet width - $APBRspWidth = APB$ response packet width

35.4 Architecture

35.4.1 Component Structure

```

axi4_to_apb_convert
├── Packet Unpacking
│   └── Extract AXI channel signals from packets
├── APB FSM (3 states)
│   ├── IDLE: Waiting for AXI transaction
│   ├── READ: Processing read burst
│   └── WRITE: Processing write burst
├── Response FSM (2 states)
│   ├── RSP_IDLE: Waiting for APB response
│   └── RSP_ACTIVE: Collecting responses
├── Address Generator (axi_gen_addr)
│   └── Increments address per AXI burst type
├── Width Adaptation Logic
│   ├── Data shift register (read data accumulation)
│   ├── Pointer tracking (which APB beat in AXI word)
│   └── WSTRB → PSTRB mapping
└── Side FIFO (gaxi_fifo_sync)
    └── Tracks ID, LAST, USER for response reconstruction

```

35.4.2 Data Flow

Write Path:

```

AW packet → Unpack → APB WRITE FSM
W packet  → Unpack → Data slicing → APB PWDATA
                        → Width pointer tracking

```

→ Burst counter decrement
→ APB command generation
Side info → {ID, last, user} → Side FIFO
APB response → Error accumulation → B packet

Read Path:

AR packet → Unpack → APB READ FSM
→ Address generation
→ Burst counter decrement
→ APB command generation
Side info → {ID, last, user} → Side FIFO
APB response → Data accumulation → R packet
→ Shift register fill

35.5 Behavior

35.5.1 Burst Decomposition

AXI Burst → APB Beats:

Single AXI beat may require multiple APB beats when `AXI_DATA_WIDTH > APB_DATA_WIDTH`:

Example: 64-bit AXI → 32-bit APB (2:1 ratio)

AXI Write Burst (AWLEN=3, 4 beats):
Beat 0: 64 bits → 2 APB transfers (addr+0, addr+4)
Beat 1: 64 bits → 2 APB transfers (addr+8, addr+12)
Beat 2: 64 bits → 2 APB transfers (addr+16, addr+20)
Beat 3: 64 bits → 2 APB transfers (addr+24, addr+28)
Total: 8 APB transfers for 4 AXI beats

Pointer Tracking: - `r_axi_wr_data_pointer`: Tracks which APB slice of current AXI write beat -
`r_axi_rd_data_pointer`: Tracks which APB slice to write for read accumulation -
`r_axi_rsp_data_pointer`: Tracks response data accumulation position

35.5.2 Address Generation

Supported Burst Types: - **INCR (0x01)**: Sequential incrementing (most common) - **WRAP (0x10)**: Wrapping at boundary (less common) - **FIXED (0x00)**: Same address (APB limitation: converted to INCR)

Address Alignment:

```
// APB addresses are always aligned to APB_DATA_WIDTH
w_alignment_mask = APB_WSTRB_WIDTH - 1;
w_apb_cmd_pkt_paddr = r_apb_paddr & ~w_alignment_mask;
```

35.5.3 Width Adaptation

Write Data Slicing (AXI → APB):

```
// For 64b AXI → 32b APB:  
// Pointer = 0 → Take bits [31:0]  
// Pointer = 1 → Take bits [63:32]  
w_apb_cmd_pkt_pwdata = (axi2abpratio == 1) ?  
    r_s_axi_wdata[APBDW-1:0] :  
    r_s_axi_wdata[r_axi_wr_data_pointer*APBDW +: APBDW];
```

Read Data Accumulation (APB → AXI):

```
// For 32b APB → 64b AXI:  
// Pointer = 0 → Fill bits [31:0]  
// Pointer = 1 → Fill bits [63:32], then return complete 64b word  
r_axi_data_shift[r_axi_rsp_data_pointer*APBDW +: APBDW] <=  
r_apb_rsp_pkt_prdata;
```

35.5.4 Error Handling

Error Accumulation: - Multiple APB PSLVERR errors OR'd together - Entire AXI transaction marked with error if any APB beat fails - Read errors: RRESP = 2'b10 (SLVERR) - Write errors: BRESP = 2'b10 (SLVERR)

Error Propagation:

```
r_pslverr <= r_pslverr | w_pslverr; // Sticky error  
w_resp_rd = (w_pslverr) ? 2'b10 : 2'b00;  
w_resp_wr = (w_pslverr | r_pslverr) ? 2'b10 : 2'b00;
```

35.5.5 Side Information Tracking

Side FIFO Contents:

```
// For each AXI transaction, track:  
{operation,      // 1=write, 0=read  
  ID,            // Transaction ID  
  last,          // RLAST/BLAST flag  
  user}          // User sideband
```

Purpose: - Preserves ID for out-of-order response reconstruction - Maintains LAST flag for burst completion signaling - Passes USER sideband through the bridge

35.6 Packet Formats

35.6.1 APB Command Packet

$\text{APBCmdWidth} = \text{APBAW} + \text{APBDW} + \text{APBSW} + 4 \text{ bits}$

[APBCmdWidth-1]	last	(end of AXI burst)
[APBCmdWidth-2]	first	(start of AXI burst)
[APBCmdWidth-3]	pwrite	(1=write, 0=read)
[APBCmdWidth-4:...]]	pprot	(3 bits)
[...]	pstrb	(APBSW bits)
[...]	paddr	(APBAW bits)
[APBDW-1:0]	pdata	(APBDW bits)

35.6.2 APB Response Packet

$\text{APBRspWidth} = \text{APBDW} + 3 \text{ bits}$

[APBRspWidth-1]	last	(end of sequence)
[APBRspWidth-2]	first	(start of sequence)
[APBRspWidth-3]	pslverr	(error flag)
[APBDW-1:0]	prdata	(read data)

35.7 Usage Examples

35.7.1 Example 1: Standalone Conversion (Same Width)

// 32b AXI → 32b APB conversion (same clock)

```
axi4_to_apb_convert #(
    .SIDE_DEPTH(4),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(32),
    .AXI_USER_WIDTH(1),
    .APB_ADDR_WIDTH(32),
    .APB_DATA_WIDTH(32)
) u_convert (
    .aclk(clk),
    .aresetn(rst_n),

    // From AXI slave stub
    .r_s_axi_aw_pkt(aw_pkt),
    .r_s_axi_awvalid(aw_valid),
    .w_s_axi_awready(aw_ready),
    // ... other AXI channels ...
```

```

    // To APB master stub
    .w_cmd_valid(apb_cmd_valid),
    .r_cmd_ready(apb_cmd_ready),
    .r_cmd_data(apb_cmd_data),
    .r_rsp_valid(apb_rsp_valid),
    .w_rsp_ready(apb_rsp_ready),
    .r_rsp_data(apb_rsp_data)
);

```

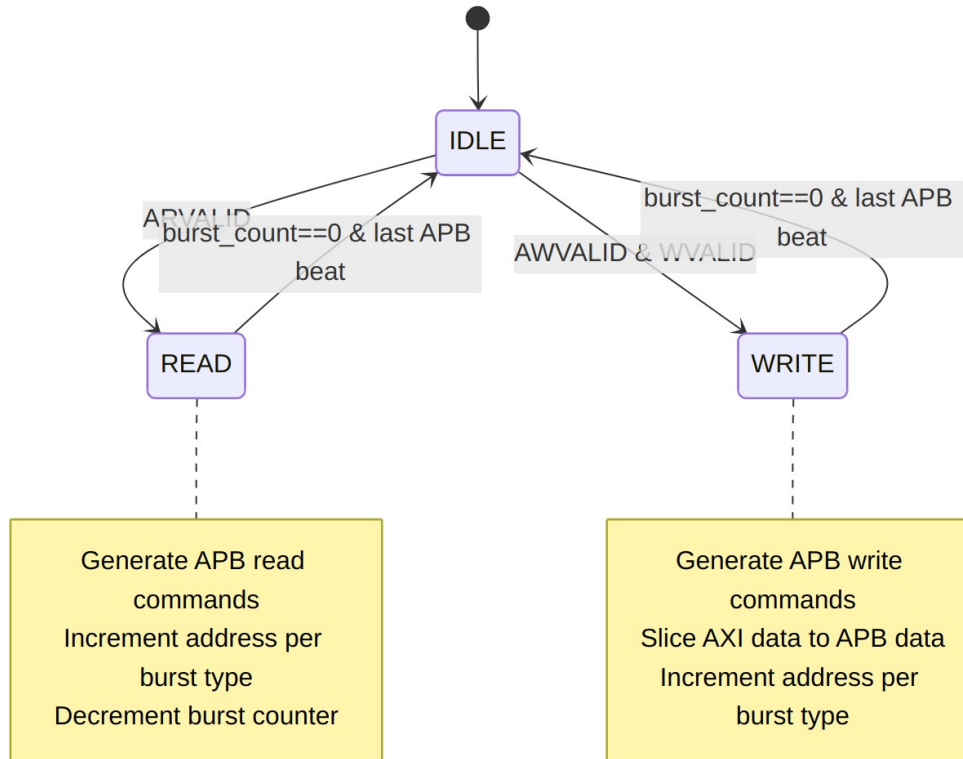
35.7.2 Example 2: Width Conversion (64b → 32b)

```

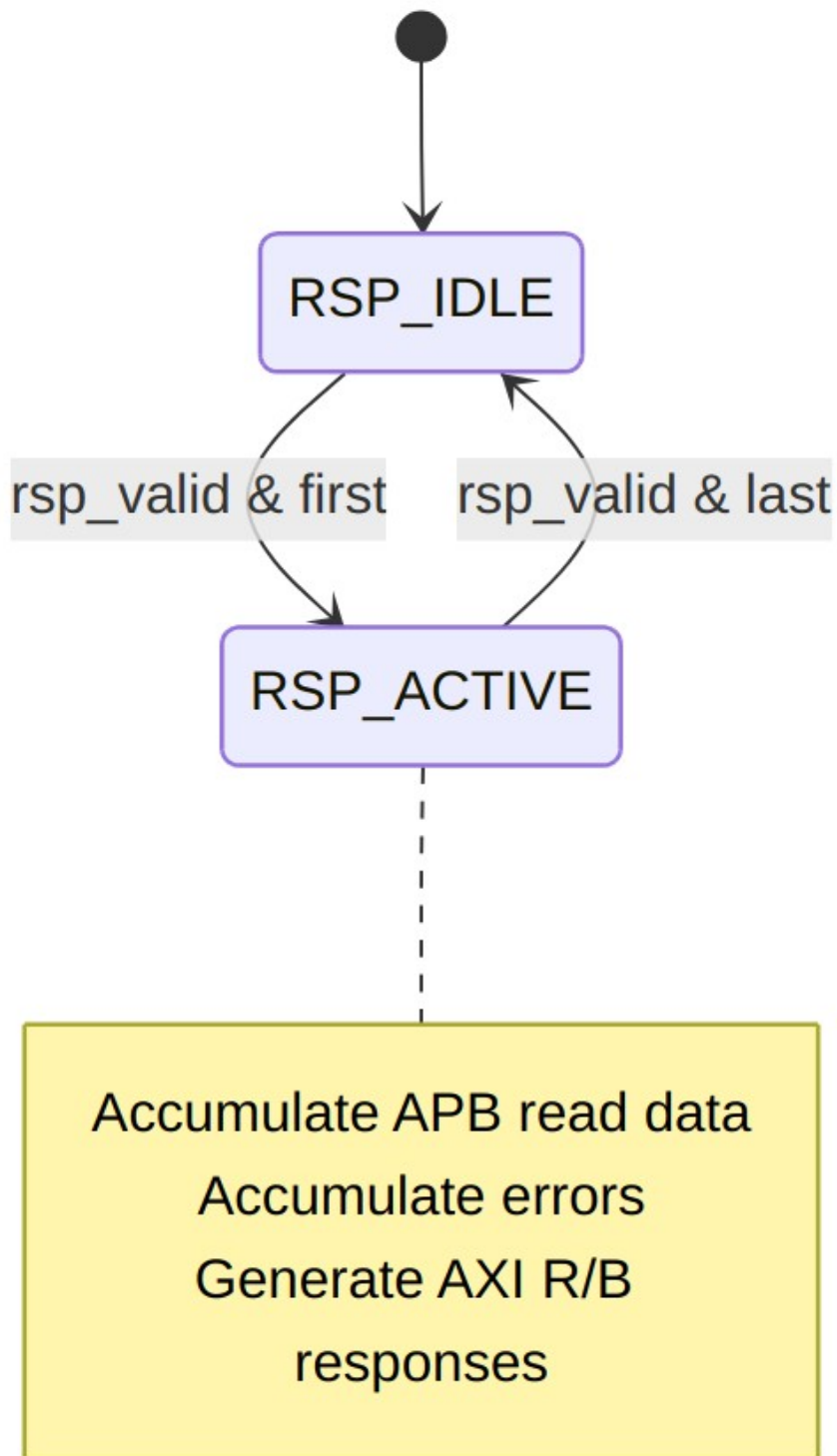
// Width adaptation: 64b AXI → 32b APB
axi4_to_apb_convert #(
    .SIDE_DEPTH(8),           // Deeper for burst traffic
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),      // 64-bit AXI
    .AXI_USER_WIDTH(4),
    .APB_ADDR_WIDTH(32),
    .APB_DATA_WIDTH(32)       // 32-bit APB (2:1 ratio)
) u_convert (
    .aclk(clk),
    .aresetn(rst_n),
    // ... connections ...
);

```

35.8 State Machines



APB FSM



35.9 Performance

35.9.1 Latency (Cycles)

Operation	Latency	Notes
AXI single-beat read (1:1 width)	4-5	Unpack + APB + pack
AXI single-beat write (1:1 width)	4-5	Same
AXI burst (AWLEN=15, 1:1 width)	68-80	16 beats × 4-5 cycles
Width convert read (2:1)	8-10	2 APB beats per AXI beat
Width convert read (4:1)	16-20	4 APB beats per AXI beat

35.9.2 Throughput

Maximum Throughput (same width): - 1 APB transfer every 2 cycles (SETUP + ACCESS phases) - Pipelined: ~0.5 transfers/cycle

Width Conversion Impact: - 2:1 ratio: Throughput halved (2 APB beats per AXI beat) - 4:1 ratio: Throughput quartered

35.10 Testing

Core conversion tested via integration test
`pytest val/integ_amba/test_axi2apb_shim.py -v`

Test with burst traffic
`pytest "val/integ_amba/test_axi2apb_shim.py::test_burst" -v`

Test width conversion
`pytest "val/integ_amba/test_axi2apb_shim.py::test_width[64-32]" -v`

35.11 Constraints and Limitations

Parameter Constraints: - $APB_DATA_WIDTH \leq AXI_DATA_WIDTH - AXI_DATA_WIDTH /$
APB_DATA_WIDTH must be power of 2 (1, 2, 4, 8) - SIDE_DEPTH must be $\geq$ maximum AXI burst length (typically 16)

Protocol Limitations: - No support for AXI exclusive access (AWLOCK/ARLOCK ignored) - No support for AXI cache/QoS hints (AWCACHE/ARCACHE/AWQOS/ARQOS ignored) - APB protocol is serialized (no concurrent transactions)

Width Adaptation: - Only supports narrowing (AXI $\rightarrow$ APB), not widening - APB_DATA_WIDTH must be at least 8 bits

35.12 Synthesis Notes

35.12.1 Resource Usage

Configuration	LUTs	FFs	Notes
32b/32b	~400	~300	No width conversion
64b/32b	~600	~450	2:1 width conversion
128b/32b	~900	~650	4:1 width conversion

Critical Logic: - Address generation (axi_gen_addr instantiation) - Data shift register (read accumulation) - FSM next-state logic

35.13 Related Modules

- [axi4_to_apb_shim](#) - Top-level shim (instantiates this module)
 - [axi4_slave_stub](#) - Provides input packet format
 - [apb_master_stub](#) - Consumes output packet format
 - [axi_gen_addr](#) - Address generation utility
 - [gaxi_fifo_sync](#) - Side information FIFO
-

35.14 When to Use

✓ **Use Standalone When:** - Same clock domain for AXI and APB - Want minimal latency (no CDC overhead) - Custom integration with different buffering

✓ **Use via axi4_to_apb_shim When:** - Different clock domains (need CDC) - Want complete turnkey solution - Standard AXI/APB interfaces

Last Updated: 2025-10-20

35.15 Navigation

- [← Back to Shims Index](#)
- [← Back to RTLAmba Index](#)

36 axi4_to_apb_shim

Module: axi4_to_apb_shim.sv **Location:** rtl/amba/shims/ **Status:** ✓ Production Ready

36.1 Overview

Top-level bridge module that converts AXI4 memory-mapped transactions to APB peripheral bus accesses with full dual-clock domain crossing (CDC) support. This shim integrates multiple components to provide a complete, robust bridge between AXI4 and APB protocols.

36.1.1 Key Features

- ✓ **Full AXI4 to APB Bridge:** Complete protocol conversion with burst decomposition
- ✓ **Dual-Clock Support:** Independent AXI (aclk) and APB (pclk) clock domains
- ✓ **CDC-Safe Handshakes:** Asynchronous clock domain crossing with proper synchronization
- ✓ **Configurable Buffering:** Skid buffers on all AXI channels for timing closure
- ✓ **Width Adaptation:** Automatic data width conversion (AXI → APB)
- ✓ **Burst Decomposition:** AXI bursts → APB single-beat transfers

- ✓ **Error Propagation:** APB PSLVERR mapped to AXI RRESP/BRESP
-

36.2 Module Declaration

```

module axi4_to_apb_shim #(
    // Skid buffer depths (AXI side)
    parameter int DEPTH_AW      = 2,    // AW channel depth
    parameter int DEPTH_W       = 4,    // W channel depth
    parameter int DEPTH_B       = 2,    // B channel depth
    parameter int DEPTH_AR      = 2,    // AR channel depth
    parameter int DEPTH_R       = 4,    // R channel depth

    // Internal buffering
    parameter int SIDE_DEPTH    = 4,    // Side info FIFO depth
    parameter int APB_CMD_DEPTH = 4,    // APB command FIFO depth
    parameter int APB_RSP_DEPTH = 4,    // APB response FIFO depth

    // AXI parameters
    parameter int AXI_ID_WIDTH  = 8,
    parameter int AXI_ADDR_WIDTH = 32,
    parameter int AXI_DATA_WIDTH = 32,
    parameter int AXI_USER_WIDTH = 1,

    // APB parameters
    parameter int APB_ADDR_WIDTH = 32,
    parameter int APB_DATA_WIDTH = 32
) (
    // Clock and Reset
    input logic          aclk,          // AXI clock
    domain
    input logic          aresetn,      // AXI reset
    (active-low)
    input logic          pclk,          // APB clock
    domain
    input logic          presetn,      // APB reset
    (active-low)

    // AXI4 Slave Interface (all 5 channels)
    // Write address channel (AW)
    input logic [AXI_ID_WIDTH-1:0]    s_axi_awid,
    input logic [AXI_ADDR_WIDTH-1:0]   s_axi_awaddr,
    input logic [7:0]                  s_axi_awlen,
    input logic [2:0]                  s_axi_awsz,
    input logic [1:0]                  s_axi_awburst,
    input logic                        s_axi_awlock,
    input logic [3:0]                  s_axi_awcache,

```

```

input  logic [2:0]
input  logic [3:0]
input  logic [3:0]
input  logic [AXI_USER_WIDTH-1:0]
input  logic
output logic

s_axi_awprot,
s_axi_awqos,
s_axi_awregion,
s_axi_awuser,
s_axi_awvalid,
s_axi_awready,

// Write data channel (W)
input  logic [AXI_DATA_WIDTH-1:0]
input  logic [AXI_DATA_WIDTH/8-1:0]
input  logic
input  logic [AXI_USER_WIDTH-1:0]
input  logic
output logic

s_axi_wdata,
s_axi_wstrb,
s_axi_wlast,
s_axi_wuser,
s_axi_wvalid,
s_axi_wready,

// Write response channel (B)
output logic [AXI_ID_WIDTH-1:0]
output logic [1:0]
output logic [AXI_USER_WIDTH-1:0]
output logic
input  logic

s_axi_bid,
s_axi_bresp,
s_axi_buser,
s_axi_bvalid,
s_axi_bready,

// Read address channel (AR)
input  logic [AXI_ID_WIDTH-1:0]
input  logic [AXI_ADDR_WIDTH-1:0]
input  logic [7:0]
input  logic [2:0]
input  logic [1:0]
input  logic
input  logic [3:0]
input  logic [2:0]
input  logic [3:0]
input  logic [3:0]
input  logic [AXI_USER_WIDTH-1:0]
input  logic
output logic

s_axi_arid,
s_axi_araddr,
s_axi_arlen,
s_axi_arsize,
s_axi_arburst,
s_axi_arlock,
s_axi_arcache,
s_axi_arprot,
s_axi_arqos,
s_axi_arregion,
s_axi_aruser,
s_axi_arvalid,
s_axi_arready,

// Read data channel (R)
output logic [AXI_ID_WIDTH-1:0]
output logic [AXI_DATA_WIDTH-1:0]
output logic [1:0]
output logic
output logic [AXI_USER_WIDTH-1:0]
output logic
input  logic

s_axi_rid,
s_axi_rdata,
s_axi_rresp,
s_axi_rlast,
s_axi_ruser,
s_axi_rvalid,
s_axi_rready,

// APB Master Interface

```

```

output logic m_apb_PSEL,
output logic [APB_ADDR_WIDTH-1:0] m_apb_PADDR,
output logic m_apb_PENABLE,
output logic m_apb_PWRITE,
output logic [APB_DATA_WIDTH-1:0] m_apb_PWDATA,
output logic [APB_DATA_WIDTH/8-1:0] m_apb_PSTRB,
output logic [2:0] m_apb_PPROT,

input logic [APB_DATA_WIDTH-1:0] m_apb_PRDATA,
input logic m_apb_PREADY,
input logic m_apb_PSLVERR
);

```

36.3 Parameters

36.3.1 Skid Buffer Depths

Parameter	Default	Description
DEPTH_AW	2	AW channel skid buffer depth (power of 2)
DEPTH_W	4	W channel skid buffer depth
DEPTH_B	2	B channel skid buffer depth
DEPTH_AR	2	AR channel skid buffer depth
DEPTH_R	4	R channel skid buffer depth

Recommendation: Use deeper depths (4-8) for high-latency paths, shallow (2) for low-latency.

36.3.2 Internal Buffering

Parameter	Default	Description
SIDE_DEPTH	4	Side information FIFO depth (tracks ID, last, user)
APB_CMD_DEPTH	4	APB command CDC FIFO depth

Parameter	Default	Description
APB_RSP_DEPTH	4	APB response CDC FIFO depth

36.3.3 AXI Interface

Parameter	Default	Description
AXI_ID_WIDTH	8	Transaction ID width (1-16)
AXI_ADDR_WIDTH	32	Address width (12-64)
AXI_DATA_WIDTH	32	Data width (32, 64, 128, 256, 512)
AXI_USER_WIDTH	1	User sideband signal width

36.3.4 APB Interface

Parameter	Default	Description
APB_ADDR_WIDTH	32	APB address width (must match AXI)
APB_DATA_WIDTH	32	APB data width (8, 16, 32, 64)

Note: APB_DATA_WIDTH must be $\leq$ AXI_DATA_WIDTH. Width adaptation is automatic.

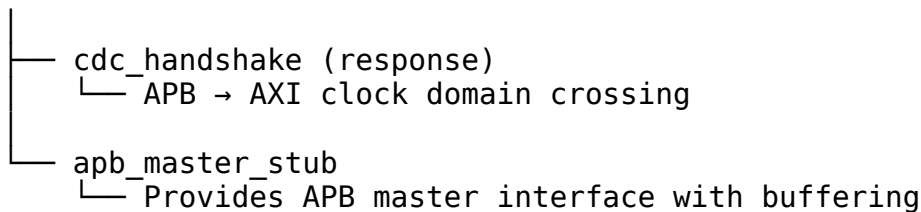
36.4 Architecture

36.4.1 Component Hierarchy

```

axi4_to_apb_shim (this module)
├── axi4_slave_stub
│   └── Provides AXI slave interface with skid buffers
├── axi4_to_apb_convert
│   ├── Core conversion logic
│   ├── Burst decomposition
│   ├── Width adaptation
│   └── gaxi_fifo_sync (side info)
└── cdc_handshake (command)
    └── AXI → APB clock domain crossing

```

36.4.2 Data Flow

Write Transaction:

AXI Master → AW/W skid buffers (aclk)
 → axi4_to_apb_convert (burst decomposition, aclk)
 → cmd CDC handshake (aclk → pclk)
 → apb_master_stub (APB protocol, pclk)
 → APB Slave (pclk)
 → rsp CDC handshake (pclk → aclk)
 → B channel response (aclk)

Read Transaction:

AXI Master → AR skid buffer (aclk)
 → axi4_to_apb_convert (burst decomposition, aclk)
 → cmd CDC handshake (aclk → pclk)
 → apb_master_stub (APB protocol, pclk)
 → APB Slave (pclk)
 → rsp CDC handshake (pclk → aclk)
 → R channel response + data (aclk)

36.5 Behavior

36.5.1 AXI to APB Conversion

Burst Decomposition: - AXI burst (AWLEN > 0) → Multiple APB single-beat transfers - Each beat addressed individually (address increments per burst type) - WLAST signal tracked to determine burst completion

Width Adaptation: - When AXI_DATA_WIDTH > APB_DATA_WIDTH: - Single AXI beat → Multiple APB beats - Data sliced according to APB_DATA_WIDTH - WSTRB mapped to PSTRB per beat - When AXI_DATA_WIDTH == APB_DATA_WIDTH: - Direct 1:1 mapping

Protection Signals: - AWPROT/ARPROT → PPROT (direct mapping)

Error Handling: - PSLVERR → BRESP[1] (write errors) - PSLVERR → RRESP[1] (read errors) - Multiple errors accumulated (OR'd) for burst transactions

36.5.2 Clock Domain Crossing

CDC Handshake Protocol: - Uses cdc_handshake module (Gray-code pointer-based) - Command path: aclk → pclk - Response path: pclk → aclk - Zero data loss, proper synchronization

Timing: - Command CDC latency: 3-5 cycles (depending on clock ratio) - Response CDC latency: 3-5 cycles - Total round-trip latency: AXI skid + convert + 2×CDC + APB protocol

36.6 Usage Examples

36.6.1 Example 1: Same Clock, Same Width

```
// Simplest configuration: same clock, same data width
axi4_to_apb_shim #(
    .DEPTH_AW(2),
    .DEPTH_W(2),
    .DEPTH_B(2),
    .DEPTH_AR(2),
    .DEPTH_R(2),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(32),
    .APB_ADDR_WIDTH(32),
    .APB_DATA_WIDTH(32)
) u_shim (
    .aclk(system_clk),
    .aresetn(system_resetn),
    .pclk(system_clk),           // Same clock
    .presetn(system_resetn),    // Same reset

    // Connect AXI slave interface
    .s_axi_awid(axi_awid),
    .s_axi_awaddr(axi_awaddr),
    // ... all AXI signals ...

    // Connect APB master interface
    .m_apb_PSEL(apb_psel),
    .m_apb_PADDR(apb_paddr),
    // ... all APB signals ...
);
```

36.6.2 Example 2: Dual-Clock with Width Conversion

```
// AXI @ 200 MHz, 64-bit → APB @ 100 MHz, 32-bit
axi4_to_apb_shim #(
    .DEPTH_AW(4),                // Deeper for CDC
```

```

        .DEPTH_W(8),           // Extra depth for width conversion
        .DEPTH_B(4),
        .DEPTH_AR(4),
        .DEPTH_R(8),
        .SIDE_DEPTH(8),       // More side info for bursts
        .APB_CMD_DEPTH(8),    // Deeper CDC FIFOs
        .APB_RSP_DEPTH(8),
        .AXI_ID_WIDTH(8),
        .AXI_ADDR_WIDTH(32),
        .AXI_DATA_WIDTH(64),  // 64-bit AXI
        .APB_ADDR_WIDTH(32),
        .APB_DATA_WIDTH(32)   // 32-bit APB (2:1 ratio)
    ) u_shim (
        .aclk(axi_clk_200mhz),
        .aresetn(axi_resetn),
        .pclk(apb_clk_100mhz), // Different clock
        .presetn(apb_resetn),
        // ... connections ...
    );

```

36.6.3 Example 3: CPU-Peripheral Bridge

```

// ARM AXI CPU → APB peripherals
axi4_to_apb_shim #(
    .DEPTH_AW(2),
    .DEPTH_W(4),
    .DEPTH_B(2),
    .DEPTH_AR(2),
    .DEPTH_R(4),
    .AXI_ID_WIDTH(12),       // CPU master ID width
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(32),
    .AXI_USER_WIDTH(8),     // CPU user signals
    .APB_ADDR_WIDTH(32),
    .APB_DATA_WIDTH(32)
) u_cpu_to_periph (
    .aclk(cpu_clk),
    .aresetn(cpu_resetn),
    .pclk(periph_clk),
    .presetn(periph_resetn),

    // From CPU AXI master
    .s_axi_awid(cpu_m_axi_awid),
    .s_axi_awaddr(cpu_m_axi_awaddr),
    // ... rest of AXI interface ...

    // To APB peripheral bus
    .m_apb_PSEL(periph_psel),

```

```

        .m_apb_PADDR(periph_paddr),
        // ... rest of APB interface ...
    );

```

36.7 Performance Characteristics

36.7.1 Latency

Configuration	Typical Latency	Notes
Same clock, no width convert	5-7 cycles	Minimal overhead
Same clock, width convert	8-12 cycles	+1 cycle per APB beat
Dual-clock, no width convert	12-16 cycles	+3-5 cycles per CDC
Dual-clock, width convert	15-20 cycles	Combined overhead

Components: - AXI skid buffers: 1 cycle - Conversion logic: 2-3 cycles - CMD CDC: 3-5 cycles - APB protocol: 2 cycles minimum - RSP CDC: 3-5 cycles

36.7.2 Throughput

Maximum AXI Burst Throughput: - Limited by APB protocol (2 cycles per beat minimum) - Width conversion: N APB beats per AXI beat ($N = \text{AXI_DW} / \text{APB_DW}$) - **Example:** 64b AXI → 32b APB = 4 APB cycles per AXI beat

Sustained Bandwidth:

AXI Bandwidth → APB Bandwidth:

- Same width: $\text{APB_BW} = \text{AXI_BW} / 2$ (APB overhead)
 - 2:1 ratio: $\text{APB_BW} = \text{AXI_BW} / 4$
 - 4:1 ratio: $\text{APB_BW} = \text{AXI_BW} / 8$
-

36.8 Testing

Run integration test

```
pytest val/integ_amba/test_axi2apb_shim.py -v
```

Test with waveforms

```
pytest val/integ_amba/test_axi2apb_shim.py --vcd=shim.vcd -v
```

```
gtkwave shim.vcd
```

```
# Run with specific configuration
```

```
pytest "val/integ_amba/test_axi2apb_shim.py::test_axi2apb[64-32]" -v
```

36.9 Constraints and Limitations

Parameter Constraints: - APB_DATA_WIDTH must be $\leq$ AXI_DATA_WIDTH - AXI_DATA_WIDTH / APB_DATA_WIDTH must be power of 2 - APB_ADDR_WIDTH should match AXI_ADDR_WIDTH (or be subset) - All DEPTH_* parameters must be ≥ 2

Protocol Limitations: - APB does not support outstanding transactions (serialized) - APB does not support bursts (AXI bursts decomposed) - No support for AXI exclusive access (AWLOCK/ARLOCK ignored) - No support for AXI cache hints (AWCACHE/ARCACHE ignored)

CDC Requirements: - Clock domains must be asynchronous (no phase relationship assumed) - Resets must be properly synchronized in each domain - Metastability-hardened flip-flops recommended for CDC paths

36.10 Synthesis Notes

36.10.1 Resource Usage (Typical)

Configuration	LUTs	FFs	BRAM	Notes
32b/32b, shallow	~800	~600	0	Minimal buffering
64b/32b, moderate	~1200	~900	0	Width conversion
64b/32b, deep	~1500	~1100	0	CDC + deep buffers

Breakdown: - axi4_slave_stub: ~300 LUTs, ~200 FFs (skid buffers) - axi4_to_apb_convert: ~400 LUTs, ~300 FFs (FSMs, width logic) - 2x cdc_handshake: ~200 LUTs, ~150 FFs (synchronizers) - apb_master_stub: ~100 LUTs, ~100 FFs (APB protocol)

36.10.2 Timing Closure

Critical Paths: - AXI packet unpacking (combinatorial) - Width conversion muxing - CDC synchronizers (use proper constraints)

Recommended Constraints:

```
# Set false path between clock domains
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
set_false_path -from [get_clocks pclk] -to [get_clocks aclk]

# Constrain CDC paths
set_max_delay -from [get_pins */cdc_handshake/src_*] \
               -to [get_pins */cdc_handshake/dst_*] 10.0
```

36.11 Related Modules

- [axi4_to_apb_convert](#) - Core conversion logic (instantiated internally)
 - [axi4_slave_stub](#) - AXI slave interface wrapper
 - [apb_master_stub](#) - APB master interface wrapper
 - [cdc_handshake](#) - Clock domain crossing handshake
-

36.12 When to Use

✓ **Use This Shim When:** - Bridging AXI4 system interconnect to APB peripherals - Different clock domains for AXI and APB buses - Need protocol conversion with full CDC support - CPU/DMA master accessing peripheral registers

✓ **Consider Alternatives When:** - Both sides are AXI (use AXI crossbar/interconnect instead) - Both sides are APB (use APB crossbar) - Need high-performance memory access (APB is slow, use AXI4)

Last Updated: 2025-10-20

36.13 Navigation

- [← Back to Shims Index](#)
- [← Back to RTLAmba Index](#)

37 peakrdl_to_cmdrsp

Module: peakrdl_to_cmdrsp.sv **Location:** rtl/amba/shims/ **Status:** ✓ Production Ready

37.1 Overview

Protocol adapter that bridges between the RTL Design Sherpa standard cmd/rsp interface and the PeakRDL register block “passthrough” cpuif interface. This shim enables seamless integration of PeakRDL-generated register blocks with RTL Design Sherpa’s APB infrastructure.

37.1.1 Key Features

- ✓ **Protocol Conversion:** cmd/rsp valid/ready ↔ PeakRDL passthrough cpuif
 - ✓ **Stall Handling:** Proper support for PeakRDL req_stall signals
 - ✓ **Strobe Conversion:** Byte strobes → bit enables (PeakRDL requirement)
 - ✓ **Error Propagation:** PeakRDL errors → cmd/rsp PSLVERR
 - ✓ **FSM-Based:** Robust state machines for cmd and rsp channels
 - ✓ **Assertions Included:** Comprehensive protocol checking (simulation only)
-

37.2 Background: PeakRDL Integration

PeakRDL is a register description language compiler that generates SystemVerilog register blocks from .rdl files. The generated blocks use a “passthrough” cpuif (CPU interface) that differs from standard bus protocols.

This adapter solves: - Integration with RTL Design Sherpa’s cmd/rsp-based APB infrastructure - Byte strobe → bit enable conversion (PeakRDL uses bit-level enables) - Stall handling (PeakRDL can stall requests separately for read/write) - Response channel synchronization

See Also: rtl/amba/shims/peakrdl_adapter_README.md for PeakRDL workflow guide

37.3 Module Declaration

```
module peakrdl_to_cmdrsp #(
    parameter int ADDR_WIDTH = 12, // Address width (match PeakRDL
generation)
    parameter int DATA_WIDTH = 32 // Must match PeakRDL (typically
32)
) (
    // Clock and Reset
    input logic          aclk,
    input logic          aresetn,
```

```

//
=====
===
// CMD/RSP Interface (rtlDesignSherpa standard)
//
=====
===
// Command Channel
input logic cmd_valid,
output logic cmd_ready,
input logic cmd_pwrite, // 1=write,
0=read
input logic [ADDR_WIDTH-1:0] cmd_paddr, // Byte
address
input logic [DATA_WIDTH-1:0] cmd_pwdata, // Write data
input logic [DATA_WIDTH/8-1:0] cmd_pstrb, // Byte
strokes

// Response Channel
output logic rsp_valid,
input logic rsp_ready,
output logic [DATA_WIDTH-1:0] rsp_prdata, // Read data
output logic rsp_pslverr, // Error flag

//
=====
===
// PeakRDL Passthrough Interface
//
=====
===
output logic regblk_req, // Request
strobe
output logic regblk_req_is_wr, // Write flag
output logic [ADDR_WIDTH-1:0] regblk_addr, // Address
output logic [DATA_WIDTH-1:0] regblk_wr_data, // Write data
output logic [DATA_WIDTH-1:0] regblk_wr_biten, // Write bit
enables

input logic regblk_req_stall_wr, // Write
stall
input logic regblk_req_stall_rd, // Read stall
input logic regblk_rd_ack, // Read
acknowledge
input logic regblk_rd_err, // Read error
input logic [DATA_WIDTH-1:0] regblk_rd_data, // Read data

```



```

    input logic regblk_wr_ack, // Write
    acknowledge
    input logic regblk_wr_err // Write error
);

```

37.4 Parameters

Parameter	Default	Description
ADDR_WIDTH	12	Address width for cmd/rsp interface (4 KB register space)
DATA_WIDTH	32	Data width (must match PeakRDL generation, typically 32)

Note: These must match the PeakRDL register block generation parameters.

37.5 Ports

37.5.1 CMD/RSP Interface (RTL Design Sherpa Standard)

Command Channel:

Port	Direction	Width	Description
cmd_valid	Input	1	Command valid
cmd_ready	Output	1	Ready to accept command
cmd_pwrite	Input	1	Write operation (1=write, 0=read)
cmd_paddr	Input	ADDR_WIDTH	Byte address
cmd_pwdata	Input	DATA_WIDTH	Write data
cmd_pstrb	Input	DATA_WIDTH/8	Byte strobes (1=valid byte)

Response Channel:

Port	Direction	Width	Description
rsp_valid	Output	1	Response valid
rsp_ready	Input	1	Ready to consume response
rsp_prdata	Output	DATA_WIDTH	Read data
rsp_pslverr	Output	1	Error flag (1=error)

37.5.2 PeakRDL Passthrough Interface**Request Signals:**

Port	Direction	Width	Description
regblk_req	Output	1	Request strobe (valid for 1 cycle)
regblk_req_is_wr	Output	1	Operation type (1=write, 0=read)
regblk_addr	Output	ADDR_WIDTH	Register address
regblk_wr_data	Output	DATA_WIDTH	Write data
regblk_wr_biten	Output	DATA_WIDTH	Write bit enables (1=update bit)

Stall/Acknowledge Signals:

Port	Direction	Width	Description
regblk_req_stall_wr	Input	1	Write request stalled (retry later)
regblk_req_stall_rd	Input	1	Read request stalled (retry later)
regblk_rd_ack	Input	1	Read completed (1 cycle pulse)
regblk_wr_ack	Input	1	Write completed (1 cycle pulse)

Port	Direction	Width	Description
			pulse)
regblk_rd_data	Input	DATA_WIDT H	Read data (valid with rd_ack)
regblk_rd_err	Input	1	Read error (valid with rd_ack)
regblk_wr_err	Input	1	Write error (valid with wr_ack)

37.6 Behavior

37.6.1 Command State Machine

States:

CMD_IDLE → Waiting for command
 CMD_WAIT_ACK → Waiting for register block ack
 CMD_STALLED → Stalled, retry when stall clears

Transitions:

IDLE:

- cmd_valid && !stall → WAIT_ACK
- cmd_valid && stall → STALLED

WAIT_ACK:

- regblk_wr_ack || regblk_rd_ack → IDLE

STALLED:

- !stall → WAIT_ACK (retry request)

37.6.2 Response State Machine

States:

RSP_IDLE → No response pending
 RSP_VALID → Response valid, waiting for rsp_ready

Transitions:

IDLE:

- regblk_wr_ack || regblk_rd_ack → VALID

VALID:

- rsp_ready → IDLE

37.6.3 Strobe to Bit Enable Conversion

PeakRDL requires bit-level write enables:

```
function automatic logic [DATA_WIDTH-1:0] strb_to_biten(
    input logic [STRB_WIDTH-1:0] strb
);
    for (int i = 0; i < STRB_WIDTH; i++) begin
        biten[i*8 +: 8] = {8{strb[i]}};
    end
    return biten;
endfunction
```

Example (32-bit):

```
cmd_pstrb = 4'b1100
           ↓
regblk_wr_biten = 32'hFFFF_0000
                  (upper 16 bits enabled)
```

37.6.4 Stall Handling

Separate stalls for read and write: - regblk_req_stall_wr: Write request cannot be accepted - regblk_req_stall_rd: Read request cannot be accepted

Behavior: 1. If stall asserted when cmd_valid: - Register command locally - Enter CMD_STALLED state - Keep regblk_req deasserted 2. When stall clears: - Assert regblk_req with registered command - Move to CMD_WAIT_ACK

Purpose: Allows PeakRDL register block to implement internal arbitration or serialization.

37.7 Usage Examples

37.7.1 Example 1: Basic Integration with PeakRDL Register Block

```
// Generate PeakRDL register block
// $ peakrdl regblock my_regs.rdl -o my_regs.sv
```

```
// Instantiate adapter
peakrdl_to_cmdrsp #(
    .ADDR_WIDTH(12),    // 4 KB register space
    .DATA_WIDTH(32)
) u_adapter (
    .clk(apb_clk),
```

```

    .aresetn(apb_resetn),

    // From APB master stub (cmd/rsp interface)
    .cmd_valid(apb_cmd_valid),
    .cmd_ready(apb_cmd_ready),
    .cmd_pwrite(apb_cmd_pwrite),
    .cmd_paddr(apb_cmd_paddr),
    .cmd_pwdata(apb_cmd_pwdata),
    .cmd_pstrb(apb_cmd_pstrb),

    .rsp_valid(apb_rsp_valid),
    .rsp_ready(apb_rsp_ready),
    .rsp_prdata(apb_rsp_prdata),
    .rsp_pslverr(apb_rsp_pslverr),

    // To PeakRDL register block
    .regblk_req(my_regs_req),
    .regblk_req_is_wr(my_regs_req_is_wr),
    .regblk_addr(my_regs_addr),
    .regblk_wr_data(my_regs_wr_data),
    .regblk_wr_biten(my_regs_wr_biten),

    .regblk_req_stall_wr(my_regs_req_stall_wr),
    .regblk_req_stall_rd(my_regs_req_stall_rd),
    .regblk_rd_ack(my_regs_rd_ack),
    .regblk_rd_err(my_regs_rd_err),
    .regblk_rd_data(my_regs_rd_data),
    .regblk_wr_ack(my_regs_wr_ack),
    .regblk_wr_err(my_regs_wr_err)
);

// Instantiate PeakRDL register block
my_regs u_regs (
    .clk(apb_clk),
    .rst(~apb_resetn),

    // Passthrough cpuif
    .cpuif_req(my_regs_req),
    .cpuif_req_is_wr(my_regs_req_is_wr),
    .cpuif_addr(my_regs_addr),
    .cpuif_wr_data(my_regs_wr_data),
    .cpuif_wr_biten(my_regs_wr_biten),
    .cpuif_req_stall_wr(my_regs_req_stall_wr),
    .cpuif_req_stall_rd(my_regs_req_stall_rd),
    .cpuif_rd_ack(my_regs_rd_ack),
    .cpuif_rd_err(my_regs_rd_err),
    .cpuif_rd_data(my_regs_rd_data),

```

```

        .cpuif_wr_ack(my_regs_wr_ack),
        .cpuif_wr_err(my_regs_wr_err),

        // Field interfaces
        .hwif_in(...),
        .hwif_out(...)
    );

```

37.7.2 Example 2: APB to PeakRDL Complete Path

// APB Master → APB Slave Stub → peakrdl_to_cmdrsp → PeakRDL Register Block

```

// APB slave stub (APB protocol → cmd/rsp)
apb_slave_stub #(
    .DATA_WIDTH(32),
    .ADDR_WIDTH(12)
) u_apb_slave (
    .pclk(apb_clk),
    .presetn(apb_resetn),

    // APB slave interface
    .s_apb_PSEL(apb_psel),
    .s_apb_PADDR(apb_paddr),
    .s_apb_PENABLE(apb_penable),
    .s_apb_PWRITE(apb_pwrite),
    .s_apb_PWDATA(apb_pwdata),
    .s_apb_PSTRB(apb_pstrb),
    .s_apb_PREADY(apb_pready),
    .s_apb_PRDATA(apb_prdata),
    .s_apb_PSLVERR(apb_pslverr),

    // cmd/rsp interface
    .cmd_valid(cmd_valid),
    .cmd_ready(cmd_ready),
    .cmd_pwrite(cmd_pwrite),
    .cmd_paddr(cmd_paddr),
    .cmd_pwdata(cmd_pwdata),
    .cmd_pstrb(cmd_pstrb),
    .rsp_valid(rsp_valid),
    .rsp_ready(rsp_ready),
    .rsp_prdata(rsp_prdata),
    .rsp_pslverr(rsp_pslverr)
);

// PeakRDL adapter (cmd/rsp → passthrough cpuif)
peakrdl_to_cmdrsp #(
    .ADDR_WIDTH(12),

```

```

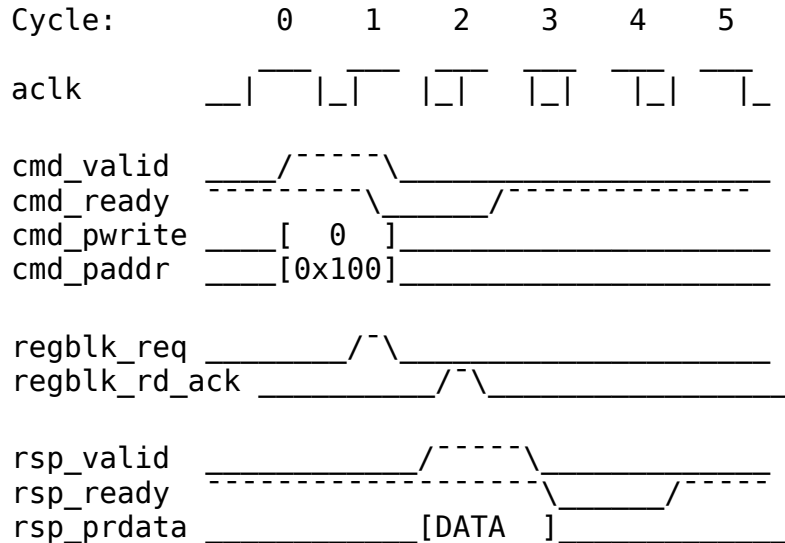
        .DATA_WIDTH(32)
    ) u_adapter (
        .aclk(apb_clk),
        .aresetn(apb_resetn),
        .cmd_valid(cmd_valid),
        .cmd_ready(cmd_ready),
        // ... cmd/rsp interface ...
        .regblk_req(regblk_req),
        // ... passthrough cpuif ...
    );

    // PeakRDL register block
    my_regs u_regs (
        .clk(apb_clk),
        .rst(~apb_resetn),
        .cpuif_req(regblk_req),
        // ... passthrough cpuif ...
    );

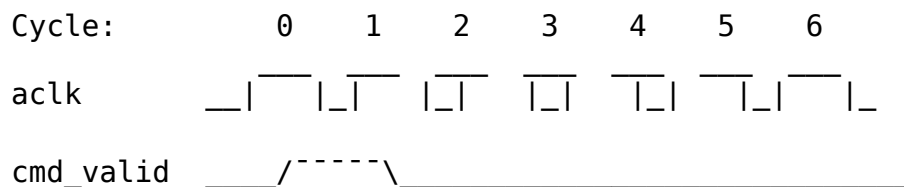
```

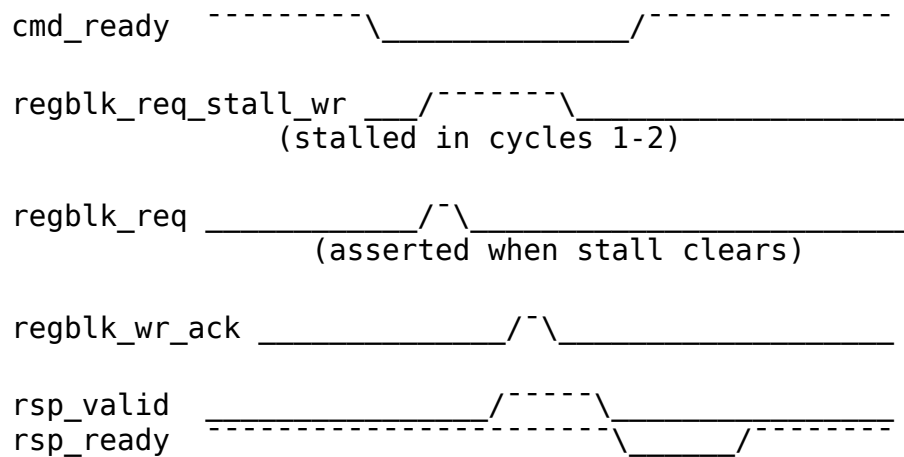
37.8 Timing Diagrams

37.8.1 Read Transaction (No Stall)



37.8.2 Write Transaction with Stall





37.9 Testing

Run adapter test

```
pytest val/amba/test_peakrdl_to_cmdrsp.py -v
```

Test with waveforms

```
pytest val/amba/test_peakrdl_to_cmdrsp.py --vcd=peakrdl.vcd -v
gtkwave peakrdl.vcd
```

37.10 Assertions

Included assertions (simulation only):

1. **cmd_valid_stable:** cmd_valid must remain high until cmd_ready
2. **cmd_data_stable:** cmd signals must remain stable until cmd_ready
3. **rsp_valid_stable:** rsp_valid must remain high until rsp_ready
4. **rsp_data_stable:** rsp signals must remain stable until rsp_ready

Purpose: Catch protocol violations during simulation.

37.11 Performance

37.11.1 Latency

Operation	Typical Latency	Notes
Read (no stall)	2 cycles	req → rd_ack

Operation	Typical Latency	Notes
Write (no stall)	2 cycles	→ rsp req → wr_ack → rsp
Read (with stall)	2 + stall cycles	Depends on PeakRDL arbitration
Write (with stall)	2 + stall cycles	Depends on PeakRDL arbitration

37.11.2 Throughput

- **Maximum:** 1 transaction per 2 cycles (no stalls)
- **Typical:** 1 transaction per 3-5 cycles (APB overhead)

37.12 Synthesis Notes

37.12.1 Resource Usage

Typical (ADDR_WIDTH=12, DATA_WIDTH=32): - ~50 LUTs - ~100 FFs - 0 BRAM

Components: - Command FSM: ~20 LUTs, ~40 FFs - Response FSM: ~10 LUTs, ~35 FFs - Strobe conversion: ~20 LUTs (combinatorial)

37.13 Related Modules

- **apb_slave_stub** - APB → cmd/rsp conversion (upstream)
- **apb_master_stub** - cmd/rsp → APB conversion

External: - PeakRDL: <https://github.com/SystemRDL/PeakRDL> - PeakRDL regblock: <https://github.com/SystemRDL/PeakRDL-regblock>

37.14 When to Use

✓ **Use This Adapter When:** - Integrating PeakRDL-generated register blocks - Using RTL Design Sherpa APB infrastructure - Need cmd/rsp ↔ passthrough cpuif conversion

✓ **Alternatives:** - PeakRDL native APB cpuif (if not using cmd/rsp) - Custom register implementation (if not using PeakRDL)

37.15 PeakRDL Workflow

Complete workflow documented in: `rtl/amba/shims/peakrdl_adapter_README.md`

Quick summary: 1. Write `.rdl` register description 2. Generate SystemVerilog: `peakrdl regblock my_regs.rdl -o my_regs.sv` 3. Select passthrough cpuif during generation 4. Use this adapter to bridge to APB infrastructure

Last Updated: 2025-10-20

37.16 Navigation

- [← Back to Shims Index](#)
- [← Back to RTLamba Index](#)